

Charon Documentation

version 2026.4.0

Denis Zykov

September 02, 2026

Contents

Data-Driven Game Design with Charon	1
What is Data-Driven Game Design?	1
Who Is This For?	1
Deployment Options	1
Is It Free?	2
Further Reading	2
Unreal Engine Plugin Overview	2
What is it?	3
Which problem does it solve?	3
Who is it for?	3
Key Features	4
Getting Started	4
Prerequisites	4
Installation from Marketplace	5
Building from Source Code	5
Core Concepts	5
Data-Driven Design Principles	5
Understanding the Plugin's Architecture	6
Working with the Plugin	6
Creating Game Data	6
Editing Game Data	6
Referencing Game Data in Blueprints	6
Advanced Features	7
Localization and Multi-Language Support	7
Referencing Unreal Engine Assets	7
Feedback	7
See also	7
How to Create Game Data File	8
Step By Step	8
Throubleshooting	8
See also	9
Unity Plugin Overview	9
What is it?	9
Which problem does it solve?	10
Who is it for?	10
Key Features	10
Getting Started	11
Prerequisites	11
Installation from OpenUPM (recommended)	11
Installation from Unity Asset Store	11
Installation from GitHub	12
Core Concepts	12
Data-Driven Design Principles	12
Understanding the Plugin's Architecture	12

Working with the Plugin	13
Creating Game Data	13
Editing Game Data	13
Advanced Features	13
Localization and Multi-Language Support	13
Referencing Game Data in Scenes	13
Referencing Unity Assets	14
Work & Build Automation	14
Feedback	14
See also	14
CharonCli Overview	15
Game Data Management	15
Import and Export	15
Localization (I18N)	15
Patching and Backup	15
Validation and Code Generation	15
Tool Utilities	16
See also	16
Migration from Legacy Version (Before 2025.1.*)	16
Automated Migration	16
Manual Migration	16
See also	17
Migrating to Web Application	17
Migration with Connection	17
See also	17
Standalone & CLI Overview	17
1. What is it?	18
2. Which problem does it solve?	18
3. For whom?	18
Prerequisites	19
Installation and Updates	19
dnx (recommended, .NET SDK 10+)	19
dotnet tool (.NET SDK 8+)	20
Bootstrap Script	20
Creating and Editing Game Data	21
See also	21
Web Application Overview	21
Starting with a new Project	22
See also	22
CLI Access to charon.live	22
Installing the CLI	23
Finding Your Project URL	23
Generating an API Key	23
Authentication	24
Common Operations	24
Export Data	24

Import Data	24
Generate Source Code	25
Validate Data	25
Backup	25
See also	25
Migrating to Web Application	26
Step 1: Backup Data Step by Step	26
Step 2: Restoring Backup in the Web Application	26
See also	26
Roles and Permissions	27
Where Roles Are Assigned	27
What Each Role Can Do	28
Inviting a Member	28
Which Role to Give	29
See also	29
REST API	29
Authentication	29
Request and Response Formats	30
Finding Your IDs	30
Response Envelope	30
Error Responses	31
Error code reference	31
See also	34
Workspaces, Projects, and Branches	34
Hierarchy	35
Workspace	35
Project	35
Branch	35
Creating a Project	36
Inviting Collaborators	36
Managing Branches in the Editor	36
Working with Branches via CLI	37
Branch-based Release Pipeline	38
How a push works	38
Typical branch setup	39
CI/CD pipeline example	39
Advanced: Patch-based Feature Merging	39
Cloning a Branch Locally	40
See also	40
Server Edition	40
Why Teams Need It	40
Feature Comparison	41
Collaboration Model	41
Authentication and Enterprise Integration	41
Access Control	42
Deployment Options	42

See also	42
On-Premises Deployment	42
When to Self-Host	42
Prerequisites	43
Data Compatibility	43
Contingency Plan	43
Getting Help	43
See also	43
Basic Navigation and User Interface Overview	44
The Editor Window	44
Side Navigation	44
Dashboard	45
Global Search	45
First Steps	46
The Working Area	46
Tips and Keyboard Shortcuts	46
See also	46
Creating Document Type (Schema)	47
Schema	47
Benefits of Structured Data	47
Data Organization	47
Data Validation	47
Data Consistency	47
Data Interoperability	47
Analyzing Game Requirements	47
Identifying Schemas and Relationships	48
Defining Schemas and Properties	48
Data Types	49
Date	49
Example	49
See also	49
Document	49
Example	50
See also	50
Document Collection	50
Example	51
See also	51
Formula	51
Example	52
See also	52
Integer	52
Example	52
See also	52
Localized Text	53
Example	53
See also	53

Logical	53
Use Cases	53
Comparison with MultiPickList	53
Example	54
See also	54
Multi-Pick List	54
Example	55
See also	55
Number	55
Example	56
See also	56
Pick List	56
Example	56
See also	57
Reference	57
Example	57
See also	57
Reference Collection	58
Example	58
See also	58
Text	59
Example	59
Sub-Types	59
See also	59
Time	59
Example	60
See also	60
Asset Path	60
Specification	60
UI Behavior	60
Validation Behavior	60
Asset Localization	60
Path Origin	61
Example	61
See also	61
Rectangle	61
UI Behavior	61
Validation Behavior	61
Example	61
Parsing	61
See also	62
Integer Rectangle	62
UI Behavior	62
Validation Behavior	62
Example	62
Parsing	62

See also	63
Tags	63
UI Behavior	63
Validation Behavior	63
Relation to Multi-Pick List	63
Example	64
See also	64
Vector 2/3/4	64
UI Behavior	64
Validation Behavior	64
Example	64
Parsing	64
See also	65
Integer Vector 2/3/4	65
UI Behavior	65
Validation Behavior	65
Example	65
Parsing	65
See also	66
Data Types	66
Composite Types	67
Use Cases and Guidance	67
See also	67
Display Text Template	67
Template Syntax	68
Property Access	68
Supported Data Types	68
Unary Operators	68
Binary Operators	68
Ternary Operator	69
Operator Precedence	69
Boolean Coercion	69
Format Specifiers	69
Collection Accessors	69
String Manipulation Methods	70
See also	70
Schema	70
Where Schemas Are Edited	70
Name	71
Display Name	71
Type	71
Display Text Template	71
Icon	71
Description	71
Group	71
Menu Visibility	71

Id Generator	71
Properties	71
See also	72
Schema Types	72
Normal	72
Component	72
Settings	73
Union (Tagged Union)	73
Why Union instead of the other inheritance patterns	73
Defining a Union schema	74
Storage format	74
Generated source code	74
Malformed Union values	74
Repairing malformed unions	74
See also	75
Specification	75
Overview	75
Use Cases	75
Encoding Guide	76
Encoding in JavaScript	76
Encoding in Excel	76
Best Practices	76
See also	76
Id Property	76
Generated Id	77
Generated ObjectId	77
Generated Guid	77
Generated Sequence	77
Generated Global Sequence	78
Non-Generated Id	78
Id Placeholder	78
See also	78
Schema Property	78
Where Properties Are Edited	79
Adding a Property	79
Name	80
Display Name	80
Description	80
Sync Name	80
Data Type	80
Requirement	80
Uniqueness	80
Default Value	80
Size In Bits	81
Generated Type Name	81
Pick Options	81

Specification	81
Id Property	81
See also	81
Shared Property	81
Replicated Parameters	81
Impact on Generated Source Code	82
See also	82
See also	82
Filling Documents	82
Where Documents Live	83
The Actions Menu	83
Creating Documents	84
Editing in the Grid	85
Finding Documents	86
Filtering by Column	86
Writing Filters by Hand	87
Sorting and Column Width	87
Choosing Columns	87
Changing What the Columns Are	87
Deleting Documents	88
Deleting a Schema	88
Bringing Data In From Files	89
Automating from the CLI	89
See also	90
Editing a Document	90
Opening the Form	90
Moving Between Documents	91
Fields	91
Reordering and Reusing Embedded Documents	91
Saving	91
Validation Errors	92
View Modes	92
JSON	93
Layout	93
Graph	94
The Actions Menu	94
When the Form Is Read-Only	95
See also	95
Implementing Inheritance	95
1. Composition	96
2. Merging	96
3. Aggregation	97
4. Tagged Union (Recommended)	98
Conclusion	98
See also	99
Schema	99

Where Schemas Are Edited	99
Name	100
Display Name	100
Type	100
Display Text Template	100
Icon	100
Description	100
Group	100
Menu Visibility	100
Id Generator	100
Properties	100
See also	101
Schema Types	101
Normal	101
Component	101
Settings	102
Union (Tagged Union)	102
Why Union instead of the other inheritance patterns	102
Defining a Union schema	103
Storage format	103
Generated source code	103
Malformed Union values	103
Repairing malformed unions	103
See also	104
Display Text Template	104
Template Syntax	105
Property Access	105
Supported Data Types	105
Unary Operators	105
Binary Operators	105
Ternary Operator	106
Operator Precedence	106
Boolean Coercion	106
Format Specifiers	106
Collection Accessors	106
String Manipulation Methods	107
See also	107
Specification	107
Overview	107
Use Cases	107
Encoding Guide	108
Encoding in JavaScript	108
Encoding in Excel	108
Best Practices	108
See also	108
Schema Property	108

Where Properties Are Edited	108
Adding a Property	109
Name	110
Display Name	110
Description	110
Sync Name	110
Data Type	110
Requirement	110
Uniqueness	110
Default Value	110
Size In Bits	110
Generated Type Name	111
Pick Options	111
Specification	111
Id Property	111
See also	111
Id Property	111
Generated Id	111
Generated ObjectId	112
Generated Guid	112
Generated Sequence	112
Generated Global Sequence	112
Non-Generated Id	112
Id Placeholder	113
See also	113
Shared Property	113
Replicated Parameters	113
Impact on Generated Source Code	114
See also	114
Data Types	114
Date	114
Example	114
See also	114
Document	115
Example	115
See also	115
Document Collection	116
Example	116
See also	116
Formula	116
Example	117
See also	117
Integer	117
Example	118
See also	118
Localized Text	118

Example	118
See also	118
Logical	119
Use Cases	119
Comparison with MultiPickList	119
Example	119
See also	119
Multi-Pick List	120
Example	120
See also	120
Number	121
Example	121
See also	121
Pick List	121
Example	122
See also	122
Reference	122
Example	123
See also	123
Reference Collection	123
Example	124
See also	124
Text	124
Example	124
Sub-Types	124
See also	125
Time	125
Example	125
See also	125
Asset Path	125
Specification	126
UI Behavior	126
Validation Behavior	126
Asset Localization	126
Path Origin	126
Example	126
See also	126
Rectangle	127
UI Behavior	127
Validation Behavior	127
Example	127
Parsing	127
See also	128
Integer Rectangle	128
UI Behavior	128
Validation Behavior	128

Example	128
Parsing	128
See also	129
Tags	129
UI Behavior	129
Validation Behavior	129
Relation to Multi-Pick List	129
Example	129
See also	129
Vector 2/3/4	129
UI Behavior	129
Validation Behavior	130
Example	130
Parsing	130
See also	130
Integer Vector 2/3/4	130
UI Behavior	131
Validation Behavior	131
Example	131
Parsing	131
See also	131
Data Types	132
Composite Types	132
Use Cases and Guidance	132
See also	133
Generating Source Code	133
Language Support Tiers	133
The Generation Wizard	134
Step 1. Target Language	134
Step 2. Customize Generation	135
Optimizations	137
Step 3. Summary	137
From the Unity and Unreal Engine Plugins	138
From the CLI	138
See also	139
Working with Source Code (C# 7.3)	139
Loading Game Data	139
Accessing Documents	140
Formulas	140
Calling a Formula	140
Formula Scope	140
Formula Syntax	141
Generated Code Contract	141
Extension of Generated Code	141
Partial Classes and Methods	141
Customizing Attributes	142

See also	142
Working with Source Code (C# 4.0)	142
Loading Game Data	143
Accessing Documents	143
Formulas	143
Generated Code Contract	143
Generated Code Extensions	144
See also	144
Working with Source Code (Type Script)	144
Loading Game Data	145
Accessing Documents	145
Formulas	145
Extension of Generated Code	146
Customizing Decorators	146
See also	146
Working with Source Code (UE C++)	146
Loading Game Data	147
Accessing Documents	147
Formulas	147
Calling a Formula	148
Formula Scope	148
Formula Syntax	148
Lambda Expressions	149
Generated Code Contract	149
Extension of Generated Code	149
Customizing Metadata	149
See also	150
Working with Source Code (Haxe)	150
Loading Game Data	150
Accessing Documents	150
Formulas	150
Extension of Generated Code	151
Customizing Metadata	151
See also	151
Working with Source Code (Lua)	151
Loading Game Data	152
Accessing Documents	152
Formulas	152
See also	152
Publishing Game Data	152
Data Version	153
Publishing from the Editor	153
Step 1. Collections Review	154
Step 2. Pick Format	154
Step 3. Languages	155
Step 4. Summary	155

Publishing from a Game Engine Plugin	156
Unity	156
Unreal Engine	157
Where the data ends up	157
Publishing from the CLI	158
See also	158
Importing and Exporting Data	158
Where Import and Export Live	159
Exporting Data	160
Step 1. Select Collections	160
Step 2. Pick Format	161
Step 3. Summary	161
Quick Export Without the Wizard	162
Importing Data	162
Step 1. Import Mode	162
Step 2. From	163
Step 3. File Input / Clipboard Input	163
Step 4. Select Collections	163
Step 5. Summary	164
Faster Paths	165
Validation During Import	166
Working with Exported Data	167
Modifying Exported Data	167
Creating Data from Scratch	167
Document Identification	167
Partial Updates	167
See also	167
Document Merge Logic	168
Core Principle	168
Import Mode	168
Property-Level Merge Rules	169
Scalar Properties	169
LocalizedString Properties	169
Inline Document Properties	169
Document Collections	170
Null optional collections	170
The Partial-Document Resurrection Caveat	170
Merge in Context	171
See also	171
Internationalization (i18n)	171
Overview	171
Configuring Languages	172
Localization (l10n)	173
Exporting Translatable Text	174
Step 1. Pick Format	174
Step 2. Languages	174

Step 3. Select Collections	174
Step 4. Summary	175
Importing Translated Data	175
Step 1. From	175
Step 2. File Input	175
Step 3. Select Collections	176
Step 4. Languages	176
Step 5. Summary	176
Machine Translation	177
Translating the Whole Project	177
Translating a Single Document	178
Working with Translation Teams	178
Best Practices	179
Unsupported Formats	179
Additional Resources	179
Validation	179
Where Validation Appears in the Editor	180
Errors in the Document Form	180
Finding Documents with Errors	181
Reviewing Before Publication	182
When Validation Runs	184
Validation Flags	184
Integrity check flags	184
Repair flags	185
Validation Report Format	185
Exit Code Behaviour	186
Common Recipes	186
Skip all checks (import prototype data)	186
Auto-repair + full check on import	186
Fix union conflicts after schema refactor	186
Check for untranslated strings	186
Gate a build on a clean publication review	186
Dry Run vs Validate	187
See also	187
Backup and Restore	187
Where Backup and Restore Live	187
Creating a Backup	188
Step 1. Destination	188
Step 2. Pick Format	188
Step 3. Summary	189
What is included	190
What is not included	190
Restoring from a Backup	190
Step 1. From	190
Step 2. File Input	191
Step 3. Summary	191

Backup vs Export	191
Scheduled Automated Backups	192
Pre-Publish Safety Snapshot	192
See also	193
Patch and Diff Workflow	193
Overview	193
Creating a Patch	194
Patch file format	194
Applying a Patch	194
Validation on apply	195
End-to-End Example: Feature Branch Merge	195
Storing Patches in Version Control	195
See also	195
Modding Support	196
Overview	196
Shipping the Editor with Your Game	196
Capturing Mod Changes as a Patch	197
Loading Mods at Runtime	197
How Patches Are Merged at Load Time	197
See also	198
CI/CD Integration	198
Installation in CI	198
Version Pinning and Compatibility	199
Tool version vs. format version	199
Authentication	199
Exit Codes	200
Common Pipeline Recipes	200
Validate on every pull request	200
Export and publish on tag	201
Apply a patch on merge	202
Export translation files for a localization vendor	202
Import translated files from a vendor	202
Recommended Pipeline Structure	203
See also	203
Command Line Interface (CLI)	203
Requirements	203
Installation	203
Option 1: dnx (recommended, .NET SDK 10+)	203
Option 2: dotnet tool (.NET SDK 8+)	204
Adding the global tool to PATH	204
Option 3: Bootstrap scripts (.NET SDK 8+)	205
Verifying the installation	205
Choosing how to invoke Charon	206
Command Syntax	206
Running on Windows, macOS and Linux	206
Absolute and relative paths	207

Understanding Paths	207
Usage in Terminals	207
Getting Help Text	207
Apply Patch (Merge)	208
Command	208
Parameters	208
See also	209
Create Backup	209
Command	210
Parameters	210
Output	211
See also	211
Create Document	211
Command	211
Parameters	212
Input Data Schema	214
Output	214
See also	214
Create Patch (Diff)	214
Command	215
Parameters	215
See also	216
Delete Document	216
Command	217
Parameters	217
Output	218
See also	218
Examine Game Data	219
Command	219
Parameters	219
Output	221
See also	221
Export Data	221
Command	222
Parameters	222
Output	225
Modifying Exported Data with <i>yq</i>	226
See also	226
Find Document	227
Command	227
Parameters	227
Output	228
See also	228
Find Game Data Files	229
Command	229
Parameters	229

Output	230
See also	231
Add Translation Languages	231
Command	231
Parameters	231
See also	232
Export Translated Data	232
Command	232
Parameters	233
Output	234
See also	234
Importing Translated Data	235
Command	235
Parameters	235
See also	238
List Translation Languages	239
Command	239
Parameters	239
See also	240
Import Data	241
Command	241
Parameters	241
Input Data Structure	245
See also	245
List Documents	245
Command	246
Parameters	246
Output	249
See also	249
List Schemas	249
Command	249
Parameters	250
Output	251
See also	252
Project Settings	252
Command	252
Parameters	253
Output	253
See also	254
Restore from Backup	254
Command	254
Parameters	254
See also	255
Search Documents	255
Command	256
Parameters	256

Output	257
See also	258
Update Document	258
Command	259
Parameters	259
Input Data Schema	261
Output	262
See also	262
Update Project Settings	262
Command	263
Available Properties	263
Parameters	264
Output	265
See also	265
Validate Game Data	265
Command	266
Parameters	266
Output Data Schema	267
See also	269
Generate C# Source Code	269
Command	269
Parameters	269
See also	272
Generate Haxe Source Code	272
Command	272
Parameters	272
See also	274
Generate Lua Source Code	275
Command	275
Parameters	275
See also	277
Export Code Generation Templates (Obsolete)	278
Command	278
Parameters	278
See also	278
Generate Text from Templates (Obsolete)	278
See also	279
Generate TypeScript Source Code	279
Command	279
Parameters	279
See also	281
Generate Unreal Engine C++ Source Code	282
Command	282
Parameters	282
See also	284
Initialize Game Data	285

Command	285
Parameters	285
See also	285
URL input/output parameters	285
Supported URL Schemes	285
Authentication	285
Examples	286
See also	286
Start in Standalone Mode	286
Command	286
Parameters	287
See also	287
Universal parameters	287
Environment variables	287
See also	288
Get Charon Version	288
Command	288
Parameters	288
See also	288
See also	288
MCP Server (AI Assistant Integration)	288
Prerequisites	289
Starting the MCP Server	289
Logging	290
Configuring AI Clients	290
Claude Desktop	290
Claude Code (CLI)	290
Cursor	290
VS Code (GitHub Copilot)	290
Windsurf	291
JetBrains IDEs (Rider, IntelliJ IDEA, WebStorm, etc.)	291
Cline (VS Code Extension)	291
Zed	291
Authentication for Remote Sources	292
Available Tools	292
Limitations and Remote-Only Use Case	293
Game Data Structure	293
Game Data	293
Project Settings	294
Schema	295
Schema Property	295
See also	297
Formula Expression AST	297
Structure	300
TypeReference	300
Expression Types	300

Constant	300
MemberResolve	301
Unary	302
Binary	302
Condition	303
Convert / TypeAs	303
Default	304
TypeIs	304
TypeOf	305
Index	305
Invoke	305
Lambda	306
New	306
NewArrayBounds	307
NewArrayInit	307
MemberInit	307
ListInit	308
Member Bindings	309
AssignmentBinding	309
MemberBinding	309
ListBinding	309
Implementing an Interpreter	309
See also	310
UI Extensions	310
What an Extension Can Add	310
How Extensions Are Declared	311
Choosing an Editor for a Property	312
Installing an Extension	312
Consent and Trust	313
When an Extension Fails to Load	313
Packaging and Publishing	314
Examples	314
See also	315
Implementing CharonPropertyEditorElement	315
ValueControl	315
Purpose of ValueControl	315
Reacting to Value Changes	316
Handling Focus Events	316
Providing Custom Validators	316
Reaching Host Services	317
See also	317
Implementing CharonSchemaEditorElement	317
Declaring the Editor	318
The Element Contract	318
Working with the Document	319
Read-Only Documents	320

Document Services	320
Undo and Redo	320
Development	320
See also	321
Custom Actions	321
Declaring an Action	321
Where Actions Appear	322
Grouping Side Navigation Entries	322
The Action Context	323
Reporting Progress	323
See also	324
Custom Pages	324
Declaring a Page	325
The Page Element	325
The Page Context	326
Persisting Page State	326
See also	326
Host Services	327
What Each Context Carries	327
Game Data	328
Dialogs and Wizards	329
Progress	329
Your Own Dialog	329
Charon's Wizards	329
Notifications	330
Navigation	330
Persisted UI State	330
Current User, Project, and Workspace	331
See also	331
Creating a Custom Editor with React	331
Introduction to Web Components	331
Step-by-Step: Building a Logical Toggle Editor	331
1. Create a New React Project	332
2. Install Required Dependencies	332
3. Write and Export Your Web Component	332
4. Modify <code>vite.config.ts</code> for Packaging	332
5. Update <code>package.json</code> for Packaging	333
Debugging & Local Testing	333
Test Locally in Charon	333
Disable Optimizations for Browser Debugging	334
Versioning	334
Publishing	334
See also	334
Creating a Custom Editor with Angular	335
Introduction to Web Components	335
Step-by-Step: Building a Color Picker Editor	335

1. Install the Latest Angular CLI	335
2. Create a New Angular Project	335
3. Install Required Dependencies	335
4. Write and Export Your Web Component	335
5. Modify <code>angular.json</code> for Packaging	336
6. Update <code>package.json</code> for Packaging	336
7. Enable Standalone/Zoneless Mode (Performance)	337
Debugging & Local Testing	337
Test Locally in Charon	337
Disable Optimizations for Browser Debugging	337
Versioning	337
Publishing	338
See also	338
Troubleshooting Common Issues	338
Editor Fails to Launch or Gets Stuck Loading	339
Web UI Shows "Http failure response for api/v1/context/: 0 Unknown Error"	339
Blank Page or Extension Fails to Load	339
Game Data File Is Corrupted After Version Control Operations	340
Validation Errors on System Schemas (Schema, ProjectSettings, SchemaProperty)	340
".NET SDK is not installed" During Setup	340
Exported XLSX File Contains an "Errors" Sheet	341
Game Data Is Read-Only in the Web Editor	341
Unreal Engine Compile Errors After an Engine Upgrade	341
Import Regenerates IDs of Nested Documents	341
Nothing Helped?	341
See also	342
Working with Logs	342
Logging Levels	342
See also	342
Resetting UI Preferences	342
See also	343
Apply Patch (Merge)	343
Command	343
Parameters	343
See also	344
Create Backup	344
Command	345
Parameters	345
Output	346
See also	346
Create Document	346
Command	346
Parameters	347
Input Data Schema	349
Output	349
See also	349

Create Patch (Diff)	349
Command	350
Parameters	350
See also	351
Delete Document	351
Command	352
Parameters	352
Output	353
See also	353
Examine Game Data	354
Command	354
Parameters	354
Output	356
See also	356
Export Data	356
Command	357
Parameters	357
Output	360
Modifying Exported Data with <i>yq</i>	361
See also	361
Find Document	362
Command	362
Parameters	362
Output	363
See also	363
Find Game Data Files	364
Command	364
Parameters	364
Output	365
See also	366
Add Translation Languages	366
Command	366
Parameters	366
See also	367
Export Translated Data	367
Command	367
Parameters	368
Output	369
See also	369
Importing Translated Data	370
Command	370
Parameters	370
See also	373
List Translation Languages	374
Command	374
Parameters	374

See also	375
Import Data	376
Command	376
Parameters	376
Input Data Structure	380
See also	380
List Documents	380
Command	381
Parameters	381
Output	384
See also	384
List Schemas	384
Command	384
Parameters	385
Output	386
See also	387
Project Settings	387
Command	387
Parameters	388
Output	388
See also	389
Restore from Backup	389
Command	389
Parameters	389
See also	390
Search Documents	390
Command	391
Parameters	391
Output	392
See also	393
Update Document	393
Command	394
Parameters	394
Input Data Schema	396
Output	397
See also	397
Update Project Settings	397
Command	398
Available Properties	398
Parameters	399
Output	400
See also	400
Validate Game Data	401
Command	401
Parameters	401
Output Data Schema	403

See also	404
Generate C# Source Code	404
Command	405
Parameters	405
See also	407
Generate Haxe Source Code	408
Command	408
Parameters	408
See also	410
Generate Lua Source Code	411
Command	411
Parameters	411
See also	413
Export Code Generation Templates (Obsolete)	414
Command	414
Parameters	414
See also	414
Generate Text from Templates (Obsolete)	414
See also	415
Generate TypeScript Source Code	415
Command	415
Parameters	415
See also	417
Generate Unreal Engine C++ Source Code	418
Command	418
Parameters	418
See also	420
Initialize Game Data	421
Command	421
Parameters	421
See also	421
URL input/output parameters	421
Supported URL Schemes	421
Authentication	421
Examples	422
See also	422
Start in Standalone Mode	422
Command	422
Parameters	423
See also	423
Universal parameters	423
Environment variables	423
See also	424
Get Charon Version	424
Command	424
Parameters	424

See also	424
Frequently Asked Questions (FAQ)	424
What kind of game data can be modeled in Charon?	424
What data types are supported in Charon?	424
Is schema inheritance supported?	425
Is Charon free to use?	425
Are there limitations in the Unity/Unreal plugins or CLI version?	425
Does Charon support localization?	425
How does Charon integrate with Unity or Unreal Engine?	425
Can I define relationships between documents (e.g., parent-child or references)?	425
Can I generate custom editors or tools with Charon?	425
Does Charon support formulas or computed fields?	425
Can Charon be integrated into CI/CD pipelines or custom toolchains?	425
What export formats does Charon support?	425
Can I control user access and editing permissions?	426
Is the editor's source code available?	426
What happens if the project is discontinued or no longer maintained?	426
Glossary	426

Data-Driven Game Design with Charon

Every game team hits the wall eventually.

It starts small - a spreadsheet for item stats, another for enemy data, a few config files for quests. Then the game grows. Tabs multiply. Formulas break silently. A programmer renames a constant and three designer files now reference something that doesn't exist. A localization request arrives and no one knows where all the strings actually live.

This is the spreadsheet ceiling - and most teams don't realize they've hit it until something breaks in production.

Charon exists to push that ceiling out of your way.

What is Data-Driven Game Design?

Data-driven game design means your game's content lives in *structured, validated data* - not in source code, not in spreadsheets, not in a programmer's head. Designers define what a quest looks like. Programmers consume it through generated, type-safe code. The pipeline enforces that they stay in sync.

When it works:

- A designer changes a boss's phase thresholds without filing a ticket
- Deleting a deprecated item immediately surfaces every quest that still references it
- Adding a language is a data task, not a code task
- Game balance is version-controlled, diff-able, and rollback-able

Charon is the tool that makes this practical. It gives your team a shared language between design and engineering - a structured database, a visual editor, and a code generator that keeps both sides moving independently.

TLDR: Charon is an in-game database for your game. It replaces spreadsheets and config files with a validated, schema-driven data layer that generates ready-to-use code for your engine.

Who Is This For?

Game Designers

You get a real editor - schemas you define, forms that match your data, and validation that catches mistakes before the next build does. No more pasting values into cells and hoping the formula still works.

Programmers

You get generated, type-safe classes for your data. Instead of parsing raw JSON and casting strings, you write `gameData.Heroes.Find("Arbalest").Stats.Hp` with full IDE autocomplete. The loading, validation, and localization plumbing is already handled.

Technical Leads

You get a pipeline. Charon has a CLI, a REST API, and CI/CD hooks. Validate data on every commit. Diff game data between versions. Patch live data without a full rebuild. Distribute the editor with your game for community modding.

Deployment Options

Charon adapts to your workflow, not the other way around:

- Unity Plugin - edit game data directly inside the Unity editor

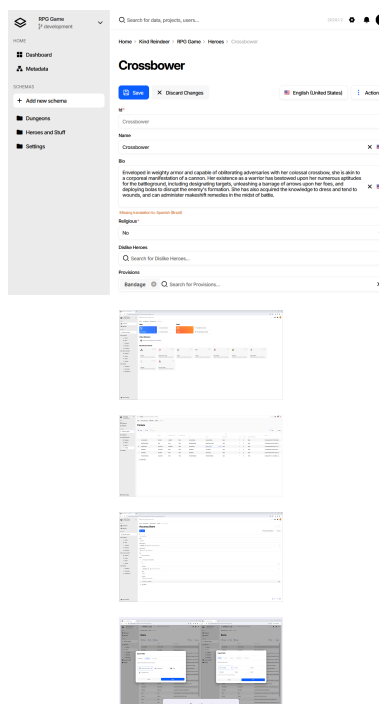
Is It Free?

- Unreal Engine Plugin - native integration with the Unreal content pipeline
 - Standalone App - run the editor locally for any engine (C#, TypeScript, Haxe)
 - Web / Cloud Edition - collaborative editing for distributed teams
-

Is It Free?

The offline CLI, standalone app, and engine plugins are **completely free** with no usage restrictions. They are open-licensed and can be bundled with your shipped game - so your players can build mods using the same tools you used to build the game.

The online collaborative edition, which enables shared editing across large teams with cloud hosting, requires a subscription.



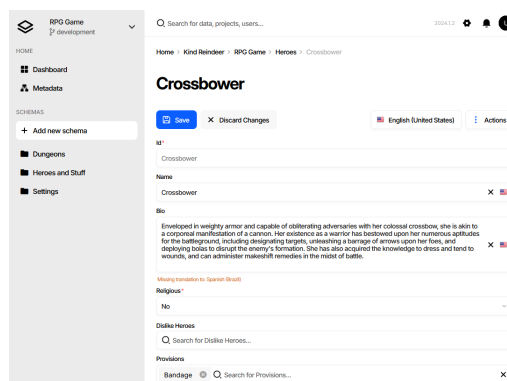
Further Reading

Unreal Engine Plugin Overview

DataTables work - until they don't.

A flat CSV-backed table handles a list of weapons just fine. But the moment a Quest needs to reference three Items, each with variant stats that inherit from a base type, with display text in four languages - DataTables hit a wall. You end up with parallel tables held together by string row names, a Blueprint full of GetDataTableRow nodes, and struct definitions that go stale the moment a designer adds a field.

Unreal's data tools were built for simplicity, not for scale. The [Charon Unreal Plugin](#) is built for exactly the kind of complex, relational, multi-language game data that real productions demand - with native C++ and Blueprint access and zero boilerplate.



What is it?

The **Charon Unreal Plugin** is a relational game database embedded in the Unreal Editor. It gives you:

- A **schema-driven data model** - define your Hero, Item, Quest types once, with typed fields, inheritance, and cross-references between them
- A **visual editor** that opens directly from the Content Drawer - same workflow designers already know, no new tools to learn
- **Automatic C++ and Blueprint generation** - every time your schema changes, your struct definitions and access code update themselves

It is the data layer your game needs without the infrastructure you'd have to build to get there.

Which problem does it solve?

Pain Point	How Charon Fixes It
DataTable limitations	Full relational model: nesting, inheritance, and cross-table references without string row-name fragility.
Struct boilerplate	C++ structs and Blueprint nodes are generated automatically - change a field in the editor, not in code.
Broken cross-references	Referencing a deleted row is a validation error, caught before the build, not at runtime.
Designer bottleneck	A purpose-built UI lets designers tune values and write content without touching C++ or Blueprints.
Localization complexity	Localizable Text fields export directly to your translation pipeline and re-import cleanly.
Live-ops friction	Dynamic data loading via TryLoad supports hot updates and A/B testing without a full rebuild.

Who is it for?

Technical Directors & Programmers

Generated C++ structs give you type-safe data access with full IDE support and no parsing logic. The CLI and REST API plug directly into your CI/CD pipeline - validate data on every commit, diff between versions, automate patch builds.

Game & Narrative Designers

Build complex item economies, branching quest trees, and character progressions in a structured editor - without writing a line of C++ or creating Blueprint spaghetti. Validation catches broken references before they reach a programmer.

Live-Ops Teams

Load updated game data at runtime without a store submission. Roll back a bad balance change in minutes. Ship localization updates as data patches, not engine builds.

Key Features

Feature	Benefit to Your Workflow
C++/Blueprint Generation	Access your data instantly with full IDE auto-complete and Blueprint nodes.
Relational Modeling	Interconnect tables (Items, Quests, NPCs) with ease within a single UI.
Hot Updates	Support for dynamic loading of game data for live-tuning and hotfixes.
Modding Support	Give your community the same professional tools to create game mods.
Localization	Seamlessly export and import translation keys for global releases.

Getting Started

To begin using [this](#) plugin, the initial step involves installing the plugin from the Unreal Engine Marketplace. Once installed, you'll need to [enable the plugin](#) for your project through the project settings. Following this, a rebuild of your project's C++ code is necessary. The final step in the setup process is the creation of your first game data file.

Prerequisites

The Unreal Engine plugin is written in C++ but relies on the dotnet-charon tool, a .NET application built for .NET 8. Install the [.NET SDK 10 or later](#); SDK 8 is the minimum supported version.

Note

The plugin requires **Unreal Engine 5.3 or later**. The generated C++ code depends on the plugin's runtime module, so the plugin is the only supported integration path - generated code cannot be used standalone on older engine versions.

Windows

1. Download and install [.NET SDK 10+](#).
2. Make sure you have write access to %PROGRAMDATA%/Charon.

MacOS

1. Download and install [.NET SDK 10+](#).
2. Make sure you have write access to /Users/<username>/ .config/Charon.
3. Make sure dotnet is available from \$PATH.

Linux

1. Download and install [.NET SDK 10+](#).
2. Make sure you have write access to /home/<username>/ .config/Charon.
3. Make sure dotnet is available from \$PATH.

Checking Available .NET SDK Versions

```
# check for dotnet already installed
dotnet --list-sdks
```

```
# output for dotnet --list-sdks
7.0.120 [C:\Program Files\dotnet\sdk]
```

```
8.0.206 [C:\Program Files\dotnet\sdk]
8.0.414 [C:\Program Files\dotnet\sdk] # <- this one is fine
9.0.300 [C:\Program Files\dotnet\sdk] # <- this one too
```

Installation from Marketplace

1. Add to cart Charon plugin [[Epic Launcher](#)] / [[Fab.com Asset Page](#)] in the Unreal Engine Marketplace.
2. Follow the [instruction](#) on installing plugin into your project:
 - a. Click **Install to Engine** and select the engine version.
 - b. Open your project and go to **Edit** → **Plugins...** window.
 - c. Type **Charon** in the **Search** bar.
 - d. Check the checkbox near the plugin's name to enable it.
3. Rebuild project C++ code.

Building from Source Code

1. Clone or download the [plugin source code](#) from the GitHub repository.
2. Create a <project-dir>/Plugins/Charon directory.
3. Copy the plugin files into this directory. Ensure **Charon.uplugin** is located at the path <project-dir>/Plugins/Charon/Charon.uplugin after copying.
4. Remove the "EngineVersion" attribute if your engine doesn't match the plugin's engine version.
5. Rebuild the project's C++ code.
6. Enable the plugin in **Edit** → **Plugins...** if needed.

Core Concepts

Data-Driven Design Principles

The rule is straightforward: **if a designer might want to change it, it belongs in data - not in C++ and not in a Blueprint.**

Damage formulas, AI behavior thresholds, loot table weights, narrative branch conditions - when these live in code, every balance pass requires an engineer. When they live in structured data, a designer can iterate in an afternoon without touching the build.

Unreal Engine has always encouraged this approach. DataTables, UDeveloperSettings, and config files are all steps in that direction. Charon takes it further: a full relational model with validation, generated access code, and a UI that non-technical team members can use confidently.

The payoff compounds over time:

- **During production:** balance changes and content additions don't block engineering
- **At ship:** every game value is version-controlled, auditable, and rollback-able
- **Post-launch:** mods, live patches, and A/B tests are data operations, not code releases

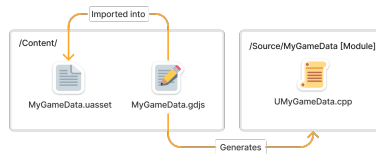
Further reading:

- [Data Driven Gameplay Elements \(UE Documentation\)](#)
- [Modify Everything! Data-Driven Dynamic Gameplay Effects on 'For Honor' \(Video\)](#)
- [Data-driven Design in Unreal \(Article\)](#)

Understanding the Plugin's Architecture

Plugin Assets

Working with data in this plugin is akin to how the built-in *DataTable* functions. There is a data source file, a module containing the code required to load the data, and an asset that will be utilized in the game. Whenever you edit a data source file, you need to re-import this data into the asset. Should the data structure in the source file change, then the C++ code must be regenerated.



For scenarios requiring dynamic loading of game data, this can be accomplished through the `TryLoad` method on the game data class, which accepts the source JSON file.

Plugin Modules

The Charon plugin is structured into two modules:

- `CharonEditor` module acts as an Unreal Engine Editor extension. Extension points for the module are declared in the `ICharonEditorModule` class, and automation of game data processing is facilitated through the `FCharonCli` class.
- `Charon` module, houses the core logic and shared code crucial for handling game data files.

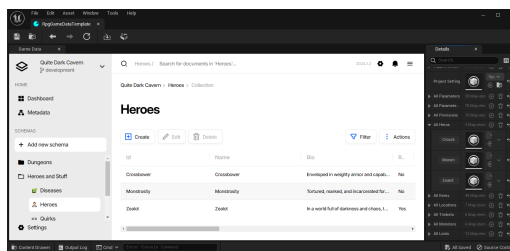
Working with the Plugin

Creating Game Data

To create a new game data file within the Unreal Engine Editor, open the **Content Drawer**, right-click in the desired folder, and select in the **Create Advanced Assets** section **Miscellaneous** → **Game Data** menu option. Name your game data file and proceed according to the instructions in the dialog window that appears.

Detailed guide on how to create game data.

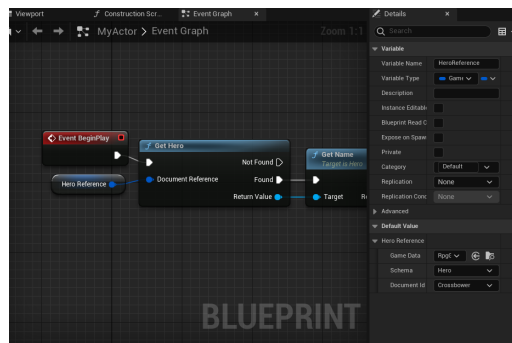
Editing Game Data



To edit a game data file in the Unreal Engine Editor, navigate to the **Content Drawer**, find the corresponding .uasset file, and double-click it. This action opens a new window featuring a user interface for editing the game data. Remember to reimport and, if necessary, regenerate the source code after completing your edits.

Referencing Game Data in Blueprints

Similar to the `DataTable`'s `FDataTableRowHandle`, the Charon plugin introduces a specific type for referencing documents within Blueprints, named `FGameDataDocumentReference`. This type is housed within the Charon module. Here is an example of **Game Data Document Reference** used to resolve *Hero* document:



Advanced Features

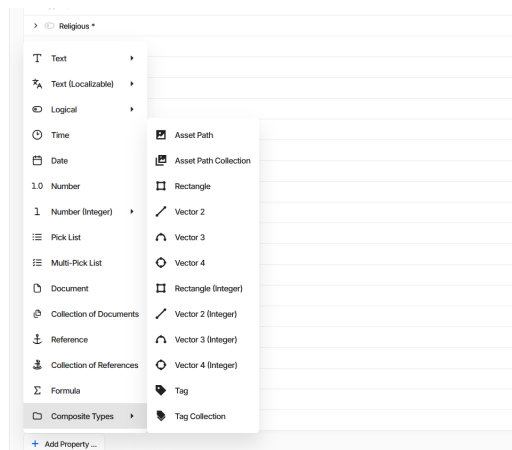
Localization and Multi-Language Support

Charon facilitates multi-language text support through the Localizable Text data type. When creating a *Schema*, properties can be defined with various data types, including Localizable Text. Initially, all localizable text defaults to EN-us (US English). Additional languages can be added via **Project Settings** → **Internationalization** → **Translation Languages** in the Charon UI.

Exporting/importing localizable data.

Referencing Unreal Engine Assets

To reference assets within the game, you can use a special Asset composite type. Create a property via Composite Types -> Asset Path menu options.



Feedback

We welcome and encourage feedback, particularly bug reports and suggestions, to help improve our tool. If you have any questions or would like to share your thoughts, please join our [Discord community](#) or reach out to us via email at support@gamedevware.com.

See also

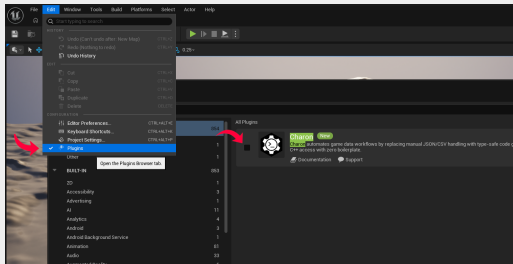
- Basic Navigation and User Interface Overview
- Creating Document Type (Schema)
- Filling Documents
- Frequently Asked Questions (FAQ)
- Glossary

How to Create Game Data File

To create a new game data file within the Unreal Engine Editor, open the **Content Drawer**, right-click in the desired folder, and select in the **Create Advanced Assets** section **Miscellaneous** → **Game Data** menu option. Name your game data file and proceed according to the instructions in the dialog window that appears.

Note

The plugin must be enabled after installation before the **Game Data** menu option becomes available. Open **Edit** → **Plugins...**, find **Charon** in the list, and check its checkbox.



Step By Step

1. **Open Content Drawer:** Open the *Content Drawer* window in the Unreal Engine Editor.
2. **Select Folder:** Right-click in the desired folder where you want to create the game data file.
3. **Create Game Data:** Navigate to **Create Advanced Assets** → **Miscellaneous** → **Game Data** from the context menu.
4. **Name the File:** In the *Content Drawer* window that appears, enter a name for your game data file.
5. **Check for Errors:** Ensure there are no error messages in the dialog window that opens, then press *Next*.
6. **Wait for Module Generation:** Allow time for the new module to be generated, watching the wizard in the dialog proceed to the next step automatically.
7. **Review Summary:** Check the summary and verify there are no suspicious errors in the *Output Log* window.
8. **Recompile C++ Code:** Use your IDE of choice to recompile the C++ code. Restart Unreal Engine Editor if needed.
9. **Import Game Data:** Reopen the *Content Drawer* window and click the **Import** button.
- 10 **Select .gdjs File:** Locate and select the *.gdjs* game data file you named in step 4, then click *Ok*.
- 11 **Choose Game Data Class:** Select the *Game Data* class, which should match the game data file name. If it's not listed, return to step 7.
- 12 **Save .uasset File:** Save the newly created *.uasset* file after completing the import process.

Throubleshooting

Game data creation or code generation/compilation may encounter issues under certain circumstances:

Module Generation Fails on a Blueprint-Only Project - Problem: The project has no C++ source yet (Blueprint-only), so there is no *.Target.cs*/IDE project for the wizard to add the generated module to, and module generation fails. - **Solution:** Convert the project to a C++ project first, then retry. The easiest way is via **Tools** → **New C++ Class...**, picking any class (e.g. *Actor*), then *Next* → *Create Class*. This generates the missing C++ project files; the new class itself isn't needed and can be deleted afterward.

Insufficient File System Rights or File Creation Errors - Problem: Lack of sufficient rights to the OS file system, or errors during file creation (e.g., file name too long, antivirus block). - **Solution:** Check the *Output Log* window for errors or the most recent log file in <project-dir>\Intermediate\Charon\logs and attempt to resolve them.

Class Name Collision Within Project - Solution 1 (Game Data Class Name Collision): Delete the newly created .gdjs game data file and the generated module. Then, start over with a new name and clean your .Target.cs files from the generated module name. - **Solution 2** (Schema Class Name Collision): Open the game data in another editor (Online, Standalone), rename the schema, and try again.

No Game Data Class in Import Window - Problem: The generated game data module is not being compiled. - **Solution:** Ensure it's added to your <project-name>.Target.cs and <project-name>Editor.Target.cs files as an extra module. If missing, include following expression in both target files:

```
ExtraModuleNames.Add("<module-name>");
```

Additionally, verify that your .uproject file includes the generated module definition. If it's absent, add the following module definition to the **Modules** list:

```
{
  "Name": "<module-name>",
  "Type": "Runtime",
  "LoadingPhase": "Default"
}
```

See also

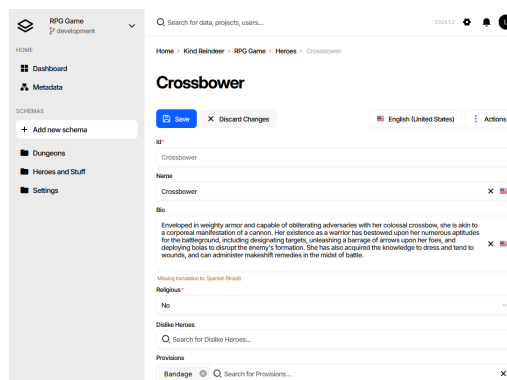
- Basic Navigation and User Interface Overview
- Creating Document Type (Schema)
- Filling Documents
- Frequently Asked Questions (FAQ)
- Glossary

Unity Plugin Overview

You know the moment. Your project starts small - a few ScriptableObjects for enemy stats, a couple of prefabs with Inspector overrides. Clean, manageable, very Unity.

Then six months in, you have 300 .asset files scattered across a dozen folders. A designer needs to change a damage multiplier and can't tell which one to open. A programmer renames a field and three of those assets silently break. A new team member spends an afternoon just trying to understand where the quest data actually lives.

ScriptableObjects were never designed to be a game database. The [Charon Unity Plugin](#) replaces that fragile patchwork with a structured, validated game database - embedded directly in your Unity project.



What is it?

The **Charon Unity Plugin** is a full data-driven design workflow embedded in the Unity Editor. It gives you:

- A **structured game database** - one organized place for all your items, characters, quests, and abilities, with enforced relationships between them
- A **visual editor** that opens from the Unity menu and runs in your browser - no external tools, no switching context
- A **C# code generator** that turns your schema into type-safe classes, so your code reads `hero.Stats.Hp` instead of `(float)data["stats"]["hp"]`

Designers own the data. Programmers own the code. Charon keeps both in sync automatically.

Which problem does it solve?

Pain Point	How Charon Fixes It
ScriptableObject sprawl	All entities live in one database file. No more hunting through hundreds of .asset files.
Broken references	Deleting an item that a quest still references is a validation error, not a silent runtime bug.
Boilerplate data code	Generated C# classes handle all loading, deserialization, and lookups - zero hand-written plumbing.
Designer bottleneck	Designers edit data in a purpose-built UI without waiting on a programmer to change a value.
Localization sprawl	All translatable text lives in one place, exportable to any translation pipeline in one click.

Who is it for?

Unity Developers

Stop writing data-loading boilerplate. Generated C# classes give you full IntelliSense, compile-time safety, and zero magic strings. Plug into CI/CD with the CLI - validate data on every commit.

Game & Narrative Designers

A real editor, not a spreadsheet workaround. Define your own schemas, fill in data, and see validation errors immediately - without touching a line of code or filing a ticket.

Production Teams

Building an RPG, CCG, strategy game, or anything with hundreds of balanced entities? Charon scales with you. It also supports modding out of the box - ship the editor alongside your game so your community builds with the same tools you used.

Key Features

Feature	Description
In-Editor Server	Launch the data editor directly from the Unity menu bar.
Auto-Generation	C# code is updated automatically whenever you change your data schema.
Asset Integration	Link your game data directly to Unity Assets (like Prefabs, Sprites, etc.).
Memory Efficient	Optimized loading routines designed specifically for mobile/console performance.

Getting Started

Prerequisites

Unity plugin uses the dotnet-charon tool, a .NET application built for .NET 8. Install the [.NET SDK 10 or later](#); SDK 8 is the minimum supported version.

Note

The plugin requires **Unity 2022.3.62f3 or later**. The generated C# code itself is far less demanding - the C# 4.0 generator produces code that works even on Unity 3.4 - but there is no plugin support for such legacy versions. For older Unity versions, take the “other engines” path instead: edit game data with the standalone editor or CLI, generate C# source code, and wire up loading with your own tooling and helpers.

Windows

1. Download and install [.NET SDK 10+](#).
2. Make sure you have write access to %PROGRAMDATA%/Charon.

MacOS

1. Download and install [.NET SDK 10+](#).
2. Make sure you have write access to /Users/<username>/ .config/Charon.
3. Make sure dotnet is available from \$PATH.

Linux

1. Download and install [.NET SDK 10+](#).
2. Make sure you have write access to /usr/share/Charon.
3. Make sure dotnet is available from \$PATH.

Checking Available .NET SDK Versions

```
# check for dotnet already installed
dotnet --list-sdks
```

```
# output for dotnet --list-sdks
```

```
7.0.120 [C:\Program Files\dotnet\sdk]
8.0.206 [C:\Program Files\dotnet\sdk]
8.0.414 [C:\Program Files\dotnet\sdk] # <- this one is fine
9.0.300 [C:\Program Files\dotnet\sdk] # <- this one too
```

Installation from OpenUPM (recommended)

1. Install the required software for your operating system.
2. Ensure your Unity version is 2022.3.62f3 or later.
3. Open the [OpenUPM](#) page for the plugin.
4. Click the **Manual Installation** button in the upper right corner and follow the instructions.

Installation from Unity Asset Store

1. Install the required software for your operating system.
2. Ensure your Unity version is 2022.3.62f3 or later.
3. Open the [Charon plugin](#) in the Unity Asset Store.
4. Click **Add To My Assets**.

5. Open the Unity Package Manager by navigating to **Window → Package Manager**.
6. Wait for the package manager to populate the list.
7. Select **My Assets** from the dropdown in the top left corner.
8. Select **Charon** from the list and click **Download**. If it's already downloaded, you will see an **Import** option.

Installation from GitHub

1. Install the required software for your operating system.
2. Clone or download the [plugin source code](#) from the GitHub repository.
3. Create a <project-dir>/Packages/com.gamedevware.charon directory.
4. Copy the plugin files from src/GameDevWare.Charon.Unity/Packages/com.gamedevware.charon into this directory.
5. Restart Unity if necessary.

Core Concepts

Data-Driven Design Principles

The core idea is simple: **keep data out of code, and keep code out of data.**

Instead of baking a weapon's damage range into a C# constant or a MonoBehaviour field, you define it in a schema and let designers own it. Instead of checking a quest's completion condition in a hardcoded switch statement, you describe it in structured data that the engine evaluates at runtime.

This separation pays off at every stage:

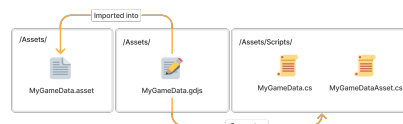
- **During production:** designers iterate without waiting for programmers; balance changes don't require rebuilds
- **At launch:** every value is traceable, auditable, and diffable in version control
- **Post-launch:** mods and live updates become data operations, not code deployments

Further reading:

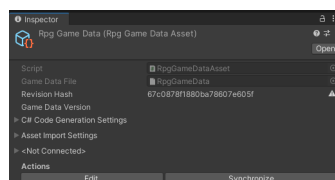
- [Modify Everything! Data-Driven Dynamic Gameplay Effects in 'For Honor' \(Video\)](#)
- [Data-driven Design in Unreal \(Article\)](#)

Understanding the Plugin's Architecture

Plugin Assets



All game data information is stored in a JSON file within your project. The generated source code is used to load this data into the game. Additionally, a ScriptableObject asset will be created, which can be used to access game data from your scenes.



Whenever there is a modification in the data structure within a JSON file, it is necessary to regenerate the C# source code and reimport the .asset file. To do this, select the .asset file and press the **Synchronize** button.

Working with the Plugin

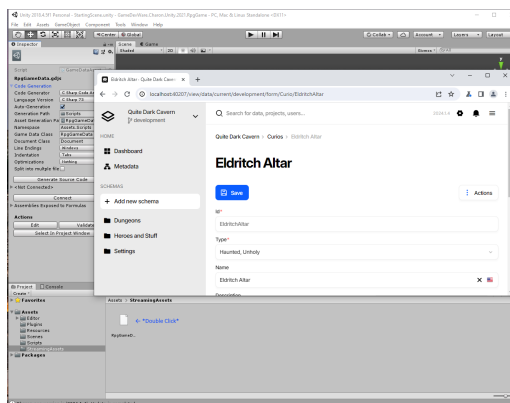
Creating Game Data

To create a new game data file within the Unity Editor, open the **Project** window, right-click in the desired folder, and select the **Create** → **Game Data** menu option.

1. Open the **Project** window and navigate to the desired folder.
2. Right-click in the **Project** window and select **Create** → **Game Data**.
3. Name your game data file and click the **Create** button.
4. Wait for the source code and assets to be created in the specified folder and for the editor to recompile the scripts.
5. Double-click the created .asset or .gdjs file to start editing.

Editing Game Data

To edit a game data file in the Unity Editor, open the **Project** window, find the corresponding .gdjs, .gdmp, or .asset file, and double-click it. This action opens a new web browser window featuring a user interface for editing the game data. Remember to **Synchronize** assets from the Inspector window after completing your edits.



Advanced Features

Localization and Multi-Language Support

Charon facilitates multi-language text support through the Localizable Text data type. When creating a *Schema*, properties can be defined with various data types, including Localizable Text. Initially, all localizable text defaults to EN-us (US English). Additional languages can be added via **Project Settings** → **Internationalization** → **Translation Languages** in the Charon UI.

Exporting/importing localizable data.

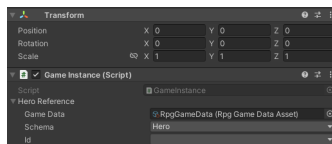
Referencing Game Data in Scenes

The Charon plugin introduces a specific type for referencing documents within scenes, named GameDataDocumentReference. This type is part of the Charon package. To create such a reference, add a field with the GameDataDocumentReference type to your component class.

```
public class HeroComponent : MonoBehaviour
{
    public GameDataDocumentReference heroReference;
}
```

Is It Free?

You can then configure it in the Inspector. Here is an example of a **Game Data Document Reference** used to point to a *Hero* document:

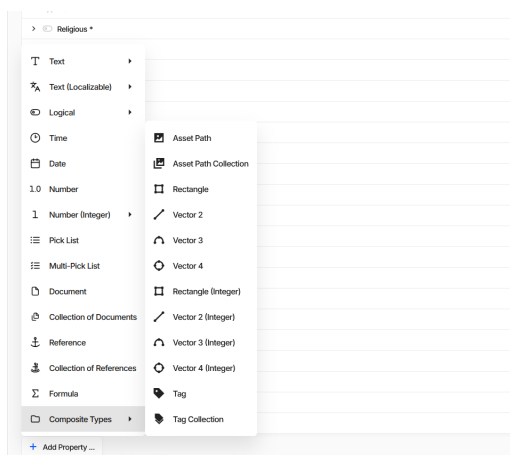


To get an instance of a document in your game code, call the `GameDataDocumentReference.GetReferencedDocument<Hero>()` method.

```
private void OnEnable()  
{  
    var hero = this.heroReference.GetReferencedDocument<Hero>();  
    Debug.Log(hero.Name);  
}
```

Referencing Unity Assets

To reference assets within the game, you can use a special Asset composite type. Create a property via Composite Types -> Asset Path menu options.



Work & Build Automation

To facilitate automation of work or builds, a programmatic interface for working with game data is provided. You can read more about it on the [CharonCli class documentation page](#).

Feedback

We welcome and encourage feedback, particularly bug reports and suggestions, to help improve our tool. If you have any questions or would like to share your thoughts, please join our [Discord community](#) or reach out to us via email at support@gamedevware.com.

See also

- Basic Navigation and User Interface Overview
- Creating Document Type (Schema)
- Filling Documents
- Frequently Asked Questions (FAQ)
- Glossary

CharonCli Overview

The [CharonCli](#) class provides a convenient interface for running dotnet charon command-line operations. It simplifies interactions with the Charon tool, enabling developers to manage game data, automate workflows, and integrate with Unity projects. Below is an overview of its methods grouped by purpose.

Game Data Management

- **InitGameDataAsync**: Initializes a GameData file.
- **CreateDocumentAsync**: Creates a document in the specified GameData URL.
- **UpdateDocumentAsync**: Updates a document in the specified GameData URL.
- **DeleteDocumentAsync**: Deletes a document in the specified GameData URL (by document or ID).
- **FindDocumentAsync**: Finds a document in the specified GameData URL by ID.
- **ListDocumentsAsync**: Lists documents in the specified GameData URL with optional filters and sorting.

Import and Export

- **ImportAsync**: Imports documents grouped by schema into a specified GameData URL.
- **ImportFromFileAsync**: Imports documents from a file into a specified GameData URL.
- **ExportAsync**: Exports documents from a GameData URL.
- **ExportToFileAsync**: Exports documents from a GameData URL to a file.

Localization (I18N)

- **I18NImportAsync**: Imports translated documents grouped by schema into a specified GameData URL.
- **I18NImportFromFileAsync**: Imports translated documents from a file into a specified GameData URL.
- **I18NExportAsync**: Exports documents for localization from a GameData URL.
- **I18NExportToFileAsync**: Exports documents for localization from a GameData URL to a file.
- **I18NAddLanguageAsync**: Adds translation languages to a GameData URL.

Patching and Backup

- **CreatePatchAsync**: Compares documents in two GameData URLs and creates a patch representing the difference.
- **CreatePatchToFileAsync**: Compares documents in two GameData URLs and saves the patch to a file.
- **ApplyPatchAsync**: Applies a patch to a specified GameData URL.
- **ApplyPatchFromFileAsync**: Applies a patch from a file to a specified GameData URL.
- **BackupAsync**: Backs up game data with all documents and metadata.
- **BackupToFileAsync**: Backs up game data to a file with all documents and metadata.
- **RestoreAsync**: Restores game data from a backup.
- **RestoreFromFileAsync**: Restores game data from a backup file.

Validation and Code Generation

- **ValidateAsync**: Validates all documents in a GameData URL and returns a report with issues.
- **GenerateCSharpCodeAsync**: Generates C# source code for loading game data into a game's runtime.
- **DumpTemplatesAsync**: Dumps T4 code generation templates into a specified directory.

Tool Utilities

- **GetVersionAsync**: Gets the version number of the Charon tool executable.
- **GetGameDataToolVersionAsync**: Gets the version of the Charon tool used to create a GameData URL.
- **RunCharonAsync**: Runs a specified command with the Charon tool.
- **RunT4Async**: Processes T4 templates using the dotnet - t4 command-line tool.

See also

- Unity Plugin Overview
- [CharonCli class](#)
- [Examples of CharonCli class](#)

Migration from Legacy Version (Before 2025.1.*)

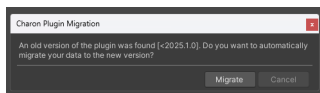
Warning

Before proceeding with the migration, ensure your project is under a source control system (e.g., Git) or that a full backup of your project has been created. Migration involves modifying and deleting files, and having a backup or version control ensures you can recover in case of unexpected issues.

Install the package with the new version of the plugin via the [Unity Asset Store](#) or using [OpenUPM](#) (recommended). After installing plugin package you have two options:

Automated Migration

A window will appear offering to perform the migration automatically.



1. Click the **Migrate** button and wait for the process to complete.
2. Once the migration is finished, close the window if everything is successful.
3. If an error occurs, check the **Console** window for details and consider using the *Manual Migration* approach.

Manual Migration

To migrate manually, you will need to remove the old plugin, convert, and configure the game data files:

1. Navigate to the Assets/Editor/GameDevWare.Charon folder and delete it.
2. Temporarily move all **.gdjs** and **.gdmp** files from Assets/StreamingAssets/ to Assets/.
3. Select each **.gdjs** or **.gdmp** file and click the **Reimport** button in the **Inspector** window.
4. Replace the old **.asset** file with the newly generated one. If the file did not exist previously, place it anywhere within the boundaries of the **.asmdef** file.
5. Replace the old source code files (**.cs**) with the newly generated ones.

Warning

Preserve the original **.meta** files for **.cs** and **.asset** assets to maintain Unity resource associations and links.

See also

- Unity Plugin Overview

Migrating to Web Application

To migrate to the <https://charon.live>, you can do it through a *backup* <../web/migrating_to_web> or through the “Connection” mechanism.

In short: you need to create an empty project in at <https://charon.live>, in Unity Editor in Inspector window click **Connect**, and specify that you want to upload data to the <https://charon.live>.

Migration with Connection

Be sure to back up your local data before making any connections.

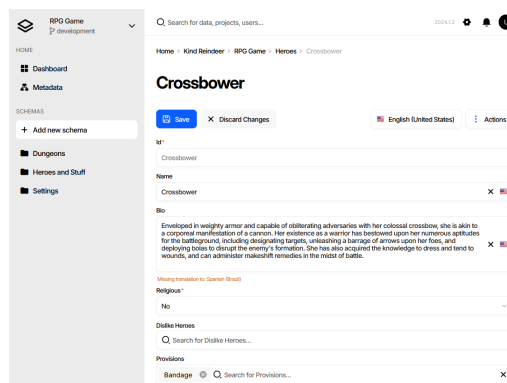
1. At <https://charon.live>: on the Home screen, click on **Create Project**.
2. Specify the project name, tags, and script language.
3. Click the **Create** button.
4. In the Unity Editor: select your game data **.asset** file in the **Project** window.
5. In the **Inspector** window, expand **<Not Connected>** foldout, click **Connect** button.
6. In the dialog that opens, click on the **Profile** → **API Keys** link.
7. At <https://charon.live>: a page of your profile on the *API Keys* management page should have opened in your browser.
8. Click **Create API Key...** button.
9. Fill in the name and expiration time, then click the **Create** button.
- 10 Click the **Copy** button in the list of keys next to the newly created key labeled “New!”.
- .
- 11 In the Unity Editor: paste the *API Key* into the corresponding field in the **Connect Game Data** window.
- .
- 12 Check the *Upload local data...* checkbox, it is only available when the selected *Project* is empty and does not contain any data.
- 13 Click the **Upload** button”.
- .
- 14 Close **Connect Game Data** window
- .

See also

- Basics
- Web Application Overview
- CLI Access to charon.live
- Workspaces, Projects and Branches
- [Charon Website](#)

Standalone & CLI Overview

For developers who require maximum control over their environment, Charon offers a flexible standalone distribution. This version is built for those who prefer working outside of a specific game engine or need to automate their data workflows.



1. What is it?

The standalone version is a **cross-platform .NET CLI application**. It operates as a dual-purpose tool:

- **Headless CLI:** A powerful command-line interface for data manipulation and code generation.
- **Local HTTP Server:** By launching a local server, it provides a high-performance **browser-based UI** for visual data modeling and editing without requiring an internet connection.

2. Which problem does it solve?

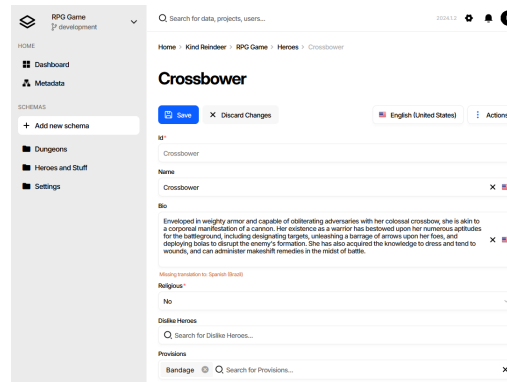
While the engine plugins are great for standard development, the Standalone/CLI version addresses specialized technical requirements:

- **Custom Pipeline Integration:** Easily integrate game data validation and code generation into custom build scripts or tools.
- **CI/CD Automation:** Run data integrity checks and generate source code automatically on build servers (GitHub Actions, Jenkins, etc.).
- **Modding Toolsets:** Distribute a lightweight, engine-agnostic tool to your players, enabling them to create and edit game content safely.
- **Engine-Agnostic Workflows:** Work on game data for custom engines or projects that don't use Unity or Unreal.

3. For whom?

This version is tailored for **advanced users and technical leads** who are comfortable with the terminal. It is ideal for:

- **Tools Engineers** building custom development pipelines.
- **Technical Designers** who need a standalone environment for content creation.
- **Build Engineers** automating game data processing in the cloud.



Prerequisites

Standalone application uses the [dotnet-charon](#) tool, which is distributed as a .NET tool. The [.NET SDK 10 or later](#) is recommended - it provides dnx, which runs Charon without installing anything. The .NET SDK 8 and 9 are also supported, but require an explicit install (see Installation).

Windows

1. Download and install the [.NET SDK 10+](#).
2. Make sure you have write access to %PROGRAMDATA%/Charon.

MacOS

1. Download and install the [.NET SDK 10+](#).
2. Make sure you have write access to /Users/<username>/ .config/Charon.
3. Make sure dotnet is available from \$PATH.

Linux

1. Download and install the [.NET SDK 10+](#).
2. Make sure you have write access to /home/<username>/ .config/Charon.
3. Make sure dotnet is available from \$PATH.

Checking Available .NET Versions

```
# check which SDKs are installed
dotnet --list-sdks
```

```
# output for dotnet --list-sdks
8.0.404 [C:\Program Files\dotnet\sdk] # supported, no dnx
10.0.100 [C:\Program Files\dotnet\sdk] # <- this one is what you want
```

Installation and Updates

Either run the tool on demand with dnx, install it once as a global tool, or use a bootstrap script that checks dependencies, then downloads and runs the tool for you.

dnx (recommended, .NET SDK 10+)

Nothing to install - dnx downloads and runs the tool on demand:

```
# make empty game data file
dnx dotnet-charon -- INIT ./gamedata.json

# run editor
dnx dotnet-charon -- ./gamedata.json
```

The tool is cached by NuGet after the first run, so subsequent invocations start quickly.

dotnet tool (.NET SDK 8+)

On .NET SDK 8 or 9 dnx is unavailable. Install the tool globally instead, which also lets you type charon directly:

```
# install charon globally (run it once)
dotnet tool install -g dotnet-charon

# update global tool
dotnet tool update -g dotnet-charon

# make empty game data file
charon INIT ./gamedata.json

# run editor
charon ./gamedata.json
```

On macOS and Linux the charon command only works once \$HOME/.dotnet/tools is on your PATH - see Adding the global tool to PATH.

Bootstrap Script

Two bootstrap scripts which download and run the latest version of Charon on your PC:

- RunCharon.bat for Windows
- RunCharon.sh for Linux or MacOS

Warning

Bootstrap scripts require the **.NET SDK** to run, not only the bare .NET Runtime.

Both scripts require the [dotnet](#) tool to be available in PATH.

1. Download the script for your OS into a local folder Charon:
 - a. [RunCharon.bat \(Windows\)](#)
 - b. [RunCharon.sh \(Linux, MacOS\)](#)
2. Navigate into that folder with `cd Charon`.
3. Run the script with no arguments. It downloads and upgrades the dotnet-charon tool, then displays the help text.
4. Create an empty game data file: `.\RunCharon.bat INIT gamedata.json` (Windows) or `./RunCharon.sh INIT gamedata.json`.
5. Run in standalone mode: `.\RunCharon.bat gamedata.json` (Windows) or `./RunCharon.sh gamedata.json`.

Or copy the whole sequence:

Windows

```
REM ##### Load and run bootstrap script #####
```

```
mkdir Charon
```

```
cd Charon
```

```
curl -O https://raw.githubusercontent.com/gamedevware/charon/main/scripts/bootstrap/RunCharon.bat
.\RunCharon.bat INIT .\gamedata.json
```

```
REM ##### Start editor #####
```

```
.\RunCharon.bat .\gamedata.json --log out
```

Linux, MacOS

```
##### Load and run bootstrap script #####
```

```
mkdir Charon
```

```
cd Charon
```

```
curl -O https://raw.githubusercontent.com/gamedevware/charon/main/scripts/bootstrap/RunCharon.sh
```

```
chmod +x RunCharon.sh
```

```
./RunCharon.sh INIT ./gamedata.json
```

```
##### Start editor #####
```

```
./RunCharon.sh ./gamedata.json --log out
```

Creating and Editing Game Data

Any empty **gamedata.json** file could be used as starting point for standalone application launch. The editor will automatically fill the empty file with the initial data.

Windows

```
# dnx dotnet-charon -- INIT .\gamedata.json
```

```
# or
```

```
# copy /y NUL .\gamedata.json >NUL
```

```
charon .\gamedata.json --log out
```

Linux, MacOS

```
# dnx dotnet-charon -- INIT ./gamedata.json
```

```
# or
```

```
# touch ./gamedata.json
```

```
charon ./gamedata.json --log out
```

After finishing your work, you could just terminate the process with CTRL+C keyboard shortcut or close terminal window.

See also

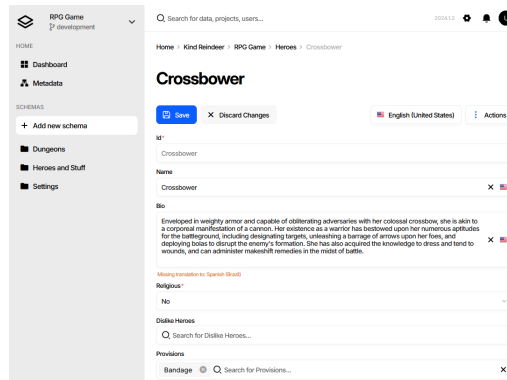
- [Nuget Package](#)
- [Bootstrap Scripts](#)
- Basic Navigation and User Interface Overview
- Creating Document Type (Schema)
- Filling Documents
- Publication of Game Data
- Generating Source Code
- Frequently Asked Questions (FAQ)
- Glossary

Web Application Overview

The web version of the Charon provides a collaborative work environment where game designers can work together to create engaging gameplay experiences. The core concepts of collaborative work include workspaces and projects. A workspace is a virtual location where projects are located. The subscription and all limitations are bound to the workspace, meaning that all projects within the workspace are subject to the same subscription and limitations.

A project is a virtual location for storing game data, localization settings, backups, branches, and members. When a project is created,

the user becomes its owner and can invite other members to join.



Starting with a new Project

1. Visit the charon.live website and click on the *Register* button to create a new account.
2. Fill out the registration form with your desired username and password, and click on the “Create” button to create your account.
3. After successfully logging in, you will be directed to the workspace page.
4. If you're a new user, the workspace page will be empty, with no projects listed. Click on the *Create project* button to create your first project.
5. On the “Create Project” page, fill in the name of your project and any other basic information you want to include, and click on the “Create” button to create your project.
6. After creating the project, you'll be redirected to the project's dashboard page, which provides an overview of the project and allows you to start modelling game data.

See also

- [Charon Website](#)
- Workspaces, Projects, and Branches
- CLI Access to Web Project
- Migrating to Web Application
- Basic Navigation and User Interface Overview
- Creating Document Type (Schema)
- Filling Documents
- Publication of Game Data
- Generating Source Code
- Frequently Asked Questions (FAQ)
- Glossary

CLI Access to charon.live

The web version of Charon provides a REST API and CLI for accessing and modifying game data remotely. To authenticate CLI commands against a web project, users need to generate an API Key in the *API Keys* section of their *User Profile*.

With an API Key, Charon integrates into any existing workflow - exporting data into a local repository, importing designer edits back to the server, generating source code, or running validation in CI.

[Installing the CLI](#)

23

[Finding Your Project URL](#)

23

Generating an API Key	23
Authentication	24
Common Operations	24
Export Data	24
Import Data	24
Generate Source Code	25
Validate Data	25
Backup	25
See also	25

Installing the CLI

Charon requires the [.NET SDK 10 or later](#). There is nothing to install - dnx downloads and runs the tool on demand:

```
dnx dotnet-charon -- DATA EXPORT --help
```

On .NET SDK 8 or 9 dnx is unavailable; install dotnet-charon as a global tool instead and call it as charon:

```
dotnet tool install -g dotnet-charon
charon DATA EXPORT --help
```

See Command Line Interface (CLI) for full installation options.

Finding Your Project URL

Every web project has a stable URL that can be passed to the `--dataBase` parameter. You can find it in your browser's address bar while viewing any page inside the project on charon.live - it has the form:

```
https://charon.live/view/data/<Project>/
https://charon.live/view/data/<Project>/<Branch>/
```

For example:

```
https://charon.live/view/data/My_Game/
https://charon.live/view/data/My_Game/develop/
```

Use the branch-specific URL when you want to target a particular branch (e.g. `main`, `develop`).

Generating an API Key

1. Navigate to the *API Keys* section in your *User Profile* → *API Keys* and click **Create API Key....**
2. Copy the generated API Key - it is shown only once.
3. Store the key securely (e.g. in a password manager or CI secret store).

Use the API Key in:

- The `--credentials` parameter of any CLI command.
 - The `CHARON_API_KEY` environment variable (the CLI reads it automatically).
 - The `Authenticate` HTTP header when calling the REST API directly.
-

Authentication

Pass the API Key either as an explicit parameter or as an environment variable:

```
# Explicit parameter
dnx dotnet-charon -- DATA EXPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --output "./gamedata.json" \
  --outputFormat json \
  --credentials "<API-Key>"

# Environment variable - picked up automatically by all commands
export CHARON_API_KEY="<API-Key>"
dnx dotnet-charon -- DATA EXPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --output "./gamedata.json" \
  --outputFormat json
```

The environment variable takes precedence when both are supplied. In CI pipelines, store the key as a secret and inject it via CHARON_API_KEY rather than hard-coding it in pipeline files.

Common Operations

Export Data

Export all documents (or a subset by schema) from a web project into a local file:

```
# Export all schemas
dnx dotnet-charon -- DATA EXPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --output "./gamedata.json" \
  --outputFormat json \
  --credentials "<API-Key>"

# Export a specific schema
dnx dotnet-charon -- DATA EXPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schemas Character \
  --output "./characters.json" \
  --outputFormat json \
  --credentials "<API-Key>"

# Export publication-ready data (strips non-essential fields)
dnx dotnet-charon -- DATA EXPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --mode publication \
  --output "./StreamingAssets/gamedata.json" \
  --outputFormat json \
  --credentials "<API-Key>"
```

Import Data

Push local edits back to the web project:

```
# Update existing documents (safe - no creates or deletes)
dnx dotnet-charon -- DATA IMPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schemas Character \
  --input "./characters.json" \
```

```
--inputFormat json \
--mode safeUpdate \
--credentials "<API-Key>"

# Create and update documents
dnx dotnet-charon -- DATA IMPORT \
--dataBase "https://charon.live/view/data/My_Game/develop/" \
--input "./gamedata.json" \
--inputFormat json \
--mode createAndUpdate \
--credentials "<API-Key>"
```

Generate Source Code

Generate C# classes directly from the web project without downloading the data first:

```
dnx dotnet-charon -- GENERATE CSHARPCODE \
--dataBase "https://charon.live/view/data/My_Game/develop/" \
--namespace "MyGame.Parameters" \
--outputDirectory "./Assets/Scripts/Generated" \
--clearOutputDirectory \
--credentials "<API-Key>"
```

Validate Data

Check data integrity - useful before a release or as a CI gate:

```
# Print a validation report to stdout
dnx dotnet-charon -- DATA VALIDATE \
--dataBase "https://charon.live/view/data/My_Game/develop/" \
--validationOptions checkRequirements checkReferences checkFormat \
--credentials "<API-Key>"

# Exit with code 1 if any errors are found (CI-friendly)
dnx dotnet-charon -- DATA VALIDATE \
--dataBase "https://charon.live/view/data/My_Game/develop/" \
--validationOptions checkRequirements checkReferences checkFormat \
--output err \
--credentials "<API-Key>"
```

Backup

Create a local backup archive of a web project:

```
dnx dotnet-charon -- DATA BACKUP \
--dataBase "https://charon.live/view/data/My_Game/develop/" \
--output "./backup_my_game.zip" \
--credentials "<API-Key>"
```

See also

- Command Line Interface (CLI)
- Migrating to Web
- REST API
- CI/CD Integration
- DATA EXPORT Command

- DATA IMPORT Command
- DATA VALIDATE Command
- DATA BACKUP Command
- GENERATE CSHARPCODE Command
- URL-based Input/Output

Migrating to Web Application

To migrate to the web application, you only need to backup your game data in your current editor and restore it in the web application.

In short: project settings include a backup and restore feature. You make a backup in one place, and you can restore it in another. This is how you can transfer game data to another project.

Step 1: Backup Data Step by Step

1. Open your game data in the editor (Standalone, Unity ...)
2. In the bottom left corner, click on the gear icon “Settings”
3. In the left menu, select “Backup”
4. On the backup management page, click the “Backup” button
5. Choose “File” as the destination to save the backup and click “Next”
6. On the “Format” step, select “JSON” and click “Backup”
7. On the “Summary” step, click the link with a file name like `backup_2023_11_10_11_27.json` and save it to your computer.

Step 2: Restoring Backup in the Web Application

1. On the Home screen, click on “Create Project”
2. Specify the project name, tags, and script language
3. Click the “Create” button
4. In the left menu, select “Backup”
5. On the backup management page, click the “Restore” button
6. Choose “File” as the source of the restore and click “Next”
7. Select the input file, the one you have after the backup, such as `backup_2023_11_10_11_27.json`, and click “Restore”
8. Done

Any data that was in this project at the time before the restore will be lost.

See also

- Basic Navigation and User Interface Overview
- Publication of Game Data
- Generating Source Code
- Frequently Asked Questions (FAQ)
- Glossary

Roles and Permissions

Access to a project in the web editor is granted per member, as one of four roles. The roles are cumulative: each one can do everything the roles above it can, plus more. Two further roles come from the workspace and are not assigned per project.

[Where Roles Are Assigned](#) 27

[What Each Role Can Do](#) 28

[Inviting a Member](#) 28

[Which Role to Give](#) 29

[See also](#) 29

Where Roles Are Assigned

Open **Project Settings** → **Members**.

Home > Kind Reindeer > My RPG Game > Settings > Members



My RPG Game
 development

General

Backup

Publication

Extensions

Features

Source Code

Internationalization

Branches

Members

Member Management

Members are users who have been invited by the project's owner to participate in the creation of the game. They can have different access levels to the project's data. Please refer to the [documentation](#) for further details.

Workspace owners and workspace administrators are automatically invited and could not be expelled.

 Invite Member...

Member	Access to the project
 Dan Smith	Workspace Owner ...
 John Doe	Designer ...

The **Access to the project** column is a drop-down per member - changing it takes effect immediately. The ... menu at the end of the row expels the member from the project.

Only an **Administrator** can change roles or invite people. Workspace owners and workspace administrators are members of every project in the workspace automatically, and cannot be expelled or demoted from here.

What Each Role Can Do

Capability	Viewer	Editor	Designer	Admin
View documents	yes	yes	yes	yes
Export data	yes	yes	yes	yes
View project settings	yes	yes	yes	yes
Edit documents	-	yes	yes	yes
Import data	-	yes	yes	yes
Change document structure (schemas and properties)	-	-	yes	yes
Change project settings, including the game data version	-	-	-	yes
Create and restore backups	-	-	-	yes
Invite members and change their roles	-	-	-	yes

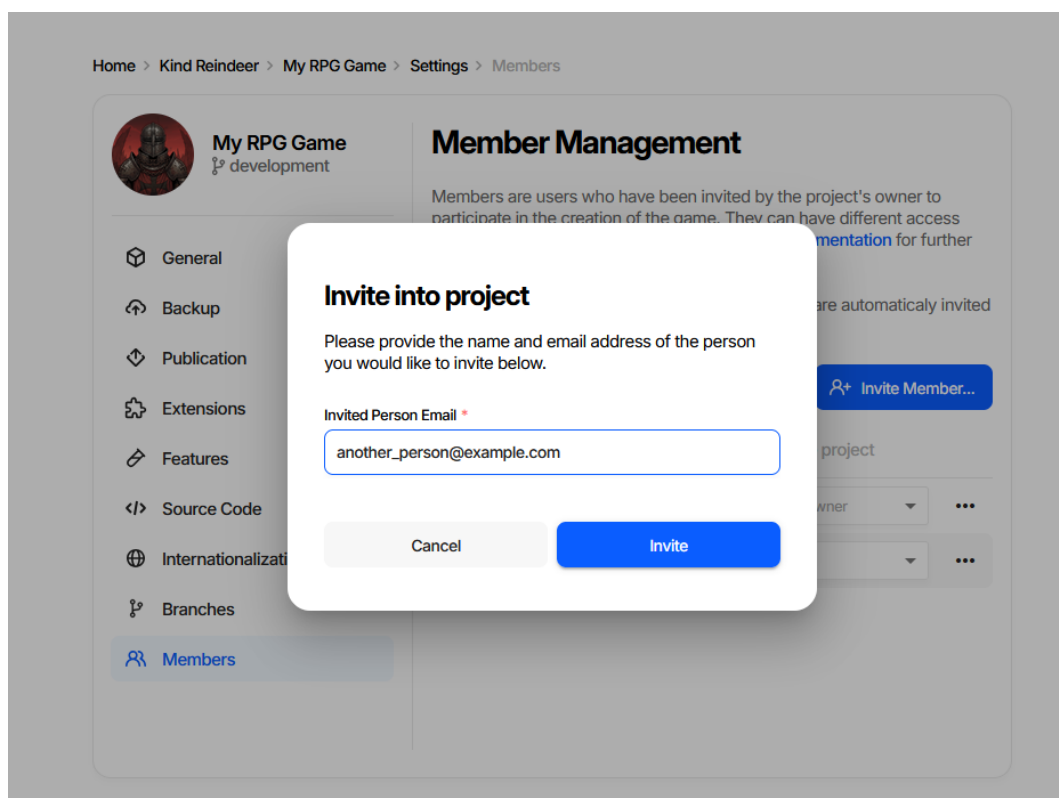
Delete or transfer a project is a workspace-level action, available to the **Workspace Owner** and to **Workspace Administrators** only. Within a project they have everything an Administrator has.

Note

The four project roles map onto the permission levels the API and the CLI use - view, edit, design, and administer. A feature that requires design is available to Designers and Administrators, and so on.

Inviting a Member

Invite Member... on the Members page asks for an email address.



The invited person appears in the list right away with the invitation state next to them - *sent*, *email sent*, or *declined* - and picks up their role as soon as they accept. New members start as Viewers unless you change the drop-down.

The number of members a project can have depends on the subscription plan; the page warns when the limit is reached.

Which Role to Give

- **Viewer** - people who need to read the data but not change it: QA, external partners, anyone consuming an export.
 - **Editor** - the default for content work. Fills documents, imports spreadsheets, cannot change the shape of the data.
 - **Designer** - the people who model the data. Adding a property or a schema affects everyone's generated code, which is why it is a separate role rather than part of editing.
 - **Administrator** - a small number of people. Backups, restores, project settings, and the member list.
-

See also

- Workspaces, Projects, and Branches
- Overview
- Backup and Restore
- Publishing Game Data
- [Web-based Application](#)

REST API

The web edition of Charon exposes a REST API at <https://charon.live/api/v1>. It covers the full feature set of the application - data CRUD, import/export, branch management, code generation, validation, and more.

An interactive API explorer (Swagger UI) is available at <https://charon.live/api/index.html>.

Authentication	29
Request and Response Formats	30
Finding Your IDs	30
Response Envelope	30
Error Responses	31
Error code reference	31
See also	34

Authentication

All endpoints require a bearer token in the Authorization header. Two credential types are accepted:

API Key (recommended for CI/CD and scripts)

Obtain an API key from **User Profile** → **API Keys** → **Create API Key**. Use it directly as the bearer value:

Authorization: Bearer <api-key>

See CLI Access to charon.live for step-by-step instructions on generating one.

Request and Response Formats

All endpoints default to `application/json`. Use the `Accept` and `Content-Type` headers to switch between supported formats.

Most data endpoints also support these formats for both request bodies and responses. Prefer `MessagePack` for large payloads in performance-sensitive pipelines.

Finding Your IDs

Most data endpoints require `projectId` and/or `dataSourceId`.

`projectId`

Visible in the browser URL when you open a project:
`https://charon.live/view/project/<projectId>/`.

`dataSourceId`

Each branch has its own data source ID. Retrieve it via `GET /project/{projectId}/`:

```
curl -s "https://charon.live/api/v1/project/{projectId}/" \
  -H "Authorization: Bearer $CHARON_API_KEY" | jq '.result.branches[]'
```

The response includes each branch name and its `id`, which is the `dataSourceId` used in all `/datasource/` endpoints.

Response Envelope

Every response - success or failure - shares the same top-level JSON structure:

```
{
  "result": { ... },
  "errors": []
}
```

One of fields are always present in the response body:

- `result` - contains the operation's output on success. The shape varies by endpoint (a single document, a list, a backup payload, etc.). On failure the field is absent.
- `errors` - an array of error objects. Absent on success; contains one or more entries on failure.

The intended semantics are a discriminated union - either `result` carries meaningful data **or** `errors` is non-empty.

Success example - `GET /datasource/{id}/collection/Item/`

```
{
  "result": {
    "document": { "Id": "Sword", "Name": "Iron Sword", "Damage": 20 },
    "meta": { "schemaId": "592fc86c983a36266c0912a0", "path": "/Item/Sword" }
  }
}
```

No-content success - some write operations (create branch, commit transaction, etc.) return HTTP 204 with no body.

The simplest way to detect success in code is to check the HTTP status code first. A 2xx status means the operation succeeded; anything else means at least one entry will be present in `errors`.

Error Responses

Every failed response uses the same JSON envelope regardless of the HTTP status code:

```
{
  "errors": [
    {
      "code": "resourceNotFound",
      "message": "Document with id 'Sword' not found in collection 'Item'."
    }
  ]
}
```

Fields:

- `result` - is absent on error.
- `errors` - array of one or more error objects. Each object has:
 - `code` (*string*) - machine-readable error code from the `ErrorCode` enum (see below).
 - `message` (*string*) - human-readable description of the problem.
 - Additional fields may be present for specific error types (e.g. `parameterName`, `path`).

HTTP status codes and their typical error codes:

HTTP	Typical code value	When it occurs
400	<code>invalidRequestBody</code>	Malformed JSON body
	<code>invalidParameterValue</code>	Query/route parameter has wrong type or value
	<code>missingParameterValue</code>	Required parameter absent
	<code>invalidImportData</code>	Imported document fails schema validation
401	<code>authenticationFailed</code>	Missing or invalid <code>Authorization</code> header
403	<code>authorizationFailed</code>	Authenticated but lacks required permission
	<code>resourceReadOnly</code>	Target resource is read-only
404	<code>resourceNotFound</code>	Project, branch, or document does not exist
409	<code>duplicateTransaction</code>	Transaction ID already in use
	<code>abortedTransaction</code>	Transaction was rolled back by the server
	<code>completedTransaction</code>	Transaction already committed or rolled back
	<code>duplicateUniqueValue</code>	Document violates a unique-value constraint
429	<code>tooManyRequests</code>	Rate limit exceeded
503	<code>serviceUnavailable</code>	Server temporarily unavailable
5xx	<code>genericError</code>	Unexpected server-side error

Error code reference

The full set of code values, grouped by category:

General

Code	Meaning
<code>genericError</code>	Unexpected server error.
<code>resourceNotFound</code>	Requested resource does not exist.

serviceUnavailable	Server is temporarily unavailable.
tooManyRequests	Too many concurrent requests from this client.
operationNotSupported	Operation is not supported by this resource.
authenticationFailed	Credentials are missing or invalid.
authorizationFailed	Permission denied for this operation.
resourceReadOnly	Target resource is read-only.

Request validation

Code	Meaning
invalidRequestBody	Request body could not be parsed.
invalidParameterValue	Parameter has an unacceptable value.
missingParameterValue	Required parameter is absent.
invalidParameterPattern	String parameter does not match the required pattern.
parameterBelowMinValue parameterBelowMinValueInclusive	Numeric parameter is below the minimum (exclusive / inclusive).
parameterAboveMaxValue parameterAboveMaxValueInclusive	Numeric parameter exceeds the maximum (exclusive / inclusive).
parameterBelowMinLength	String parameter is shorter than minimum length.
parameterAboveMaxLength	String parameter exceeds maximum length.
parameterBelowMinSize	Collection parameter has fewer items than the minimum.
parameterAboveMaxSize	Collection parameter exceeds maximum item count.
duplicateParameterItems	Collection parameter contains duplicate values.
invalidImportData	Import payload does not match the expected schema.

JSON Patch

Code	Meaning
invalidJsonPatchParameter	Patch document is malformed.
jsonPatchTestFailed	A test operation in the patch failed.
``forbiddenJsonPatchOperation``	Patch attempts a prohibited operation.

Data source / transaction

Code	Meaning
referencedSchemaDeleted	Operation references a schema that has been deleted.
protectedDocument	Document is protected and cannot be modified or deleted.
nonRootDocument	Operation requires a root document but an embedded document was supplied.
abortedTransaction	Transaction was aborted by the server (e.g. timeout or conflict).
completedTransaction	Transaction has already been committed or rolled back.
duplicateTransaction	A transaction with this ID already exists.
missingTransaction	The referenced transaction ID does not exist.
failedMetadataLoad	Schema/metadata could not be loaded.

Document value validation

These codes appear inside import or create/update responses when individual field values fail validation. They are also returned by the POST /validity/ endpoint.

Code	Meaning
invalidTimeFormat	Value does not match the time format.
invalidDateFormat	Value does not match the date format.
invalidLocalizedTextFormat	Localized text value is malformed.
invalidTextFormat	Text value is malformed.
invalidLogicalFormat	Boolean value is malformed.
invalidNumberFormat	Decimal number value is malformed.
invalidIntegerFormat	Integer value is malformed.
invalidPickListFormat	Pick list value does not match any defined option.
invalidMultiPickListFormat	Multi-pick list value is malformed.
invalidDocumentFormat	Embedded document value is malformed.
invalidDocumentCollectionFormat	Embedded document collection is malformed.
invalidReferenceFormat	Reference value is malformed.
invalidReferenceCollectionFormat	Reference collection is malformed.
invalidFormulaFormat	Formula expression is malformed.
nullValue	Required field is null.
emptyValue	Required field is empty.
overflowedValue	Numeric value exceeds the type's range.
exceededValueLength	String value exceeds maximum length.
exceededCollectionSize	Collection exceeds maximum item count.
brokenReference	Reference points to a non-existent document.
missingTranslation	Localized text is missing a required translation.
duplicateUniqueValue	Value violates a unique constraint.
invalidIdFormat	Document Id value is malformed.
invalidGeneratedIdFormat	Auto-generated ID does not match the schema's ID generator format.
documentsInRootCollection	Embedded documents were supplied as root documents.
missingSettingsDocument	A required settings document is absent.
duplicateSettingsDocument	More than one settings document exists.
emptyUnion	Tagged union has no selected option.
conflictingUnionOptions	Tagged union has more than one selected option.

Schema / metadata validation

These codes appear when a schema definition itself is invalid (e.g. during schema create or update operations).

Code	Meaning
invalidNameFormat	Schema or property name is not a valid C identifier.
reservedName	Name conflicts with a reserved keyword.
invalidSize	Size value is out of the allowed range.
invalidLanguage	Language code is not a valid BCP-47 tag.
recursiveReferenceType	Schema has a circular reference that is not allowed.
invalidIdDataType	The Id property uses a data type that is not allowed for identifiers.

<code>invalidDataTypeChange</code>	Changing a property's data type in a way that would corrupt existing data.
<code>invalidIdGeneratorChange</code>	Changing the ID generator in an incompatible way.
<code>invalidRequirementForDataType</code>	Requirement level is incompatible with the property's data type.
<code>nonOptionalTaggedUnionProperty</code>	A tagged-union property must be optional.

Schema specification validation

Code	Meaning
<code>invalidIconFormat</code>	Icon specification is malformed.
<code>invalidDisplayTextTemplate</code>	Display text template expression is invalid.
<code>invalidPickName</code>	Pick list option name is invalid.
<code>duplicatePickName</code>	Pick list has duplicate option names.
<code>invalidPickValue</code>	Pick list option value is invalid.
<code>invalidParameterName</code>	Formula parameter name is invalid.
<code>duplicateParameterName</code>	Formula has duplicate parameter names.
<code>invalidTypeName</code>	Type name in specification is invalid.
<code>invalidTypeReference</code>	Type reference cannot be resolved.
<code>invalidPrecision</code>	Decimal precision value is out of range.
<code>invalidRowCount</code>	Row count specification is out of range.

See also

- [CLI Access to charon.live](#)
- [Workspaces, Projects, and Branches](#)
- [Roles and Permissions](#)
- [Backup and Restore](#)
- [Importing and Exporting Data](#)
- [CI/CD Integration](#)
- [Patch and Diff Workflow](#)

Workspaces, Projects, and Branches

Charon organises collaborative game data in a three-level hierarchy. Understanding this model is the foundation for managing teams, permissions, and parallel design workflows.

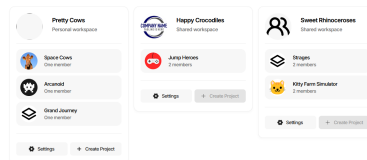
Hierarchy	35
Workspace	35
Project	35
Branch	35
Creating a Project	36
Inviting Collaborators	36
Managing Branches in the Editor	36
Working with Branches via CLI	37

Branch-based Release Pipeline	38
How a push works	38
Typical branch setup	39
CI/CD pipeline example	39
Advanced: Patch-based Feature Merging	39
Cloning a Branch Locally	40
See also	40

Hierarchy

Workspace

- ■ ■ ■ Project
 - ■ ■ ■ Branch (e.g. main)
 - ■ ■ ■ Branch (e.g. develop)
 - ■ ■ ■ Branch (e.g. feature-loot-rework)



The home page shows this as cards: one per workspace - personal or shared - listing the projects it holds and how many members each has.

Workspace

A **Workspace** is the top-level organisational container for your studio or team.

- A workspace holds one or more projects.
- A workspace has one **Owner** and can have multiple **Administrators**.
- Deleting a workspace permanently removes all projects and branches inside it.

Project

A **Project** is a named container for a single game's data.

- Every project has at least one branch (main by default).
- Projects can be shared by inviting team members; each invitation carries a specific role.
- Projects can be cloned locally at any time - the resulting file is fully compatible with the standalone CLI.

Branch

A **Branch** is a named, independently editable snapshot of game data inside a project.

- Branches are addressed in CLI calls via their URL: `https://charon.live/view/data/<project>/<branch>/`
 - Branches enable parallel development: designers work on feature branches, then merge changes back to main using Charon's patch/diff tooling.
-

Creating a Project

1. Sign in to charon.live.
2. Find your workspace on the home page.
3. Click **Create Project** on the workspace card and choose a name.
4. The project is created with a single branch, which is marked *Primary*.

Inviting Collaborators

1. Open the project.
2. Navigate to **Project Settings** → **Members**.
3. Press **Invite Member...** and enter the person's email address.
4. They receive an email with a link to accept the invitation, and appear in the list straight away with their invitation state next to them.
5. Set their role in the **Access to the project** column - new members start as Viewers.

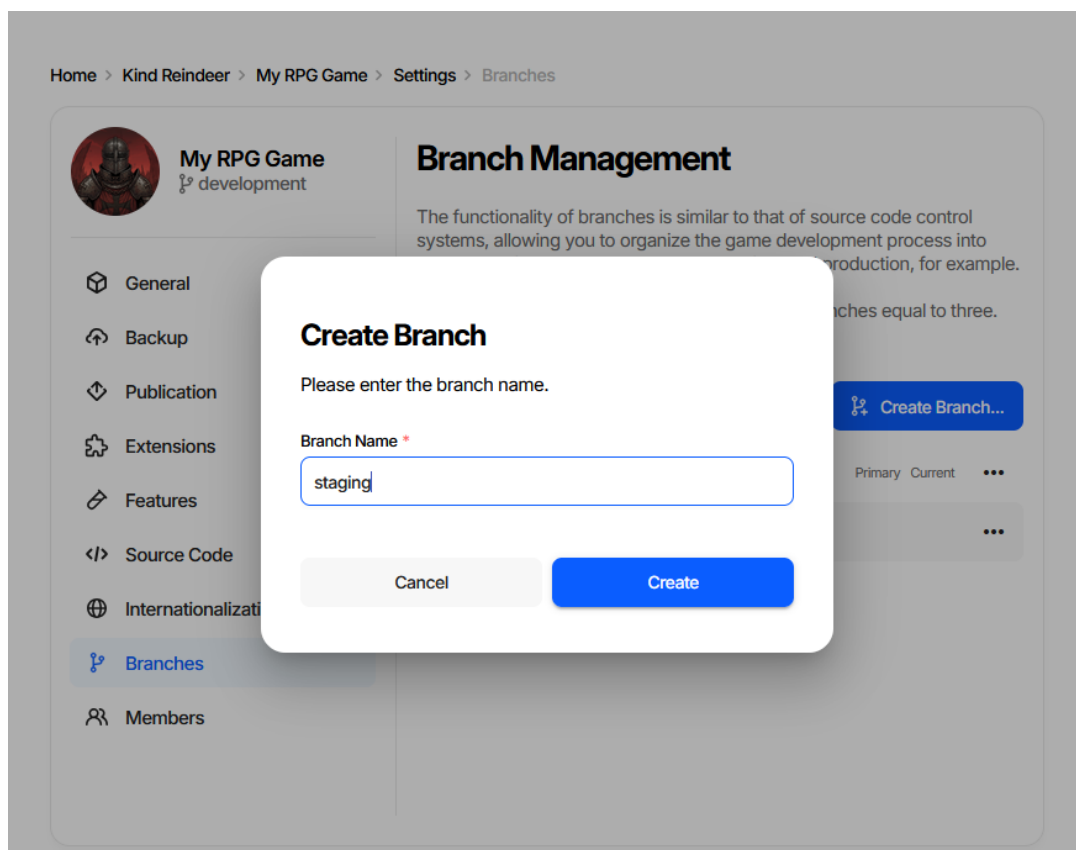
Available roles: Viewer, Editor, Designer, Administrator. Only an Administrator can invite members or change roles. See Roles and Permissions for the full permission matrix and for the screenshots of both steps.

Managing Branches in the Editor

Project Settings → **Branches** lists the branches of the project.

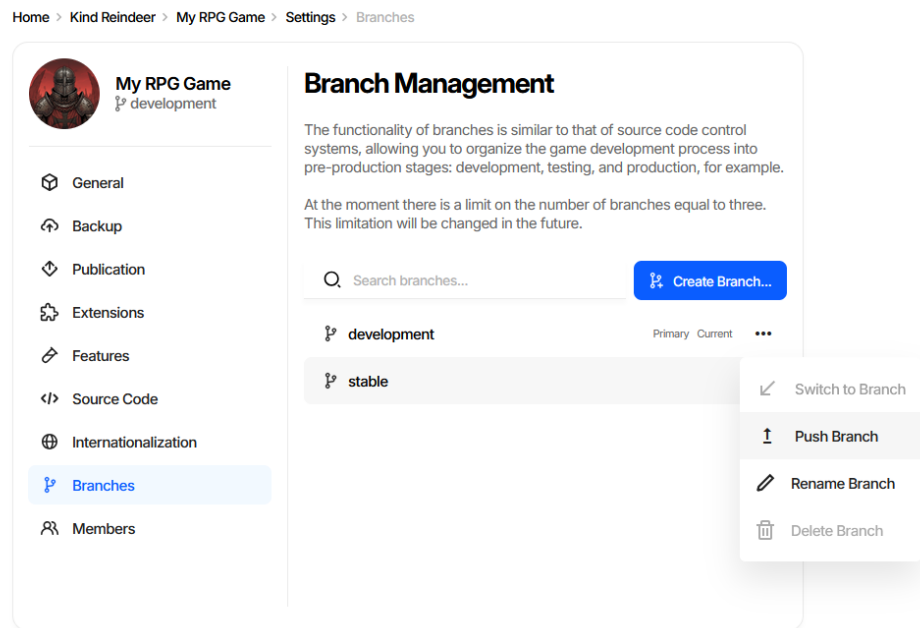
Primary marks the branch a project cannot be without - it cannot be deleted - and *Current* marks the one open in the editor. A project is limited to three branches at the time of writing.

Create Branch... asks only for a name:



Is It Free?

The . . . menu at the end of a row holds the rest:



Switch to Branch opens it in the editor, and is disabled on the branch already open. **Push Branch** copies data into another branch, see [How a push works](#). **Rename Branch** renames it, which changes the URL the CLI addresses it by. **Delete Branch** is disabled on the primary branch.

Working with Branches via CLI

All CLI DATA and GENERATE commands accept a branch URL as `--dataBase`:

```
# Export from the develop branch
dnx dotnet-charon -- DATA EXPORT \
  --dataBase "https://charon.live/view/data/MyGame/develop/" \
  --output items.json --outputFormat json \
  --credentials "$CHARON_API_KEY"

# Import into a feature branch
dnx dotnet-charon -- DATA IMPORT \
  --dataBase "https://charon.live/view/data/MyGame/feature-loot/" \
  --input new_items.json --inputFormat json \
  --mode createAndUpdate \
  --credentials "$CHARON_API_KEY"

# Generate code from main
dnx dotnet-charon -- GENERATE CSHARPCODE \
  --dataBase "https://charon.live/view/data/MyGame/main/" \
  --namespace MyGame.Parameters \
  --outputDirectory ./Generated \
  --credentials "$CHARON_API_KEY"
```

Branch-based Release Pipeline

The recommended workflow is a linear promotion pipeline. Work flows in one direction - from dev toward stable - by **pushing** (copying) the entire branch into the next stage. A push replaces the destination branch completely; it is not a merge.

dev (design) ■■■■ push ■■■■ → staging (QA) ■■■■ push ■■■■ → stable (release) ■■■■ publish ■■ → game

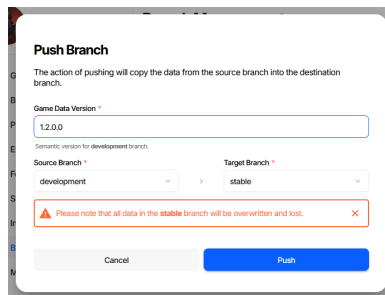
How a push works

A push copies the complete content of one branch into another, replacing the destination entirely. There is no partial merge - every document in the destination is overwritten.

Three ways to trigger a push are available:

Web UI

Go to **Project Settings** → **Branches**, open the . . . menu next to the source branch, and choose **Push Branch**.



Source and target are both drop-downs, so the direction can be corrected here, and the dialog names the branch about to be overwritten in red. **Game Data Version** stamps a semantic version onto the source branch as part of the push - the same field as Data Version in the project settings, so a promotion and a version bump are one action.

REST API

Send a PUT request to the branch endpoint with the destination branch name in the request body:

```
curl -X PUT \
  "https://charon.live/api/project/{projectId}/branch/dev/" \
  -H "Authorization: Bearer $CHARON_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"destinationBranchName": "staging"}'
```

The operation is synchronous; a 200 response means the push completed successfully.

CLI (BACKUP + RESTORE)

Use this approach when the push is part of a larger CLI script, or when you need to inspect the snapshot before promoting it:

```
# Step 1 - snapshot the source branch
dnx dotnet-charon -- DATA BACKUP \
  --dataBase "https://charon.live/view/data/MyGame/dev/" \
  --output dev_snapshot.json \
  --credentials "$CHARON_API_KEY"

# Step 2 - overwrite the destination branch with the snapshot
dnx dotnet-charon -- DATA RESTORE \
  --dataBase "https://charon.live/view/data/MyGame/staging/" \
  --input dev_snapshot.json \
  --credentials "$CHARON_API_KEY"
```

Repeat the same steps to promote from staging to stable.

Warning

All three methods **replace all content** in the target branch. Double-check that you are targeting the correct branch before triggering a push.

Typical branch setup

Branch	Purpose	Who writes to it
dev	Active design and iteration	Designers, game designers
staging	Integration and QA testing	CI pipeline (push from dev)
stable	Release-ready data; source for publish	CI pipeline (push from staging)

CI/CD pipeline example

The REST API push is the simplest option for CI pipelines - a single HTTP call with no intermediate files. The example below uses GitHub Actions, but the same pattern works with any CI system.

```
# .github/workflows/promote.yml
name: Promote dev → staging

on:
  workflow_dispatch:  # triggered manually or on a schedule

jobs:
  promote:
    runs-on: ubuntu-latest
    steps:
      - name: Push dev branch into staging
        run: |
          curl -X PUT \
            "https://charon.live/api/project/${{ secrets.CHARON_PROJECT_ID }}/branch/dev/" \
            -H "Authorization: Bearer ${{ secrets.CHARON_API_KEY }}" \
            -H "Content-Type: application/json" \
            -d '{ "destinationBranchName": "staging" }'
```

See CI/CD Integration for a full CI/CD walkthrough including validation and publish steps.

Advanced: Patch-based Feature Merging

For teams that need to integrate isolated feature branches selectively - without promoting the entire dev branch - Charon provides a patch/diff workflow. A *patch* is a JSON file that captures only the differences between two branches. It can be reviewed, stored in version control, and applied to any compatible target.

```
# Compute what changed between dev and a feature branch
dnx dotnet-charon -- DATA CREATEPATCH \
  --dataBase1 "https://charon.live/view/data/MyGame/dev/" \
  --dataBase2 "https://charon.live/view/data/MyGame/feature-loot/" \
  --output loot.patch.json --credentials "$CHARON_API_KEY"

# Apply only those changes to the target branch
dnx dotnet-charon -- DATA APPLYPATCH \
  --dataBase "https://charon.live/view/data/MyGame/dev/" \
  --input loot.patch.json --credentials "$CHARON_API_KEY"
```

This approach requires a custom build pipeline and additional scripting to handle conflict resolution. It is best suited for large teams with long-lived feature branches or for shipping incremental live-service updates.

See Patch and Diff Workflow for the full reference.

Cloning a Branch Locally

Any branch can be downloaded as a standalone file using DATA BACKUP:

```
dnx dotnet-charon -- DATA BACKUP \
  --dataBase "https://charon.live/view/data/MyGame/main/" \
  --output myGame_main.json \
  --credentials "$CHARON_API_KEY"
```

The resulting file can be opened with the standalone CLI:

```
charon myGame_main.json
```

The standalone tool and the server share the same file format - full round-trip compatibility is guaranteed.

See also

- Web Application Overview
- Roles and Permissions
- CLI Access to charon.live
- REST API
- Backup and Restore
- CI/CD Integration
- Patch and Diff Workflow

Server Edition

The **Charon Server Edition** is the team collaboration layer built on top of the standalone editor. It gives studios a shared, version-controlled game data repository that every designer, developer, and QA engineer accesses through a browser - no local installation required.

It is available as a hosted cloud service at charon.live or self-hosted on your own infrastructure for studios with data-residency or security requirements.

Why Teams Need It	40
Feature Comparison	41
Collaboration Model	41
Authentication and Enterprise Integration	41
Access Control	42
Deployment Options	42
See also	42

Why Teams Need It

The standalone editor is designed for a single user working locally. As soon as a second designer touches the same data file, the team is managing conflicts in spreadsheets or shared drives. The server edition eliminates that problem:

- **Single source of truth** - all designers edit the same live data; no more “who has the latest version?” questions.

- **Branch-based workflows** - feature branches let designers work in isolation and merge changes only when ready, the same way engineers use Git.
- **Full audit trail** - every data change is attributed to a user with a timestamp, satisfying compliance and post-mortem requirements.
- **Works with existing pipelines** - the CLI and REST API are the same as the standalone edition, so CI/CD scripts, engine plugins, and code generation workflows require no changes.

Feature Comparison

Feature	Standalone	Server
Game data editing UI	✓	✓
CLI access (all DATA / GENERATE commands)	✓	✓
Local file storage	✓	✓
Multi-user real-time collaboration		✓
Workspaces, Projects, Branches		✓
Role-based access control		✓
Enterprise SSO (OAuth2, SAML, AD)		✓
API key management		✓
Audit logs (auth events, command traces)		✓
On-premises / private cloud deployment		✓

Collaboration Model

Game data is organised in a three-level hierarchy: **Workspace** → **Project** → **Branch**.

Designers work on named branches (e.g. feature/new-enemies) and merge changes to a shared main branch using Charon's patch/diff tooling - a workflow familiar to any engineer who uses Git, adapted for non-technical designers who never touch the terminal.

See Workspaces, Projects, and Branches for the full collaboration guide.

Authentication and Enterprise Integration

The server edition supports multiple identity models to fit into your studio's existing infrastructure:

- **Email + password** with optional **TOTP two-factor authentication** - suitable for small teams or studios without a central directory.
- **OAuth2** - delegate authentication to any standards-compliant identity provider, including Google Workspace, GitHub, or an in-house IdP.
- **SAML 2.0 / Active Directory** - for enterprise studios that require single sign-on through their corporate directory. Users log in with their existing company credentials; no separate Charon account needed.
- **API keys** - generated per user for CLI and REST API access in automated pipelines.

Contact support@gamedevware.com to discuss SSO integration options for your studio.

Access Control

Fine-grained roles control what each team member can do within a project: **Viewer**, **Editor**, **Designer**, **Administrator**.

See Roles and Permissions for the full role and permission matrix.

Deployment Options

Cloud ([charon.live](#))

The fastest way to get started. No infrastructure to manage. Game data is stored on [charon.live](#) servers. Suitable for teams without strict data-residency requirements.

On-premises / Private Cloud

Deploy the server on your own infrastructure - on-premises servers, private cloud VPC, or an air-gapped environment. You retain full control over where game data is stored and who can reach the service. See On-Premises Deployment for deployment guidance.

Note

Files created by the server edition are fully compatible with the standalone CLI and engine plugins. Any branch can be downloaded at any time and opened locally - teams are never locked in to the hosted service.

See also

- [Web Application Overview](#)
- [Workspaces, Projects, and Branches](#)
- [Roles and Permissions](#)
- [REST API](#)

On-Premises Deployment

The Charon server edition can be self-hosted on your own infrastructure. This gives teams complete control over data residency, network access, and configuration, while providing the same collaboration features as the hosted `charon.live` service.

[When to Self-Host](#) 42

[Prerequisites](#) 43

[Data Compatibility](#) 43

[Contingency Plan](#) 43

[Getting Help](#) 43

[See also](#) 43

When to Self-Host

Consider on-premises deployment when:

- **Data residency requirements** prohibit storing game data on third-party cloud services.
- **NDA or IP restrictions** require all data to stay within a private network.

- **Network isolation** - studios working on unannounced titles with air-gapped environments.
 - **Cost at scale** - large teams where a self-hosted MongoDB instance is more economical than the hosted subscription.
-

Prerequisites

- A Linux, Windows, or macOS server with .NET 8 runtime installed.
 - A running **MongoDB** instance (v6.0 or later recommended), accessible from the server.
 - Optional: an SMTP server or email relay for transactional emails (registration, 2FA, etc.).
 - Optional: a reverse proxy (nginx, Caddy, or similar) to provide HTTPS termination.
-

Data Compatibility

Files created by the server edition are **fully compatible** with the standalone CLI and engine plugins. A branch can be downloaded at any time with DATA BACKUP and opened locally:

```
dnx dotnet-charon -- DATA BACKUP \
  --dataBase "https://charon.example.com/view/data/MyGame/main/" \
  --output myGame_main.zip \
  --credentials "$CHARON_API_KEY"
```

```
unzip myGame_main.zip -d myGame/
dnx dotnet-charon -- SERVER START myGame/gamedata.json --launchDefaultBrowser
```

This portability is a deliberate design decision - teams are never locked in to the hosted service.

Contingency Plan

In the unlikely event that the Charon project is discontinued or key contributors become unavailable, the full server source code and private Docker images will be published under the **MIT license** by a designated successor. This guarantees that self-hosted deployments can continue to be maintained by the community indefinitely.

The CLI and engine plugins rely on published NuGet packages, which are already public and will remain unaffected regardless of the server's status.

Getting Help

For licensing, installation, and support:

- Email: support@gamedevware.com
- GitHub issues: <https://github.com/gamedevware/charon/issues>

See also

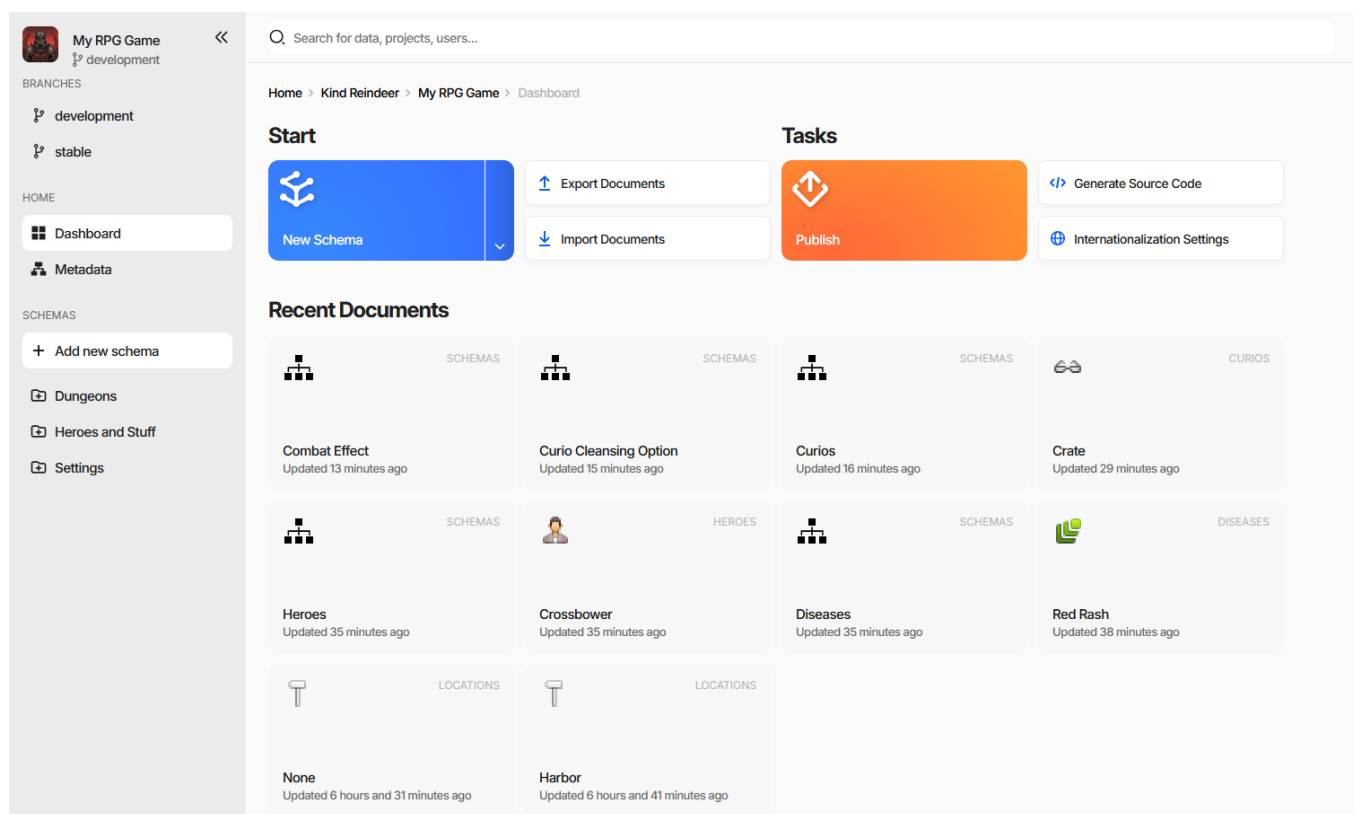
- Server Edition
- Workspaces, Projects, and Branches
- Roles and Permissions

Basic Navigation and User Interface Overview

The editor is the same in every edition - the standalone application, the Unity and Unreal Engine plugins, and the web editor. This page is a tour of what is on the screen; the pages it links to explain the work done there.

The Editor Window	44
Side Navigation	44
Dashboard	45
Global Search	45
First Steps	46
The Working Area	46
Tips and Keyboard Shortcuts	46
See also	46

The Editor Window



Three regions, and they stay put:

- **The side navigation** on the left - branches, the dashboard, and every schema in the project.
- **The search field** across the top.
- **The working area** in the middle - the dashboard, a document collection, or a document form, with a breadcrumb above it that tracks where you are.

Side Navigation

From the top:

- **The project** - its icon, its name, and the branch currently open.
- **Branches** - the other branches of the project, one click to switch. Web editor only, see Workspaces, Projects, and Branches.
- **Home** - the **Dashboard**, and **Metadata**, which is the list of schemas.
- **Schemas** - every schema that is visible in the menu, arranged into the folders its **Group** field names, plus **Add new schema**.
- **Project Settings** at the bottom.

Installed extensions can add entries of their own here, either in the **Home** group or in a section of their own.

The « button collapses the whole panel to a narrow rail of icons, and » brings it back - worth doing on a laptop screen when a collection has many columns.

Dashboard

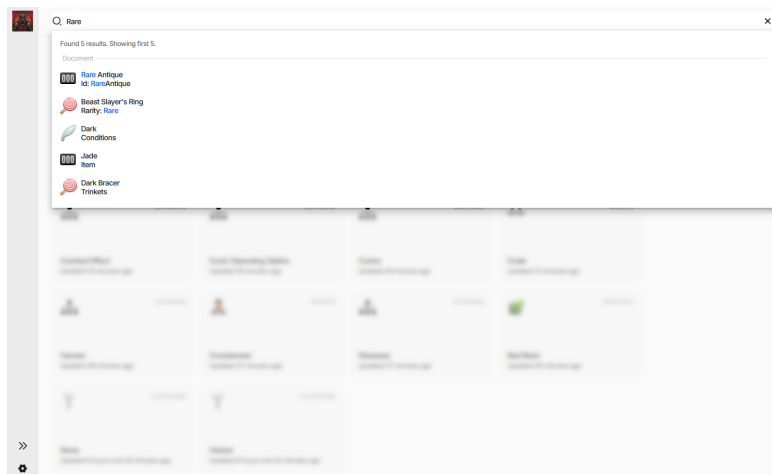
The landing page of a project, and what the side navigation returns to.

- **Start** - create a schema, export, import.
- **Tasks** - publish, generate source code, internationalization settings.
- **Recent Documents** - what you had open lately, newest first, each card naming the collection it belongs to.

In the web editor the dashboard also shows which teammates are in the project at the same time.

Global Search

The field at the top searches the whole project rather than the current page.



It matches documents on their display text **and** on their field values - the screenshot finds an item called *Rare Antique* and, separately, a ring whose *Rarity* field is set to *Rare*. Results are grouped by what they are, the header says how many were found, and the matched part of each hit is highlighted. In the web editor the same field also finds projects and users.

Inside a collection the field narrows its scope to that collection and gains filtering; see Finding Documents.

Note

The screenshot also shows the side navigation collapsed to its icon rail.

First Steps

A new project opens with a guide docked to the right of the dashboard.



Five steps - create a schema, open and manage documents, generate source code, publish, load the game data - each ticked off as you do it, with the current one expanded into instructions. The header chevron collapses the card, **Skip Tutorial** dismisses it for good.

The Working Area

Selecting a schema in the side navigation opens its **document collection**, a grid of every document of that schema. Opening one of those documents replaces the grid with the **document form**.

Those two screens are where nearly all the work happens, and each has a page of its own: Filling Documents and Editing a Document.

Tips and Keyboard Shortcuts

- **Jump to a referenced document** - hold Ctrl and click any reference value to open the referenced document's form. This works on references throughout the UI.
- **Move between documents without going back** - Alt/⬛+← and Alt/⬛+→ walk the collection from inside the form, filters and sorting included.
- **Reorder or copy an embedded document** - inside a document form, drag a collection entry by its panel header to move it, or hold Ctrl while dropping to copy it. See Editing a Document.
- The collection grid and the document form have shortcuts of their own, listed on their pages.

See also

- Creating Document Type (Schema)
- Filling Documents
- Editing a Document
- Publishing Game Data
- Generating Source Code

Creating Document Type (Schema)

Schema

Schema is essential for organizing and defining game data in a structured framework. In the context of game data modeling, a schema serves as a blueprint or template that establishes the structure and properties of a particular type of data entity in a game. It defines the columns or fields that represent the various attributes of the entity, similar to how a table has columns or a spreadsheet has cells.

Enemy

Name → Text	HP → Number	Attack → Number
Zombie	10	5
Skeleton	5	2

← Schema

Benefits of Structured Data

Data Organization

The organization of game data into logical entities and attributes is facilitated by the schema. This blueprint or template defines the structure, properties, and relationships of different entities and attributes within the game data, ensuring efficient storage, retrieval, and management.

Data Validation

The integrity and adherence to predefined rules of the game data are ensured through the validation capabilities of the schema. Constraints and validations, such as data types, allowed values, and dependencies, can be defined, preventing errors and inconsistencies in the game data.

Data Consistency

Consistency across the game data is achieved through the standardized structure and rules provided by the schema. It enforces a consistent naming convention, attribute definitions, and relationships between entities, thereby enhancing coherence and simplifying collaboration.

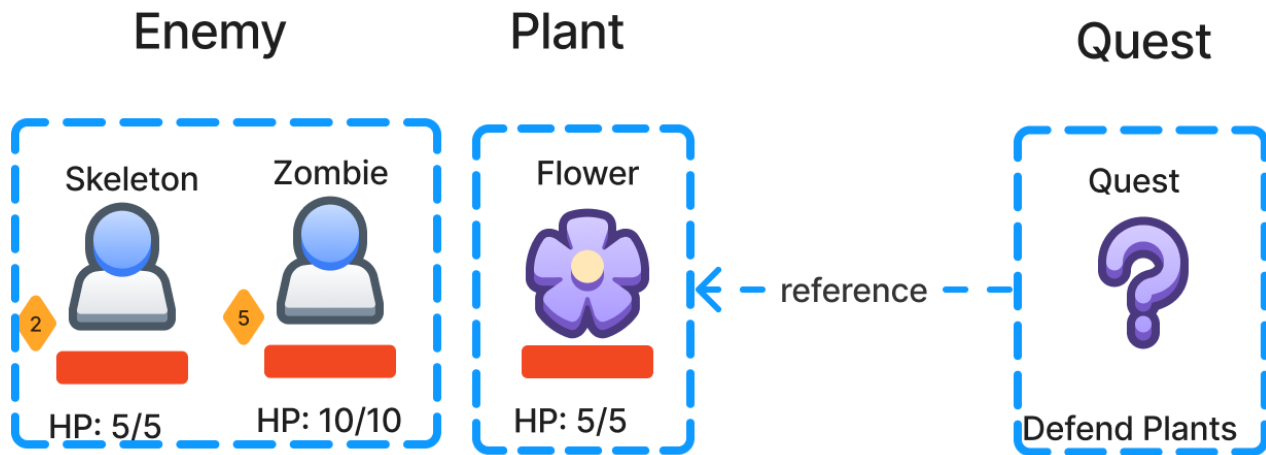
Data Interoperability

Interoperability and integration with external systems or tools are facilitated by a well-defined schema. By establishing a common language and structure, the schema enables seamless data exchange and collaboration with localization tools, analytics platforms, and asset pipelines.

Analyzing Game Requirements

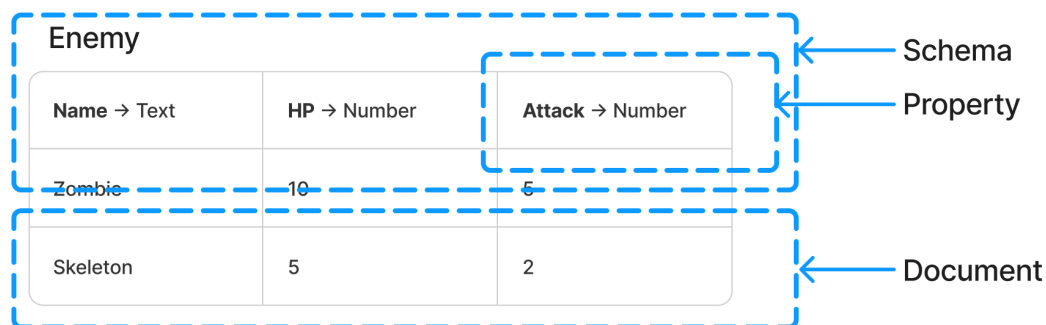
This step involves analyzing the game requirements to understand the design and functionality of the game. It includes studying the game design document and identifying key features, gameplay mechanics, and data elements that need to be captured and represented in the game.

Identifying Schemas and Relationships



In this step, schemas and their relationships within the game are identified. Schemas can be objects, characters, locations, items, quests, or any other significant element in the game. Relationships define how these entities are connected or interact with each other.

Defining Schemas and Properties



This step involves defining schemas to represent the structure and properties of the game data. A schema serves as a blueprint or template for a specific type of data entity, specifying its properties, attributes, and relationships. Properties describe the characteristics and attributes of an entity, such as its name, description, stats, or any other relevant information.

Data Types

Date

The Date data type is used to store dates in [ISO 8601](#) format, which includes the year, month, day, and time with [UTC](#) time zone. This data type is particularly useful for storing information about events that occur on specific dates or for tracking the age of entities. Since dates are stored with [UTC](#) time zone, the data can be consistently interpreted across different time zones.

Source Code Type

Language	Type
C#	System.DateTime
TypeScript	Date
C++ (Unreal Engine)	FDateTime
Haxe	Date

Uniqueness

Date cannot be marked as unique.

Format

yyyy-MM-ddTHH:mm:ss.fffZ

Example

```
"2017-12-27T00:00:00.000Z"
```

```
// Avoid dates before the Unix epoch for cross-platform compatibility:
// some runtimes and serialization libraries do not handle pre-1970 dates correctly.
"1970-01-01T00:00:00.000Z"
```

See also

- [Data Types](#)
- [Time](#)
- [Text](#)

Document

The Document data type in game data schema is used to represent complex structures. A document can contain multiple properties of different data types, including other documents or document collections, allowing for hierarchical data modeling. It is important to note that the lifetime of sub-documents is tied to the lifetime of the parent document, meaning that any changes (e.g. deletion) to the parent document will affect all of its sub-documents.

Source Code Type

Language	Type
C#	class
TypeScript	class
C++ (Unreal Engine)	UCLASS
Haxe	class

Uniqueness

Document cannot be marked as unique.

Example

For example, in a **Dialog**, each node can be a Document with dialog text, response options, and actions that occur after a response is chosen. Each response option can be a sub-document that is another **Dialog** node.

```
{
  "Text": "Welcome to the game. What's your name?",
  "Options": [
    {
      "Text": "My name is John.",
      "Options": [
        {
          "Text": "Hello John! What brings you here?",
          "Options": [
            {
              "Text": "I'm looking for adventure.",
              "Action": "dialog.GiveQuestAndEnd()"
            },
            {
              "Text": "I'm on a mission.",
              "Options": [/* ... */]
            }
          ]
        }
      ]
    },
    {
      "Text": "I prefer not to say.",
      "Action": "dialog.End()"
    }
  ]
}
```

See also

- Data Types
- Document Collection
- Reference

Document Collection

The DocumentCollection data type is used to store an array of sub-documents, which are used to represent complex structures. It is important to note that the lifetime of sub-documents is tied to the lifetime of the parent document, meaning that any changes (e.g. deletion) to the parent document will affect all of its sub-documents.

Source Code Type

Language	Type
C#	ReadOnlyCollection{T}
TypeScript	ReadOnlyArray{T}
C++ (Unreal Engine)	TArray{T}
Haxe	ReadOnlyArray{T}

Uniqueness

Document Collection cannot be marked as unique.

Size

May be limited in number of items. 0 - no limit.

Example

One example use case for DocumentCollection is storing a list of items in a game, such as a chest and its contents. Each item in the chest could be represented by a sub-document containing information such as reference to an item and its quantity.

```
{
  "Name": "Silver Chest",
  "Loot": [
    {
      "Item": { "Id": "Sword" },
      "Quantity": 1
    },
    {
      "Item": { "Id": "Silver" },
      "Quantity": 100
    }
  ]
}
```

See also

- Data Types
- Document
- Reference Collection

Formula

The **Formula** data type allows storing and evaluating C# expressions within game data. It enables developers to define logic that performs calculations based on dynamic inputs and parameters not known until runtime. A formula can be any valid C# expression that returns a value of a supported data type.

Formulas are especially useful in scenarios involving complex calculations, such as determining weapon damage based on factors like target resistance and attack type. By encapsulating such logic in formulas, developers can fine-tune gameplay mechanics without modifying or recompiling the source code.

At runtime, formulas are evaluated using arguments passed to them. These arguments can reference other data properties or objects, providing a high level of flexibility in defining game rules and balancing.

For example, a damage formula might look like this:

```
(weaponPower * (1.0 - targetResistance)) * attackMultiplier
```

This expression multiplies weapon power by a factor based on the target's resistance and then scales the result by an attack multiplier. The final result is the computed damage.

Source Code Type

Language	Type
C#	class
TypeScript	class
C++ (Unreal Engine)	UCLASS
Haxe	class

Uniqueness

Formulas cannot be marked as unique.

Specification

Formulas support the following specification parameters: - param.<name> - Defines a formula parameter and its type.

Example: param.id=System.Int32

- `resultType` - Specifies the return type of the formula.

Example: `resultType=System.String`

- `knownType` - Declares types known to the formula. Only static members of known types can be accessed.

Example: `knownType=System.Math`

- `typeName` - Custom type name for the generated class. If omitted, a name is derived from the containing schema and property.

Example: `typeName=MyDamageCalculatingFormula`

Example

```
"target.HP < 100"
"x != 0"
"target.DoDamage(100)"
"(weapon.Damage / target.DamageResistance) / 2"
```

See also

- Data Types
- Formula Expression AST

Integer

The Integer data type is a whole number data type that is limited to 64 bits. It is used to represent integers without a fractional component. It can be used in cases where you need to store a large range of positive or negative whole numbers, such as in-game currency or player levels.

Unlike the Number data type, integers do not have any precision caveats since they do not store decimal values. Therefore, they are suitable for calculations that require exact values.

Source Code Type

Language	Type
C#	System.Int32 or System.Int64
TypeScript	number or bigint
C++ (Unreal Engine)	int32 or int64
Haxe	Int or Int64

Uniqueness

Integers can be marked as unique.

Size

32 or 64bit

Example

```
0
-1
100
```

See also

- Data Types
- Number
- Pick List

Localized Text

The `LocalizedText` data type is used to store text that needs to be displayed in multiple languages. Unlike the `Text` data type, the `LocalizedText` data type allows the storage of the same text in multiple languages. It supports the whole range of UTF symbols, just like the `Text` data type. The `LocalizedText` data type is essential for games that require localization support, and it makes it easy for game developers to manage text that needs to be displayed in multiple languages.

Source Code Type

Language	Type
C#	<code>LocalizedText</code> or <code>System.String</code>
TypeScript	<code>LocalizedText</code> or <code>string</code>
C++ (Unreal Engine)	<code>FLocalizedText</code> or <code>FText</code>
Haxe	<code>LocalizedText</code> or <code>String</code>

Uniqueness

Localized Text cannot be marked as unique.

Size

May be limited in number of characters. 0 - no limit.

Example

```
{"en-US": "Hello", "fr-FR": "Bonjour"}
```

See also

- Data Types
- Text
- Internationalization

Logical

The `Logical` data type is used to represent boolean values - values that can be either `true` or `false`. It is suitable for modeling simple binary states, conditions, or flags commonly used throughout game data.

Use Cases

The `Logical` type is ideal for:

- Representing **simple state**: Examples include *IsActive*, *IsPublished*, *IsLatest*, *IsReligious*, or *IsOptional*.
- Indicating **ownership or structural conditions**: Examples include *HasPartner*, *HasSecondPhase*.
- Controlling **behavioral expectations**: Examples include *ShouldFinishTutorial*, *ShouldGoFirst*.
- Defining **capabilities or permissions**: Examples include *CanBlock*, *CanRun*, *CanBeSold*.

This makes the `Logical` type a versatile and lightweight choice for many decision-driven properties in schema design.

Comparison with MultiPickList

While the `MultiPickList` data type can represent combinations of multiple states, the `Logical` type is recommended when the property has exactly two possible values. Using `Logical` ensures simplicity, clarity, and optimal editor behavior for binary flags.

Source Code Type

Language	Type
----------	------

C#	System.Boolean
TypeScript	boolean
C++ (Unreal Engine)	bool
Haxe	Bool

Uniqueness

Logical can be marked as unique.

Example

```

true    // canonical JSON boolean
false

// alternative accepted input formats during import:
True    // case-insensitive spelling
False
1        // non-zero integer = true
0        // zero = false
"Yes"    // accepted string aliases
"No"

```

See also

- Data Types
- Integer
- Pick List

Multi-Pick List

The `MultiPickList` data type allows a property to represent multiple values selected from a predefined list of options. It is designed for cases where multiple attributes, traits, or capabilities can be applied to a single entity simultaneously. Unlike `PickList`, which limits the property to a single choice, `MultiPickList` enables the combination of several boolean-like flags into a single compact representation. In the UI, `MultiPickList` is typically presented as a dropdown menu, allowing users to select or deselect multiple options independently.

Storage on Disk

On disk, the `MultiPickList` is stored as an integer bitmask. Each option is assigned a specific bit position. When multiple options are selected, their bits are combined using bitwise OR operations. The number of options is limited by the bit width of the storage format—commonly 32 or 64 options for a 32-bit or 64-bit integer, respectively.

Example

An item in an RPG might be marked as equippable, sellable, and destructible using a single `MultiPickList` field instead of three separate boolean flags.

Source Code Type

Language	Type
C#	[Flags] enum
TypeScript	enum
C++ (Unreal Engine)	UENUM(meta = (Bitflags, UseEnumValuesAsMaskValuesInEditor = "true"))
Haxe	abstract

Uniqueness

Multi-Pick Lists can be marked as unique.

Size

32 or 64bit

Specification

Multi-Pick Lists support the following specification parameters:

- `typeName` - Custom type name for the generated class. If omitted, a name is derived from the containing schema and property.

Example: `typeName=MyEnum`

Example

```
// Single value selected:
1 // stored as integer (bit position 0 = first option)
"Apple" // string name also accepted as input

// Multiple values selected (bitwise OR of individual bits):
3 // Apple(1) | Banana(2) - both selected
"Apple, Banana" // comma-separated string names also accepted
```

See also

- Data Types
- Pick List
- Reference Collection

Number

The Number data type is used to represent decimal numbers. It conforms to the [IEEE 754](#) floating-point standard and can represent both positive and negative numbers, as well as zero. However, due to the limitations of the floating-point representation, precision may be lost when performing certain arithmetic operations. Therefore, it is recommended to use the Integer data type for financial calculations and other scenarios that require high precision.

Some use cases for the Number data type include representing quantities, such as the count of items or the amount of gold reward in the chest, or representing percentages, such as the chance of an event occurring. It can also be used to represent measurements, such as the height of a character or the length of a weapon.

When working with Numbers in game data, it is important to ensure that the precision is appropriate for the use case. Additionally, it may be necessary to round numbers to a certain number of decimal places to avoid displaying unnecessarily precise values to players.

Source Code Type

Language	Type
C#	System.Single or System.Double
TypeScript	number
C++ (Unreal Engine)	float or double
Haxe	Float

Uniqueness

Numbers can be marked as unique.

Size

32 or 64bit

Specification

Numbers support the following specification parameters:

- `precision` - Specify decimal precision of stored number.

Example: `precision=2`

- `displayPrecision` - Specify decimal precision of displayed in UI number.

Example: `displayPrecision=2`

Example

```
3.14
0.21
-3.14
```

See also

- Data Types
- Integer
- Formula

Pick List

The `PickList` data type defines a property that can take one value from a predefined set of options. It is best suited for fields where only a single choice is valid at any given time. This ensures consistency and restricts inputs to known, valid values. This type is commonly used for categorical selections such as character class, item rarity, or region. By standardizing the input, `PickList` helps reduce errors and supports better validation and filtering within game systems. In user interfaces, a `PickList` is typically presented as a dropdown menu, allowing users to choose only one option from the list.

Storage on Disk

On disk, `PickList` values are stored as integers. Each integer corresponds directly to one of the predefined options.

Example

In a character creator, the player might choose “Mage” as their class from a list of available options like Warrior, Mage, or Rogue.

Source Code Type

Language	Type
C#	enum
TypeScript	enum
C++ (Unreal Engine)	UENUM
Haxe	abstract

Uniqueness

Pick Lists can be marked as unique.

Size

32 or 64bit

Specification

Pick Lists support the following specification parameters:

- `typeName` - Custom type name for the generated class. If omitted, a name is derived from the containing schema and property.

Example: `typeName=MyEnum`

Example

```
1 // on-disk stored as integers
"Apple" // string values also valid
```

See also

- Data Types
- Multi Pick List
- Integer

Reference

The Reference data type enables the creation of non-embedded relationships between documents. A reference acts as a pointer to another document by using that document's Id as a key. This approach facilitates linking between related documents without the need to duplicate or embed data.

When using a Reference, the target documents are not stored within the parent document but are instead referenced externally. This is particularly beneficial when working with large or complex data sets, where referencing is more efficient and maintains separation of concerns. It also promotes data integrity by ensuring that only valid document references are maintained.

For example, in a game scenario, a **Chest** with a loot table might hold a reference to an **Item** document, rather than embedding the entire **Item** directly within the **Chest**. This separation allows independent management of the loot logic and item definitions while preserving their relationships.

Source Code Type

Language	Type
C#	DocumentReference{T} or T
TypeScript	DocumentReference{T} or T
C++ (Unreal Engine)	FGameDataDocumentReference
Haxe	DocumentReference{T} or T

Uniqueness

Reference cannot be marked as unique.

Specification

References support the following specification parameter:

- `displayTextTemplate` - Defines a template string for how the referenced value is displayed in the UI.
Example: `displayTextTemplate=Item%3A+%7BName%7D%2C+Count%3A+%7BCount%7D` (renders as: *Item: {Name}, Count: {Count}*)
- `localOnly` - Limits the selection to documents defined within the current document. References from other documents will be excluded from the drop-down list.
Example: `localOnly=true`
- `pathFilter` - Filters available documents based on their path (RFC 6901 JSON Pointer). Use `*` to include all paths, or an empty string `""` to include only root-level documents.
Example: `pathFilter=%2FItem` (documents under `/Item`) `pathFilter=*` (all documents)
`pathFilter=` (only root documents)

Example

```
{ "Id": "Sword" }
"Sword" // just raw Id is also accepted
```

See also

- Data Types
- Document
- Reference Collection

Reference Collection

The `ReferenceCollection` data type is used to create non-embedded, one-to-many relationships between documents. It allows a document to reference multiple documents of the same type without embedding them directly.

When using a `ReferenceCollection`, the referenced documents are stored as references to their original locations, rather than being embedded in the parent document. This approach is particularly useful when dealing with large or complex datasets where referencing is more efficient and scalable than embedding. It also helps maintain data integrity by ensuring that all references point to valid documents.

For example, in a game, each **Quest** may reference a collection of **Objectives**. These objectives can be defined separately for reuse across multiple quests. The `ReferenceCollection` data type enables the quest document to maintain a list of references to objective documents.

Source Code Type

Language	Type
C#	<code>ReadOnlyCollection{T}</code>
TypeScript	<code>ReadOnlyArray{T}</code>
C++ (Unreal Engine)	<code>TArray{T}</code>
Haxe	<code>ReadOnlyArray{T}</code>

Uniqueness

Reference Collection cannot be marked as unique.

Size

May be limited in number of items. 0 - no limit.

Specification

References support the following specification parameter:

- `displayTextTemplate` - Defines a template string for how the referenced value is displayed in the UI.
Example: `displayTextTemplate=Item%3A+%7BName%7D%2C+Count%3A+%7BCount%7D` (renders as: *Item: {Name}, Count: {Count}*)
- `localOnly` - Limits the selection to documents defined within the current document. References from other documents will be excluded from the drop-down list.
Example: `localOnly=true`
- `pathFilter` - Filters available documents based on their path (RFC 6901 JSON Pointer). Use `*` to include all paths, or an empty string `""` to include only root-level documents.
Example: `pathFilter=%2FItem` (documents under `/Item`) `pathFilter=*` (all documents)
`pathFilter=` (only root documents)

Example

```
[{ "Id": "Sword" }, { "Id": "Gold" }]
["Sword", "Gold"] // just raw Ids are also accepted
```

See also

- Data Types
- Reference
- Document Collection

Text

The `Text` data type is used to store simple text values in game data. Unlike the `LocalizedText` data type, `Text` does not have support for multiple translations of the same text. Instead, it allows for the storage of any UTF symbol in a single language. This data type is useful for fields that do not require localization, such as ids, system names, tags or asset paths.

Source Code Type

Language	Type
C#	<code>System.String</code>
TypeScript	<code>string</code>
C++ (Unreal Engine)	<code>FString</code>
Haxe	<code>String</code>

Uniqueness

Text can be marked as unique (case sensitive for uniqueness purposes).

Size

May be limited in number of characters. 0 - no limit.

Example

```
"Hello world!"
```

Sub-Types

- Asset Path
- Vector 2/3/4
- Integer Vector 2/3/4
- Rectangle
- Integer Rectangle
- Tags

See also

- Data Types
- Localized Text

Time

The `Time` data type in game data is equivalent to the `TimeSpan` data type in C#. It is used to store a duration or a time interval, such as the time it takes to complete a task or the length of a cutscene in a game. The `Time` data type is represented as a string in the format `[DD.]HH:mm:ss`, where `DD` is an optional number of days, `HH` is the number of hours, `mm` is the number of minutes, and `ss` is the number of seconds. It can also be stored as a plain integer representing the total number of seconds. Negative durations are supported.

For example, if a task takes 2 hours and 30 minutes to complete, the `Time` data type value would be `02:30:00`.

Source Code Type

Language	Type
C#	<code>System.TimeSpan</code>
TypeScript	<code>TimeSpan</code> (Bundled)
C++ (Unreal Engine)	<code>FTimespan</code>

Haxe	TimeSpan (Bundled)
------	--------------------

Uniqueness

Time cannot be marked as unique.

Format

[DD.]HH:mm:ss or <number-of-seconds>

Example

```
"02:30:00" // 2 hours and 30 minutes
"1.00:00:00" // 1 day
60 // 60 seconds
120 // two minutes

"-00:30:00" // could be negative
```

See also

- Data Types
- Date
- Integer

Asset Path

The `Asset Path` is a sub-type of the `Text` data type and is used to store a path to a game asset within game data.

The `Asset Path Collection` is a collection variant of this type and it is used to store multiple paths. Each path is separated by the `|` (pipe) character.

Paths are typically relative to the project root and can be used to load assets directly at runtime.

Specification

`Asset Path Collection` supports the following specification parameters:

- `assetType` - Filters the types of assets to select. For Unity and Unreal Engine, this refers to class names. For standalone builds, it refers to file extensions. Example: `assetType=Texture` or `assetType=Texture2D`

UI Behavior

This field is represented in the editor by a multi-select searchable input. Users can type to search for assets and add multiple items to the collection using auto-complete.

The search input supports the `t:<TypeName>` filter pattern to limit search results by asset type.

Examples:

- Unity: `swing t:AudioClip`
- Unreal Engine: `swing t:SoundWave`
- Standalone: `swing t:wav`

Validation Behavior

The Charon editor does not validate asset paths during game data loading (i.e., at runtime). You are responsible for ensuring the existence of assets at the provided paths. To improve reliability, consider writing a custom editor extension that validates all asset references before packaging the game.

Asset Localization

Localizable asset paths are not currently supported. If this is a required feature, you are encouraged to suggest it in Charon's [Discord](#) channel.

Path Origin

The meaning of “relative path” for asset references depends on how the game data editor is launched:

- **Unity Editor:** Paths are relative to the Unity project root directory, typically the directory containing the Assets folder. Asset paths will usually start with Assets/. Example: Assets/Textures/MyTexture.png
- **Unreal Engine Editor:** Paths are relative to the Unreal project root directory, usually containing the Content folder. Asset paths often start with /Game/ because it is Unreal Engine package path, not physical path. Example: /Game/Textures/MyTexture
- **Standalone Execution** (via CLI): Paths are relative to the game data file path by default. This behavior can be customized using either:
 - The `--gameAssetsPath` CLI argument:

```
dotnet tool charon ./gamedata.gdjs --gameAssetsPath "C:/Projects/MyGame/"
```
 - The `STANDALONE__GAMEASSETSPATH` environment variable.

Example

```
"Assets/Textures/MyTexture.png" // Unity
"/Game/Textures/MyTexture"      // Unreal Engine
"Textures/MyTexture.png"        // Standalone
```

See also

- Data Types
- Text

Rectangle

The Rectangle is a sub-type of the Text data type used to store four decimal values representing a rectangle. The components represent X, Y, Width, and Height, in that order. Values are separated by spaces and must be parsed from the string before use at runtime.

UI Behavior

This field is represented in the editor as a group of input fields labeled X, Y, Width, and Height. Each component accepts decimal numbers.

Validation Behavior

The editor ensures that a valid rectangle value is entered and only stores properly formatted values. However, there is no runtime validation or built-in parsing logic, so values must be parsed manually in game code.

Example

```
"50.0 100.0 200.5 25.0"
```

Parsing

Unity (C#) example:

```
var uiElement = gameData.AllUIElements.Get("HealthBar"); // -> UIElement
var boundsString = uiElement.Bounds; // -> "50.0 100.0 200.5 25.0"
var components = boundsString.Split(' ');
var bounds = new Rect(
    float.Parse(components[0]), // X
    float.Parse(components[1]), // Y
    float.Parse(components[2]), // Width
    float.Parse(components[3]) // Height
);
```

Unreal Engine (C++) example:

```
auto UIElement = GameData->AllUIElements->Find(TEXT("HealthBar")); // -> UIElement*
FString BoundsString = UIElement->Bounds; // -> "50.0 100.0 200.5 25.0"
TArray<FString> Components;
BoundsString.ParseIntoArray(Components, TEXT(" "));
FSlateRect Bounds(
    FString::Atof(*Components[0]), // X
    FString::Atof(*Components[1]), // Y
    FString::Atof(*Components[2]), // Width
    FString::Atof(*Components[3]) // Height
);
```

TypeScript example:

```
let uiElement = gameData.uiElementsAll.find("HealthBar"); // -> UIElement
let boundsString = uiElement.Bounds; // -> "50.0 100.0 200.5 25.0"
let parts = boundsString.split(" ").map(x => parseFloat(x));
let bounds = { x: parts[0], y: parts[1], width: parts[2], height: parts[3] };
```

Haxe example:

```
var uiElement = gameData.uiElementsAll.get("HealthBar"); // -> UIElement
var boundsString = uiElement.Bounds; // -> "50.0 100.0 200.5 25.0"
var parts = boundsString.split(" ").map(Std.parseFloat);
var bounds = { x: parts[0], y: parts[1], width: parts[2], height: parts[3] };
```

See also

- Data Types
- Rectangle Int
- Vector

Integer Rectangle

The `Integer Rectangle` (`RectangleInt`) is a sub-type of the `Text` data type used to store four integer values representing a rectangle. The components represent X, Y, Width, and Height, in that order. Values are separated by spaces and must be parsed from the string before use at runtime.

UI Behavior

This field is represented in the editor as a group of input fields labeled X, Y, Width, and Height. Each component accepts only whole numbers (integers).

Validation Behavior

The editor ensures that a valid integer rectangle is entered and only stores properly formatted values. However, there is no runtime validation or built-in parsing logic, so values must be parsed manually in game code.

Example

```
"50 100 200 25"
```

Parsing**Unity (C#) example:**

```
var uiElement = gameData.AllUIElements.Get("HealthBar"); // -> UIElement
var boundsString = uiElement.Bounds; // -> "50 100 200 25"
var components = boundsString.Split(' ');
var bounds = new RectInt(
    int.Parse(components[0]), // X
```



```

    int.Parse(components[1]), // Y
    int.Parse(components[2]), // Width
    int.Parse(components[3]) // Height
);

```

Unreal Engine (C++) example:

```

auto UIElement = GameData->AllUIElements->Find(TEXT("HealthBar")); // -> UIElement*
FString BoundsString = UIElement->Bounds; // -> "50 100 200 25"
TArray<FString> Components;
BoundsString.ParseIntoArray(Components, TEXT(" "));
FIntRect Bounds(
    FString::Atoi(*Components[0]), // X
    FString::Atoi(*Components[1]), // Y
    FString::Atoi(*Components[2]), // Width
    FString::Atoi(*Components[3]) // Height
);

```

TypeScript example:

```

let uiElement = gameData.uiElementsAll.find("HealthBar"); // -> UIElement
let boundsString = uiElement.Bounds; // -> "50 100 200 25"
let parts = boundsString.split(" ").map(x => parseInt(x, 10));
let bounds = { x: parts[0], y: parts[1], width: parts[2], height: parts[3] };

```

Haxe example:

```

var uiElement = gameData.uiElementsAll.get("HealthBar"); // -> UIElement
var boundsString = uiElement.Bounds; // -> "50 100 200 25"
var parts = boundsString.split(" ").map(Std.parseInt);
var bounds = { x: parts[0], y: parts[1], width: parts[2], height: parts[3] };

```

See also

- Data Types
- Rectangle
- Vector Int

Tags

The Tag data type is a sub-type of the Text data type and is used to store a reusable set of tag values (e.g., “fire”, “enemy”, “magic”) that can be shared across multiple documents.

The Tag Collection is a collection variant of this type and it is used to store multiple tags. Each tag is separated by the `` (space) character.

UI Behavior

In the editor, this field is represented as a list of chips/pills/tokens with support for free-form input and autocompletion of existing tags. Note that autocompletion relies on scanning all documents of the same type to extract available tags, so performance considerations should be made when using this data type at scale.

Validation Behavior

The Charon editor does not validate tags during game data loading (i.e., at runtime). Ensure that your game logic handles any missing or unexpected tags appropriately.

Relation to Multi-Pick List

Tags are similar to the Multi-Pick List data type but offer more flexibility by allowing free-form input. C-style named tags can be easily converted to a Multi-Pick List by changing the property type. Likewise, Multi-Pick List values can be converted back to Tags if strict value control is no longer needed.

Example

```
"fire"
"poison magical non-resistable"
```

See also

- Data Types
- Text
- Multi Pick List

Vector 2/3/4

The Vector 2/3/4 is a sub-type of the Text data type used to store a group of 2, 3, or 4 decimal values. These values are space-separated and must be parsed from the string before use at runtime.

UI Behavior

This field is represented in the editor as a group of input fields, each allowing the user to enter one component of the vector (e.g., X, Y, Z).

Validation Behavior

The editor ensures that a valid vector is entered and only stores properly formatted values. However, there is no runtime validation or built-in parsing logic, so the value must be manually parsed in game code.

Example

```
"120.1 56.3 30.0"
```

Parsing**Unity (C#) example:**

```
var hero = gameData.AllHeroes.Get("Knight"); // -> Hero
var startingLocationString = hero.StartingLocation; // -> "120.1 56.3 30.0"
var components = startingLocationString.Split(' ');
var startingLocation = new Vector3(
    float.Parse(components[0]),
    float.Parse(components[1]),
    float.Parse(components[2])
);
```

Unreal Engine (C++) example:

```
auto Hero = GameData->AllHeroes->Find(TEXT("Knight")); // -> UHero*
FString StartingLocationString = Hero->StartingLocation; // -> "120.1 56.3 30.0"
TArray<FString> Components;
StartingLocationString.ParseIntoArray(Components, TEXT(" "));
FVector StartingLocation = FVector(
    FString::Atof(*Components[0]),
    FString::Atof(*Components[1]),
    FString::Atof(*Components[2])
);
```

TypeScript example:

```
let hero = gameData.heroesAll.find("Knight"); // -> Hero
let startingLocationString = hero.StartingLocation; // -> "120.1 56.3 30.0"
let parts = startingLocationString.split(" ").map(parseFloat);
let startingLocation: [number, number, number] = [parts[0], parts[1], parts[2]];
```

Haxe example:

```

var heroById = gameData.heroesAll.get("Knight"); // -> Hero
var startingLocationString = heroById.StartingLocation; // -> "120.1 56.3 30.0"
var parts = startingLocationString.split(" ").map(Std.parseFloat);
var startingLocation = { x: parts[0], y: parts[1], z: parts[2] };

```

See also

- Data Types
- Vector Int
- Rectangle

Integer Vector 2/3/4

The Integer Vector 2/3/4 is a sub-type of the Text data type used to store a group of 2, 3, or 4 integer (whole number) values. Values are separated by spaces and must be parsed from the string before use at runtime.

UI Behavior

This field is represented in the editor as a group of input fields, allowing the user to enter each component of the vector (e.g., X, Y, Z) as whole numbers.

Validation Behavior

The editor ensures that a valid integer vector is entered and only stores properly formatted values. However, there is no runtime validation or built-in parsing logic, so the value must be manually parsed in game code.

Example

```
"10 20 5"
```

Parsing**Unity (C#) example:**

```

var hero = gameData.AllHeroes.Get("Knight"); // -> Hero
var gridPositionString = hero.StartingLocation; // -> "10 20 5"
var components = gridPositionString.Split(' ');
var startingLocation = new Vector3Int(
    int.Parse(components[0]),
    int.Parse(components[1]),
    int.Parse(components[2])
);

```

Unreal Engine (C++) example:

```

auto Hero = GameData->AllHeroes->Find(TEXT("Knight")); // -> UHero*
FString GridPositionString = Hero->StartingLocation; // -> "10 20 5"
TArray<FString> Components;
GridPositionString.ParseIntoArray(Components, TEXT(" "));
FIntVector StartingLocation = FIntVector(
    FString::Atoi(*Components[0]),
    FString::Atoi(*Components[1]),
    FString::Atoi(*Components[2])
);

```

TypeScript example:

```

let hero = gameData.heroesAll.find("Knight"); // -> Hero
let gridPositionString = hero.StartingLocation; // -> "10 20 5"

```

```
let parts = gridPositionString.split(" ").map(x => parseInt(x, 10));
let startingLocation: [number, number, number] = [parts[0], parts[1], parts[2]];
```

Haxe example:

```
var heroById = gameData.heroesAll.get("Knight"); // -> Hero
var gridPositionString = heroById.StartingLocation; // -> "10 20 5"
var parts = gridPositionString.split(" ").map(Std.parseInt);
var startingLocation = { x: parts[0], y: parts[1], z: parts[2] };
```

See also

- Data Types
- Vector
- Rectangle Int

Choosing the appropriate data type is essential for ensuring that game data is accurately structured, efficiently stored, and correctly interpreted by both the editor and runtime systems. Each data type serves a specific purpose and is designed to represent particular kinds of values or relationships.

For instance, use the Text data type for simple string values, or the LocalizedText type when the same text must support multiple languages. For numeric data, Number is suited for decimal values, while Integer should be used for whole numbers.

When a property should be selected from a defined set of options, PickList and MultiPickList are recommended for single or multiple selections, respectively. For more complex structures, use Document to define an embedded object, or DocumentCollection to represent a list of such objects.

By selecting data types according to their intended purpose, developers can ensure better validation, clearer data organization, and more predictable behavior across tools and runtime systems.

Data Types

Data Type	Description	Example
Text	A line of text.	"Hello, world!"
LocalizedText	A localized text for multiple languages.	{ "en-US": "Hello", "fr-FR": "Bonjour" }
Logical	A true/false value.	true
Time	A time duration (days, hours, etc.).	"1.00:00:00"
Date	A calendar date and time.	"2017-12-27T00:00:00.000Z"
Number	A decimal number.	3.14
Integer	A whole number.	42
PickList	A single value from a predefined list.	"Red"
MultiPickList	Multiple values from a predefined list.	"Apple, Banana, Cherry"
Document	An embedded structured object.	{ "Id": "Sword", "Name": "Rusty Sword" }
DocumentCollection	A list of embedded structured objects.	[{ "Id": "Sword", "Name": "Rusty Sword" }]
Reference	A pointer to another document by ID.	{ "Id": "Sword" }
ReferenceCollection	A list of pointers to other documents.	[{ "Id": "Sword" }]
Formula	A C# expression evaluated at runtime.	"target.HP < 100"

Composite Types

Composite Type	Base Type	Description
Asset Path	Text	A relative path to a single game asset.
Vector 2/3/4	Text	A group of 2, 3, or 4 decimal values (e.g., position or scale).
Integer Vector 2/3/4	Text	A group of 2, 3, or 4 whole numbers.
Rectangle	Text	A rectangle with floating-point position and size.
Integer Rectangle	Text	A rectangle with integer position and size.
Tags	Text	A reusable set of tags shared across documents.

Use Cases and Guidance

- Use **Text** and **LocalizedText** for titles, descriptions, and UI content.
- Choose **Integer** or **Number** for quantities, rates, and stats.
- Use **Document** or **Reference** to include structured data or link to another entity.
- Use **PickList** or **MultiPickList** when the value should be selected from a known set.
- Use **Asset Path** or **Asset Path Collection** when referencing assets like textures, sounds, or prefabs by path.
- Use **Formula** to define runtime-calculated values, like damage formulas or dynamic prices.
- Use **Tags** when multiple documents share common descriptors (e.g., “fire”, “enemy”, “magic”).

See also

- Game Data Structure
- Text
- Integer
- Reference

Display Text Template

When a document needs to be shown or referenced in the UI, a text representation is required. By default, Charon will use one of the following properties, in order of precedence: `DisplayName`, `Name`, `Title`, or `Id`. This fallback behavior can often result in uninformative or unclear document labels.

For example, consider the following document:

```
{
  "Id": 1,
  "Description": "Bring 40 wolf skins",
  "Type": "RepeatableQuest",
  "Tasks": [ /* ... */ ]
}
```

By default, this document would be displayed in UI lists simply as "1". This can be improved by defining a custom **Display Name Template** for the Schema.

For instance, the template:

```
Quest #{Id}: {Type} - {Description}
```

would render the document's label as:

```
Quest #1: RepeatableQuest - Bring 40 wolf skins
```

Template Syntax

The display name template is a string that may include *replaceable expressions* enclosed in curly braces (`{}`). These expressions support a subset of C-style syntax and can reference any top-level or nested property of the document.

Property Access

- Top-level properties can be accessed directly by name.
- Nested properties in embedded documents can be accessed using dot notation, e.g., `Item.Name`.

Supported Data Types

Type	Example
String	"a string"
Number	1 or 3.14
Boolean	true or false
null	null

Unary Operators

Operator	Example
Logical NOT	<code>!CanWalk</code>
Bitwise Complement	<code>~ResistanceTypes</code>
Negation	<code>-Damage</code>

Binary Operators

The following binary expressions are supported:

Operator	Example
Subtract	<code>TotalPrice - BasePrice</code>
Add	<code>BaseDamage + BonusDamage</code>
Concatenate	<code>"Quest #" + Id</code>
Division	<code>CriticalChance / CurseDebuff</code>
Multiplication	<code>DropRate * 100.0</code>
Power	<code>Damage ** CriticalChance</code>
Modulo	<code>Slots % SlotPerPage</code>
Bitwise AND	<code>DamageFlags & AdditionalDamageFlags</code>
Bitwise OR	<code>DamageTypes ResistanceTypes</code>
Bitwise XOR	<code>ResistanceTypes ^ HealingTypes</code>
Bitwise Left Shift	<code>Flags << 1</code>
Bitwise Right Shift	<code>Flags >> 1</code>
Logical AND (short-circuit)	<code>CanWalk && CanRun</code>
Logical OR (short-circuit)	<code>HasHead CanBeHeadshot</code>
Greater Than	<code>Count > MaxCount</code>
Greater Than or Equal	<code>Count >= MaxCount</code>
Less Than	<code>HP < MaxHP</code>

Less Than or Equal	HP <= MaxHP
Equal To	Type == "RepeatableQuest"
Not Equal To	Description != null
Coalesce	Nickname ?? Name

Ternary Operator

Ternary expressions are supported using the standard syntax:

```
{Damage >= 0 ? "Total Damage" : "Total Heal"}: {Damage}
```

This will result in either “Total Damage: X” or “Total Heal: X”, depending on the value of Damage.

Operator Precedence

You can control expression evaluation order using parentheses. For example:

```
Max Damage: {BaseDamage * (BaseCriticalChance + BonusCriticalChance)}
```

Boolean Coercion

Many types can be implicitly coerced to a Boolean value:

- **Numeric values** are considered `true` if they are not equal to 0.
- **Documents, References, and Formulas** are considered `true` if they are not null.
- **Collections** (Document or Reference), **Text**, and **Localized Text** are `true` if they are not empty.

Other types must be explicitly compared to a value to be evaluated as Boolean. These coercion rules apply to ternary expressions and unary logical negation (!).

```
Has Items: {Items ? "Yes" : "No"}
```

Format Specifiers

Template expressions can include format specifiers, similar to C#'s formatting syntax. Add a colon and format string at the end of the substitution expression:

```
Resist Chance: {ResistChance * 100.0 :F2}
```

This limits the precision to two decimal places.

Supported format specifiers:

- **F**: Fixed-point (decimal precision), e.g., `F2` for two decimal places.
- **X / x**: Uppercase / lowercase hexadecimal (integers only), e.g., `X16` for hexadecimal string padded with zeroes to length of 16 characters.
- **B**: Binary representation (integers only), e.g., `B32` for binary string padded with zeroes to length of 32 characters.
- **L**: String length limitation, e.g., `L12` for a string shorter than 12 characters nothing will change, but if it is longer than 12 it will be truncated to 12 and an ellipsis character ... will be added.

Collection Accessors

Collections can be accessed by index using square brackets. If the index is out of bounds, the result is `null`.

```
First Item: {Items[0]}
```

The number of items in a collection can be accessed using the `.Count` property:

```
Item Count: {Items.Count}
```

To create a joined string from all items in a collection, use the `Join(separator: string)` method:

```
Items: {Items.Join(" and ")}
```

This would produce output like:

Items: Sword and Shield and Armor

String Manipulation Methods

String expressions support several methods:

- `Trim()` - Removes leading and trailing whitespace.
- `Substring(start: int, count: int)` - Extracts a substring from a given position.
- `ToString()` - Converts any non-null value to its string representation.

String concatenation is done using the `+` operator:

```
Damage: {Damage + " [" + DamageType + "]"}
```

Result:

Damage: 100 [Fire]

See also

- Schema
- Property
- Id Property
- Shared Property
- All Data Types
- Creating Document Type (Schema)

Schema

A **Schema** defines the structure of a specific type of game data entity. It acts as a blueprint that outlines the fields (properties) associated with the entity, much like columns in a database table or cells in a spreadsheet. Schemas are central to organizing and modeling structured game data.

Where Schemas Are Edited

A schema is a document like any other, edited in the same document form as game data. Open it with **Actions** → **Design** from a collection page, or from **Metadata** in the side menu; the breadcrumb marks the form as **[Design]**.

The screenshot shows a web-based form for editing a schema named 'Combat Effect'. The breadcrumb trail at the top is 'Home > Kind Reindeer > My RPG Game > Schemas > Combat Effect [Design]'. The form includes a 'Save' button and an 'Actions' menu. Fields include: 'Name' (CombatEffect), 'Display Name' (Combat Effect), 'Type' (Component — Embedded-only document, hidden from navigation menu), 'Display Text Template' (Changes: (Changes), Duration: (Duration) (DurationUnit)), 'Icon' (Fugue — Raster graphics 18x18px icon set), 'Description', 'Group' (Folder/Subfolder/...), 'Menu Visibility' (Not Applicable), 'Id Generator' (Numeric Sequence), and a 'Properties' section with expandable fields for 'Id * (auto-generated)', 'Changes *', 'Duration', and 'Duration Unit *'. An 'Add Property ...' button is at the bottom.

The screenshot has **Actions** → **Show advanced options** turned on, which reveals the rarely used fields. Without it the form is shorter.

Schema documents also have a **Layout** view mode, which arranges these properties on the document form that editors will use. See [Editing a Document](#).

Name

The **Name** of a schema is used as an alternative identifier to the *Id* and is also used in generated source code as the name of the resulting class. Therefore, it must be a valid C-style identifier: it should start with a letter and include only Latin letters, digits, or underscores.

Schema names are also used as the name of the collection when storing or exporting data. It is recommended to use PascalCase and singular form for schema names.

Display Name

The **Display Name** is used in the UI and can contain any characters. There are no restrictions on alphabet or character set.

Type

The **Type** setting defines how instances (documents) of this schema are created and managed:

- **Normal**: Standard behavior. Documents can be created in the root collection, embedded in other documents, or may be absent entirely.
- **Component**: These documents can only be embedded within other documents. They do not appear in the main navigation menu.
- **Settings**: A singleton-style schema. Only one root-level document can exist, and it cannot be embedded in other documents.

Display Text Template

The **Display Text Template** defines how documents of this schema are presented in the UI when a textual representation is needed-e.g., in dropdowns, reference lists, or relationship views.

Icon

The **Icon** is displayed next to the schema name in the navigation menu or selection dialogs within the UI.

Description

The **Description** helps other users understand the purpose of the schema. It is shown in the UI and is also included as a documentation comment in the generated source code for the corresponding class.

Group

The **Group** defines the hierarchical path of the schema in the left-hand navigation menu. It behaves similarly to a folder path, with sections separated by the / character.

Menu Visibility

This flag determines whether the schema appears in the left-hand navigation menu in the UI.

Id Generator

The **Id Generator** setting defines how the **Id** Property is managed for this schema. If set to *None*, the **Id** property must be configured manually.

Properties

A list of **Properties** that define the fields associated with this schema. Each row expands into the property's own fields, **Add Property** . . . appends a new one, and the bin icon removes it.

An Id property managed by the schema's **Id Generator** is marked (*auto-generated*) and offers a **Hide** checkbox, which keeps the field out of the document form for editors who never need to see it. The checkbox writes Display=Hidden into the property's Specification.

See also

- Display Text Template
- Property
- Id Property
- Shared Property
- Specification
- All Data Types
- Creating Document Type (Schema)
- Editing a Document

Schema Types

Every Schema has a **Type** field that controls how its documents are created, stored, and navigated within the data model. Choosing the right type is fundamental to a well-structured data model.

Normal	72
Component	72
Settings	73
Union (Tagged Union)	73
Why Union instead of the other inheritance patterns	73
Defining a Union schema	74
Storage format	74
Generated source code	74
Malformed Union values	74
See also	75

Normal

Normal is the default type. Documents of a Normal schema can be:

- Created as standalone root-level entries, visible in the left navigation menu.
- Embedded inside other documents as Document or DocumentCollection property values.
- Referenced from other documents via Reference or ReferenceCollection properties.

Use **Normal** for any entity that has an independent identity and may need to be listed, searched, and referenced across the project - items, characters, quests, abilities, etc.

```
{ "Id": "Sword", "Name": "Iron Sword", "Damage": 15 }
```

Component

Component schemas model sub-objects that are *always embedded* inside another document and never exist as standalone root-level entries.

- Do **not** appear in the left navigation menu.

- Can only be used as a value for a Document or DocumentCollection property.
- Useful for reusable sub-structures with no independent identity.

Typical examples: a Stat block inside a Character, a DropEntry inside a LootTable, a BonusEffect inside a Weapon.

Warning

Documents of a **Component** schema are never exported or published as root-level collection entries - export and publication skip them unless they are embedded inside another document. If documents of a schema must live in the root collection *and* appear in exported data, the schema type should be **Normal**. Normal documents are exported in both cases: standalone and embedded.

```
{
  "Id": "Hero_Arbalest",
  "BaseStats": { "HP": 100, "Speed": 4, "Damage": 12 }
}
```

Settings

Settings is a singleton type. Exactly **one** document of this schema may exist in the root-level collection. It cannot be embedded inside other documents.

- Created automatically on first access and cannot be deleted.
- Appears in the navigation menu as a direct link (no list page).
- Suited for project-wide configuration: GameSettings, BalanceConfig, GlobalRules.

```
{ "Id": "settings", "DifficultyMultiplier": 1.5, "MaxPlayersPerRoom": 4 }
```

Union (Tagged Union)

A **Union** schema - also known as a Discriminated Union, Sum Type, or \$oneOf - lets a single collection slot or property hold documents of **different shapes** in a type-safe way.

A Union schema does not define properties itself. Instead it declares a set of **variant schemas** (any Normal or Component schema).

Why Union instead of the other inheritance patterns

See Implementing Inheritance for a full comparison, but in short:

Pattern	Storage cost	UI	Schema refactoring
Composition	Extra Item schema level	Standard form	Move fields to Item
Merging	All fields on every doc	All fields always visible	Edit one big schema
Aggregation	Nullable ref per variant	Multiple nullable refs	Add a new ref field
Union	Only active variant's fields	Type picker, sparse form	Add a new variant

Key properties of Union:

- **Sparse storage** - unset variant fields are not written to disk; only the active variant's properties are stored.
- **First-class UI** - when editing a Union field the editor shows a type-selector dropdown and only renders the form for the chosen variant.

Defining a Union schema

1. Create the variant schemas (Normal or Component) with their own properties.
2. Create a new schema, set **Type** to **Union**, and list the variant schemas in **Variants**.
3. Use the Union schema as the ReferenceType of a Reference, ReferenceCollection, Document, or DocumentCollection property on another schema.

Storage format

Unlike many serialisation libraries that use a "\$type" or "@class" string field to tag the variant, Charon uses **property-key presence** as the discriminator. Each union slot is serialised as a **single-key wrapper object** whose key is the variant schema name and whose value is the document body:

```
{ "Weapon": { "Id": "Sword01", "Damage": 15, "WeaponType": "Melee" } }
{ "Armor": { "Id": "Plate01", "Defense": 8 } }
{ "Shield": { "Id": "Shield01", "BlockChance": 0.25 } }, /* Weapon: null, "Armo
```

The variant is identified by whichever key is present in the wrapper - "Weapon", "Armor", "Shield", etc. There is no "\$type" string anywhere in the stored data; the schema name *is* the discriminator.

A full collection of union documents therefore looks like:

```
{
  "Collections": {
    "ShopItem": [
      { "Weapon": { "Id": "Sword01", "Damage": 15, "WeaponType": "Melee" } },
      { "Armor": { "Id": "Plate01", "Defense": 8 } },
      { "Shield": { "Id": "Shield01", "BlockChance": 0.25 } }
    ]
  }
}
```

Generated source code

Union schemas generate a container type in the target language. Each variant is a nullable field inside that container; exactly one field is non-null at runtime.

Malformed Union values

A Union value is **well-formed** when the wrapper object contains exactly two keys: Id and exactly one variant key. Two malformed states can occur - usually as a result of a bulk import, a hand-crafted JSON payload, or a patch applied to a Union field without accounting for the wrapper format.

Empty union - the Id is present but no variant key is set:

```
{ "Id": "item_01" }
```

Conflicting union - two or more variant keys are set simultaneously:

```
{ "Id": "item_01", "Weapon": { "Damage": 15 }, "Potion": { "HP": 5 } }
```

Both states are detected by the checkConstraints validation option (enabled by default) and reported as separate error codes in the validation report:

- Empty union → emptyUnion error
- Conflicting union → conflictingUnionOptions error, listing the conflicting variant names

Repairing malformed unions

The resolveConflictingUnions repair option fixes both cases automatically. It can be used with DATA VALIDATE or DATA IMPORT:

```
# repair in-place
dnx dotnet-charon -- DATA VALIDATE --dataBase "gamedata.json" \
```

```
--validationOptions repair resolveConflictingUnions
```

```
# repair while importing
```

```
dnx dotnet-charon -- DATA IMPORT --dataBase "gamedata.json" --input "./patch.json" \
--validationOptions repair resolveConflictingUnions
```

Repair behaviour:

Malformed state	What the repair does
Empty union	Erases the union value entirely (sets it to null).
Conflicting union	Keeps the last variant in schema-property order; all other variant values are discarded.

Note

The “last variant in schema-property order” is determined by the order of variant schemas as listed in the Union schema definition, not by the order of keys in the stored JSON. If the outcome of the automatic repair is not acceptable, correct the source data manually before importing.

Note

The Union type is the recommended approach when you need heterogeneous collections and want first-class editor support. Prefer it over Aggregation for new designs.

See also

- Schema
- Implementing Inheritance
- Specification
- Document
- Reference

Specification

The Specification field is a flexible extension point available on both Schema and Property documents. It enables developers to attach additional metadata in a standardized format that can be used for custom logic in UI field editors, source code generation, or other tooling integrations.

Overview

The Specification field stores a string formatted using the [application/x-www-form-urlencoded](#) standard (commonly referred to as [URLSearchParams](#) format). It contains key-value pairs separated by ampersands (&), where each key and value are URL-encoded.

For example:

```
icon=table&group=Combat
```

Use Cases

The *Specification* field is commonly used for:

- **Schema metadata:** Define an icon, group behavior, or UI preferences:

```
icon=table&category=Gameplay&hideInMenu=true
```

- **Property editor extensions:** Pass configuration values to custom editors:

colorFormat=RGB&alpha=true

- **Source code generation:** Customize how types or methods are generated:

typeName=MyEnum&csAttribute=Obsolete

Encoding Guide

All keys and values must be properly URL-encoded to ensure compatibility with parsing libraries and avoid format-breaking characters.

Encoding in JavaScript

In any browser console or Node.js REPL:

```
const params = new URLSearchParams({
  icon: "sword",
  group: "Items and Stuff",
  noThumbnail: true
});
console.log(params.toString());
// Output: icon=sword&group=Items%20and%20Stuff&noThumbnail=true
```

Encoding in Excel

To encode values manually using Excel formulas:

- Build a key-value pair:

=ENCODEURL("noThumbnail") & "=" & ENCODEURL("true")

- Combine multiple:

=ENCODEURL("icon") & "=" & ENCODEURL("sword") & "&" & ENCODEURL("group") & "=" & ENCODEURL("noThumbnail")

Best Practices

- **Use lowercase or camelCase for keys** to ensure consistency.
- **Avoid spaces or special characters** in keys; values should always be encoded.

See also

- Display Text Template
- Property
- Schema
- All Data Types
- Creating Document Type (Schema)

Id Property

Each schema includes an Id property that defines the unique identifier for a document. This property is always named Id and must satisfy the following constraints:

- Must be unique.
- Required and must be non-empty (for Text data type).
- Must use a data type with a stable text representation and comparison support, such as:
 - Time

- Date
- Number
- Integer
- Text
- MultiPickList
- PickList

Generated Id

Schemas can specify that the Id property should be automatically generated. Based on the selected Id Generator type, an appropriate Id property will be added to the schema.

Supported Id Generators:

- ObjectId
- Guid
- Sequence
- GlobalSequence

Generated ObjectId

A BSON ObjectId is a 96-bit unique identifier derived from the machine name, process ID, and timestamp. It is always increasing and well-suited for sorting documents by creation time.

Id property details:

- **Data Type:** Text
- **Size:** 24 characters
- **Requirement:** Not Empty
- **Example:** "049bc0604c363a980b000088"

Generated Guid

A GUID (Globally Unique Identifier) is a 128-bit identifier typically represented as a 32-character hexadecimal string. It ensures global uniqueness across systems.

Id property details:

- **Data Type:** Text
- **Size:** 32 characters
- **Requirement:** Not Empty
- **Example:** "6fe4202b1b9c4565b439980138524112"

Generated Sequence

A sequential numeric identifier unique to each document within a specific schema. Documents from different schemas may share the same numeric Id. For globally unique numeric IDs, use GlobalSequence.

Id property details:

- **Data Type:** Integer
- **Size:** 32-bit
- **Requirement:** Not Null
- **Example:** 1

Generated Global Sequence

A globally unique numeric identifier across all schemas and documents in the project.

Id property details:

- **Data Type:** Integer
- **Size:** 32-bit
- **Requirement:** Not Null
- **Example:** 69

Non-Generated Id

Schemas may also use a manually defined Id property by selecting the None option for the Id Generator. You can define the Id property using any supported data type listed above.

All Id constraints will be automatically validated when saving the schema. You can change a schema's Id Generator to None at any time, but you cannot switch back to a generator once this change is made.

Id Placeholder

If you want to assign a temporary value to the ID of a document being created—for example, for internal reference—you can use a placeholder string such as `_ID_XXX_ANY_UNIQUE_TEXT_HERE_XXX`. This placeholder will be automatically replaced with the actual generated ID during saving or importing.

Before save:

```
{
  "Id": "_ID_ITEM_123",
  "Name": "My New Item",
  "ItemReference": {
    "Id": "_ID_ITEM_123", // self-reference
    "DisplayName": "My New Item"
  }
}
```

After save:

```
{
  "Id": 42,
  "Name": "My New Item",
  "ItemReference": {
    "Id": 42,
    "DisplayName": "My New Item"
  }
}
```

See also

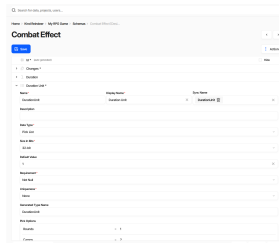
- Schema
- Property
- Shared Property
- All Data Types
- Creating Document Type (Schema)

Schema Property

Properties define which fields a document will contain.

Where Properties Are Edited

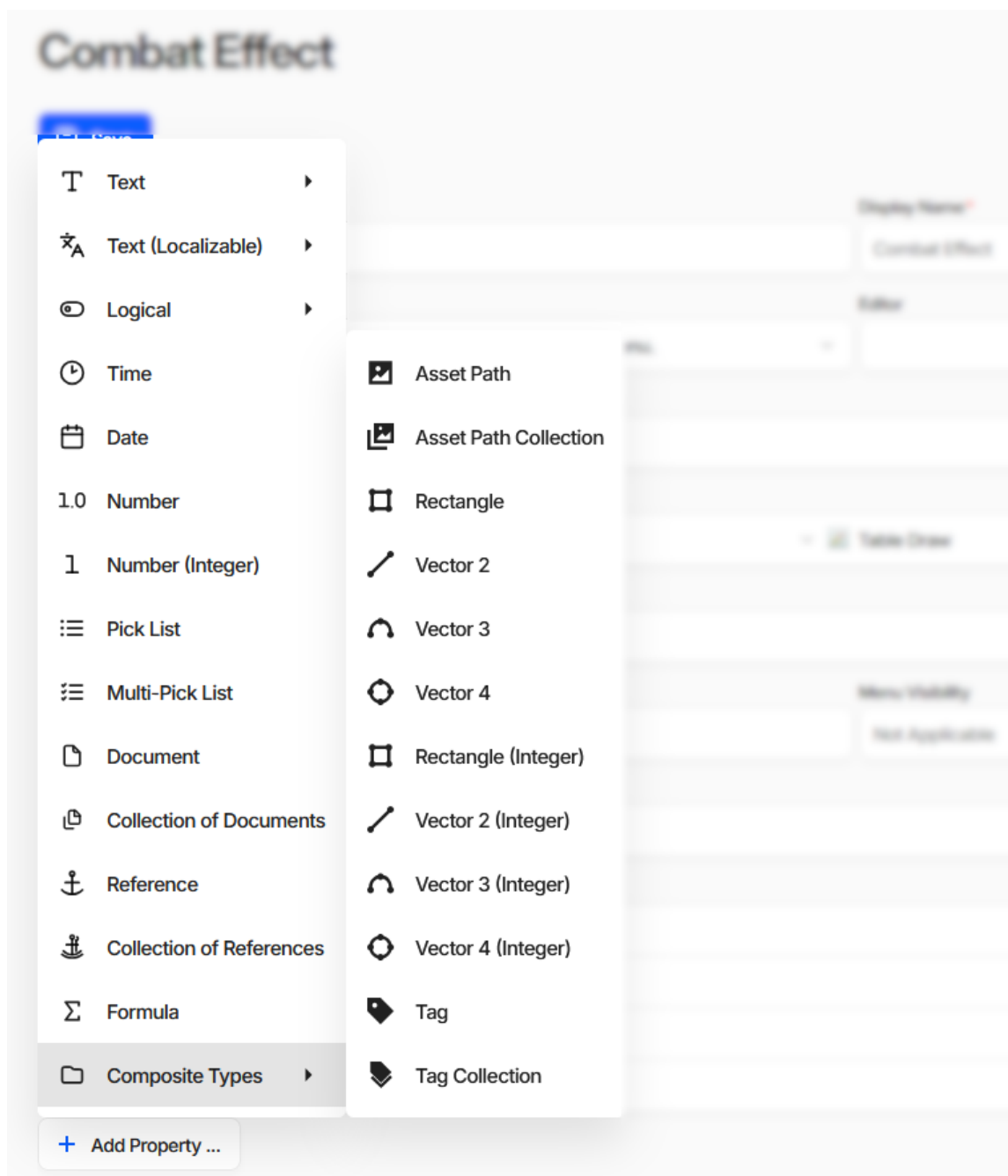
Properties live inside the schema form, as a list under **Properties**. Expanding a row opens the fields of that property.



The screenshot has Actions → Show advanced options turned on. Which fields are shown also depends on the **Data Type** - *Pick Options* below only appears for pick lists, *Size In Bits* only for the types that are stored as a number.

Adding a Property

Add Property ... asks for the data type first:



The menu is the list of data types, plus a **Composite Types** submenu of ready-made shapes - vectors, rectangles, asset paths, and tags. Those are not extra data types: each one adds a property whose structure is already filled in for that shape.

Name

The **Name** of a property is used as an alternative identifier to the *Id* and is also used in generated source code as the name of the resulting field/property. Therefore, it must be a valid C-style identifier: it should start with a letter and include only Latin letters, digits, or underscores.

Property names are also used as the name of the fields when storing or exporting data. It is recommended to use PascalCase and singular form for property names.

Display Name

The **Display Name** is used in the UI and can contain any characters. There are no restrictions on alphabet or character set.

Description

The **Description** helps other users understand the purpose of the property. It is shown in the UI and is also included as a documentation comment in the generated source code for the corresponding field/property.

Sync Name

Some properties describe data types that are common across multiple schemas. For example, a `PickList` property might benefit from shared pick options used in several schemas. These synced properties are referred to as Shared Properties.

Data Type

Each property has a data type, which determines what kind of data can be entered into the document's field, how it is stored on disk, and how it is represented at runtime. The editor UI will attempt to validate input based on the selected data type but does not strictly enforce it. At runtime, game data loading code will validate the data types and may fail if the document is malformed.

Requirement

Specifies whether a value must be provided for the field. Possible values include:

- **None** – No requirement to fill in the property. It can be absent from the document both on disk and at runtime. Typically represented as a nullable or optional type in supported source code languages.
- **Not Null** – A value must be provided. For the `Text` data type, an empty string is considered a valid value.
- **Not Empty** – The field must contain a non-empty value for `Text` or have at least one item for `Collection` data types.

Uniqueness

Specifies uniqueness constraints for field values across documents. Useful to ensure certain values, such as names, are only used once.

- **Unique** – The value must be unique across all documents.
- **Unique In Collection** – The value must be unique within the scope of the current collection.

Default Value

Defines the default value to use for newly created documents or to populate existing documents when a new required property is added. The default must be a properly formatted value consistent with the specified data type.

Size In Bits

The in-memory size of the value, for the data types stored as a number - integers and pick lists. It decides the width of the type used in the generated source code, so narrowing it is a way to keep large collections small.

Generated Type Name

The name this property's type gets in the generated source code. Leave it empty and Charon picks the name itself. It matters for the types that become a type of their own - the enum behind a pick list, for instance.

Pick Options

The list of allowed values of a `Pick List` or `Multi-Pick List`, each a name paired with the number it is stored as. In generated code they become an enumeration or the nearest equivalent the target language has, so the names must be valid identifiers.

Specification

Additional parameters that do not have a field of their own, written as a URL query string and read by the editor UI and by the code generators. See [Specification](#).

Id Property

Each document includes a required `Id` property, which has specific constraints and must be present and valid.

See also

- [Schema](#)
- [Id Property](#)
- [Shared Property](#)
- [All Data Types](#)
- [Creating Document Type \(Schema\)](#)
- [Editing a Document](#)

Shared Property

A **Shared Property** is a Schema property that syncs data type-related configuration with other properties of the same name across different schemas. When a shared property is modified in one schema, all other synced properties with the same shared name are updated accordingly.

Shared properties can be unlinked (or “decoupled”) at any time, converting them into independent, schema-specific properties.

Replicated Parameters

Shared properties replicate a defined subset of configuration parameters between themselves:

- Data Type
- Default Value
- Uniqueness
- Requirement
- Reference Type (for *Document*, *DocumentCollection*, *Reference*, and *ReferenceCollection* types)
- Size
- Specifications for *PickList*, *MultiPickList*, and *Formula* data types

These parameters ensure consistency across schemas that use the same shared property.

Impact on Generated Source Code

In cases where a property results in a distinct type in the generated source code-such as a *PickList* producing an enum-the name of the shared property is used to derive the generated type name.

For example:

- A shared *PickList* property named *Damage* Type would generate:
 - *DamageType* enum in C#, Haxe, TypeScript
 - *EDamageType* enum in Unreal Engine C++

To customize this behavior, the *Generated Type Name* field of the property can be set manually to override the default naming convention.

See also

- Schema
- Property
- Id Property
- Shared Property
- All Data Types
- Creating Document Type (Schema)

See also

- Schema
- Property
- Implementing Inheritance
- Filling Documents
- Publishing Game Data
- Generating Source Code

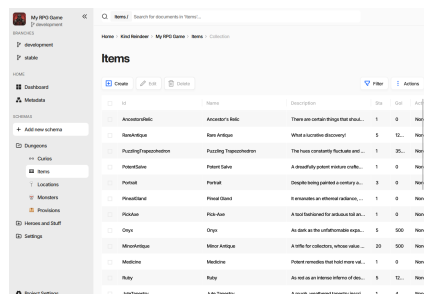
Filling Documents

Schemas describe what a document looks like. The collection page is where documents are actually typed in - a spreadsheet-like grid where rows are documents and columns are schema properties. Most day-to-day data entry never leaves it; larger batches arrive through import instead.

Where Documents Live	83
The Actions Menu	83
Creating Documents	84
Editing in the Grid	85
Finding Documents	86
Filtering by Column	86
Writing Filters by Hand	87
Sorting and Column Width	87
Choosing Columns	87
Changing What the Columns Are	87
Deleting Documents	88
Deleting a Schema	88
Bringing Data In From Files	89
Automating from the CLI	89

Where Documents Live

Every schema appears in the left side menu, grouped the way its **Group** field says. Selecting one opens its collection page.



The page has four parts:

- **The search field** at the top, scoped to the current collection - the chip in front of it (Items /) shows the scope. It does plain-text search and holds filters, see [Finding Documents](#).
- **The toolbar** with Create, Edit, and Delete. Save and Discard join them as soon as something is changed. When the game data is read-only, Edit reads View and the editing buttons are hidden.
- **The grid**, one row per document. It is virtualized, so a collection of any size scrolls in a single page.
- **Filter** and **Actions** on the right.

Note

In the web editor the avatars of teammates viewing the same collection appear next to the Filter button, and the grid marks the fields they are editing.

The Actions Menu

Everything that is not a per-document edit lives in Actions.



Entry	What it does
Design	Opens the schema of this collection in the schema editor. This is where columns come from - see Changing What the Columns Are .
Clone	Copies the selected document and opens the copy for editing.
Delete	Same as the toolbar button, see Deleting Documents .
Refresh	Fetches the latest version of the collection from the server.
Export...	Opens the export wizard with this collection preselected.
Export To	Exports the collection straight to a JSON, BSON, Message Pack, or spreadsheet file, skipping the wizard.
Import...	Opens the import wizard.
Customize Columns...	Picks which properties are shown as columns, see Choosing Columns .

Extensions installed in the project add their own entries below the separator - see Custom Actions.

Creating Documents

Create opens an empty document form for the collection's schema. Fill in the fields, press Save, and the new document appears in the grid.

For entering many short documents in a row, stay in the grid instead. The + New <Schema> button under the last row adds a row in place:

The row appears immediately but is not written to the database until you save it, exactly like any other grid edit.

Editing in the Grid

Double-click a cell, or select it and press Enter, to edit it in place. The editor that opens matches the property's data type - a text box, a number field, a date picker, a reference picker, and so on. Arrow keys and Tab move between cells.

☐	Id	Name	Effects
☐	BadHumors	Bad Humors	Max Hit Points Percent: -0.2, Condition:
☐	CreepingCough	Creeping Cough	Damage Percent: -0.2, Condition:
☐	Ennui	Ennui	Virtue Chance: -0.15, Condition:
☐	Hemophilia	Hemophilia	Bleed Resistance: -0.5, Condition:
☐	Rabies	Rabies	Accuracy: -0.1, Condition: , Damage Perc...
☐	TheRuns	The Runs	Max Hit Points Percent: -0.1, Condition: , ...
☒	RedRashes	Red	None

+ New Disease

Edits stay local until they are saved. Changed cells and their rows are marked, and Save and Discard appear in the toolbar:

- Save writes the changes and makes them visible to everyone else. With rows selected it saves only those rows; with nothing selected it saves everything that changed.
- Discard reverts to the last saved state, following the same selection rule.
- When a changed document fails validation, Save becomes Save Anyway and carries a badge with the number of errors. Charon stores the document anyway.

Note

Broken data is allowed while you work on it. A reference pointing at a document that does not exist yet, a required field still empty, a formula that does not parse - none of that stops a save, in the editor or in the CLI. Half-finished data is a normal state during development, and a tool that refuses it forces you to invent placeholder values just to close a form.

The tolerance ends at publication. Generated code and the game-side loaders assume the data satisfies the schema, so anything the Publication wizard reports as a critical issue should be fixed before shipping, not after. See Validation for the checks, the two severities, and how to gate a build on a clean review.

Action	Shortcut
Create a document in the form	Ctrl/⬛+Alt+N
Add a row inline	Ctrl/⬛+Shift+N
Duplicate the selected document	Ctrl/⬛+Shift+D
Open the edit form	Ctrl + click on the row

Edit a cell in place	double click, or Enter
Save	Ctrl/⌘+S
Save ignoring validation errors	Shift + click on Save
Undo / Redo	Ctrl/⌘+Z / Ctrl/⌘+Y

Finding Documents

The search field above the grid does two things: plain-text search over the collection, and structured filtering by property.

Filtering by Column

Filter is a shortcut that writes the first half of a filter for you. Pick a property, then a comparison:

The screenshot shows the 'Items' collection interface. On the left is a sidebar with navigation options: My RPG Game, development, stable, Dashboard, Metadata, Add new schema, Dungeons, Curios, Items (selected), Locations, Monsters, Provisions, and Heroes and Stuff. The main area displays the 'Items' collection with a search bar and a table of items. A 'Filter' dropdown menu is open, showing a list of properties: Id, Name, Description, Stack Size, Gold Value, and Activation Effects. A comparison menu is also open, showing options: Equal, Not Equal, Contains Text, Greater Than, Greater Than Or Equal, Less Than, and Less Than Or Equal. The 'Contains Text' option is highlighted.

Choosing **Name** → **Contains Text** types `/Name contains` - with a trailing space - into the search field and puts the cursor at the end of it. Type the value and press Enter:

The screenshot shows the 'Items' collection interface after applying the filter. The search bar now contains the text `/Name contains Rare`. A red arrow points to the search bar. The table below shows the filtered results, with only the 'Rare Antique' item visible.

The finished filter becomes a chip in front of the search field and the grid narrows to the matching rows. Repeat to add more filters - they all apply together. Removing a chip removes that filter.

Properties of an embedded document open a submenu, and the path written into the field points inside the document: `/Stats/Hp > 10`.

Writing Filters by Hand

The menu only saves typing. The same filter can be typed straight into the search field as `[property] [operator] [value]` followed by Enter. Quote the value if it contains spaces - single quotes, double quotes, and backticks all work:

```
/Name contains "Ancestor Relic"
/StackSize >= 5
/GoldValue != 0
```

Comparison	From the menu	Also accepted
Equal	=	==, equal
Not Equal	!=	<>, notequal
Contains Text	contains	like
Greater Than	>	greaterthan
Greater Than Or Equal	>=	greaterthanorequal
Less Than	<	lessthan
Less Than Or Equal	<=	lessthanorequal
One of a set	-	in

Whatever is left in the field that is not a complete filter is used as plain-text search.

Sorting and Column Width

Click a column header to sort by that column, click again to reverse the direction. Column widths are dragged from the right edge of the header.

Choosing Columns

Actions → Customize Columns... decides which properties become columns and in which order.



Tick a property to show it, drag it by the handle to move it, then press Apply. **Reset** returns the grid to the schema's own order.

Three extra columns are offered besides the schema's properties: **Display Name** - the document's rendered display text, **Path**, and **Error Count**, the number of validation issues found in the document.

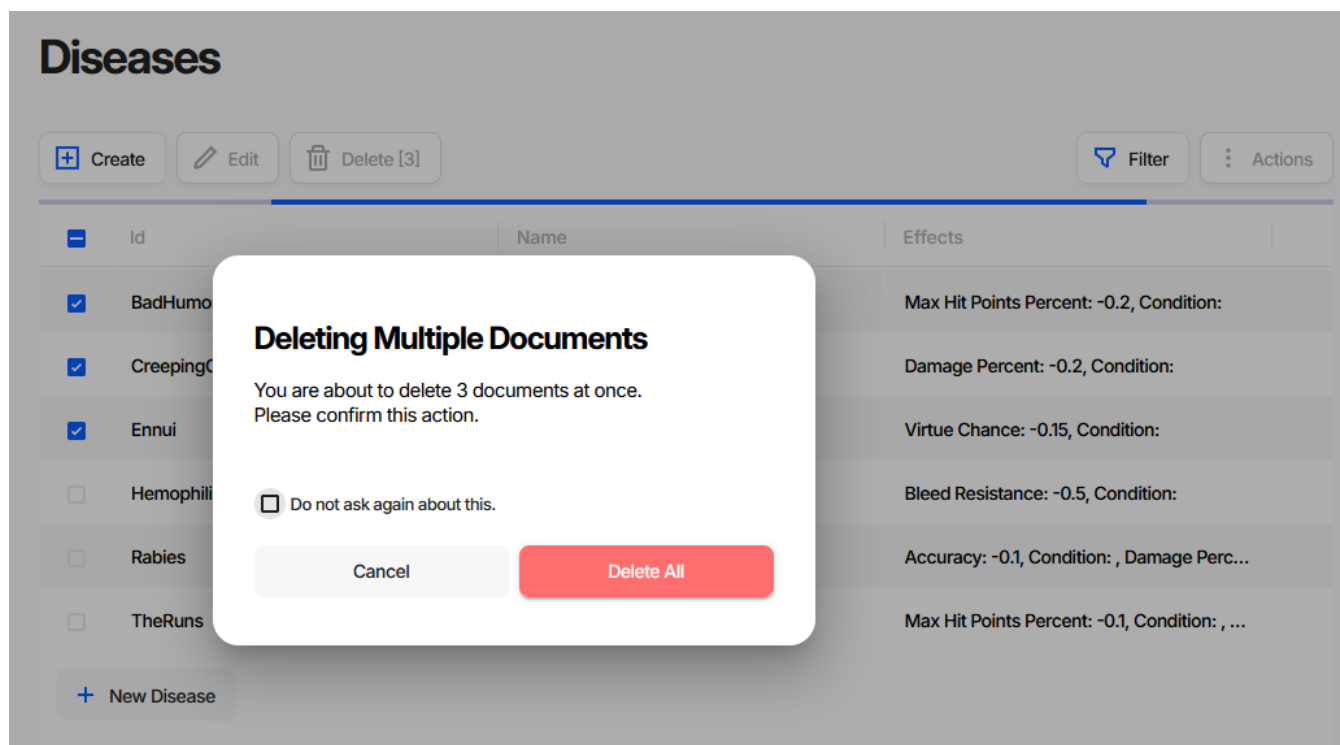
Changing What the Columns Are

Customize Columns... can only pick from properties the schema already has. To add a property, rename one, or change its data type, use Actions → Design: it opens the schema of the current collection in the schema editor. Saving there updates the grid.

See Creating Document Type (Schema) and Schema for the fields of that form.

Deleting Documents

Tick the rows and press Delete - the button carries the number of selected documents. Deleting more than one asks for confirmation first:



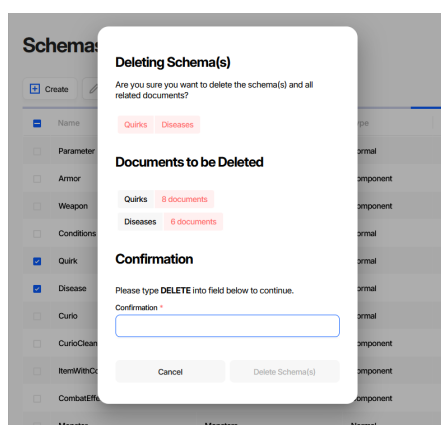
Do not ask again about this stores the answer in your personal preferences for the project, so later multi-document deletions run without the dialog.

Note

Deleting a document does not clean up references to it, and the deletion is not blocked by them. They stay behind as broken references, which validation reports - the quickest way to find everything that pointed at the removed document. Like any other invalid data, they are tolerated while you work and should be resolved before publishing.

Deleting a Schema

Deleting a schema is a different operation with a much wider blast radius, and it is done from the schema list (**Metadata** in the side menu), not from a collection page.



The dialog lists everything the deletion takes with it:

- **the schemas themselves**, as chips at the top;
- **Properties to be Deleted** - properties of *other* schemas that point at the ones being deleted. A reference, a reference collection, an embedded document, or a document collection of a deleted schema cannot outlive it, so the property is removed from that schema and its values from every document of it;
- **Documents to be Deleted** - how many documents each deleted schema takes down.

Warning

That cascade cannot be undone from the UI. The Delete Schema(s) button stays disabled until you type DELETE into the confirmation field.

Tip

Take a backup before deleting a schema that other schemas reference.

Bringing Data In From Files

Typing is one way to fill a collection. The other two are the import wizard and a plain drag and drop:

- Actions → Import... opens the wizard, which chooses between adding, replacing, and updating documents.
- Dragging a file onto the grid starts the same import for the current collection.
- Actions → Export To → Spreadsheet (.xlsx), followed by a drop of the edited file back onto the grid, makes a round trip through Excel or Google Sheets - still the fastest way to do bulk numeric balancing.

Structure requirements for imported files and the full walkthrough of both wizards are in Importing and Exporting Data.

Automating from the CLI

Every grid operation has a command line equivalent, which is what CI jobs and generator scripts use.

```
# list documents of a collection
dnx dotnet-charon -- DATA LIST \
  --dataBase "c:\my app\gamedata.json" \
  --schema Item

# create one document from a file
dnx dotnet-charon -- DATA CREATE \
  --dataBase "c:\my app\gamedata.json" \
  --schema Item \
  --input "c:\my app\item.json"

# update one document
dnx dotnet-charon -- DATA UPDATE \
  --dataBase "c:\my app\gamedata.json" \
  --schema Item \
  --input "c:\my app\item.json"
```

```
# delete one document
dnx dotnet-charon -- DATA DELETE \
  --dataBase "c:\my app\gamedata.json" \
  --schema Item \
  --id "AncestorsRelic"
```

For bulk changes use DATA IMPORT instead of a loop over DATA CREATE.

See also

- Basic Navigation and User Interface Overview
- Editing a Document
- Creating Document Type (Schema)
- Importing and Exporting Data
- Validation
- Publishing Game Data
- Generating Source Code

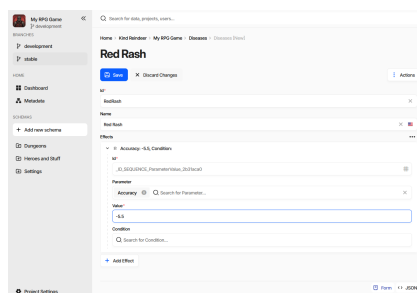
Editing a Document

The document form is the full editor for a single document. The grid on the collection page is faster for short values, but everything that does not fit a table cell - embedded collections, localized text, long descriptions - is edited here.

Opening the Form	90
Moving Between Documents	91
Fields	91
Reordering and Reusing Embedded Documents	91
Saving	91
Validation Errors	92
View Modes	92
JSON	93
Layout	93
Graph	94
The Actions Menu	94
When the Form Is Read-Only	95
See also	95

Opening the Form

From the collection page: Create for a new document, Edit for the selected one, or Ctrl + click on a row. Elsewhere in the editor, Ctrl + click on any reference value opens the form of the referenced document.



The heading is the document's rendered display text, not its Id, so it changes as you type. Save and Discard Changes sit above the fields, Actions on the right, and the view mode switch in the bottom right corner.

Moving Between Documents

The < and > buttons next to the heading open the previous and the next document without going back to the collection - Alt/■+← and Alt/■+→ do the same. They follow the order of the list the form was opened from, filters and sorting included, so reviewing a filtered set is a matter of pressing one key repeatedly.

They are disabled on the first and the last document, and when the form was reached in a way that has no position in a list - by following a reference, for instance.

Fields

Each field is the editor of the property's data type - a text box, a number field, a date picker, a reference search box, and so on. A red asterisk after the label marks a required property.

- **Localized text** carries a flag button that switches the language being edited. See Internationalization.
- **References** are search boxes; typing filters candidate documents by their display text.
- **Embedded documents and collections** appear as expansion panels with an Add button below them, so a whole subtree is edited in place without leaving the document.

Reordering and Reusing Embedded Documents

The panels of an embedded collection can be dragged by their header:

- **within the same collection** - the entries swap places, which is how the order of a collection is changed;
- **into another collection of the same schema** - anywhere on the form - the entry moves out of the first one and into the second;
- **holding** Ctrl - the entry is copied instead of moved. The copy is a clone with its identifiers cleared, so it becomes a new document rather than a second appearance of the same one.

Only collections of the same schema accept each other's entries, and nothing can be dragged while the form is read-only. This works on the collection fields of a form; the grid on the collection page is not a drop target.

Saving

Save writes the document and makes it visible to everyone else; Discard Changes reverts it to the last saved state. Both appear only when something has changed.

Action	Shortcut
Save	Ctrl/■+S
Save and return to the collection	Ctrl/■ + click on Save
Save ignoring validation errors	Shift + click on Save
Undo / Redo	Ctrl/■+Z / Ctrl/■+Y

Note

There is no three-way merge. If two people save the same document, the last save wins. In practice collisions are rarer than that sounds, because only the fields you actually changed are written - two people working on different properties of the same document do not overwrite each other.

Validation Errors

Fields that fail validation are outlined in red with the reason under them, and a counter appears in the bottom left corner. Clicking it lists every problem in the document with the field it belongs to.

Save

Discard Changes

Actions

Id*

Value can't be empty text.

Root Node

Search for Root Node...

A value is required.

Nodes

> Hello stranger!

> <EMPTY>

Add Node

Id Value can't be empty text.

Root Node A value is required.

2

Form

Graph

JSON

Save still works. When the document has errors the button becomes Save Anyway and carries a badge with their number.

Note

Broken data is allowed while you work on it. Empty required fields, references to documents that do not exist yet, formulas that do not parse - none of it blocks a save, here or in the CLI. Half-finished data is a normal state during development.

The tolerance ends at publication: generated code and the game-side loaders assume the data satisfies the schema. See Validation and the review step of the Publication wizard.

View Modes

The switch in the bottom right corner changes how the document is presented. The same modes are listed under Actions → View Mode. Two of the four are conditional - they appear only where they mean something.

Mode	Available
Form	Always.
JSON	Always.

Is It Free?

Layout	Only when the document being edited is a schema.
Graph	Only when an extension provides a custom editor for the schema.

JSON

The document as it is actually stored, in an editor with syntax highlighting and folding.

The screenshot shows a web application interface for editing a schema. On the left is a sidebar with a project name 'My RPG Game' and two branches: 'development' and 'stable'. Below this is a 'HOME' section with links to 'Dashboard' and 'Metadata'. The 'SCHEMAS' section includes a button to 'Add new schema' and a list of categories: 'Dungeons', 'Heroes and Stuff', and 'Settings'. The main area has a search bar and a breadcrumb trail: 'Home > Kind Reindeer > My RPG Game > Diseases > Diseases [New]'. The title 'Red Rash' is prominently displayed. Below the title are buttons for 'Save' and 'Discard Changes', and an 'Actions' menu. The JSON data is shown with syntax highlighting and line numbers (1-21). The JSON structure includes an 'Id', a 'Name' object with 'notes' and 'stale' properties, and an 'Effects' array containing an effect with an 'Id', 'Parameter' object, 'Value', and 'Condition'. At the bottom right, there are tabs for 'Form' and 'JSON'.

It is editable: changes made here go through the same save as the form. When the game data is read-only the JSON is shown as plain text instead. Useful for pasting a document in whole, for checking what a localized text or a reference really holds, and for reading the structure that import and export expect.

Layout

Layout mode is not a way to view data - it edits the *schema*, deciding how that schema's properties are arranged on the document form. It is therefore offered only while editing a schema, which the breadcrumb marks as [Design].

The screenshot shows the 'Layout' editor for a schema named 'Heroes'. The breadcrumb trail is 'Home > Kind Reindeer > My RPG Game > Heroes > Heroes [Design]'. The title 'Heroes' is at the top. Below it are buttons for 'Save', 'Discard Changes', and 'Reset Layout', along with an 'Actions' menu. The main area is a form with various input fields and controls. It includes a 'Name' field, a 'Bio' field, and several checkboxes for 'Religious', 'Dislike Heroes', and 'Provisions'. There are also numeric input fields for 'Provisions Count', 'Movement Forward', and 'Movement Backward'. A grid of resistance values is shown, including 'Trap Resistance', 'Debuff Resistance', 'Bleed Resistance', 'Move Resistance', 'Stun Resistance', 'Blight Resistance', 'Disease Resistance', and 'Death Blow Resistance'. At the bottom, there are buttons for 'Armors', 'Weapons', and 'Accuracy Modifier'. The bottom right corner has tabs for 'Form', 'Layout', and 'JSON'.

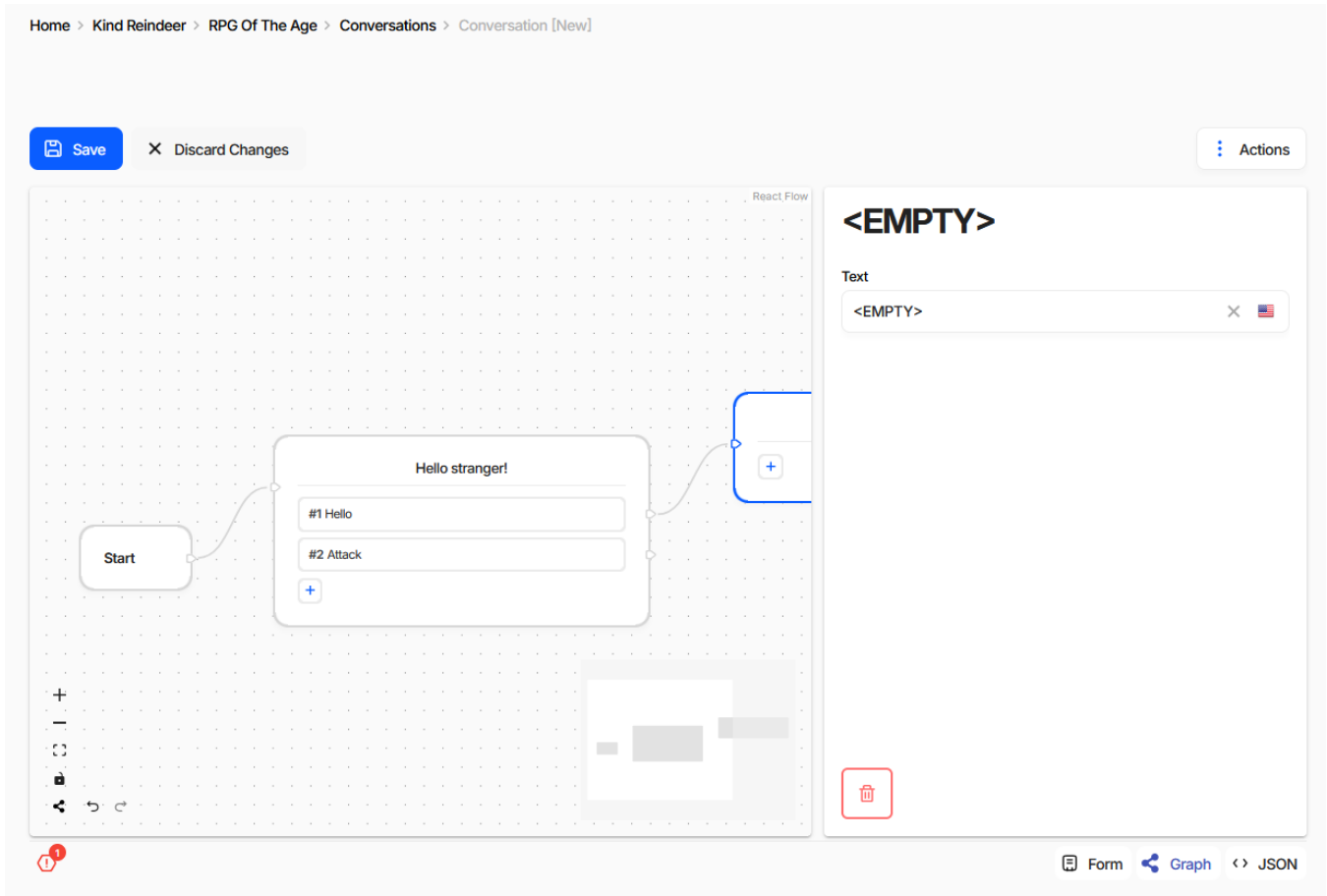
Is It Free?

Each property is a chip. Drag chips to reorder them and to group several onto one row. The area on the right takes an Asset property, which is then displayed beside the form instead of in it. Reset Layout returns the schema to the default one-property-per-row arrangement.

Every document of that schema uses the layout from then on, so it is worth arranging the schemas your team fills most often - related fields side by side beat scrolling through forty rows.

Graph

Graph mode belongs to extensions. When an extension registers a custom editor for a schema, that editor is shown here; without one the mode does not appear at all. Writing one is Implementing a Schema Editor.



The screenshot is a conversation editor extension: dialogue nodes on a canvas, links between them as edges, and the selected node's fields in the panel on the right. It edits the same document as the form - the nodes are the entries of an embedded document collection - so the two modes can be switched between freely, and the error counter keeps working in both.

The Actions Menu



Entry	What it does
Design	Opens the schema of this document in the schema editor. Hidden when the document already is a schema.
Create New	Opens an empty form for the same schema.
Clone	Opens a copy of the current document as a new one.
Translate Texts	Machine-translates the localized text of this document - everything, only what is missing, or only what is marked stale. Requires the machine translation feature, see Internationalization.
Delete	Deletes the document and returns to the collection.
Undo / Redo	Step through the changes made since the form was opened.
View Mode	The same modes as the switch in the corner.

Schema forms replace Design with Show advanced options, which reveals the rarely used fields of a schema and its properties.

When the Form Is Read-Only

The editing buttons are hidden and a banner explains why. There are three of them:

- **the game data is read-only** - insufficient permissions, a subscription plan limit, or a data source that is read-only to begin with;
- **you do not have permission to edit this document**, see Roles and Permissions;
- **the document is protected from modifications** and cannot be edited at all.

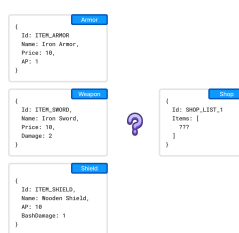
See also

- Filling Documents
- Creating Document Type (Schema)
- Data Types
- Internationalization
- Validation
- Publishing Game Data

Implementing Inheritance

Inheritance is a familiar tool for programmers when working with shared behavior or data. Unfortunately, it is not natively supported in Charon. However, you can achieve similar functionality using alternative approaches.

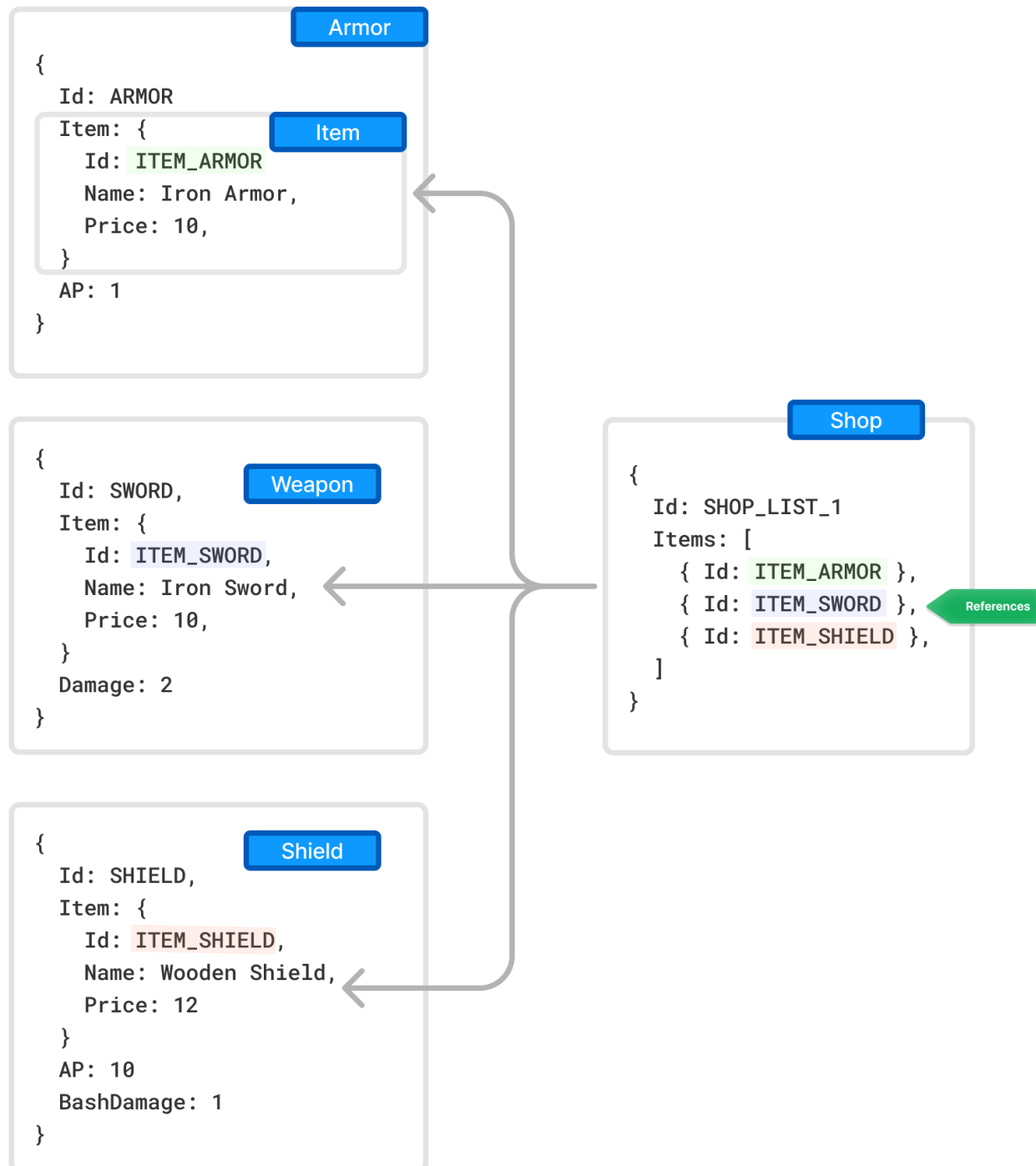
For example, imagine you have three different document types: Armor, Weapon, and Shield. You want to include all these items in a Shop list of sellable goods. In traditional inheritance, you could create a base type to unify these documents and refer to it.



Without inheritance, here are three possible approaches:

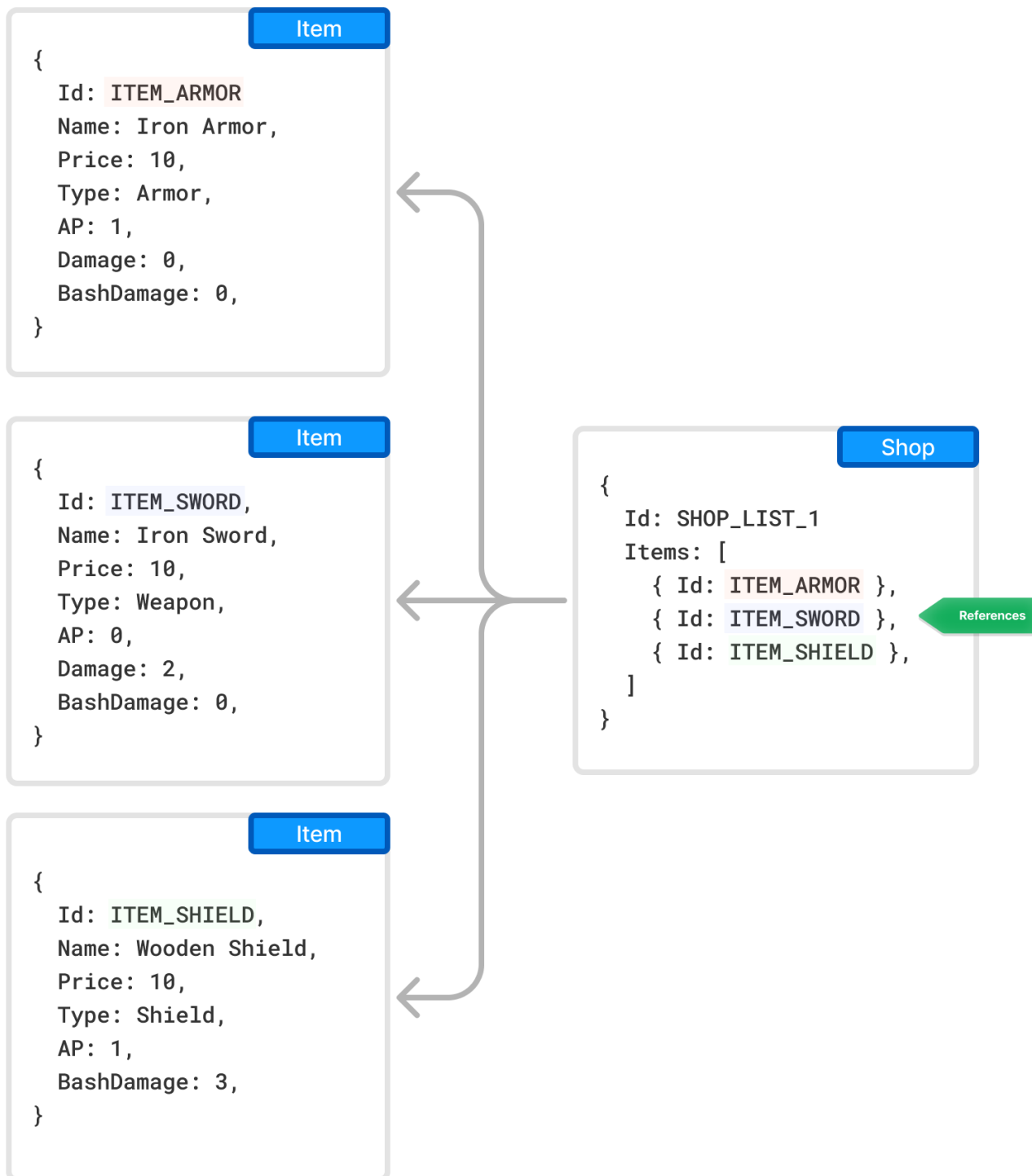
1. Composition

Extract the shared data from all sellable item types into a separate document type called `Item`. Each sellable document (e.g., `Armor`, `Weapon`, `Shield`) should include an embedded `Item` document containing store-relevant information. In the `Shop` list of sellable goods, you can store references to `Item` documents.



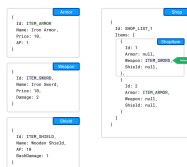
2. Merging

Alternatively, combine the three document types (`Armor`, `Weapon`, and `Shield`) into a single document type called `Item`. This document will contain fields for all three original types, along with an additional `Type` field to specify the item's category. The `Shop` list can then store references to these unified `Item` documents.



3. Aggregation

As a less elegant alternative, introduce a ShopItem type with fields referencing all possible document types (Armor, Weapon, and Shield). In each ShopItem document, only one of these fields will be filled, depending on the item's type. The Shop list can then reference ShopItem documents.



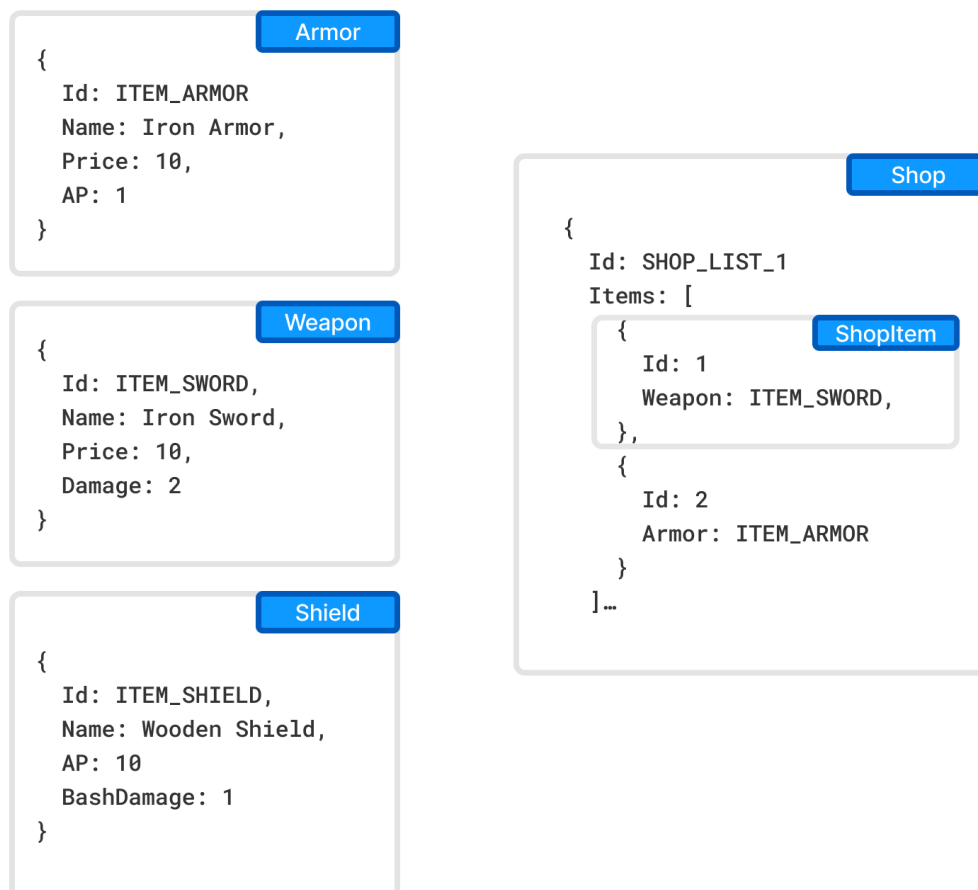
4. Tagged Union (Recommended)

A more powerful and storage-efficient alternative to Aggregation is to use a Tagged Union Schema (also known as a Discriminated Union, Sum Type, or \$oneOf). Tagged Unions allow documents of different shapes to coexist within the same collection or be embedded inside other documents, making them a natural fit when multiple heterogeneous document types must be handled together.

Union Schemas have several important distinctions from Aggregation:

- They provide first-class support in the editor UI, making them more convenient to create and maintain than other inheritance alternatives.
- Unassigned properties within a Tagged Union document are not stored and therefore take no space.

When used in this scenario, a ShopItem can be modeled as a Tagged Union that defines Armor, Weapon, and Shield as its possible variants.



Conclusion

Each of these methods has its trade-offs in terms of simplicity, flexibility, and performance. Choosing the right approach depends on your application's requirements and the expected complexity of your data model.

See also

- Creating Document Type (Schema)
- Filling Documents
- Publishing Game Data
- Generating Source Code

Schema

A **Schema** defines the structure of a specific type of game data entity. It acts as a blueprint that outlines the fields (properties) associated with the entity, much like columns in a database table or cells in a spreadsheet. Schemas are central to organizing and modeling structured game data.

Where Schemas Are Edited

A schema is a document like any other, edited in the same document form as game data. Open it with Actions → Design from a collection page, or from **Metadata** in the side menu; the breadcrumb marks the form as [Design].

Search for data, projects, users...

Home > Kind Reindeer > My RPG Game > Schemas > Combat Effect [Design]

Combat Effect

[Save](#) [Actions](#)

Name* **Display Name***

Type*

Display Text Template

Icon [Table Draw](#)

Description

Group **Menu Visibility**

Id Generator*

Properties

- ☒ **Id *** (auto-generated) ☐ Hide
- ☒ **Changes ***
- ☒ **Duration**
- ☒ **Duration Unit ***

[+ Add Property ...](#)

The screenshot has Actions → Show advanced options turned on, which reveals the rarely used fields. Without it the form is shorter.

Schema documents also have a Layout view mode, which arranges these properties on the document form that editors will use. See Editing a Document.

Name

The **Name** of a schema is used as an alternative identifier to the *Id* and is also used in generated source code as the name of the resulting class. Therefore, it must be a valid C-style identifier: it should start with a letter and include only Latin letters, digits, or underscores.

Schema names are also used as the name of the collection when storing or exporting data. It is recommended to use PascalCase and singular form for schema names.

Display Name

The **Display Name** is used in the UI and can contain any characters. There are no restrictions on alphabet or character set.

Type

The **Type** setting defines how instances (documents) of this schema are created and managed:

- **Normal:** Standard behavior. Documents can be created in the root collection, embedded in other documents, or may be absent entirely.
- **Component:** These documents can only be embedded within other documents. They do not appear in the main navigation menu.
- **Settings:** A singleton-style schema. Only one root-level document can exist, and it cannot be embedded in other documents.

Display Text Template

The Display Text Template defines how documents of this schema are presented in the UI when a textual representation is needed-e.g., in dropdowns, reference lists, or relationship views.

Icon

The **Icon** is displayed next to the schema name in the navigation menu or selection dialogs within the UI.

Description

The **Description** helps other users understand the purpose of the schema. It is shown in the UI and is also included as a documentation comment in the generated source code for the corresponding class.

Group

The **Group** defines the hierarchical path of the schema in the left-hand navigation menu. It behaves similarly to a folder path, with sections separated by the / character.

Menu Visibility

This flag determines whether the schema appears in the left-hand navigation menu in the UI.

Id Generator

The **Id Generator** setting defines how the Id Property is managed for this schema. If set to *None*, the Id property must be configured manually.

Properties

A list of Properties that define the fields associated with this schema. Each row expands into the property's own fields, Add Property . . . appends a new one, and the bin icon removes it.

An Id property managed by the schema's **Id Generator** is marked (*auto-generated*) and offers a **Hide** checkbox, which keeps the field out of the document form for editors who never need to see it. The checkbox writes Display=Hidden into the property's Specification.

See also

- [Display Text Template](#)
- [Property](#)
- [Id Property](#)
- [Shared Property](#)
- [Specification](#)
- [All Data Types](#)
- [Creating Document Type \(Schema\)](#)
- [Editing a Document](#)

Schema Types

Every Schema has a **Type** field that controls how its documents are created, stored, and navigated within the data model. Choosing the right type is fundamental to a well-structured data model.

Normal	101
Component	101
Settings	102
Union (Tagged Union)	102
Why Union instead of the other inheritance patterns	102
Defining a Union schema	103
Storage format	103
Generated source code	103
Malformed Union values	103
See also	104

Normal

Normal is the default type. Documents of a Normal schema can be:

- Created as standalone root-level entries, visible in the left navigation menu.
- Embedded inside other documents as Document or DocumentCollection property values.
- Referenced from other documents via Reference or ReferenceCollection properties.

Use **Normal** for any entity that has an independent identity and may need to be listed, searched, and referenced across the project - items, characters, quests, abilities, etc.

```
{ "Id": "Sword", "Name": "Iron Sword", "Damage": 15 }
```

Component

Component schemas model sub-objects that are *always embedded* inside another document and never exist as standalone root-level entries.

- Do **not** appear in the left navigation menu.
- Can only be used as a value for a Document or DocumentCollection property.
- Useful for reusable sub-structures with no independent identity.

Typical examples: a Stat block inside a Character, a DropEntry inside a LootTable, a BonusEffect inside a Weapon.

Warning

Documents of a **Component** schema are never exported or published as root-level collection entries - export and publication skip them unless they are embedded inside another document. If documents of a schema must live in the root collection *and* appear in exported data, the schema type should be **Normal**. Normal documents are exported in both cases: standalone and embedded.

```
{
  "Id": "Hero_Arbalest",
  "BaseStats": { "HP": 100, "Speed": 4, "Damage": 12 }
}
```

Settings

Settings is a singleton type. Exactly **one** document of this schema may exist in the root-level collection. It cannot be embedded inside other documents.

- Created automatically on first access and cannot be deleted.
- Appears in the navigation menu as a direct link (no list page).
- Suited for project-wide configuration: GameSettings, BalanceConfig, GlobalRules.

```
{ "Id": "settings", "DifficultyMultiplier": 1.5, "MaxPlayersPerRoom": 4 }
```

Union (Tagged Union)

A **Union** schema - also known as a Discriminated Union, Sum Type, or \$oneOf - lets a single collection slot or property hold documents of **different shapes** in a type-safe way.

A Union schema does not define properties itself. Instead it declares a set of **variant schemas** (any Normal or Component schema).

Why Union instead of the other inheritance patterns

See Implementing Inheritance for a full comparison, but in short:

Pattern	Storage cost	UI	Schema refactoring
Composition	Extra Item schema level	Standard form	Move fields to Item
Merging	All fields on every doc	All fields always visible	Edit one big schema
Aggregation	Nullable ref per variant	Multiple nullable refs	Add a new ref field
Union	Only active variant's fields	Type picker, sparse form	Add a new variant

Key properties of Union:

- **Sparse storage** - unset variant fields are not written to disk; only the active variant's properties are stored.
- **First-class UI** - when editing a Union field the editor shows a type-selector dropdown and only renders the form for the chosen variant.

Defining a Union schema

1. Create the variant schemas (Normal or Component) with their own properties.
2. Create a new schema, set **Type** to **Union**, and list the variant schemas in **Variants**.
3. Use the Union schema as the ReferenceType of a Reference, ReferenceCollection, Document, or DocumentCollection property on another schema.

Storage format

Unlike many serialisation libraries that use a "\$type" or "@class" string field to tag the variant, Charon uses **property-key presence** as the discriminator. Each union slot is serialised as a **single-key wrapper object** whose key is the variant schema name and whose value is the document body:

```
{ "Weapon": { "Id": "Sword01", "Damage": 15, "WeaponType": "Melee" } }
{ "Armor": { "Id": "Plate01", "Defense": 8 } }
{ "Shield": { "Id": "Shield01", "BlockChance": 0.25 } }, /* Weapon: null, "Armor": null
```

The variant is identified by whichever key is present in the wrapper - "Weapon", "Armor", "Shield", etc. There is no "\$type" string anywhere in the stored data; the schema name *is* the discriminator.

A full collection of union documents therefore looks like:

```
{
  "Collections": {
    "ShopItem": [
      { "Weapon": { "Id": "Sword01", "Damage": 15, "WeaponType": "Melee" } },
      { "Armor": { "Id": "Plate01", "Defense": 8 } },
      { "Shield": { "Id": "Shield01", "BlockChance": 0.25 } }
    ]
  }
}
```

Generated source code

Union schemas generate a container type in the target language. Each variant is a nullable field inside that container; exactly one field is non-null at runtime.

Malformed Union values

A Union value is **well-formed** when the wrapper object contains exactly two keys: Id and exactly one variant key. Two malformed states can occur - usually as a result of a bulk import, a hand-crafted JSON payload, or a patch applied to a Union field without accounting for the wrapper format.

Empty union - the Id is present but no variant key is set:

```
{ "Id": "item_01" }
```

Conflicting union - two or more variant keys are set simultaneously:

```
{ "Id": "item_01", "Weapon": { "Damage": 15 }, "Potion": { "HP": 5 } }
```

Both states are detected by the checkConstraints validation option (enabled by default) and reported as separate error codes in the validation report:

- Empty union → emptyUnion error
- Conflicting union → conflictingUnionOptions error, listing the conflicting variant names

Repairing malformed unions

The resolveConflictingUnions repair option fixes both cases automatically. It can be used with DATA VALIDATE or DATA IMPORT:

```
# repair in-place
dnx dotnet-charon -- DATA VALIDATE --dataBase "gamedata.json" \
```

```
--validationOptions repair resolveConflictingUnions
```

```
# repair while importing
```

```
dnx dotnet-charon -- DATA IMPORT --dataBase "gamedata.json" --input "./patch.json" \
--validationOptions repair resolveConflictingUnions
```

Repair behaviour:

Malformed state	What the repair does
Empty union	Erases the union value entirely (sets it to null).
Conflicting union	Keeps the last variant in schema-property order; all other variant values are discarded.

Note

The “last variant in schema-property order” is determined by the order of variant schemas as listed in the Union schema definition, not by the order of keys in the stored JSON. If the outcome of the automatic repair is not acceptable, correct the source data manually before importing.

Note

The Union type is the recommended approach when you need heterogeneous collections and want first-class editor support. Prefer it over Aggregation for new designs.

See also

- Schema
- Implementing Inheritance
- Specification
- Document
- Reference

Display Text Template

When a document needs to be shown or referenced in the UI, a text representation is required. By default, Charon will use one of the following properties, in order of precedence: `DisplayName`, `Name`, `Title`, or `Id`. This fallback behavior can often result in uninformative or unclear document labels.

For example, consider the following document:

```
{
  "Id": 1,
  "Description": "Bring 40 wolf skins",
  "Type": "RepeatableQuest",
  "Tasks": [ /* ... */ ]
}
```

By default, this document would be displayed in UI lists simply as "1". This can be improved by defining a custom **Display Name Template** for the Schema.

For instance, the template:

```
Quest #{Id}: {Type} - {Description}
```

would render the document's label as:

Quest #1: RepeatableQuest - Bring 40 wolf skins

Template Syntax

The display name template is a string that may include *replaceable expressions* enclosed in curly braces (`{}`). These expressions support a subset of C-style syntax and can reference any top-level or nested property of the document.

Property Access

- Top-level properties can be accessed directly by name.
- Nested properties in embedded documents can be accessed using dot notation, e.g., `Item.Name`.

Supported Data Types

Type	Example
String	"a string"
Number	1 or 3.14
Boolean	true or false
null	null

Unary Operators

Operator	Example
Logical NOT	<code>!CanWalk</code>
Bitwise Complement	<code>~ResistanceTypes</code>
Negation	<code>-Damage</code>

Binary Operators

The following binary expressions are supported:

Operator	Example
Subtract	<code>TotalPrice - BasePrice</code>
Add	<code>BaseDamage + BonusDamage</code>
Concatenate	<code>"Quest #" + Id</code>
Division	<code>CriticalChance / CurseDebuff</code>
Multiplication	<code>DropRate * 100.0</code>
Power	<code>Damage ** CriticalChance</code>
Modulo	<code>Slots % SlotPerPage</code>
Bitwise AND	<code>DamageFlags & AdditionalDamageFlags</code>
Bitwise OR	<code>DamageTypes ResistanceTypes</code>
Bitwise XOR	<code>ResistanceTypes ^ HealingTypes</code>
Bitwise Left Shift	<code>Flags << 1</code>
Bitwise Right Shift	<code>Flags >> 1</code>
Logical AND (short-circuit)	<code>CanWalk && CanRun</code>
Logical OR (short-circuit)	<code>HasHead CanBeHeadshoted</code>
Greater Than	<code>Count > MaxCount</code>

Greater Than or Equal	Count >= MaxCount
Less Than	HP < MaxHP
Less Than or Equal	HP <= MaxHP
Equal To	Type == "RepeatableQuest"
Not Equal To	Description != null
Coalesce	Nickname ?? Name

Ternary Operator

Ternary expressions are supported using the standard syntax:

```
{Damage >= 0 ? "Total Damage" : "Total Heal"}: {Damage}
```

This will result in either "Total Damage: X" or "Total Heal: X", depending on the value of Damage.

Operator Precedence

You can control expression evaluation order using parentheses. For example:

```
Max Damage: {BaseDamage * (BaseCriticalChance + BonusCriticalChance)}
```

Boolean Coercion

Many types can be implicitly coerced to a Boolean value:

- **Numeric values** are considered `true` if they are not equal to 0.
- **Documents, References, and Formulas** are considered `true` if they are not null.
- **Collections** (Document or Reference), **Text**, and **Localized Text** are `true` if they are not empty.

Other types must be explicitly compared to a value to be evaluated as Boolean. These coercion rules apply to ternary expressions and unary logical negation (!).

```
Has Items: {Items ? "Yes" : "No"}
```

Format Specifiers

Template expressions can include format specifiers, similar to C#'s formatting syntax. Add a colon and format string at the end of the substitution expression:

```
Resist Chance: {ResistChance * 100.0 :F2}
```

This limits the precision to two decimal places.

Supported format specifiers:

- **F**: Fixed-point (decimal precision), e.g., `F2` for two decimal places.
- **X / x**: Uppercase / lowercase hexadecimal (integers only), e.g., `X16` for hexadecimal string padded with zeroes to length of 16 characters.
- **B**: Binary representation (integers only), e.g., `B32` for binary string padded with zeroes to length of 32 characters.
- **L**: String length limitation, e.g., `L12` for a string shorter than 12 characters nothing will change, but if it is longer than 12 it will be truncated to 12 and an ellipsis character ... will be added.

Collection Accessors

Collections can be accessed by index using square brackets. If the index is out of bounds, the result is `null`.

```
First Item: {Items[0]}
```

The number of items in a collection can be accessed using the `.Count` property:

```
Item Count: {Items.Count}
```

To create a joined string from all items in a collection, use the `Join(separator: string)` method:

```
Items: {Items.Join(" and ")}
```

This would produce output like:

```
Items: Sword and Shield and Armor
```

String Manipulation Methods

String expressions support several methods:

- `Trim()` - Removes leading and trailing whitespace.
- `Substring(start: int, count: int)` - Extracts a substring from a given position.
- `ToString()` - Converts any non-null value to its string representation.

String concatenation is done using the `+` operator:

```
Damage: {Damage + " [" + DamageType + "]"} 
```

Result:

```
Damage: 100 [Fire]
```

See also

- Schema
- Property
- Id Property
- Shared Property
- All Data Types
- Creating Document Type (Schema)

Specification

The *Specification* field is a flexible extension point available on both Schema and Property documents. It enables developers to attach additional metadata in a standardized format that can be used for custom logic in UI field editors, source code generation, or other tooling integrations.

Overview

The *Specification* field stores a string formatted using the [application/x-www-form-urlencoded](#) standard (commonly referred to as [URLSearchParams](#) format). It contains key-value pairs separated by ampersands (&), where each key and value are URL-encoded.

For example:

```
icon=table&group=Combat
```

Use Cases

The *Specification* field is commonly used for:

- **Schema metadata:** Define an icon, group behavior, or UI preferences:
`icon=table&category=Gameplay&hideInMenu=true`
- **Property editor extensions:** Pass configuration values to custom editors:
`colorFormat=RGB&alpha=true`

- **Source code generation:** Customize how types or methods are generated:

typeName=MyEnum&csAttribute=Obsolete

Encoding Guide

All keys and values must be properly URL-encoded to ensure compatibility with parsing libraries and avoid format-breaking characters.

Encoding in JavaScript

In any browser console or Node.js REPL:

```
const params = new URLSearchParams({
  icon: "sword",
  group: "Items and Stuff",
  noThumbnail: true
});
console.log(params.toString());
// Output: icon=sword&group=Items%20and%20Stuff&noThumbnail=true
```

Encoding in Excel

To encode values manually using Excel formulas:

- Build a key-value pair:

=ENCODEURL("noThumbnail") & "=" & ENCODEURL("true")

- Combine multiple:

=ENCODEURL("icon") & "=" & ENCODEURL("sword") & "&" & ENCODEURL("group") & "=" & ENCODEURL("true")

Best Practices

- **Use lowercase or camelCase for keys** to ensure consistency.
- **Avoid spaces or special characters** in keys; values should always be encoded.

See also

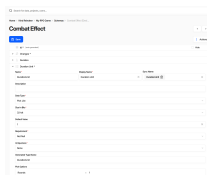
- Display Text Template
- Property
- Schema
- All Data Types
- Creating Document Type (Schema)

Schema Property

Properties define which fields a document will contain.

Where Properties Are Edited

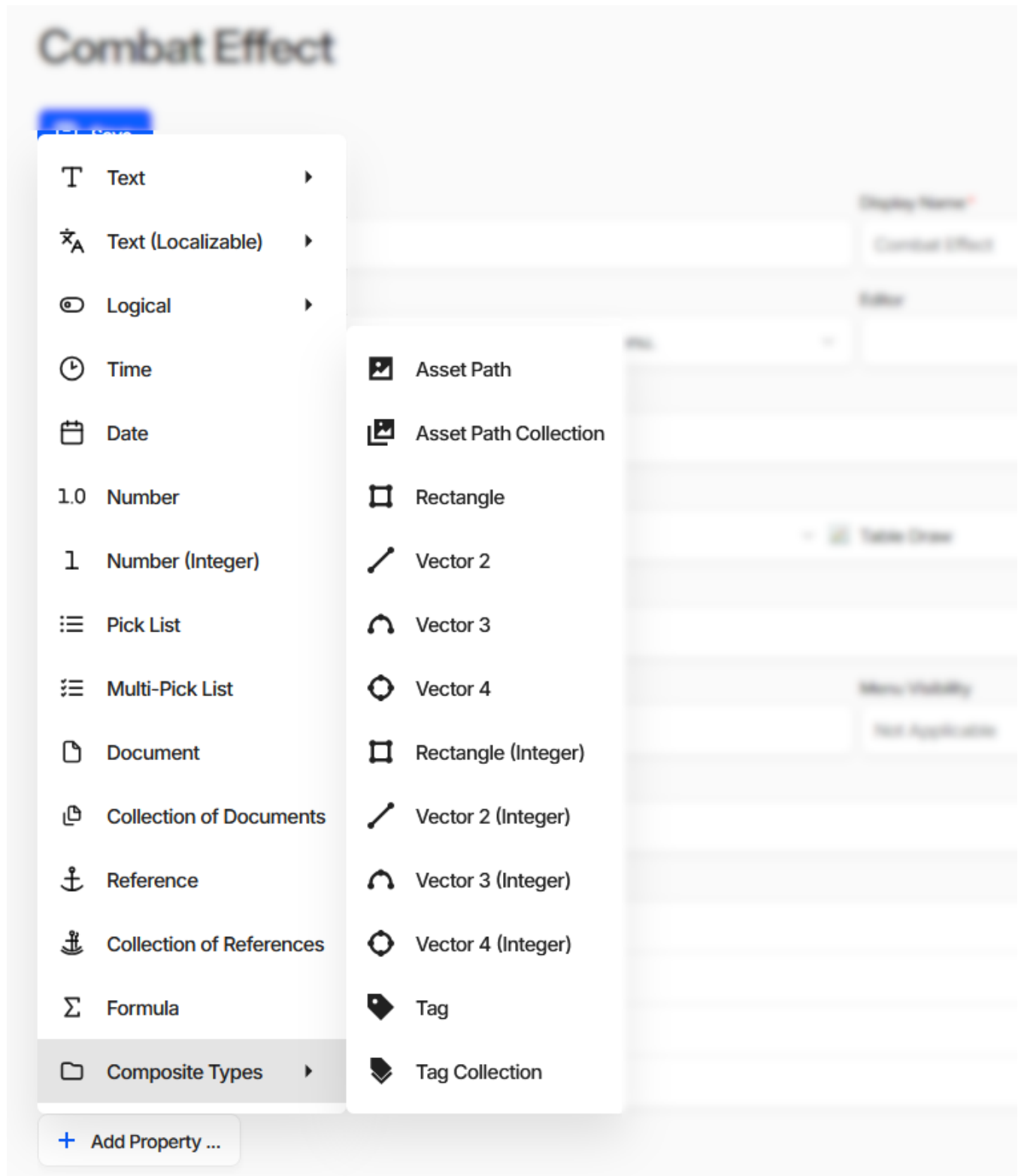
Properties live inside the schema form, as a list under **Properties**. Expanding a row opens the fields of that property.



The screenshot has Actions → Show advanced options turned on. Which fields are shown also depends on the **Data Type** - *Pick Options* below only appears for pick lists, *Size In Bits* only for the types that are stored as a number.

Adding a Property

Add Property . . . asks for the data type first:



The menu is the list of data types, plus a **Composite Types** submenu of ready-made shapes - vectors, rectangles, asset paths, and tags. Those are not extra data types: each one adds a property whose structure is already filled in for that shape.

Name

The **Name** of a property is used as an alternative identifier to the *Id* and is also used in generated source code as the name of the resulting field/property. Therefore, it must be a valid C-style identifier: it should start with a letter and include only Latin letters, digits, or underscores.

Property names are also used as the name of the fields when storing or exporting data. It is recommended to use PascalCase and singular form for property names.

Display Name

The **Display Name** is used in the UI and can contain any characters. There are no restrictions on alphabet or character set.

Description

The **Description** helps other users understand the purpose of the property. It is shown in the UI and is also included as a documentation comment in the generated source code for the corresponding field/property.

Sync Name

Some properties describe data types that are common across multiple schemas. For example, a `PickList` property might benefit from shared pick options used in several schemas. These synced properties are referred to as Shared Properties.

Data Type

Each property has a data type, which determines what kind of data can be entered into the document's field, how it is stored on disk, and how it is represented at runtime. The editor UI will attempt to validate input based on the selected data type but does not strictly enforce it. At runtime, game data loading code will validate the data types and may fail if the document is malformed.

Requirement

Specifies whether a value must be provided for the field. Possible values include:

- **None** – No requirement to fill in the property. It can be absent from the document both on disk and at runtime. Typically represented as a nullable or optional type in supported source code languages.
- **Not Null** – A value must be provided. For the Text data type, an empty string is considered a valid value.
- **Not Empty** – The field must contain a non-empty value for Text or have at least one item for Collection data types.

Uniqueness

Specifies uniqueness constraints for field values across documents. Useful to ensure certain values, such as names, are only used once.

- **Unique** – The value must be unique across all documents.
- **Unique In Collection** – The value must be unique within the scope of the current collection.

Default Value

Defines the default value to use for newly created documents or to populate existing documents when a new required property is added. The default must be a properly formatted value consistent with the specified data type.

Size In Bits

The in-memory size of the value, for the data types stored as a number - integers and pick lists. It decides the width of the type used in the generated source code, so narrowing it is a way to keep large collections small.

Generated Type Name

The name this property's type gets in the generated source code. Leave it empty and Charon picks the name itself. It matters for the types that become a type of their own - the enum behind a pick list, for instance.

Pick Options

The list of allowed values of a `Pick List` or `Multi-Pick List`, each a name paired with the number it is stored as. In generated code they become an enumeration or the nearest equivalent the target language has, so the names must be valid identifiers.

Specification

Additional parameters that do not have a field of their own, written as a URL query string and read by the editor UI and by the code generators. See [Specification](#).

Id Property

Each document includes a required `Id` property, which has specific constraints and must be present and valid.

See also

- [Schema](#)
- [Id Property](#)
- [Shared Property](#)
- [All Data Types](#)
- [Creating Document Type \(Schema\)](#)
- [Editing a Document](#)

Id Property

Each schema includes an `Id` property that defines the unique identifier for a document. This property is always named `Id` and must satisfy the following constraints:

- Must be unique.
- Required and must be non-empty (for `Text` data type).
- Must use a data type with a stable text representation and comparison support, such as:
 - `Time`
 - `Date`
 - `Number`
 - `Integer`
 - `Text`
 - `MultiPickList`
 - `PickList`

Generated Id

Schemas can specify that the `Id` property should be automatically generated. Based on the selected `Id Generator` type, an appropriate `Id` property will be added to the schema.

Supported `Id Generators`:

- `ObjectId`
- `Guid`

- Sequence
- GlobalSequence

Generated ObjectId

A BSON ObjectId is a 96-bit unique identifier derived from the machine name, process ID, and timestamp. It is always increasing and well-suited for sorting documents by creation time.

Id property details:

- **Data Type:** Text
- **Size:** 24 characters
- **Requirement:** Not Empty
- **Example:** "049bc0604c363a980b000088"

Generated Guid

A GUID (Globally Unique Identifier) is a 128-bit identifier typically represented as a 32-character hexadecimal string. It ensures global uniqueness across systems.

Id property details:

- **Data Type:** Text
- **Size:** 32 characters
- **Requirement:** Not Empty
- **Example:** "6fe4202b1b9c4565b439980138524112"

Generated Sequence

A sequential numeric identifier unique to each document within a specific schema. Documents from different schemas may share the same numeric Id. For globally unique numeric IDs, use GlobalSequence.

Id property details:

- **Data Type:** Integer
- **Size:** 32-bit
- **Requirement:** Not Null
- **Example:** 1

Generated Global Sequence

A globally unique numeric identifier across all schemas and documents in the project.

Id property details:

- **Data Type:** Integer
- **Size:** 32-bit
- **Requirement:** Not Null
- **Example:** 69

Non-Generated Id

Schemas may also use a manually defined Id property by selecting the None option for the Id Generator. You can define the Id property using any supported data type listed above.

All Id constraints will be automatically validated when saving the schema. You can change a schema's Id Generator to None at any time, but you cannot switch back to a generator once this change is made.

Id Placeholder

If you want to assign a temporary value to the ID of a document being created—for example, for internal reference—you can use a placeholder string such as `_ID_XXX_ANY_UNIQUE_TEXT_HERE_XXX`. This placeholder will be automatically replaced with the actual generated ID during saving or importing.

Before save:

```
{
  "Id": "_ID_ITEM_123",
  "Name": "My New Item",
  "ItemReference": {
    "Id": "_ID_ITEM_123", // self-reference
    "DisplayName": "My New Item"
  }
}
```

After save:

```
{
  "Id": 42,
  "Name": "My New Item",
  "ItemReference": {
    "Id": 42,
    "DisplayName": "My New Item"
  }
}
```

See also

- Schema
- Property
- Shared Property
- All Data Types
- Creating Document Type (Schema)

Shared Property

A **Shared Property** is a Schema property that syncs data type-related configuration with other properties of the same name across different schemas. When a shared property is modified in one schema, all other synced properties with the same shared name are updated accordingly.

Shared properties can be unlinked (or “decoupled”) at any time, converting them into independent, schema-specific properties.

Replicated Parameters

Shared properties replicate a defined subset of configuration parameters between themselves:

- Data Type
- Default Value
- Uniqueness
- Requirement
- Reference Type (for *Document*, *DocumentCollection*, *Reference*, and *ReferenceCollection* types)
- Size
- Specifications for *PickList*, *MultiPickList*, and *Formula* data types

These parameters ensure consistency across schemas that use the same shared property.

Impact on Generated Source Code

In cases where a property results in a distinct type in the generated source code-such as a *PickList* producing an enum-the name of the shared property is used to derive the generated type name.

For example:

- A shared *PickList* property named *Damage* Type would generate:

- *DamageType* enum in C#, Haxe, TypeScript
- *EDamageType* enum in Unreal Engine C++

To customize this behavior, the *Generated Type Name* field of the property can be set manually to override the default naming convention.

See also

- Schema
- Property
- Id Property
- Shared Property
- All Data Types
- Creating Document Type (Schema)

Data Types

Date

The *Date* data type is used to store dates in [ISO 8601](#) format, which includes the year, month, day, and time with [UTC](#) time zone. This data type is particularly useful for storing information about events that occur on specific dates or for tracking the age of entities. Since dates are stored with [UTC](#) time zone, the data can be consistently interpreted across different time zones.

Source Code Type

Language	Type
C#	System.DateTime
TypeScript	Date
C++ (Unreal Engine)	FDateTime
Haxe	Date

Uniqueness

Date cannot be marked as unique.

Format

yyyy-MM-ddTHH:mm:ss.fffZ

Example

```
"2017-12-27T00:00:00.000Z"
```

```
// Avoid dates before the Unix epoch for cross-platform compatibility:
// some runtimes and serialization libraries do not handle pre-1970 dates correctly.
"1970-01-01T00:00:00.000Z"
```

See also

- Data Types
- Time
- Text

Document

The Document data type in game data schema is used to represent complex structures. A document can contain multiple properties of different data types, including other documents or document collections, allowing for hierarchical data modeling. It is important to note that the lifetime of sub-documents is tied to the lifetime of the parent document, meaning that any changes (e.g. deletion) to the parent document will affect all of its sub-documents.

Source Code Type

Language	Type
C#	class
TypeScript	class
C++ (Unreal Engine)	UCLASS
Haxe	class

Uniqueness

Document cannot be marked as unique.

Example

For example, in a **Dialog**, each node can be a Document with dialog text, response options, and actions that occur after a response is chosen. Each response option can be a sub-document that is another **Dialog** node.

```
{
  "Text": "Welcome to the game. What's your name?",
  "Options": [
    {
      "Text": "My name is John.",
      "Options": [
        {
          "Text": "Hello John! What brings you here?",
          "Options": [
            {
              "Text": "I'm looking for adventure.",
              "Action": "dialog.GiveQuestAndEnd()"
            },
            {
              "Text": "I'm on a mission.",
              "Options": [/* ... */]
            }
          ]
        }
      ]
    }
  ],
  {
    "Text": "I prefer not to say.",
    "Action": "dialog.End()"
  }
]
```

See also

- Data Types
- Document Collection
- Reference

Document Collection

The DocumentCollection data type is used to store an array of sub-documents, which are used to represent complex structures. It is important to note that the lifetime of sub-documents is tied to the lifetime of the parent document, meaning that any changes (e.g. deletion) to the parent document will affect all of its sub-documents.

Source Code Type

Language	Type
C#	ReadOnlyCollection{T}
TypeScript	ReadOnlyArray{T}
C++ (Unreal Engine)	TArray{T}
Haxe	ReadOnlyArray{T}

Uniqueness

Document Collection cannot be marked as unique.

Size

May be limited in number of items. 0 - no limit.

Example

One example use case for DocumentCollection is storing a list of items in a game, such as a chest and its contents. Each item in the chest could be represented by a sub-document containing information such as reference to an item and its quantity.

```
{
  "Name": "Silver Chest",
  "Loot": [
    {
      "Item": { "Id": "Sword" },
      "Quantity": 1
    },
    {
      "Item": { "Id": "Silver" },
      "Quantity": 100
    }
  ]
}
```

See also

- Data Types
- Document
- Reference Collection

Formula

The **Formula** data type allows storing and evaluating C# expressions within game data. It enables developers to define logic that performs calculations based on dynamic inputs and parameters not known until runtime. A formula can be any valid C# expression that returns a value of a supported data type.

Formulas are especially useful in scenarios involving complex calculations, such as determining weapon damage based on factors like target resistance and attack type. By encapsulating such logic in formulas, developers can fine-tune gameplay mechanics without modifying or recompiling the source code.

At runtime, formulas are evaluated using arguments passed to them. These arguments can reference other data properties or objects, providing a high level of flexibility in defining game rules and balancing.

For example, a damage formula might look like this:

```
(weaponPower * (1.0 - targetResistance)) * attackMultiplier
```

This expression multiplies weapon power by a factor based on the target's resistance and then scales the result by an attack multiplier. The final result is the computed damage.

Source Code Type

Language	Type
C#	class
TypeScript	class
C++ (Unreal Engine)	UCLASS
Haxe	class

Uniqueness

Formulas cannot be marked as unique.

Specification

Formulas support the following specification parameters: - param.<name> - Defines a formula parameter and its type.

Example: param.id=System.Int32

- resultType - Specifies the return type of the formula.

Example: resultType=System.String

- knownType - Declares types known to the formula. Only static members of known types can be accessed.

Example: knownType=System.Math

- typeName - Custom type name for the generated class. If omitted, a name is derived from the containing schema and property.

Example: typeName=MyDamageCalculatingFormula

Example

```
"target.HP < 100"  
"x != 0"  
"target.DoDamage(100)"  
"(weapon.Damage / target.DamageResistance) / 2"
```

See also

- Data Types
- Formula Expression AST

Integer

The Integer data type is a whole number data type that is limited to 64 bits. It is used to represent integers without a fractional component. It can be used in cases where you need to store a large range of positive or negative whole numbers, such as in-game currency or player levels.

Unlike the Number data type, integers do not have any precision caveats since they do not store decimal values. Therefore, they are suitable for calculations that require exact values.

Source Code Type

Language	Type
C#	System.Int32 or System.Int64
TypeScript	number or bigint
C++ (Unreal Engine)	int32 or int64
Haxe	Int or Int64

Uniqueness

Integers can be marked as unique.

Size

32 or 64bit

Example

```
0
-1
100
```

See also

- Data Types
- Number
- Pick List

Localized Text

The LocalizedText data type is used to store text that needs to be displayed in multiple languages. Unlike the Text data type, the LocalizedText data type allows the storage of the same text in multiple languages. It supports the whole range of UTF symbols, just like the Text data type. The LocalizedText data type is essential for games that require localization support, and it makes it easy for game developers to manage text that needs to be displayed in multiple languages.

Source Code Type

Language	Type
C#	LocalizedText or System.String
TypeScript	LocalizedText or string
C++ (Unreal Engine)	FLocalizedText or FText
Haxe	LocalizedText or String

Uniqueness

Localized Text cannot be marked as unique.

Size

May be limited in number of characters. 0 - no limit.

Example

```
{"en-US": "Hello", "fr-FR": "Bonjour"}
```

See also

- Data Types

- Text
- Internationalization

Logical

The `Logical` data type is used to represent boolean values - values that can be either `true` or `false`. It is suitable for modeling simple binary states, conditions, or flags commonly used throughout game data.

Use Cases

The `Logical` type is ideal for:

- Representing **simple state**: Examples include *IsActive*, *IsPublished*, *IsLatest*, *IsReligious*, or *IsOptional*.
- Indicating **ownership or structural conditions**: Examples include *HasPartner*, *HasSecondPhase*.
- Controlling **behavioral expectations**: Examples include *ShouldFinishTutorial*, *ShouldGoFirst*.
- Defining **capabilities or permissions**: Examples include *CanBlock*, *CanRun*, *CanBeSold*.

This makes the `Logical` type a versatile and lightweight choice for many decision-driven properties in schema design.

Comparison with MultiPickList

While the `MultiPickList` data type can represent combinations of multiple states, the `Logical` type is recommended when the property has exactly two possible values. Using `Logical` ensures simplicity, clarity, and optimal editor behavior for binary flags.

Source Code Type

Language	Type
C#	<code>System.Boolean</code>
TypeScript	<code>boolean</code>
C++ (Unreal Engine)	<code>bool</code>
Haxe	<code>Bool</code>

Uniqueness

`Logical` can be marked as unique.

Example

```
true    // canonical JSON boolean
false

// alternative accepted input formats during import:
True    // case-insensitive spelling
False
1       // non-zero integer = true
0       // zero = false
"Yes"   // accepted string aliases
"No"
```

See also

- Data Types
- Integer
- Pick List

Multi-Pick List

The `MultiPickList` data type allows a property to represent multiple values selected from a predefined list of options. It is designed for cases where multiple attributes, traits, or capabilities can be applied to a single entity simultaneously. Unlike `PickList`, which limits the property to a single choice, `MultiPickList` enables the combination of several boolean-like flags into a single compact representation. In the UI, `MultiPickList` is typically presented as a dropdown menu, allowing users to select or deselect multiple options independently.

Storage on Disk

On disk, the `MultiPickList` is stored as an integer bitmask. Each option is assigned a specific bit position. When multiple options are selected, their bits are combined using bitwise OR operations. The number of options is limited by the bit width of the storage format—commonly 32 or 64 options for a 32-bit or 64-bit integer, respectively.

Example

An item in an RPG might be marked as equippable, sellable, and destructible using a single `MultiPickList` field instead of three separate boolean flags.

Source Code Type

Language	Type
C#	[Flags] enum
TypeScript	enum
C++ (Unreal Engine)	UENUM(meta = (Bitflags, UseEnumValuesAsMaskValuesInEditor = "true"))
Haxe	abstract

Uniqueness

Multi-Pick Lists can be marked as unique.

Size

32 or 64bit

Specification

Multi-Pick Lists support the following specification parameters:

- `typeName` - Custom type name for the generated class. If omitted, a name is derived from the containing schema and property.

Example: `typeName=MyEnum`

Example

```
// Single value selected:
1 // stored as integer (bit position 0 = first option)
"Apple" // string name also accepted as input

// Multiple values selected (bitwise OR of individual bits):
3 // Apple(1) | Banana(2) - both selected
"Apple, Banana" // comma-separated string names also accepted
```

See also

- Data Types
- Pick List
- Reference Collection

Number

The Number data type is used to represent decimal numbers. It conforms to the [IEEE 754](#) floating-point standard and can represent both positive and negative numbers, as well as zero. However, due to the limitations of the floating-point representation, precision may be lost when performing certain arithmetic operations. Therefore, it is recommended to use the Integer data type for financial calculations and other scenarios that require high precision.

Some use cases for the Number data type include representing quantities, such as the count of items or the amount of gold reward in the chest, or representing percentages, such as the chance of an event occurring. It can also be used to represent measurements, such as the height of a character or the length of a weapon.

When working with Numbers in game data, it is important to ensure that the precision is appropriate for the use case. Additionally, it may be necessary to round numbers to a certain number of decimal places to avoid displaying unnecessarily precise values to players.

Source Code Type

Language	Type
C#	System.Single or System.Double
TypeScript	number
C++ (Unreal Engine)	float or double
Haxe	Float

Uniqueness

Numbers can be marked as unique.

Size

32 or 64bit

Specification

Numbers support the following specification parameters:

- **precision** - Specify decimal precision of stored number.
Example: `precision=2`
- **displayPrecision** - Specify decimal precision of displayed in UI number.
Example: `displayPrecision=2`

Example

3.14
0.21
-3.14

See also

- Data Types
- Integer
- Formula

Pick List

The **PickList** data type defines a property that can take one value from a predefined set of options. It is best suited for fields where only a single choice is valid at any given time. This ensures consistency and restricts inputs to known, valid values. This type is commonly used for categorical selections such as character class, item rarity, or region. By standardizing the input, **PickList** helps reduce errors and supports better validation and filtering within game systems. In user interfaces, a **PickList** is typically presented as a dropdown menu, allowing users to choose only one option from the list.

Storage on Disk

On disk, `PickList` values are stored as integers. Each integer corresponds directly to one of the predefined options.

Example

In a character creator, the player might choose “Mage” as their class from a list of available options like Warrior, Mage, or Rogue.

Source Code Type

Language	Type
C#	enum
TypeScript	enum
C++ (Unreal Engine)	UENUM
Haxe	abstract

Uniqueness

Pick Lists can be marked as unique.

Size

32 or 64bit

Specification

Pick Lists support the following specification parameters:

- `typeName` - Custom type name for the generated class. If omitted, a name is derived from the containing schema and property.

Example: `typeName=MyEnum`

Example

```
1 // on-disk stored as integers
  "Apple" // string values also valid
```

See also

- Data Types
- Multi Pick List
- Integer

Reference

The Reference data type enables the creation of non-embedded relationships between documents. A reference acts as a pointer to another document by using that document's Id as a key. This approach facilitates linking between related documents without the need to duplicate or embed data.

When using a Reference, the target documents are not stored within the parent document but are instead referenced externally. This is particularly beneficial when working with large or complex data sets, where referencing is more efficient and maintains separation of concerns. It also promotes data integrity by ensuring that only valid document references are maintained.

For example, in a game scenario, a **Chest** with a loot table might hold a reference to an **Item** document, rather than embedding the entire **Item** directly within the **Chest**. This separation allows independent management of the loot logic and item definitions while preserving their relationships.

Source Code Type

Language	Type
C#	<code>DocumentReference{T}</code> or <code>T</code>
TypeScript	<code>DocumentReference{T}</code> or <code>T</code>

C++ (Unreal Engine)	FGameDataDocumentReference
Haxe	DocumentReference{T} or T

Uniqueness

Reference cannot be marked as unique.

Specification

References support the following specification parameter:

- `displayTextTemplate` - Defines a template string for how the referenced value is displayed in the UI.
Example: `displayTextTemplate=Item%3A+%7BName%7D%2C+Count%3A+%7BCount%7D` (renders as: *Item: {Name}, Count: {Count}*)
- `localOnly` - Limits the selection to documents defined within the current document. References from other documents will be excluded from the drop-down list.
Example: `localOnly=true`
- `pathFilter` - Filters available documents based on their path (RFC 6901 JSON Pointer). Use `*` to include all paths, or an empty string `""` to include only root-level documents.
Example: `pathFilter=%2FItem` (documents under `/Item`) `pathFilter=*` (all documents)
`pathFilter=` (only root documents)

Example

```
{ "Id": "Sword" }
"Sword" // just raw Id is also accepted
```

See also

- Data Types
- Document
- Reference Collection

Reference Collection

The `ReferenceCollection` data type is used to create non-embedded, one-to-many relationships between documents. It allows a document to reference multiple documents of the same type without embedding them directly.

When using a `ReferenceCollection`, the referenced documents are stored as references to their original locations, rather than being embedded in the parent document. This approach is particularly useful when dealing with large or complex datasets where referencing is more efficient and scalable than embedding. It also helps maintain data integrity by ensuring that all references point to valid documents.

For example, in a game, each **Quest** may reference a collection of **Objectives**. These objectives can be defined separately for reuse across multiple quests. The `ReferenceCollection` data type enables the quest document to maintain a list of references to objective documents.

Source Code Type

Language	Type
C#	<code>ReadOnlyCollection{T}</code>
TypeScript	<code>ReadOnlyArray{T}</code>
C++ (Unreal Engine)	<code>TArray{T}</code>
Haxe	<code>ReadOnlyArray{T}</code>

Uniqueness

Reference Collection cannot be marked as unique.

Size

May be limited in number of items. 0 - no limit.

Specification

References support the following specification parameter:

- `displayTextTemplate` - Defines a template string for how the referenced value is displayed in the UI.
Example: `displayTextTemplate=Item%3A+%7BName%7D%2C+Count%3A+%7BCount%7D` (renders as: *Item: {Name}, Count: {Count}*)
- `localOnly` - Limits the selection to documents defined within the current document. References from other documents will be excluded from the drop-down list.
Example: `localOnly=true`
- `pathFilter` - Filters available documents based on their path (RFC 6901 JSON Pointer). Use `*` to include all paths, or an empty string `""` to include only root-level documents.
Example: `pathFilter=%2FItem` (documents under */Item*) `pathFilter=*` (all documents)
`pathFilter=` (only root documents)

Example

```
[{ "Id": "Sword" }, { "Id": "Gold" }]
["Sword", "Gold"] // just raw Ids are also accepted
```

See also

- Data Types
- Reference
- Document Collection

Text

The Text data type is used to store simple text values in game data. Unlike the LocalizedText data type, Text does not have support for multiple translations of the same text. Instead, it allows for the storage of any UTF symbol in a single language. This data type is useful for fields that do not require localization, such as ids, system names, tags or asset paths.

Source Code Type

Language	Type
C#	System.String
TypeScript	string
C++ (Unreal Engine)	FString
Haxe	String

Uniqueness

Text can be marked as unique (case sensitive for uniqueness purposes).

Size

May be limited in number of characters. 0 - no limit.

Example

```
"Hello world!"
```

Sub-Types

- Asset Path
- Vector 2/3/4
- Integer Vector 2/3/4
- Rectangle
- Integer Rectangle
- Tags

See also

- Data Types
- Localized Text

Time

The `Time` data type in game data is equivalent to the `TimeSpan` data type in C#. It is used to store a duration or a time interval, such as the time it takes to complete a task or the length of a cutscene in a game. The `Time` data type is represented as a string in the format `[DD.]HH:mm:ss`, where `DD` is an optional number of days, `HH` is the number of hours, `mm` is the number of minutes, and `ss` is the number of seconds. It can also be stored as a plain integer representing the total number of seconds. Negative durations are supported.

For example, if a task takes 2 hours and 30 minutes to complete, the `Time` data type value would be `02:30:00`.

Source Code Type

Language	Type
C#	<code>System.TimeSpan</code>
TypeScript	<code>TimeSpan</code> (Bundled)
C++ (Unreal Engine)	<code>FTimespan</code>
Haxe	<code>TimeSpan</code> (Bundled)

Uniqueness

Time cannot be marked as unique.

Format

`[DD.]HH:mm:ss` or `<number-of-seconds>`

Example

```
"02:30:00" // 2 hours and 30 minutes
"1.00:00:00" // 1 day
60 // 60 seconds
120 // two minutes

"-00:30:00" // could be negative
```

See also

- Data Types
- Date
- Integer

Asset Path

The `Asset Path` is a sub-type of the `Text` data type and is used to store a path to a game asset within game data.

The `Asset Path Collection` is a collection variant of this type and it is used to store multiple paths. Each path is separated by the `|` (pipe) character.

Paths are typically relative to the project root and can be used to load assets directly at runtime.

Specification

Asset Path Collection supports the following specification parameters:

- `assetType` - Filters the types of assets to select. For Unity and Unreal Engine, this refers to class names. For standalone builds, it refers to file extensions. Example: `assetType=Texture` or `assetType=Texture2D`

UI Behavior

This field is represented in the editor by a multi-select searchable input. Users can type to search for assets and add multiple items to the collection using auto-complete.

The search input supports the `t:<TypeName>` filter pattern to limit search results by asset type.

Examples:

- Unity: `swing t:AudioClip`
- Unreal Engine: `swing t:SoundWave`
- Standalone: `swing t:wav`

Validation Behavior

The Charon editor does not validate asset paths during game data loading (i.e., at runtime). You are responsible for ensuring the existence of assets at the provided paths. To improve reliability, consider writing a custom editor extension that validates all asset references before packaging the game.

Asset Localization

Localizable asset paths are not currently supported. If this is a required feature, you are encouraged to suggest it in Charon's [Discord](#) channel.

Path Origin

The meaning of "relative path" for asset references depends on how the game data editor is launched:

- **Unity Editor:** Paths are relative to the Unity project root directory, typically the directory containing the `Assets` folder. Asset paths will usually start with `Assets/`. Example: `Assets/Textures/MyTexture.png`
- **Unreal Engine Editor:** Paths are relative to the Unreal project root directory, usually containing the `Content` folder. Asset paths often start with `/Game/` because it is Unreal Engine package path, not physical path. Example: `/Game/Textures/MyTexture`
- **Standalone Execution** (via CLI): Paths are relative to the game data file path by default. This behavior can be customized using either:

- The `dotnet tool charon ./gamedata.gdjs` CLI argument:
`dotnet tool charon ./gamedata.gdjs --gameAssetsPath "C:/Projects/MyGame/"`
- The `STANDALONE__GAMEASSETSPATH` environment variable.

Example

```
"Assets/Textures/MyTexture.png" // Unity
"/Game/Textures/MyTexture"      // Unreal Engine
"Textures/MyTexture.png"        // Standalone
```

See also

- Data Types

- Text

Rectangle

The `Rectangle` is a sub-type of the `Text` data type used to store four decimal values representing a rectangle. The components represent X, Y, Width, and Height, in that order. Values are separated by spaces and must be parsed from the string before use at runtime.

UI Behavior

This field is represented in the editor as a group of input fields labeled X, Y, Width, and Height. Each component accepts decimal numbers.

Validation Behavior

The editor ensures that a valid rectangle value is entered and only stores properly formatted values. However, there is no runtime validation or built-in parsing logic, so values must be parsed manually in game code.

Example

```
"50.0 100.0 200.5 25.0"
```

Parsing

Unity (C#) example:

```
var uiElement = gameData.AllUIElements.Get("HealthBar"); // -> UIElement
var boundsString = uiElement.Bounds; // -> "50.0 100.0 200.5 25.0"
var components = boundsString.Split(' ');
var bounds = new Rect(
    float.Parse(components[0]), // X
    float.Parse(components[1]), // Y
    float.Parse(components[2]), // Width
    float.Parse(components[3]) // Height
);
```

Unreal Engine (C++) example:

```
auto UIElement = GameData->AllUIElements->Find(TEXT("HealthBar")); // -> UIElement*
FString BoundsString = UIElement->Bounds; // -> "50.0 100.0 200.5 25.0"
TArray<FString> Components;
BoundsString.ParseIntoArray(Components, TEXT(" "));
FSlateRect Bounds(
    FString::Atof(*Components[0]), // X
    FString::Atof(*Components[1]), // Y
    FString::Atof(*Components[2]), // Width
    FString::Atof(*Components[3]) // Height
);
```

TypeScript example:

```
let uiElement = gameData.uiElementsAll.find("HealthBar"); // -> UIElement
let boundsString = uiElement.Bounds; // -> "50.0 100.0 200.5 25.0"
let parts = boundsString.split(" ").map(x => parseFloat(x));
let bounds = { x: parts[0], y: parts[1], width: parts[2], height: parts[3] };
```

Haxe example:

```
var uiElement = gameData.uiElementsAll.get("HealthBar"); // -> UIElement
var boundsString = uiElement.Bounds; // -> "50.0 100.0 200.5 25.0"
var parts = boundsString.split(" ").map(Std.parseFloat);
var bounds = { x: parts[0], y: parts[1], width: parts[2], height: parts[3] };
```

See also

- Data Types
- Rectangle Int
- Vector

Integer Rectangle

The `Integer Rectangle` (`RectangleInt`) is a sub-type of the `Text` data type used to store four integer values representing a rectangle. The components represent X, Y, Width, and Height, in that order. Values are separated by spaces and must be parsed from the string before use at runtime.

UI Behavior

This field is represented in the editor as a group of input fields labeled X, Y, Width, and Height. Each component accepts only whole numbers (integers).

Validation Behavior

The editor ensures that a valid integer rectangle is entered and only stores properly formatted values. However, there is no runtime validation or built-in parsing logic, so values must be parsed manually in game code.

Example

```
"50 100 200 25"
```

Parsing**Unity (C#) example:**

```
var uiElement = gameData.AllUIElements.Get("HealthBar"); // -> UIElement
var boundsString = uiElement.Bounds; // -> "50 100 200 25"
var components = boundsString.Split(' ');
var bounds = new RectInt(
    int.Parse(components[0]), // X
    int.Parse(components[1]), // Y
    int.Parse(components[2]), // Width
    int.Parse(components[3]) // Height
);
```

Unreal Engine (C++) example:

```
auto UIElement = GameData->AllUIElements->Find(TEXT("HealthBar")); // -> UIElement*
FString BoundsString = UIElement->Bounds; // -> "50 100 200 25"
TArray<FString> Components;
BoundsString.ParseIntoArray(Components, TEXT(" "));
FIntRect Bounds(
    FString::Atoi(*Components[0]), // X
    FString::Atoi(*Components[1]), // Y
    FString::Atoi(*Components[2]), // Width
    FString::Atoi(*Components[3]) // Height
);
```

TypeScript example:

```
let uiElement = gameData.uiElementsAll.find("HealthBar"); // -> UIElement
let boundsString = uiElement.Bounds; // -> "50 100 200 25"
let parts = boundsString.split(" ").map(x => parseInt(x, 10));
let bounds = { x: parts[0], y: parts[1], width: parts[2], height: parts[3] };
```

Haxe example:

```
var uiElement = gameData.uiElementsAll.get("HealthBar"); // -> UIElement
var boundsString = uiElement.Bounds; // -> "50 100 200 25"
var parts = boundsString.split(" ").map(Std.parseInt);
var bounds = { x: parts[0], y: parts[1], width: parts[2], height: parts[3] };
```

See also

- Data Types
- Rectangle
- Vector Int

Tags

The Tag data type is a sub-type of the Text data type and is used to store a reusable set of tag values (e.g., “fire”, “enemy”, “magic”) that can be shared across multiple documents.

The Tag Collection is a collection variant of this type and it is used to store multiple tags. Each tag is separated by the `` (space) character.

UI Behavior

In the editor, this field is represented as a list of chips/pills/tokens with support for free-form input and autocompletion of existing tags. Note that autocompletion relies on scanning all documents of the same type to extract available tags, so performance considerations should be made when using this data type at scale.

Validation Behavior

The Charon editor does not validate tags during game data loading (i.e., at runtime). Ensure that your game logic handles any missing or unexpected tags appropriately.

Relation to Multi-Pick List

Tags are similar to the Multi-Pick List data type but offer more flexibility by allowing free-form input. C-style named tags can be easily converted to a Multi-Pick List by changing the property type. Likewise, Multi-Pick List values can be converted back to Tags if strict value control is no longer needed.

Example

```
"fire"
"poison magical non-resistable"
```

See also

- Data Types
- Text
- Multi Pick List

Vector 2/3/4

The Vector 2/3/4 is a sub-type of the Text data type used to store a group of 2, 3, or 4 decimal values. These values are space-separated and must be parsed from the string before use at runtime.

UI Behavior

This field is represented in the editor as a group of input fields, each allowing the user to enter one component of the vector (e.g., X, Y, Z).

Validation Behavior

The editor ensures that a valid vector is entered and only stores properly formatted values. However, there is no runtime validation or built-in parsing logic, so the value must be manually parsed in game code.

Example

```
"120.1 56.3 30.0"
```

Parsing

Unity (C#) example:

```
var hero = gameData.AllHeroes.Get("Knight"); // -> Hero
var startingLocationString = hero.StartingLocation; // -> "120.1 56.3 30.0"
var components = startingLocationString.Split(' ');
var startingLocation = new Vector3(
    float.Parse(components[0]),
    float.Parse(components[1]),
    float.Parse(components[2])
);
```

Unreal Engine (C++) example:

```
auto Hero = GameData->AllHeroes->Find(TEXT("Knight")); // -> UHero*
FString StartingLocationString = Hero->StartingLocation; // -> "120.1 56.3 30.0"
TArray<FString> Components;
StartingLocationString.ParseIntoArray(Components, TEXT(" "));
FVector StartingLocation = FVector(
    FString::Atof(*Components[0]),
    FString::Atof(*Components[1]),
    FString::Atof(*Components[2])
);
```

TypeScript example:

```
let hero = gameData.heroesAll.find("Knight"); // -> Hero
let startingLocationString = hero.StartingLocation; // -> "120.1 56.3 30.0"
let parts = startingLocationString.split(" ").map(parseFloat);
let startingLocation: [number, number, number] = [parts[0], parts[1], parts[2]];
```

Haxe example:

```
var heroById = gameData.heroesAll.get("Knight"); // -> Hero
var startingLocationString = heroById.StartingLocation; // -> "120.1 56.3 30.0"
var parts = startingLocationString.split(" ").map(Std.parseFloat);
var startingLocation = { x: parts[0], y: parts[1], z: parts[2] };
```

See also

- Data Types
- Vector Int
- Rectangle

Integer Vector 2/3/4

The Integer Vector 2/3/4 is a sub-type of the Text data type used to store a group of 2, 3, or 4 integer (whole number) values. Values are separated by spaces and must be parsed from the string before use at runtime.

UI Behavior

This field is represented in the editor as a group of input fields, allowing the user to enter each component of the vector (e.g., X, Y, Z) as whole numbers.

Validation Behavior

The editor ensures that a valid integer vector is entered and only stores properly formatted values. However, there is no runtime validation or built-in parsing logic, so the value must be manually parsed in game code.

Example

```
"10 20 5"
```

Parsing

Unity (C#) example:

```
var hero = gameData.AllHeroes.Get("Knight"); // -> Hero
var gridPositionString = hero.StartingLocation; // -> "10 20 5"
var components = gridPositionString.Split(' ');
var startingLocation = new Vector3Int(
    int.Parse(components[0]),
    int.Parse(components[1]),
    int.Parse(components[2])
);
```

Unreal Engine (C++) example:

```
auto Hero = GameData->AllHeroes->Find(TEXT("Knight")); // -> UHero*
FString GridPositionString = Hero->StartingLocation; // -> "10 20 5"
TArray<FString> Components;
GridPositionString.ParseIntoArray(Components, TEXT(" "));
FIntVector StartingLocation = FIntVector(
    FString::Atoi(*Components[0]),
    FString::Atoi(*Components[1]),
    FString::Atoi(*Components[2])
);
```

TypeScript example:

```
let hero = gameData.heroesAll.find("Knight"); // -> Hero
let gridPositionString = hero.StartingLocation; // -> "10 20 5"
let parts = gridPositionString.split(" ").map(x => parseInt(x, 10));
let startingLocation: [number, number, number] = [parts[0], parts[1], parts[2]];
```

Haxe example:

```
var heroById = gameData.heroesAll.get("Knight"); // -> Hero
var gridPositionString = heroById.StartingLocation; // -> "10 20 5"
var parts = gridPositionString.split(" ").map(Std.parseInt);
var startingLocation = { x: parts[0], y: parts[1], z: parts[2] };
```

See also

- Data Types
- Vector
- Rectangle Int

Choosing the appropriate data type is essential for ensuring that game data is accurately structured, efficiently stored, and correctly interpreted by both the editor and runtime systems. Each data type serves a specific purpose and is designed to represent particular kinds of values or relationships.

For instance, use the `Text` data type for simple string values, or the `LocalizedText` type when the same text must support multiple languages. For numeric data, `Number` is suited for decimal values, while `Integer` should be used for whole numbers.

When a property should be selected from a defined set of options, `PickList` and `MultiPickList` are recommended for single or multiple selections, respectively. For more complex structures, use `Document` to define an embedded object, or `DocumentCollection` to represent a list of such objects.

By selecting data types according to their intended purpose, developers can ensure better validation, clearer data organization, and more predictable behavior across tools and runtime systems.

Data Types

Data Type	Description	Example
<code>Text</code>	A line of text.	"Hello, world!"
<code>LocalizedText</code>	A localized text for multiple languages.	{"en-US": "Hello", "fr-FR": "Bonjour"}
<code>Logical</code>	A true/false value.	true
<code>Time</code>	A time duration (days, hours, etc.).	"1.00:00:00"
<code>Date</code>	A calendar date and time.	"2017-12-27T00:00:00.000Z"
<code>Number</code>	A decimal number.	3.14
<code>Integer</code>	A whole number.	42
<code>PickList</code>	A single value from a predefined list.	"Red"
<code>MultiPickList</code>	Multiple values from a predefined list.	"Apple, Banana, Cherry"
<code>Document</code>	An embedded structured object.	{ "Id": "Sword", "Name": "Rusty Sword" }
<code>DocumentCollection</code>	A list of embedded structured objects.	[{ "Id": "Sword", "Name": "Rusty Sword" }]
<code>Reference</code>	A pointer to another document by ID.	{ "Id": "Sword" }
<code>ReferenceCollection</code>	A list of pointers to other documents.	[{ "Id": "Sword" }]
<code>Formula</code>	A C# expression evaluated at runtime.	"target.HP < 100"

Composite Types

Composite Type	Base Type	Description
<code>Asset Path</code>	<code>Text</code>	A relative path to a single game asset.
<code>Vector 2/3/4</code>	<code>Text</code>	A group of 2, 3, or 4 decimal values (e.g., position or scale).
<code>Integer Vector 2/3/4</code>	<code>Text</code>	A group of 2, 3, or 4 whole numbers.
<code>Rectangle</code>	<code>Text</code>	A rectangle with floating-point position and size.
<code>Integer Rectangle</code>	<code>Text</code>	A rectangle with integer position and size.
<code>Tags</code>	<code>Text</code>	A reusable set of tags shared across documents.

Use Cases and Guidance

- Use **Text** and **LocalizedText** for titles, descriptions, and UI content.
- Choose **Integer** or **Number** for quantities, rates, and stats.

- Use **Document** or **Reference** to include structured data or link to another entity.
- Use **PickList** or **MultiPickList** when the value should be selected from a known set.
- Use **Asset Path** or **Asset Path Collection** when referencing assets like textures, sounds, or prefabs by path.
- Use **Formula** to define runtime-calculated values, like damage formulas or dynamic prices.
- Use **Tags** when multiple documents share common descriptors (e.g., “fire”, “enemy”, “magic”).

See also

- Game Data Structure
- Text
- Integer
- Reference

Generating Source Code

Generated source code is how game data reaches the game. Charon reads the schemas and produces classes, enums, and a loader for the target language, so the game accesses data through typed members instead of parsing JSON by hand.

It runs in three places: the wizard in the editor, a button in the Unity and Unreal Engine plugins, and the GENERATE commands in the CLI. All three produce the same code from the same options.

Language Support Tiers	133
The Generation Wizard	134
Step 1. Target Language	134
Step 2. Customize Generation	135
Step 3. Summary	137
From the Unity and Unreal Engine Plugins	138
From the CLI	138
See also	139

Language Support Tiers

The source code generation system is organized into three support tiers:

- **Tier 3 (Basic):** Schema types/enums are generated, basic game data class, localization language selection, document references, and built-in JSON reader.
- **Tier 2 (Advanced):** Includes all Tier 3 features, plus: formulas parsed into AST (no interpreter), built-in Json/MessagePack reader, full game data class with helper functions/classes, and union types. Supported by **Haxe**.
- **Tier 1 (Complete):** Includes all Tier 2 features, plus: full formulas support (ASP interpreter), patching during load support, and language idiomatic features. Supported by **C# 4.0**, **C# 7.3**, **TypeScript**, and **UE C++**.

Features					
Feature	C#	TypeScript	C++ (UE)	Haxe	Lua
Classes	✓■	✓■	✓■	✓■	✓■
Enums	✓■	✓■	✓■	✓■	✓■
Build-in Formatters	✓■	✓■	✓■	✓■	

— JSON	✓■	✓■	✓■	✓■	
— MessagePack	✓■	✓■	✓■	✓■	
Localization	✓■	✓■	✓■	✓■	✓■
Patching	✓■	✓■	✓■	✓■	
Formulas	✓■	✓■	✓■	✓■	
— Parsed AST	✓■	✓■	✓■	✓■	
— Interpreter	✓■	✓■	✓■		
— Lambdas	✓■	✓■			

The Generation Wizard

Generate Source Code on the dashboard opens the wizard. **Project Settings** → **Source Code** has the same button.

Step 1. Target Language

Generate Source Code

1. Target Language > 2. Customize Generation > 3. Summary

Specify the programming language in which you want to generate source code:

 C# 4.0 (2010) legacy ☐

 C# 7.3 (2018) ☒

 TypeScript 4.9 (2022) ☐

 C++ Unreal Engine (5.3) ☐

 Haxe 4.3.4 beta ☐

 Lua 5.1 / LuaJIT beta ☐

If you want to know more about the process of generating source code for your game, please read the [documentation](#).

Close
Next

Each entry names the language version the generated code targets. Markers say what to expect: *legacy* on C# 4.0, which exists for old Unity versions, and *beta* on Haxe and Lua.

Two targets need something on the game side, and the wizard says so when they are selected:

Generate Source Code

1. Target Language > 2. Customize Generation > 3. Summary

Specify the programming language in which you want to generate source code:


C# 4.0 (2010) legacy ☐


C# 7.3 (2018) ☐


TypeScript 4.9 (2022) ☐


C++ Unreal Engine (5.3) ☒


Haxe 4.3.4 beta ☐


Lua 5.1 / LuaJIT beta ☐

i This option require **a plugin** to run.

If you want to know more about the process of generating source code for your game, please read the [documentation](#).

Close

Next

- **C++ Unreal Engine** requires the Charon plugin - the generated code depends on its runtime module.
- **C# 4.0** requires the [C# Eval\(\)](#) library if the data uses formulas.

Step 2. Customize Generation

Generate Source Code

1. Target Language > 2. Customize Generation > 3. Summary

Game Data Class Name * Document Class Name *

GameIDData Document

Code Namespace *

GameParameters

Define constants

Recommend: constants to define in each generated file. Use semicolon (;) to separate multiple values.

Line Endings * Indentation *

Windows Tabs

Code Bundling Optimizations

Multiple Files Raw References, Disable String Pooling O...

Close

Field	Meaning
-------	---------

Game Data Class Name	Name of the root class the game loads and then reads collections from. Must be a valid identifier.
Document Class Name	Name of the base class every document type inherits from.
Code Namespace	Namespace the generated types are placed in. Reads Code Package Name for Haxe, is required for the two C# targets, and is absent for TypeScript, UE C++, and Lua, which have no equivalent.
Define constants	Preprocessor constants written into each generated file, separated by ; or ,. Offered for the C# targets and UE C++.
Line Endings	Windows or Unix. Worth matching to whatever the repository already stores, so regeneration does not produce a diff on every line.
Indentation	Tabs, 2 spaces, or 4 spaces.
Code Bundling	One file for everything, or a file per type. UE C++ is always multiple files.
Optimizations	A multi-select, described below.

Which fields appear depends on the target. Haxe, for instance, has a package name instead of a namespace and no preprocessor constants:

Generate Source Code

1. Target Language
>
2. Customize Generation
>
3. Summary

Game Data Class Name *

Document Class Name *

Code Package Name

Line Endings *

Indentation *

Code Bundling

Optimizations

Close
Generate

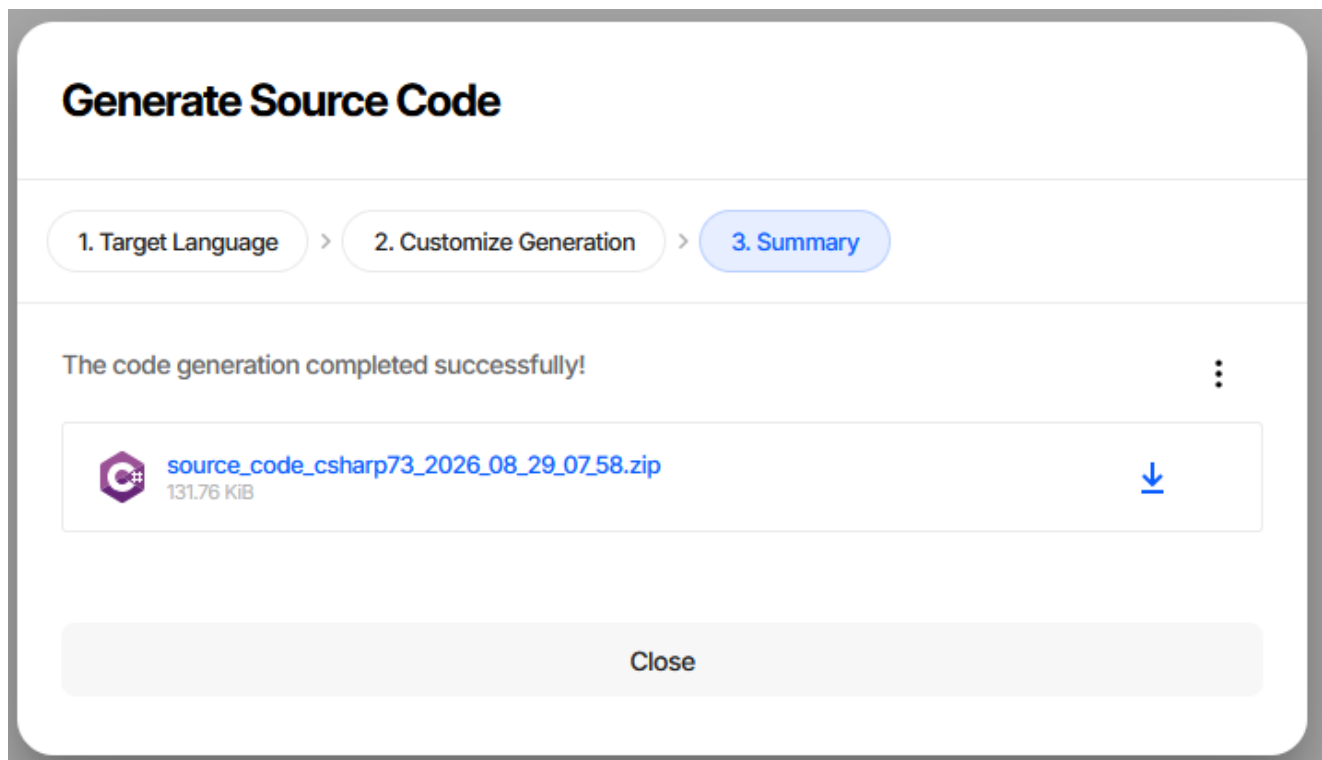
Optimizations

Two of these add work at load time and seven remove code from the output. Everything removed is code the game would otherwise carry unused, so on a large project the difference in compile time and binary size is worth having - but each one takes a feature away, and the loader will not warn you at runtime.

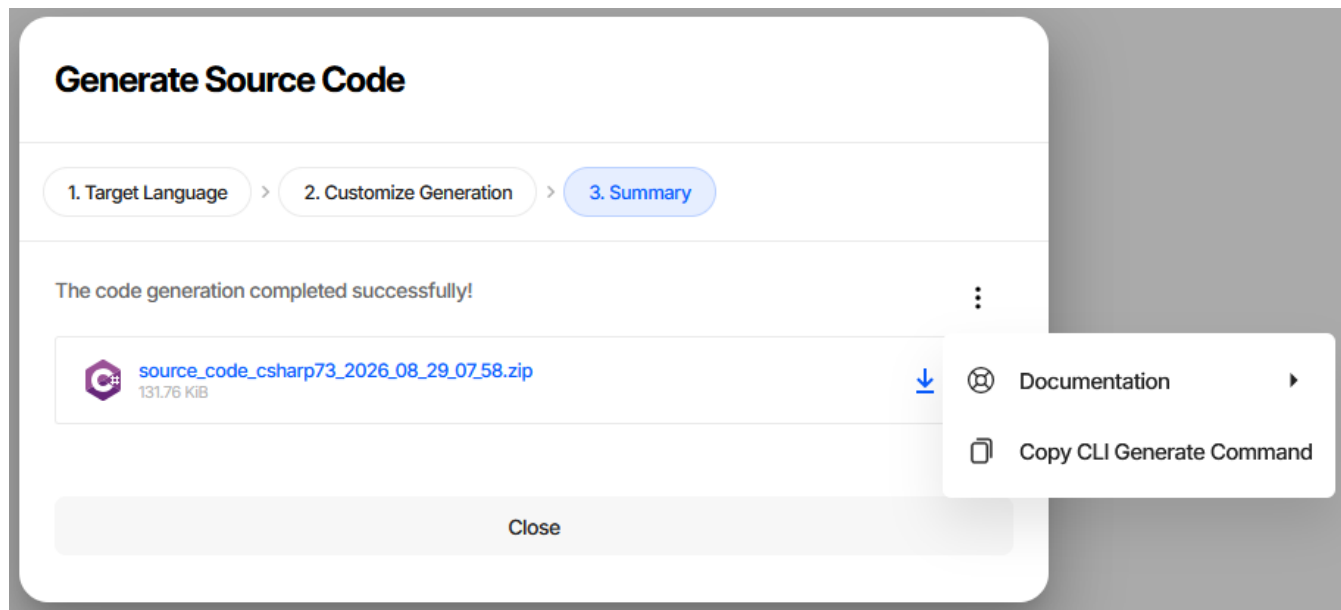
Option	Effect
Eager Reference Resolution	Resolves and validates every reference while loading, instead of on first access. Costs load time, and turns a broken reference into a failure at startup rather than mid-game.
Raw References	Drops the helper members that follow a reference to the referenced document, keeping the raw reference accessors only.
Raw Localized Texts	Drops the helpers that return the text for the current language, keeping accessors that hand back the whole set of translations.
Raw Composite Types	Drops the parsed accessors for text properties carrying a vector, rectangle or tags editor. The property keeps its original name and string type instead of moving to a <Name>Raw companion.
Disable 'String Pooling' Optimization	Stops reusing identical short strings while loading. Slightly faster, more memory.
Disable JSON Serialization	Omits the JSON reader. Only safe if the game ships Message Pack data.
Disable MessagePack Serialization	Omits the Message Pack reader. Only safe if the game ships JSON.
Disable 'Patch' Support	Omits the patch-application code, which is a large part of the output and is used for modding and patching.
Disable 'Document Id' Enums Generation	Omits the enums listing the ids of every document known at generation time. Cuts compile time noticeably on big projects, at the cost of losing the compile-time-checked id constants.

String pooling and the two serializer options are not available for Lua. Raw composite types is not available for Unreal Engine C++, which always behaves as if it were on - a UPROPERTY cannot be renamed, so those properties keep their name and get a separate GetParsed . . . accessor instead.

Step 3. Summary



The result is a zip named after the target and the moment it was generated. Download it and unpack it into the game project.



The ... menu carries **Copy CLI Generate Command**, which puts the equivalent command line on the clipboard with every option filled in exactly as the wizard was configured. It is the fastest way to move a configuration that works into a build script; **Documentation** links to the reference for the language just generated.

From the Unity and Unreal Engine Plugins

Both plugins generate code with a single button, without the wizard.

- **Unreal Engine** generates immediately, using default settings. There are no customization options in the plugin at the moment. See Creating Game Data.
- **Unity** works the same way, except the options live in the Inspector of the game data asset - set them there, then press the button. See Unity Overview.

Either way, regenerate after changing the structure of the data - adding a property, renaming a schema - and not after merely editing values.

From the CLI

One GENERATE command per language, taking the same options as the wizard:

```
dnx dotnet-charon -- GENERATE CSHARPCODE \
  --dataBase "c:\my app\gamedata.json" \
  --outputDirectory "c:\my app\scripts" \
  --namespace "MyGame.Parameters" \
  --gameDataClassName GameData \
  --splitFiles \
  --clearOutputDirectory
```

Wizard field	CLI parameter
Game Data Class Name	--gameDataClassName
Document Class Name	--documentClassName
Code Namespace / Code Package Name	--namespace

Define constants	--defineConstants
Line Endings	--lineEndings
Indentation	--indentation
Code Bundling	--splitFiles
Optimizations	--optimizations

The CLI has two parameters the wizard has no use for, because it writes into a directory rather than a zip: --outputDirectory for the destination, and --clearOutputDirectory to delete the previously generated files first. The second one only removes files carrying the <auto-generated> marker, so hand-written code in the same directory survives.

Regenerating from CI on every change to the data keeps the code and the data from drifting apart - see CI/CD.

See also

- Publishing Game Data
- Working with Source Code (C# 4.0)
- Working with Source Code (C# 7.3)
- Working with Source Code (TypeScript)
- Working with Source Code (UE C++)
- Working with Source Code (Haxe)
- Working with Source Code (Lua)
- Command Line Interface (CLI)
- GENERATE CSHARPCODE Command
- GENERATE TYPESCRIPTCODE Command
- GENERATE UECPPE Command
- GENERATE HAXE Command
- GENERATE LUA Command

Working with Source Code (C# 7.3)

Note

Tier 1 Support: The C# 7.3 code generator provides complete Tier 1 support. This includes full formula execution (interpreter), patching during load, and language idiomatic features.

Accessing game data during runtime is possible by utilizing the generated source code.

This section provides examples using default class names, but it is possible to customize class names during the source code generation process. Additionally, this customization allows to avoid naming collisions with existing code.

Loading Game Data

The following C# code creates GameData class and loads your game data into memory.

```
using System.IO;

var fileStream = File.OpenRead("RpgGameData.gdjs"); // or .json
var gameData = new GameData(fileStream, new Formatters.GameDataLoadOptions {
    Format = Formatters.Format.Json,
```

```
// Patches = new [] { patchStream1, patchStream2, ... }
});
fileStream.Dispose();
```

The file `RpgGameData.gdjs` could be published game data or original database file (`.gdjs` or `.gdmp`).

Accessing Documents

You can access your documents as a list:

```
var allHeroes = gameData.AllHeroes.AsList // -> IReadOnlyList<Hero>
var heroes = gameData.Heroes.AsList // -> IReadOnlyList<Hero>
```

Or you can access specific documents by their `Id` or *Unique* properties:

```
var heroById = gameData.AllHeroes.Get(heroId); // -> Hero
var heroByName = gameData.AllHeroes.ByName().Get(heroName); // -> Hero
```

Settings schemas are accessed by name:

```
var startingHeroes = gameData.StartingSet.Heroes; // -> IReadOnlyList<Hero>
```

Formulas

Each formula property in your schema generates a dedicated partial class - for example a property named `LevelUpConditionsCheck` with return type `bool` and one parameter `stats` of type `HeroStats` produces:

```
// Formula Signature: (HeroStats stats) -> bool
public partial class LevelUpConditionsCheckFormula
{
    public bool Invoke(HeroStats stats);
}
```

Calling a Formula

Formula properties are stored as read-only properties on the owning document. Call `Invoke` with the required arguments to evaluate the expression:

```
var hero = gameData.AllHeroes.Get("hero_01");
if (hero?.LevelUpConditionsCheck != null)
{
    // formula: stats.Experience >= this.Levels[stats.Level].ExperienceToLevelUp
    // where `this` is a reference to the Hero document
    bool result = hero.LevelUpConditionsCheck.Invoke(heroStats);
}
```

If evaluation or compilation fails, `Invoke` throws `FormulaException` with a message describing the failure and the formula expression that caused it.

Formula Scope

Every formula expression has two implicit identifiers available without a qualifier:

- `this` - the document instance that declares the formula property. For a formula property defined on the `Hero` schema, `this` refers to the `Hero` object being evaluated.
- `GameData` - the root `GameData` instance, giving access to all other documents and settings.

Consider a `Hero` schema with a numeric property `BaseDamage` and a formula property `DamageFormula` with signature `(int multiplier) -> int`. The formula expression can reference `this.BaseDamage` directly:

```
this.BaseDamage * multiplier
```

When `hero.DamageFormula.Invoke(/* multiplier */ 5)` is called on a `Hero` whose `BaseDamage` is 10, the expression evaluates `10 * 5` and returns 50.

Formula Syntax

Formula expressions use **C# 3.5 syntax** with several extensions:

- `? .` - null-conditional member access (`this.Target?.Health`)
- `?[` - null-conditional index access (`list?[0]`)
- `??` - null-coalescing operator (`value ?? 0`)
- `**` - exponentiation operator (`base ** exponent`)

Unlike the UE C++ generator, **lambda expressions and explicit generic type arguments are fully supported**:

```
items.Where(x => x.Active).Sum(x => x.Value)
```

Member and method dispatch is **dynamic**: property and method names are resolved at evaluation time against the actual object instance. This means type mismatches surface at runtime (as an `FormulaException`) rather than at compile time.

Supported value types are the standard .NET primitives (`bool`, `int`, `long`, `float`, `double`, `string`, `DateTime`, `TimeSpan`) plus any type explicitly registered in the formula's known-types list via the `Specification` field of the schema property.

Generated Code Contract

The generated C# API is **read-only by design**:

- Document classes expose only public *getters* - there are no public property setters.
- Collections are typed as `IReadOnlyList<T>` or `IReadOnlyDictionary<TKey, TValue>`.
- After the `GameData` constructor returns, all documents and collections are **frozen**. Attempting to modify internal state via reflection or unsafe casts will corrupt the object graph.
- Some properties - notably resolved cross-document references and formula results - are computed lazily on first access and cached internally. This is an implementation detail; the observable value never changes after load.

The intended usage pattern is: **load once, read many times, never write**.

Note

Do not assign to, replace, or clear any collection or document property obtained from `GameData` or its documents. To add computed or derived data, extend the generated classes with partial classes instead.

Extension of Generated Code

Partial Classes and Methods

When generating source code for game data, the resulting C# classes are declared as **partial**. This means that the classes can be extended by the programmer to add custom functionality.

For example, let's say that you have generated a `GameData` class for your game data. This class contains properties and methods for accessing and manipulating the data. However, you want to add some custom functionality to this class, such as a method for getting specific documents by criteria.

To do this, you can create a new C# file and declare a partial class with the same name as the generated `GameData` class. You can then define your custom method in this class, and it will be merged with the generated class at compile time.

Here is an example of how this could look:

```
// should be in same namespace as generated GameData class
public partial class GameData {
```

```

    public IEnumerable<Hero> GetReligiousHeroes() {
        return this.AllHeroes.AsList.Where(hero => hero.Religious);
    }
}

```

In this example, the `GameData` class is declared as partial, and two partial classes are defined with the same name: one generated by the source code generation process and one containing custom code added by the programmer.

By using partial classes in this way, you can extend the functionality of the generated classes without modifying the generated code directly. This allows you to keep your custom code separate from the generated code, making it easier to maintain and update your game data classes over time.

There is also extension point on `GameData` class:

```
partial void OnInitialize(); // Called after loading and prepping all data.
```

Customizing Attributes

You can append additional [C# Attributes](#) to the generated classes and their properties by modifying the `Specification` field of the related schema or property.

Attributes are specified using the `csAttribute` key in the `Specification` string, which uses the application/x-www-form-urlencoded format.

To help construct the correct value, you can use a spreadsheet formula (e.g., in Excel or Google Sheets):

```
# Place your attribute in cell A1
=TEXTJOIN("&", 1, IF(ISBLANK(A1), "", "&csAttribute=" & ENCODEURL(A1)))
```

Alternatively, use JavaScript to generate the encoded string:

```
const params = new URLSearchParams();
params.append("csAttribute", "Obsolete(\"Value is obsolete, do not use\")");
console.log(params.toString());
// → csAttribute=Obsolete%28%22Value%20is%20obsolete%2C%20do%20not%20use%22%29
```

After obtaining the encoded string, append it to the existing `Specification` value.

Example:

```
# Original Specification value:
icon=material&group=Metadata
```

```
# New attribute to add:
csAttribute=Obsolete%28%22Value%20is%20obsolete%2C%20do%20not%20use%22%29
```

```
# Final Specification value:
icon=material&group=Metadata&csAttribute=Obsolete%28%22Value%20is%20obsolete%2C%20do%20not%20use%22%29
```

You can add multiple attributes by including multiple `csAttribute` keys:

```
csAttribute=Serializable&csAttribute=DebuggerDisplay%28%22Id%3D%7BId%7D%22%29
```

These attributes will be emitted directly into the generated C# code, attached to the appropriate class or property.

See also

- [Generating Source Code](#)
- [GENERATE CSHARPCODE Command](#)

Working with Source Code (C# 4.0)

Note

Tier 1 Support: The C# 4.0 code generator provides complete Tier 1 support. This includes full formula execution (interpreter), patching during load, and language idiomatic features.

Warning

This is deprecated code generator and shouldn't be used in new projects.

Accessing game data during runtime is possible by utilizing the generated source code.

This section provides examples using default class names, but it is possible to customize class names during the source code generation process. Additionally, this customization allows to avoid naming collisions with existing code.

Loading Game Data

The following C# code creates `GameData` class and loads your game data into memory.

```
using System.IO;

var fileStream = File.OpenRead("gamedata.json");
var gameData = new GameData(fileStream, GameData.Format.Json);
fileStream.Dispose();
```

The file `gamedata.json` could be published game data or original database file (`.gdjs` or `.gdmp`).

Accessing Documents

You can access your documents as a list:

```
var characters = gameData.GetCharacters() // -> ReadOnlyList<Character>
var characters = gameData.GetCharacters(onlyRoot: true) // -> ReadOnlyList<Character>
```

Or you can access specific documents by their `Id` or *Unique* properties:

```
var character = gameData.GetCharacter(characterId); // -> Character
var character = gameData.GetCharacterByName(characterName); // -> Character
```

Settings schemas are accessed by name:

```
var resetTime = gameData.LootSettings.ResetTime; // -> TimeSpan
```

Formulas

Formulas are executed with `Invoke` method:

```
var reward = gameData.LootSettings.RewardFormula.Invoke() // -> int
```

Formula's parameters are passed as arguments of `Invoke` method.

Generated Code Contract

The generated C# API is **read-only by design**:

- Document classes expose only public *getters* - there are no public property setters.
- Collections are typed as `ReadOnlyList<T>`.
- After the `GameData` constructor returns, all documents and collections are **frozen**. The `OnBeforeInitialize` partial hook fires before references are resolved and documents are marked read-only; `OnInitialize` fires once everything is locked. Attempting to modify the object graph after that point will corrupt it.

- Some properties - notably resolved cross-document references - are computed lazily on first access and cached internally. This is an implementation detail; the observable value never changes after load.

The intended usage pattern is: **load once, read many times, never write.**

Note

Do not assign to, replace, or clear any collection or document property obtained from `GameData` or its documents. To add computed or derived data, extend the generated classes with partial classes instead.

Generated Code Extensions

When generating source code for game data, the resulting C# classes are declared as [partial](#). This means that the classes can be extended by the programmer to add custom functionality.

For example, let's say that you have generated a `GameData` class for your game data. This class contains properties and methods for accessing and manipulating the data. However, you want to add some custom functionality to this class, such as a method for getting specific documents by criteria.

To do this, you can create a new C# file and declare a partial class with the same name as the generated `GameData` class. You can then define your custom method in this class, and it will be merged with the generated class at compile time.

Here is an example of how this could look:

```
// should be in same namespace as generated GameData class
public partial class GameData {

    public IEnumerable<Character> GetFighterCharacters() {
        return this.GetCharacters().Where(character => character.Role == CharacterRole.Fight
    }

}
```

In this example, the `GameData` class is declared as partial, and two partial classes are defined with the same name: one generated by the source code generation process and one containing custom code added by the programmer.

By using partial classes in this way, you can extend the functionality of the generated classes without modifying the generated code directly. This allows you to keep your custom code separate from the generated code, making it easier to maintain and update your game data classes over time.

There is also two extension points on `GameData` class:

```
partial void OnBeforeInitialize(); // Called after loading the data into lists and dictionaries
partial void OnInitialize(); // Called after loading and prepping all data.
```

See also

- Generating Source Code
- GENERATE CSHARPCODE Command

Working with Source Code (Type Script)

Note

Tier 1 Support: The TypeScript code generator provides complete Tier 1 support. This includes full formula execution (interpreter), patching during load, and language idiomatic features.

Accessing game data during runtime is possible by utilizing the generated source code.

This section provides examples using default class names, but it is possible to customize class names during the source code generation process. Additionally, this customization allows to avoid naming collisions with existing code.

Loading Game Data

The following Type Script code creates `GameData` class and loads your game data into memory.

```
import { GameData } from './game.data';
import { Formatters } from './formatters';

// Node.js
import { readFileSync } from 'fs';
const gameDataStream = readFileSync(gameDataFilePath);

// Blob or File
const gameDataStream = gameDataFileBlob.arrayBuffer();

// XMLHttpRequest (XHR)
// gameDataRequest.responseType -> "arraybuffer"
const gameDataStream = gameDataRequest.response;

const gameData = new GameData(gameDataStream, {
  format: Formatters.GameDataFormat.Json,
  // patches: [patchStream1, patchStream2, ...]
});
```

The content of `gameDataStream` could be published game data or original database file (.gdjs or .gdmp).

Accessing Documents

You can access your documents as a list:

```
let heroes = gameData.heroesAll; // all heroes from all documents -> readonly Hero[]
let heroes = gameData.heroes; // heroes only from root collection -> readonly Hero[]
```

Or you can access specific documents by their `Id` or *Unique* properties:

```
let hero = gameData.heroesAll.find(heroId); // -> Hero | undefined
let hero = gameData.heroesAll.withOtherKey('Name').find(heroName); // -> Hero | undefined
```

Settings schemas are accessed by name:

```
let resetTime = gameData.lootSettings.resetTime; // -> TimeSpan
```

Formulas

Formulas inherit the `Function` type and can be invoked as-is or with `invoke` method:

```
var reward = gameData.lootSettings.rewardFormula() // -> number
// or
var reward = gameData.lootSettings.rewardFormula.invoke() // -> number
```

Formula's parameters are passed as arguments of `invoke` method.

Any non-game data related types are imported from ``formula.known.types.ts``, which should be created by the developer and have all required types exported. Here is an example of a `formula.known.types.ts` file:

```
import { MyFormulaContext } from '../my.formula.context';

// example of re-exporting MyFormulaContext type.
export MyFormulaContext;

// example of Assets.Scripts.CheckContext.
```

```
export namespace Assets.Scripts {
  export class CheckContext {
    myField: string;
  }
}
```

Extension of Generated Code

Customizing Decorators

You can append additional [TypeScript Decorators](#) to the generated classes and their properties by modifying the Specification field of the related schema or property.

Decorators are specified using the `tsAttribute` key in the Specification string, which uses the application/x-www-form-urlencoded format.

To help construct the correct value, you can use a spreadsheet formula (e.g., in Excel or Google Sheets):

```
# Place your attribute in cell A1
=TEXTJOIN("&", 1, IF(ISBLANK(A1), "", "&tsAttribute=" & ENCODEURL(A1)))
```

Alternatively, use JavaScript to generate the encoded string:

```
const params = new URLSearchParams();
params.append("tsAttribute", "deprecated(\"Value is deprecated\")");
console.log(params.toString());
// → tsAttribute=deprecated%28%22Value+is+deprecated%22%29
```

After obtaining the encoded string, append it to the existing Specification value.

Example:

```
# Original Specification value:
icon=material&group=Metadata

# New attribute to add:
tsAttribute=deprecated%28%22Value+is+deprecated%22%29

# Final Specification value:
icon=material&group=Metadata&tsAttribute=deprecated%28%22Value+is+deprecated%22%29
```

You can add multiple attributes by including multiple `tsAttribute` keys:

```
tsAttribute=@Input&tsAttribute=range%281%2C+2%29
```

These attributes will be emitted directly into the generated TypeScript code, attached to the appropriate class or property.

See also

- [Generating Source Code](#)
- [GENERATE TYPESCRIPTCODE Command](#)

Working with Source Code (UE C++)

Note

Tier 1 Support: The UE C++ code generator provides complete Tier 1 support. This includes full formula execution (interpreter), patching during load, and language idiomatic features.

Warning

The source code for Unreal Engine requires a [plugin](#) to be installed to function. If you get compilation errors, make sure the plugin is [installed and enabled](#).

Accessing game data during runtime is possible by utilizing the generated source code.

This section provides examples using default class names, but it is possible to customize class names during the source code generation process. Additionally, this customization allows to avoid naming collisions with existing code.

Loading Game Data

The following C++ code creates UGameData class and loads your game data into memory.

```
IFileManager& FileManager = IFileManager::Get();

const FString GameDataFilePath = TEXT("./RpgGameData.gdjs"); // or .json
const TUniquePtr<FArchive> GameDataStream = TUniquePtr<FArchive>(FileManager.CreateFileReader(
    GameDataFilePath));

UGameData* GameData = NewObject<UGameData>();

FGameDataLoadOptions Options;
Options.Format = EGameDataFormat::Json;
// Options.Patches.Add(PatchStream1);
// Options.Patches.Add(PatchStream2);
// ...

if (!GameData->TryLoad(GameDataStream.Get(), Options))
{
    // Handle failure
}
```

The file RpgGameData.gdjs could be published game data or original database file (.gdjs or .gdmp).

Accessing Documents

You can access your documents as a list:

```
auto AllHeroes = GameData->AllHeroes // -> TMap<string, UHero>
auto Heroes = GameData->Heroes // -> TMap<string, UHero>
```

Settings schemas are accessed by name:

```
auto StartingHeroes = GameData->StartingSet.Heroes; // -> TMap<string, UHero>
```

Formulas

Formulas are supported when the plugin has the CHARON_FEATURE_FORMULAS_V2=1 compilation constant enabled (available since plugin version 2025.2.x).

Each formula property in your schema generates a dedicated UObject subclass - for example a property named LevelUpConditionsCheck with return type System.Boolean and one parameter stats of type UObject produces:

```
// Formula Signature: (UObject stats) -> System.Boolean
UCLASS(BlueprintType)
class ULevelUpConditionsCheckFormula : public UObject
{
    GENERATED_BODY()
public:
    UFUNCTION(BlueprintCallable)
```

```
bool Invoke(UObject* Stats) const;
};
```

The generated Invoke method is BlueprintCallable, so it is usable both from C++ and from Blueprint graphs without any additional work.

Calling a Formula

Formula properties are stored as UPROPERTY pointers on the owning document. Call Invoke with the required arguments to evaluate the expression:

```
UHeroStats* Stats; // context for level condition checks
UHero* Hero = GameData->AllHeroes.FindRef(TEXT("hero_01"));
if (Hero && Hero->LevelUpConditionsCheck)
{
    // for example formula is: Stats.Experience >= this.Levels[Stats.Level].ExperienceToLevel
    // where `this` is a reference to the `UHero` document.
    bool bCheckResult = Hero->LevelUpConditionsCheck->Invoke(Stats /* UObject* Stats */);
}
```

If the formula's expression tree is missing, or if evaluation fails at runtime, Invoke logs an error to the formula's dedicated log category and returns the default value for the return type (e.g. false for bool, 0 for numeric types).

Formula Scope

Every formula expression has two implicit identifiers available without a qualifier:

- **this** - the document instance that declares the formula property. For a formula property defined on the Hero schema, this refers to the UHero object being evaluated.
- **GameData** - the root UGameData instance, giving access to all other documents and settings.

Consider a Hero schema with a numeric property BaseDamage and a formula property DamageFormula with signature (int32 Multiplier) -> int32. The formula expression can reference this.BaseDamage directly:

```
this.BaseDamage * Multiplier
```

When Hero->DamageFormula->Invoke(/* Multiplier */ 5) is called on a Hero whose BaseDamage is 10, the expression evaluates 10 * 5 and returns 50.

Formula Syntax

Formula expressions use **C# 3.5 syntax** with several extensions:

- **?.** - null-conditional member access (this.Target?.Health)
- **?[** - null-conditional index access (list?[0])
- **??** - null-coalescing operator (value ?? 0)
- ****** - exponentiation operator (base ** exponent)

The following C# features are **not supported**:

- Lambda expressions (x => expr) - see below.
- Generic type arguments (SomeMethod<T>()) - type arguments cannot be specified explicitly.

Member and method dispatch is **dynamic**: property and method names are resolved at evaluation time against the actual UObject (or UStruct) instance passed as an argument. Any UObject that has an Invoke(...) UFUNCTION is treated as a callable. This means type mismatches surface at runtime rather than at compile time.

Supported value types are the standard Blueprint primitives (bool, int32, int64, float, double, FString, FText, FTimespan, FDateTime) plus any UObject* and UStruct registered with Unreal Engine's reflection system.

Lambda Expressions

Lambda expressions (`x => expr`, `(x, y) => expr`) are parsed but are **not implemented** - calling `Invoke` on a formula that contains a lambda returns the default value and logs an `UnsupportedExpression` error.

Generated Code Contract

Document properties are declared as public `UPROPERTY` fields and collections as `TArray<T*> / TMap<FString, T*>`. These are intentionally mutable for two reasons:

- **Blueprint compatibility** - Blueprint nodes read and write `UPROPERTY` fields directly; getter-only accessors are not reachable from Blueprints.
- **UObject lifecycle** - UE's garbage collector tracks references through `UPROPERTY` metadata, which requires fields to be declared in the standard way.

Treat the loaded data as **logically read-only** even though the language does not enforce it. Do not assign to, replace, or clear any field or collection on a document obtained from a loaded `UGameData` instance - mutating loaded data produces undefined behaviour and is not persisted to the source file.

Note

To add derived or cached per-document state, subclass the relevant document class or keep a separate runtime data structure. Do not write to the generated fields after `TryLoad` returns.

Extension of Generated Code

Customizing Metadata

You can append additional or replace existing [macro](#) to the generated classes and their properties by modifying the `Specification` field of the related schema or property.

Metadata annotations are specified using the `uecppAttribute` key in the `Specification` string, which uses the `application/x-www-form-urlencoded` format.

To help construct the correct value, you can use a spreadsheet formula (e.g., in Excel or Google Sheets):

```
# Place your attribute in cell A1
=TEXTJOIN("&", 1, IF(ISBLANK(A1), "", "&uecppAttribute=" & ENCODEURL(A1)))
```

Alternatively, use JavaScript to generate the encoded string:

```
const params = new URLSearchParams();
params.append("uecppAttribute", "UPROPERTY(EditAnywhere, Meta = (Bitmask))");
console.log(params.toString());
// → uecppAttribute=UPROPERTY%28EditAnywhere%2C+Meta+%3D+%28Bitmask%29%29
```

After obtaining the encoded string, append it to the existing `Specification` value.

Example:

```
# Original Specification value:
icon=material&group=Metadata

# New attribute to add:
uecppAttribute=UCLASS%28BlueprintType%29

# Final Specification value:
icon=material&group=Metadata&uecppAttribute=UCLASS%28BlueprintType%29
```

These metadata annotations will be emitted directly into the generated C++ code, attached to the appropriate class or property.

See also

- Generating Source Code
- GENERATE UECPP Command

Working with Source Code (Haxe)

Note

Tier 2 Support: The Haxe code generator provides Tier 2 support. This includes built-in JSON/MessagePack readers, comprehensive helper classes, and formulas parsed into AST (without interpreter).

Accessing game data during runtime is possible by utilizing the generated source code.

This section provides examples using default class names, but it is possible to customize class names during the source code generation process. Additionally, this customization allows to avoid naming collisions with existing code.

Loading Game Data

The following Haxe code creates `GameData` class and loads your game data into memory.

```
import GameData;
import Formatters;
import haxe.io.Path;
import sys.io.File;

var input = File.read("RpgGameData.gdjs"); // or .json
var options = new GameDataLoadOptions();
options.format = GameDataFormat.Json;
options.leaveInputsOpen = false;
// options.patches <-- put patches here
var gameData = new GameData(input, options);
```

The file `RpgGameData.gdjs` could be published game data or original database file (`.gdjs` or `.gdmp`).

Accessing Documents

You can access your documents as a list:

```
var allHeroes = gameData.heroesAll.list // -> ReadOnlyArray<Hero>
var heroes = gameData.heroes.list // -> ReadOnlyArray<Hero>
```

Or you can access specific documents by their `id` or *Unique* properties:

```
using GameData; // load byName() and other extension methods

var heroById = gameData.heroesAll.get(heroId); // -> Hero
var heroByName = gameData.heroesAll.byName().get(heroName); // -> Hero
```

Settings schemas are accessed by name:

```
var startingHeroes = gameData.startingSet.heroes; // -> ReadOnlyArray<Hero>
```

Formulas

Formulas are currently not supported.

Extension of Generated Code

Customizing Metadata

You can append additional [metadata](#) to the generated classes and their properties by modifying the `Specification` field of the related schema or property.

metadata annotations are specified using the `haxeAttribute` key in the `Specification` string, which uses the `application/x-www-form-urlencoded` format.

To help construct the correct value, you can use a spreadsheet formula (e.g., in Excel or Google Sheets):

```
# Place your attribute in cell A1
=TEXTJOIN("&", 1, IF(ISBLANK(A1), "", "&haxeAttribute=" & ENCODEURL(A1)))
```

Alternatively, use JavaScript to generate the encoded string:

```
const params = new URLSearchParams();
params.append("haxeAttribute", ":deprecated");
console.log(params.toString());
// → haxeAttribute=%3Adeprecated
```

After obtaining the encoded string, append it to the existing `Specification` value.

Example:

```
# Original Specification value:
icon=material&group=Metadata

# New attribute to add:
haxeAttribute=%3Adeprecated

# Final Specification value:
icon=material&group=Metadata&haxeAttribute=%3Adeprecated
```

You can add multiple metadata annotations by including multiple `haxeAttribute` keys:

```
haxeAttribute=broken&haxeAttribute=range%281%2C+2%29
```

These metadata annotations will be emitted directly into the generated Haxe code, attached to the appropriate class or property.

See also

- [Generating Source Code](#)
- [GENERATE HAXE Command](#)

Working with Source Code (Lua)

Note

Tier 3 Support: The Lua code generator provides Tier 3 support. This includes basic game data class, schema types, enums, and JSON/Message Pack reading (via external table deserialization).

Accessing game data during runtime is possible by utilizing the generated source code.

This section provides examples using default class names, but it is possible to customize class names during the source code generation process. Additionally, this customization allows to avoid naming collisions with existing code.

Loading Game Data

The following Lua code creates `GameData` class and loads your game data into memory.

```
local cJSON = require("cjson")
local GameData = require('game_data') -- if splitFiles = true, otherwise `require('game_data')

-- Read file content (e.g., from 'gamedata.json')
local file = io.open("gamedata.json", "r")
local json_string = file:read("*a")
file:close()

local decodedTable = cJSON.decode(json_string)
local gameData = GameData.new(decodedTable) -- data should be deserialized into table before
```

The file `gamedata.json` could be published game data or original database file (`.gdjs` or `.gdmp`).

Accessing Documents

You can access your documents as a map/table by ID:

```
local heroes = gameData:getHeroes() -- heroes only from root collection by id -> table<any,
```

Or you can access specific documents by their `Id` or *Unique* properties:

```
local hero = heroes[heroId] -- -> Hero | nil
```

Settings schemas are accessed via their getter:

```
local lootSettings = gameData:getLootSettings() -- -> LootSetting | nil
local resetTime = lootSettings.resetTime -- -> number
```

Formulas

Formulas are currently not supported.

See also

- [Generating Source Code](#)
- [GENERATE LUA Command](#)

Publishing Game Data

Publication turns your working game data into a game-ready file. Localization for the languages you did not select is stripped out, and the result is written as JSON or MessagePack that the generated code can load directly.

This is the last step of the content pipeline, and where it ends depends on where you run it from: in the web editor you download a file, while the Unity and Unreal Engine plugins import the result into the engine for you and regenerate the source code in the same pass.

Data Version	153
Publishing from the Editor	153
Step 1. Collections Review	154
Step 2. Pick Format	154
Step 3. Languages	155
Step 4. Summary	155
Publishing from a Game Engine Plugin	156
Unity	156
Unreal Engine	157
Where the data ends up	157

Data Version

Project Settings → **Publication** carries a **Version** field - a semantic version for the game data itself, separate from the Charon tool version and from your game's build number. It identifies the data snapshot, which is what you quote when a designer asks which data a shipped build was made from.



The accepted shape is three dot-separated numbers, an optional fourth, and an optional -suffix, up to 16 characters - 1.4.0, 1.4.0.2 and 1.4.0-rc1 are all valid. The hint under the field names the branch being edited, because every branch carries its own version.

Update saves the change and stays disabled until the field is edited. Editing requires the **Designer** role or higher.

The version lives on the project settings document, so it travels with the data - into backups, ordinary exports, and the published file alike.

At runtime the generated game data class carries the value next to the revision hash and change number. It is read straight out of the project settings document while the file is deserialized, so the game can log or display which data snapshot it loaded:

```
Debug.Log($"Game data version: {gameData.GameDataVersion}");
```

Target	Accessor
C# 7.3	gameData.GameDataVersion
UE C++	GameData->GameDataVersion - also BlueprintReadOnly, so it can be read from Blueprints without any C++ code
Haxe	gameData.gameDataVersion
Lua	gameData.gameDataVersion
TypeScript	gameData.gameDataVersion
C# 4.0	Not generated

CLI equivalent

Added in version 2026.4.0: DATA UPDATEPROJECTSETTINGS

```
dnx dotnet-charon -- DATA UPDATEPROJECTSETTINGS \
  --dataBase "https://charon.live/view/data/MyGame/develop/" \
  --property Version \
  --value "1.4.0" \
  --credentials "$CHARON_API_KEY"
```

Stamping the version from CI just before the publish step keeps every shipped file traceable to the build that produced it; see CI/CD for a worked example and DATA UPDATEPROJECTSETTINGS for the full parameter list.

Publishing from the Editor

Publish on the dashboard opens the Publication wizard.



Step 1. Collections Review

A validation pass over every collection, so you find out about broken data before it reaches the game rather than after.

Publication

1. Collections Review > 2. Pick Format > 3. Languages > 4. Summary

Review document collections before publication:

- Dungeons (89 missed translations)
 - Curios (10 missed translations)
 - Items (46 missed translations)
 - Locations (7 missed translations)
 - Monsters (6 missed translations)
 - Provisions (20 missed translations)
- Heroes and Stuff (44 missed translations)
- Settings (18 missed translations)

Close Next

N issues counts integrity errors, N missed translations counts untranslated text. Clicking a collection opens it filtered to just the documents with errors.

Publication is never blocked - **Next** stays enabled whatever the review finds - but the two severities deserve different treatment. Missing translations ship as visible gaps in the affected languages. Integrity errors are broken data: generated code assumes the data satisfies the schema, so a dangling reference or a missing required value usually surfaces at runtime as a null reference or a failed load. Fix those before publishing.

See Validation for the full description of this step and how to reproduce it in CI.

Step 2. Pick Format

Publication

1. Collections Review > 2. Pick Format > 3. Languages > 4. Summary

Specify the data format you want to publish to:

☒ JSON (recommended) ☐ MTP Message Pack

Close Next

Format	When to use
JSON (recommended)	Default. Readable, diffable, debuggable - keep it unless load time or file size is a measured problem.
Message Pack	Compact binary. Smaller files and faster parsing, at the cost of not being human-readable.

Both formats carry the same data and are loaded by the same generated code, so switching later costs nothing but a re-publish.

Step 3. Languages

Publication

1. Collections Review > 2. Pick Format > **3. Languages** > 4. Summary

Choose languages to include in the published data:

- ☒ (all)
- ☒ English (United States)
- ☒ Georgian (Georgia)
- ☒ Hindi (India)
- ☒ Spanish (Spain)

Close Publish

Tick **(all)** to publish every language, or select individual ones. Languages left unticked are stripped from the output entirely - this is how you keep a Japanese-only build from shipping the German strings.

At least one language is required. **Publish** on this step starts the process.

Step 4. Summary

Publication

1. Collections Review > 2. Pick Format > 3. Languages > **4. Summary**

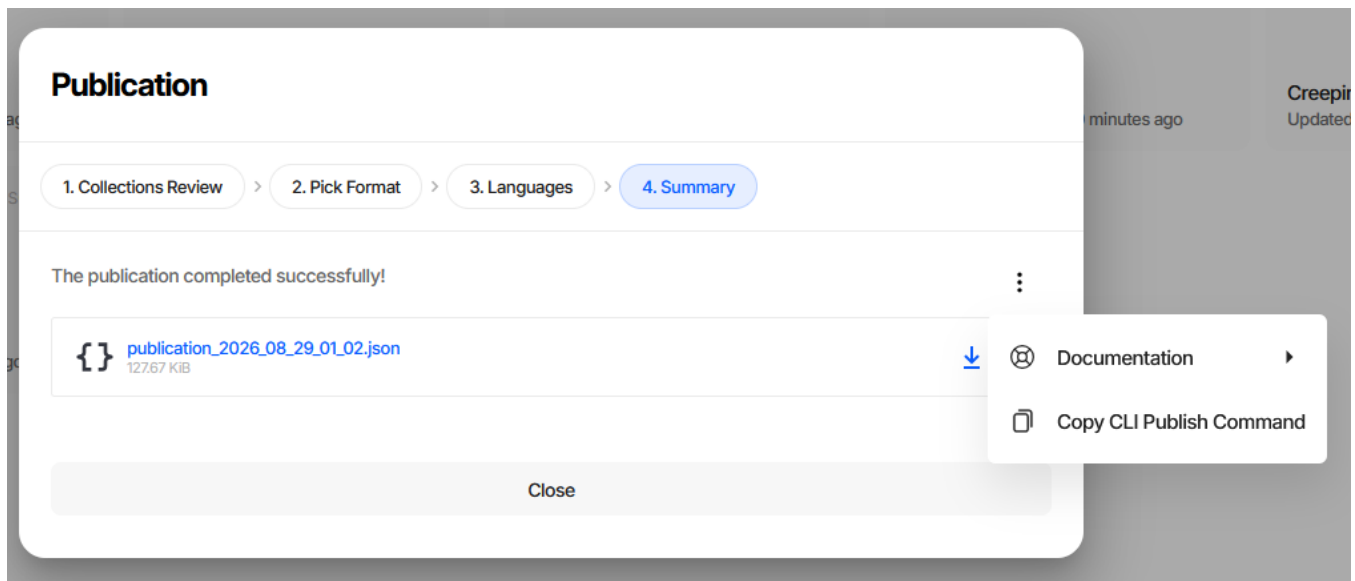
The publication completed successfully!

{ } publication_2026_08_29_01_02.json 127.67 KiB ↓

Close

Is It Free?

The published file is offered as a download, named after the date and time of publication and annotated with its size.

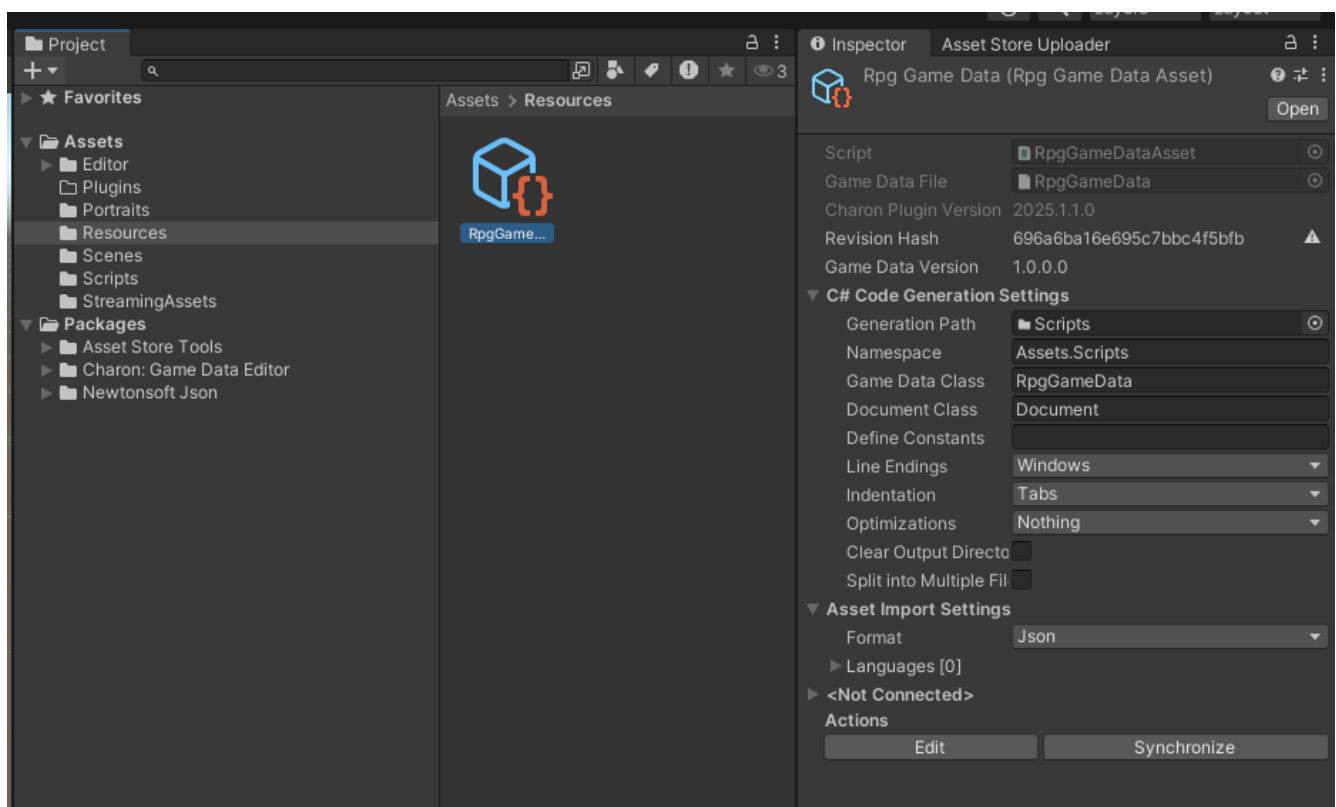


The  menu has **Copy CLI Publish Command**, which puts the exact CLI equivalent of the choices you just made on the clipboard. It is the quickest way to move a publication you configured by hand into a build script.

Publishing from a Game Engine Plugin

The Unity and Unreal Engine plugins embed the same editor and the same wizard. The difference is the last step: instead of handing you a file to download, the plugin imports the published data into the engine's own asset format and regenerates the source code. Step 4 says *"The publication process has been started. Monitor the process in the editor's window"* rather than showing a download link.

Unity



Is It Free?

The published data is imported into the game data .asset file as a BLOB in the format chosen during publication - JSON or Message Pack - and the C# source code is regenerated. At runtime the generated code reads the game data out of that BLOB, so the .asset is the only thing your build needs to ship.

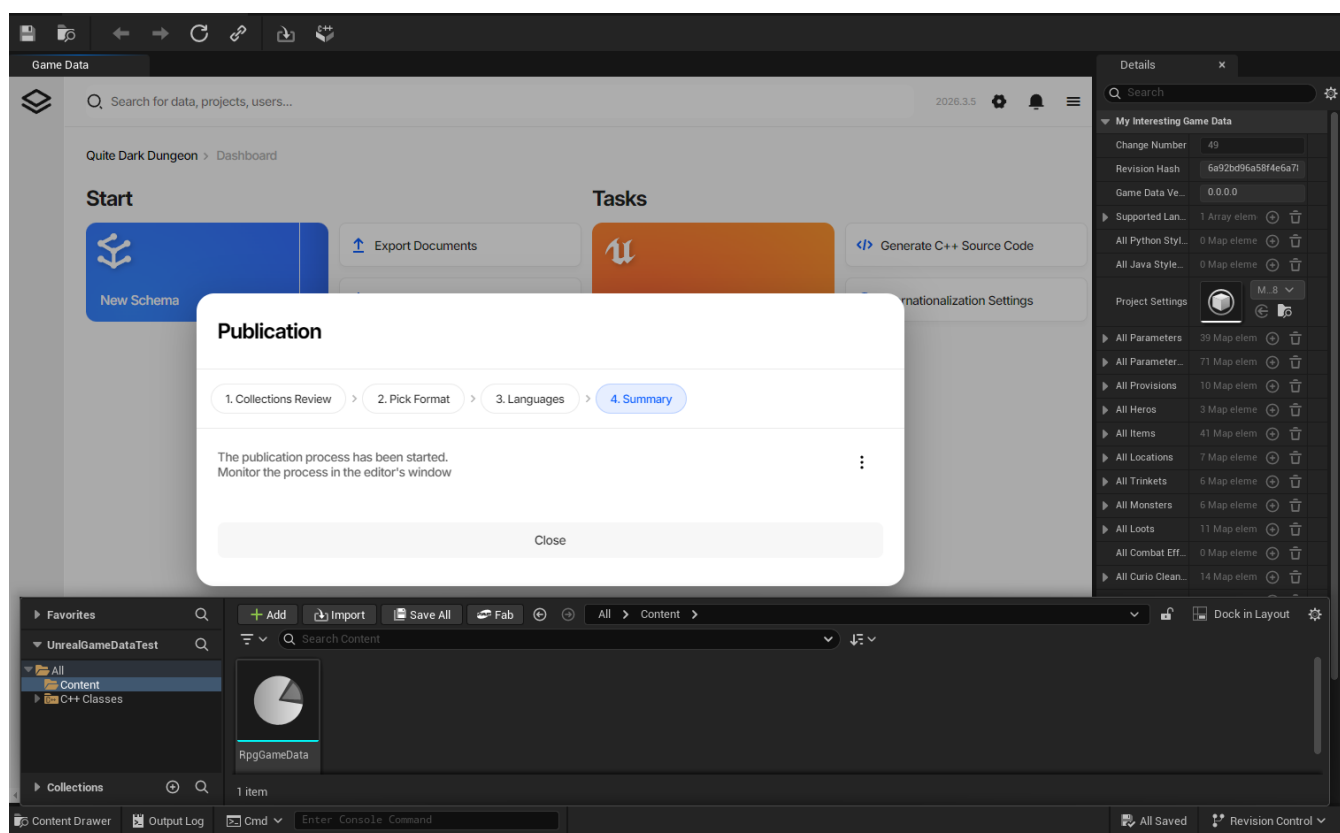
The Inspector for the asset holds the settings the process uses:

- **Asset Import Settings** - Format and Languages, the same two choices as wizard steps 2 and 3.
- **C# Code Generation Settings** - output path, namespace, class names, indentation and the rest of the code generation options.
- **Actions** - **Edit** opens the editor UI, **Synchronize** re-runs publication and code generation with the settings above.

Use **Synchronize** whenever the data structure changes: it republishes the data and regenerates the C# code in one step, which is what keeps the asset and the code in agreement.

See Unity Plugin Overview for installation and the rest of the plugin.

Unreal Engine



The published data is imported into the game data .uasset and stored in Unreal's own binary format - the screenshot shows it in the **Details** panel as ordinary Unreal properties - and the C++ source code is regenerated. Nothing about the intermediate JSON survives into the packaged game.

Because the data is converted into Unreal's representation on import, the format step of the wizard has nothing to decide and is disabled; the language selection still applies.

See Unreal Engine Plugin Overview for installation and the rest of the plugin.

Where the data ends up

Target	Output	Stored as	Format choice
Web editor	downloaded file	JSON or MessagePack file	yours
Unity	game data .asset	BLOB in the chosen format	yours

Unreal Engine	game data .uasset	Unreal's binary format	fixed
---------------	-------------------	------------------------	-------

In every case the source code is generated from the same schema, so game code written against it does not change when you switch targets.

Publishing from the CLI

Publication is DATA EXPORT with `--mode publication`:

```
dnx dotnet-charon -- DATA EXPORT \
  --dataBase ".\gamedata.json" \
  --mode publication \
  --languages {en-US} \
  --output ".\StreamingAssets\gamedata_pub.json" \
  --outputFormat json
```

Key parameters:

- `--mode publication` - keeps the system schemas and the project settings document that the generated code needs, and strips localization for the languages you did not select. Without it the export omits the system schemas and the generated code cannot load it.
- `--languages` - the languages to include, in braces. Omit the parameter to publish all languages.
- `--outputFormat` - json or msgpack, matching the wizard's format step.

Tip

Rather than assembling this by hand, configure the publication once in the wizard and use **Copy CLI Publish Command** on the Summary step.

Publishing on every commit is the usual way to keep a build pipeline in sync with the data; see CI/CD for a worked example, and DATA EXPORT for the other export modes.

See also

- Validation
- Generating Source Code
- Working with Source Code (C# 4.0)
- Working with Source Code (C# 7.3)
- Working with Source Code (TypeScript)
- Unity Plugin Overview
- Unreal Engine Plugin Overview
- Command Line Interface (CLI)
- DATA EXPORT Command
- DATA UPDATEPROJECTSETTINGS Command

Importing and Exporting Data

Export writes documents out of a project into a file; import reads them back in. The round trip is how game data leaves the editor for version control, bulk edits in external tools, or a script, and how externally produced data gets in.

Both directions are wizards in the editor, and both have a CLI equivalent for automation. See CI/CD for pipeline examples.

Note

Export is not publication. An export is a full-fidelity copy that Charon can read back. A publication strips everything the game does not need and is meant for the generated code, not for re-import.

Where Import and Export Live

159

Exporting Data

160

Step 1. Select Collections

160

Step 2. Pick Format

161

Step 3. Summary

161

Quick Export Without the Wizard

162

Importing Data

162

Step 1. Import Mode

162

Step 2. From

163

Step 3. File Input / Clipboard Input

163

Step 4. Select Collections

163

Step 5. Summary

164

Faster Paths

165

Validation During Import

166

Working with Exported Data

167

Modifying Exported Data

167

Creating Data from Scratch

167

Document Identification

167

Partial Updates

167

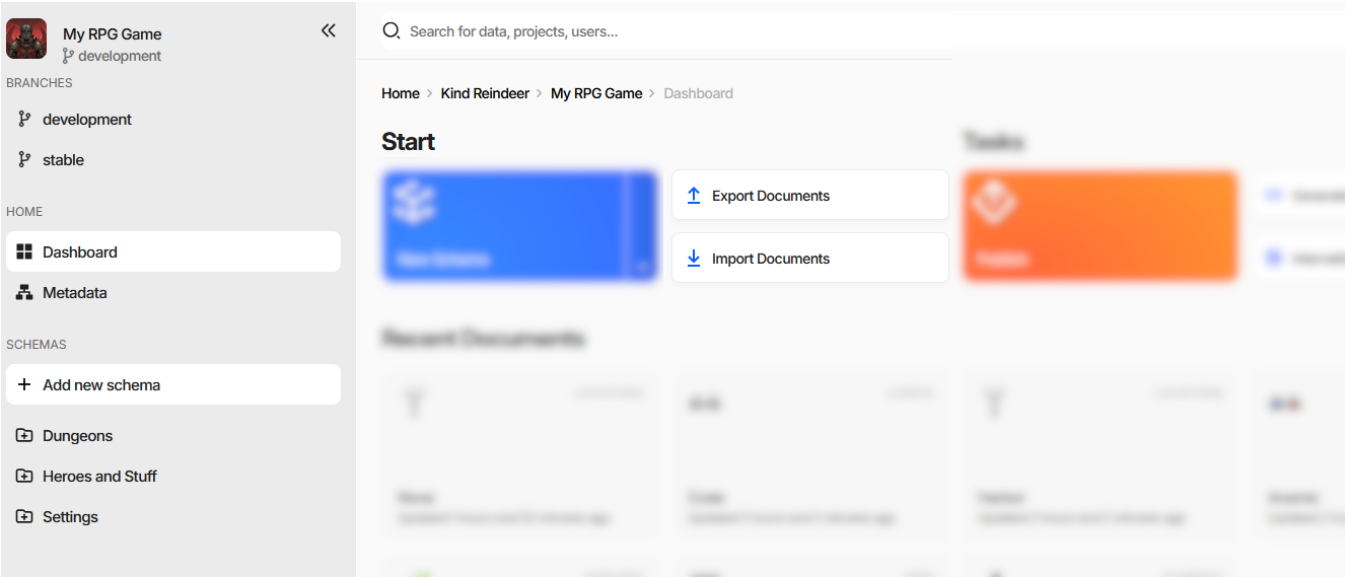
See also

167

Where Import and Export Live

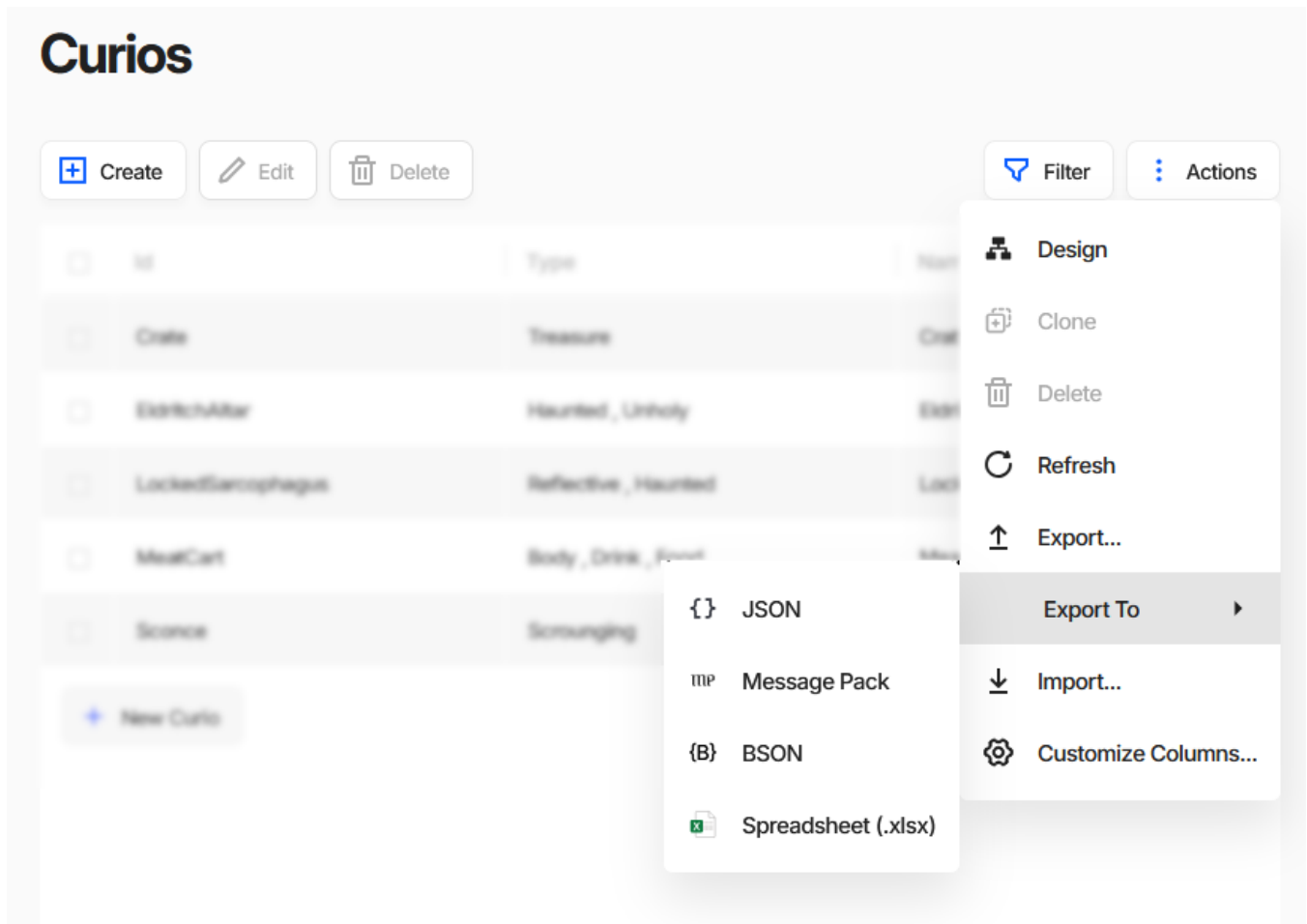
Both operations have two entry points.

The **Start** panel on the dashboard works on the whole project:



Export Documents and **Import Documents** open the wizards with nothing preselected, so you choose the collections inside. **Import Documents** is disabled when the data source is read-only.

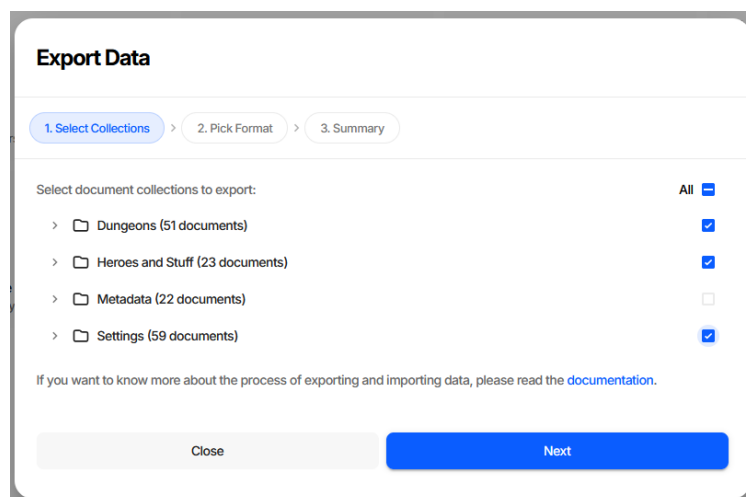
A document list has the same two operations scoped to one collection, in its **Actions** menu:



Export... and **Import...** open the same wizards with the current collection already selected. **Export To** skips the wizard entirely - see [Quick Export Without the Wizard](#).

Exporting Data

Step 1. Select Collections



Collections are shown as a tree, grouped into folders, each with its document count. The **All** checkbox at the top toggles everything, and shows a dash when the selection is partial. Expand a folder to pick individual collections. At least one is required to continue.

The **Metadata** folder holds the project's schema definitions. Leave it ticked when the file should describe its own structure, untick it to export documents alone.

Step 2. Pick Format

Export Data

1. Select Collections

2. Pick Format

3. Summary

Specify the data format you want to export to:

 JSON(recommended) ☒

 Message Pack ☐

 BSON ☐

 Spreadsheet (.xlsx) ☐

Close

Export

Format	When to use
JSON (<i>recommended</i>)	Default. Readable, diffable, and accepted by every external tool. Keep it unless you have a reason not to.
Message Pack	Compact binary. Smaller and faster to parse, not human-readable.
BSON	Binary JSON, for pipelines that already speak it.
Spreadsheet (.xlsx)	One sheet per collection, for review or editing by people who do not work in JSON. Round-trips back through import.

Step 3. Summary

Export Data

1. Select Collections

2. Pick Format

3. Summary

The export completed successfully!

 export_2026_08_29_01_51.json

81.08 KIB



Close

The file is offered as a download, named `export_<year>_<month>_<day>_<hour>_<minute>.<extension>` and annotated with its size. The download arrow next to it turns into a green check once you have taken it.

The **■** menu holds **Copy CLI Export Command**, which puts the exact CLI equivalent of the choices you just made on the clipboard, and a **Documentation** submenu linking to this page and the command reference.

Tip

The wizard remembers the collections and format you picked last time, so a repeated export is **Next, Next, Export**.

CLI equivalent

```
dnx dotnet-charon -- DATA EXPORT \
  --dataBase "gamedata.json" \
  --schemas Item Character \
  --output "items.json" \
  --outputFormat json
```

Key parameters:

- `--schemas` - collections to export. Supports wildcards (*Item*) and exclusion (!Deprecated). Omit it to export everything.
- `--properties` - narrows the export to specific properties. There is no equivalent in the wizard, which always exports whole documents.
- `--outputFormat` - json, bson, msgpack or xlsx.

See DATA EXPORT for the full parameter list.

Quick Export Without the Wizard

Export To in a collection's **Actions** menu downloads that collection immediately in the chosen format - JSON, Message Pack, BSON or Spreadsheet - with no steps to click through. It is the fast path when you just want the current collection on disk.

Importing Data

Step 1. Import Mode

Import Data

1. Import Mode > 2. From > 3. Input > 4. Select Collections > 5. Summary

Specify how imported data should be applied to your project:

Create and Update	Create Only
Update Only	Safe Update
Replace All	Delete

☐ Ignore consistency errors in imported data.

☐ Perform a dry run and don't persist changes.

If you want to know more about the process of importing and exporting data, please read the [documentation](#).

Close Next

The mode decides how incoming documents meet the ones already in the project.

Mode	--mode value	Effect
Create and Update	createAndUpdate	Default. Creates documents that do not exist yet, updates the ones that do.
Create Only	create	Creates new documents. Existing documents are never touched.
Update Only	update	Updates existing documents. Nothing new is added.
Safe Update	safeUpdate	Updates existing documents without adding, removing or reordering embedded documents. The mode to use when the document structure must survive intact - imported translations, for example.
Replace All	replace	Replaces the collection with the imported documents. Everything currently in it is removed.
Delete	delete	Deletes the documents listed in the file. Nothing is created or updated.

Warning

Replace All removes every document in the selected collections that is not in the file. Run it as a dry run first.

Two checkboxes on this step change how the import behaves:

- **Ignore consistency errors in imported data** - drops the integrity checks and keeps only the repairs. See [Validation During Import](#) for exactly what this trades away.
- **Perform a dry run and don't persist changes** - runs the entire import, produces the full report, and writes nothing. Worth doing before any import you have not run before.

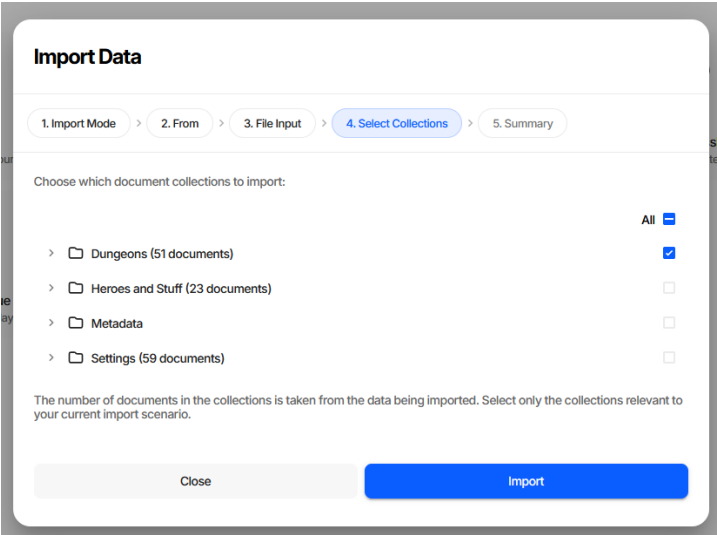
Step 2. From

Choose **File** to pick a file from your device, or **Clipboard** to paste JSON text directly into the wizard.

Step 3. File Input / Clipboard Input

For **File**, browse to the file. Accepted formats are JSON (.json), Message Pack (.msgpack), BSON (.bson) and Spreadsheet (.xlsx); anything else is rejected on this step with an *Unknown file format* error. For **Clipboard**, paste a JSON object into the text area - it is checked as valid JSON when you leave the field.

Step 4. Select Collections



Import Data

1. Import Mode > 2. From > 3. File Input > **4. Select Collections** > 5. Summary

Choose which document collections to import:

☒ All

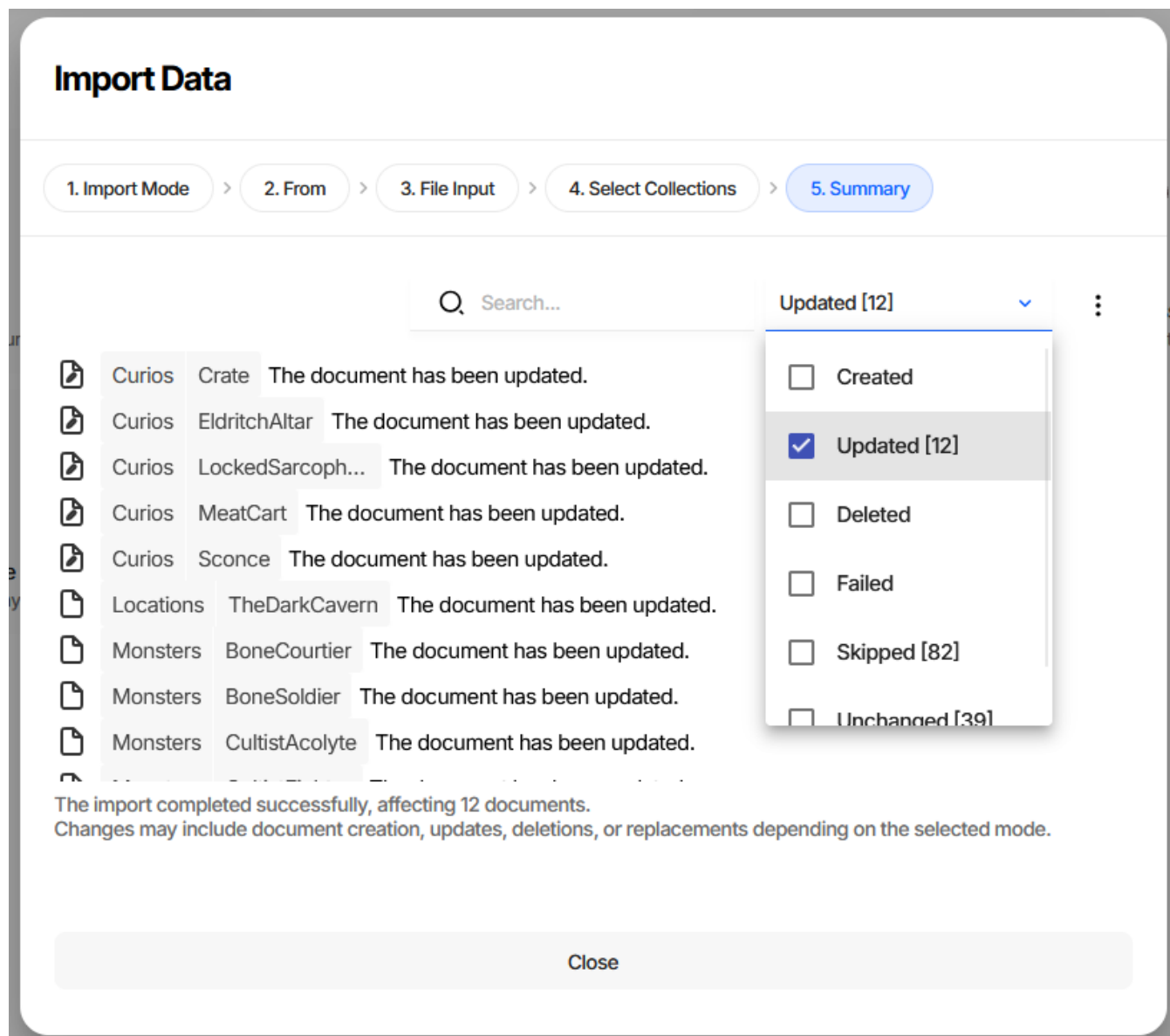
- > ☒ Dungeons (51 documents)
- > ☐ Heroes and Stuff (23 documents)
- > ☐ Metadata
- > ☐ Settings (59 documents)

The number of documents in the collections is taken from the data being imported. Select only the collections relevant to your current import scenario.

The same collection tree as in the export wizard, with one difference worth knowing: the document counts come from **the file being imported**, not from the project. Charon reads the file's structure to build this list, which doubles as a sanity check - in the screenshot above, **Metadata** carries no count because the file contains no schema definitions.

If the file cannot be read, this step shows *Failed to peek into imported data* with the server error, plus **Retry** and **Ignore and Continue**. Continuing is valid - the import itself may still succeed - but you lose the ability to narrow the collection list.

Step 5. Summary



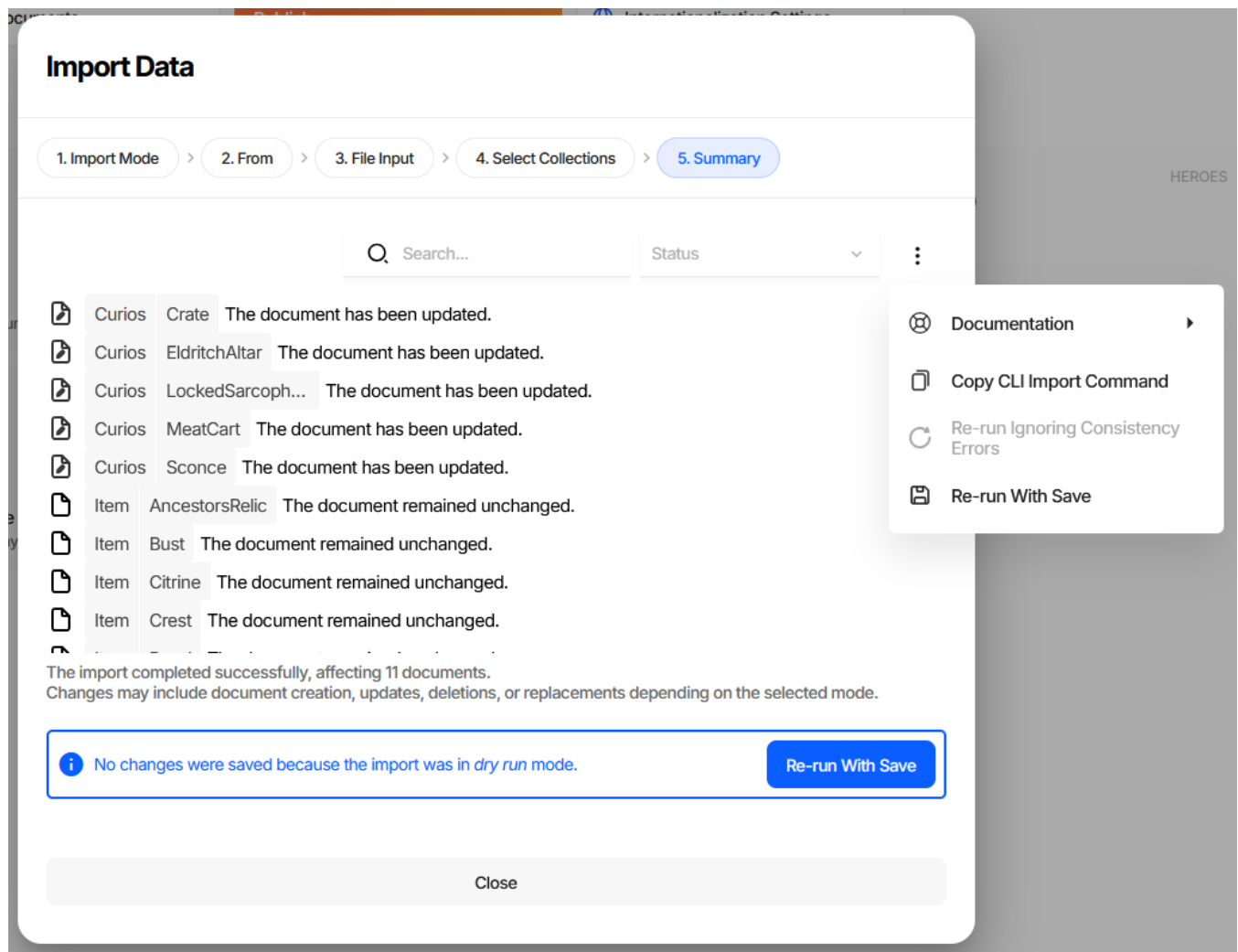
Every affected document is listed with its status, searchable by name and filterable by status. The filter carries counts, so the shape of the import is visible before you read a single row - the screenshot above is filtered to the 12 updated documents out of a run that also skipped 82 and left 39 unchanged.

Status	Meaning
Created	The document did not exist and was added.
Updated	An existing document was changed.
Deleted	The document was removed - delete or replace mode.
Failed	The document was rejected. The row carries the reason.
Skipped	The mode did not apply to this document - a new document in update mode, for instance.

Unchanged	The document matched what was already stored.
-----------	---

An import that finishes entirely as *Unchanged* usually means the file matched the project exactly, not that something went wrong.

After a dry run the report is the same, with a banner making clear that nothing was written and a **Re-run With Save** button to commit the same import for real:

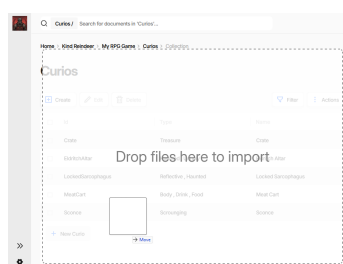


The  menu offers:

- **Copy CLI Import Command** - the CLI equivalent of this import, ready for a build script.
- **Re-run Ignoring Consistency Errors** - retries without integrity checks. Available only when the run actually produced failures and the checks were on, which is why it is greyed out above.
- **Re-run With Save** - re-runs a dry run for real. Available only after a dry run.

Faster Paths

Dragging a data file onto a document list - the grid turns into a *Drop files here to import* target - opens the import wizard with the file, the collection and **Create and Update** already filled in, so it starts at *Select Collections*:



Pasting a JSON array of documents while a document list is focused does the same with the clipboard text. Neither shortcut is a way to write data without seeing the report - both still finish on the summary step.

CLI equivalent

```
dnx dotnet-charon -- DATA IMPORT \
  --dataBase "gamedata.json" \
  --schemas Item \
  --input "items.json" \
  --inputFormat auto \
  --mode safeUpdate \
  --dryRun
```

Key parameters:

- `--inputFormat` - auto by default, detected from the file extension.
- `--mode` - see the table above.
- `--dryRun` - the CLI equivalent of *Perform a dry run*. Drop it to persist the changes.
- `--output` - where the import report goes. Without it the report is discarded **and the command fails if any document failed to import**, which is what makes an import usable as a CI step.

See DATA IMPORT for the full parameter list.

Validation During Import

Imported documents pass through the same validation engine as documents edited in the UI, but the profile differs between the wizard and the command line - and the two defaults are opposites.

Situation	Checks and repairs applied
Import wizard, default	repair, repairRequiredWithDefaults, and all six integrity checks: checkRequirements, checkFormat, checkUniqueness, checkReferences, checkSpecification, checkConstraints
Import wizard, <i>Ignore consistency errors</i> checked	repair, repairRequiredWithDefaults, deduplicateIds, resolveConflictingUnions - no integrity checks
DATA IMPORT, default	repair, repairRequiredWithDefaults, deduplicateIds, resolveConflictingUnions - no integrity checks

Warning

DATA IMPORT defaults to repairs without integrity checks. A file with dangling references or missing required values is accepted quietly, and the problem surfaces later as an error count in the editor or a failure in the game. Whenever the input is not fully trusted, ask for the checks explicitly:

```
dnx dotnet-charon -- DATA IMPORT \
  --dataBase "gamedata.json" \
  --input "items.json" \
  --validationOptions repair repairRequiredWithDefaults \
    checkRequirements checkFormat checkUniqueness \
    checkReferences checkSpecification checkConstraints
```

Following the import with a DATA VALIDATE run achieves the same thing in two steps.

To go the other way in the UI - accept data that fails the checks - tick *Ignore consistency errors in imported data*, or use **Re-run Ignoring Consistency Errors** on the summary once you have seen what failed.

Note

Schema definitions are always validated in full. The relaxed profiles apply to documents only.

See Validation for what each flag does.

Working with Exported Data

Modifying Exported Data

Exported data can be edited by hand or with a script. A single value with `yq`:

```
yq -i '.Collections.Item[0].Damage = 5' items.json
```

The same file can be converted to another format, restructured, or generated from a spreadsheet, as long as what comes back matches the expected shape.

Creating Data from Scratch

Data files can be authored externally and imported, provided they conform to the required structure: a `Collections` object holding an array of documents per schema.

```
{
  "Collections": {
    "Item": [
      { "Id": "BronzeSword", "Name": "Bronze Sword", "Damage": 4 }
    ]
  }
}
```

See Game Data Structure for the full format specification.

Document Identification

- Omit Id and one is generated.
- Provide Id if the document is referenced elsewhere in the data.
- Use a temporary identifier of the form `_ID_<UNIQUE>` when a document needs to be referenced by another document in the same file - including itself, as in a tree node pointing at its own parent. Every occurrence of the same placeholder resolves to the same generated identifier across the whole import.

Partial Updates

In the update modes documents need not be complete. A file containing only the identifier and the changed properties is enough:

```
[
  { "Id": "IronSword", "Damage": 10 }
]
```

Everything not mentioned keeps its current value.

See also

- Validation
- Publishing Game Data

- Internationalization (i18n)
- Game Data Structure
- CI/CD
- DATA EXPORT
- DATA IMPORT

Document Merge Logic

When DATA IMPORT or DATA APPLYPATCH merges incoming (modified) data with the existing (original) data, it follows a set of deterministic rules per property type. Understanding these rules is essential for predicting the outcome of partial updates, patch application, and SafeUpdate operations.

Core Principle	168
Import Mode	168
Property-Level Merge Rules	169
Scalar Properties	169
LocalizedText Properties	169
Inline Document Properties	169
Document Collections	170
The Partial-Document Resurrection Caveat	170
Merge in Context	171
See also	171

Core Principle

Every merge operation takes two document trees - **original** (left) and **modified** (right) - and produces a **merged** result. The fundamental rule is:

- If **modified** provides a value for a property → the merged result uses the modified value.
- If **modified** does not provide a value for a property → the merged result keeps the original value.
- Both absent → the property is omitted from the result.

The rule applies recursively: properties that are themselves documents are merged property-by-property using the same rules, and collections are merged item-by-item using the document's Id as the key.

Note

“Modified does not provide a value” means the property key is absent from the incoming document. A property present in modified with an explicit null value is treated as an intentional deletion, not a missing value.

Import Mode

The merge behaviour is governed by the **import mode** passed to the operation. Two modes affect merge in particular:

- createAndUpdate (**default**)
Modified values always win. Items absent from the modified collection are treated as deleted. This is the mode used by DATA APPLYPATCH.
- safeUpdate

A restricted mode that prevents deletions. Items absent from the modified collection are left unchanged. The `Id` field of a document is never overwritten regardless of what modified supplies.

create and update control *which* documents are eligible (insert-only / update-only respectively), but once a document is selected the merge rules below apply exactly as in `createAndUpdate`.

delete does not merge at all - the matched document is removed outright.

replace does not merge against the existing document either: the modified document is always merged against an **empty** document (only for schema normalization), so any property the modified document omits is **not** carried over from the original - it is simply absent from the result. This is a genuine whole-document replacement, not a property-preserving merge.

Property-Level Merge Rules

Scalar Properties

Applies to: Text, Time, Date, Logical, Reference, Formula, Number, Integer, PickList, MultiPickList.

If modified supplies a value, it overwrites the original unconditionally. If modified does not supply a value, the original is kept.

Special case - ``Id`` in SafeUpdate mode: The `Id` property is never overwritten when the import mode is `safeUpdate`. Even if modified provides an `Id` value, the original `Id` is preserved. This prevents accidental re-identification of documents when patching subsets of fields.

LocalizedText Properties

LocalizedText values are merged **per language** rather than replaced wholesale:

- For each language configured in Project Settings:
 - If modified contains a value for that language → use the modified value.
 - If modified does not contain that language key → keep the original value for that language.
- If modified cannot be interpreted as a LocalizedText at all → the result is `null`.
- If original cannot be interpreted as a LocalizedText (e.g. property was empty before) → the modified value is used as-is with no per-language merging.

Translation notes are merged separately:

- The **comment** field: modified wins if it has a comment; otherwise the original comment is kept.
- Stale translation flags:** the union (OR) of both sets - a language that was stale in either original or modified remains stale. Stale flags are then automatically cleared for any language whose value was explicitly supplied by modified **and differs from the original value**. If modified re-supplies the same text as original, the existing stale flag for that language is left untouched.

Inline Document Properties

A property of type Document (an embedded sub-document) is merged recursively using the same rules applied to top-level documents.

The four cases:

Original	Modified	Result
absent / null	document	Merge(empty, modified) - see caveat
document	absent / null	null (deleted), or original in safeUpdate
document	document	Recursive merge, property by property
absent / null	absent / null	Property omitted from result

Document Collections

`DocumentCollection` and `ReferenceCollection` properties are merged **by document Id**. The set of Ids to iterate depends on the mode:

- **createAndUpdate** - iterates the **modified** Ids. Documents present in original but absent from modified are treated as deleted and dropped from the result.
- **safeUpdate** - iterates the **original** Ids. Documents present in original but absent from modified are left unchanged. Items in modified not in original are ignored.

Per-item resolution:

Original item	Modified item	Result
exists	absent (id not listed)	Kept unchanged
absent	exists	Created (added to result)
exists	null / not a doc	Deleted (dropped), kept in safeUpdate
exists	exists	<code>DocumentCollection</code> : recursive merge <code>ReferenceCollection</code> : modified wins

For `ReferenceCollection` items (which store a reference object, not full data), when both sides exist the modified reference object replaces the original without recursion - there are no meaningful sub-properties to merge.

Null optional collections

If modified explicitly supplies null for an optional (non-required) collection property, the result is null regardless of what original contained. In `safeUpdate` mode, an empty modified collection means “no changes”, not “clear the collection”.

The Partial-Document Resurrection Caveat

A subtle but important edge case arises when:

1. A document was **deleted** from the original (original is null or absent).
2. A patch was created that **adds or modifies** that document (modified contains a partial document - only the properties that changed).
3. `DATA APPLYPATCH` (`createAndUpdate` mode) applies the patch.

In this case the merge calls `Merge(empty, partialModified, schema)`. Since `empty` provides no values, every property takes its value from `partialModified` where present - and is simply **omitted** where modified does not supply it.

The resulting document exists in the data, but **only contains the properties included in the patch**. Required properties that were absent in the original document - and were not listed in the patch - will be missing from the result.

Example:

Original `Item/IronSword` was deleted. A patch contains:

```
{
  "Collections": {
    "Item": {
      "IronSword": { "Id": "IronSword", "Damage": 20 }
    }
  }
}
```

If the `Item` schema has a required `Name` field, the merged document will be:

```
{ "Id": "IronSword", "Damage": 20 }
```

Name is absent. Subsequent validation (checkRequirements) will report this as an error.

How to avoid it:

- When generating a patch for a *new* document, include **all required fields** in the patch, not only the diff.
- When applying a patch that resurrects deleted documents, validate right after applying it - DATA APPLYPATCH has no --validationOptions parameter of its own, so the check is a separate step:

```
dnx dotnet-charon -- DATA APPLYPATCH \
  --dataBase target.json \
  --input changes.patch.json
```

```
dnx dotnet-charon -- DATA VALIDATE \
  --dataBase target.json \
  --validationOptions repair checkRequirements checkReferences \
  --output err
```

- If the resurrection is intentional but the document is expected to be incomplete, skip the validation step and fix the document via a follow-up import.

Merge in Context

The merge logic is invoked by:

- **``DATA IMPORT``** - for every document that matches both the import input and the target data (matched by Id). The mode is set by --mode.
- **``DATA APPLYPATCH``** - always runs in createAndUpdate mode. The patch format uses null entries to express deletions, which interact directly with the merge rules described above.
- **``DATA UPDATE`` (single document)** - runs in createAndUpdate mode on a single document.

See also

- Patch and Diff Workflow
- Importing and Exporting Data
- Validation
- DATA IMPORT Command
- DATA APPLYPATCH Command

Internationalization (i18n)

Charon supports localization by enabling text data to be stored in multiple languages using the Localized Text data type. This allows game data to be exported for translation, modified externally, and imported back after localization.

Supported interchange formats are [XLIFF](#) (XML Localization Interchange File Format) version 1 and 2, [XLSX](#) spreadsheets and [JSON](#). These formats are suitable for translation workflows involving external tools or localization teams.

Overview

Everything related to translation lives on one page: **Settings** → **Internationalization**. It has three sections, which also correspond to the three stages of a localization workflow:

- **Internationalization (i18n)** - which languages the project has.
- **Localization (l10n)** - export text for translators, import it back.

- **Machine Translation** - optional automatic pre-translation, when the feature is enabled.

Every action on this page has a CLI counterpart except machine translation, so the same workflow can be automated in a CI/CD pipeline. See CI/CD.

Common use cases include:

- Managing multi-language game content.
- Sending text data to third-party localization vendors.
- Automating translation pipelines with CLI tools.

Configuring Languages

Open **Settings** → **Internationalization** and use the first card to declare the project's languages.

Home > Kind Reindeer > My RPG Game > Settings > Internationalization

The screenshot shows the 'Internationalization (i18n)' settings page for a project named 'My RPG Game' in a 'development' environment. On the left is a sidebar with navigation links: General, Backup, Publication, Extensions, Features, Source Code, Internationalization (highlighted), Branches, and Members. The main content area has a title 'Internationalization (i18n)'. Below the title, there is a 'Primary Language *' section with a dropdown menu currently set to 'English (United States) [en-US]'. A note below this states: 'It is used as the display language in UI and as the source language during translation export/import.' Underneath is a 'Translation Languages' section with a list containing 'Spanish (Spain)' with a close button (X) and a text input field with the placeholder 'Translation languages..'. At the bottom of this section is a disabled 'Update Languages' button.

- **Primary Language** - the language content is authored in. It is the display language in the UI and the default source language for translation export and import.
- **Translation Languages** - the target languages. Start typing in the field and pick a language from the autocomplete; remove one with the cross button on its chip.
- **Update Languages** - saves the change. The button stays disabled until something is modified.

Adding a language does not create any text - it makes the language available as a slot on every Localized Text field and as a target in the export, import and machine translation wizards.

Note

Removing a translation language hides its text from the UI. Export the language before removing it if the translated content still matters.

CLI equivalent

Is It Free?

List the configured languages (the primary language always comes first):

```
dnx dotnet-charon -- DATA I18N LANGUAGES \
  --dataBase "gamedata.json" \
  --output out --outputFormat table
```

Add target languages:

```
dnx dotnet-charon -- DATA I18N ADDLANGUAGE \
  --dataBase "gamedata.json" \
  --languages "es-ES" "fr-FR"
```

Language values are [BCP 47](#) tags, e.g. en-US, es-ES, fr.

Localization (I10n)

The **Localization (I10n)** card is the entry point for the export/import round trip.

Home > Kind Reindeer > My RPG Game > Settings > Internationalization

The screenshot shows the 'Internationalization (i18n)' settings page for a project named 'My RPG Game' in 'development' mode. On the left is a sidebar with navigation links: General, Backup, Publication, Extensions, Features, Source Code, Internationalization (highlighted), Branches, and Members. The main content area has a title 'Internationalization (i18n)' and a blurred form with fields for 'Primary language' and 'Target languages'. Below this is a section titled 'Localization (I10n)' with a description: 'Localization involves extraction of all text that needs to be translated within the game game data, and import the localized text back into the game data once translations are complete.' It also includes a warning: 'Make sure that the structure of the documents hasn't changed much between export and import otherwise some of the translated data may be lost.' At the bottom are two blue buttons: 'Export Text' and 'Import Text'.

- **Export Text** - collects every localizable text entry and writes it to a file for translators.
- **Import Text** - reads a translated file back into the game data.

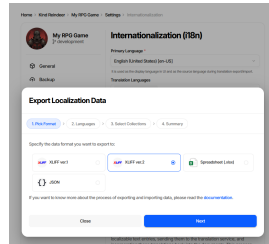
Warning

Keep the document structure stable between export and import. If schemas and documents change significantly in the meantime, some translated entries can no longer be matched and will be lost.

Exporting Translatable Text

Click **Export Text** to open the **Export Localization Data** wizard.

Step 1. Pick Format



Format	When to use
XLIFF ver.2	Default choice for professional translation tools and vendors.
XLIFF ver.1	Older tools that do not accept XLIFF 2.
Spreadsheet (.xlsx)	Translators working directly in Excel or Google Sheets.
JSON	Custom scripts and in-house translation pipelines.

Step 2. Languages

Pick the **Source Language** (the text translators will read, usually the primary language) and one or more **Target Languages**. Existing translations for the target languages are included in the file, so translators see what is already done.

Step 3. Select Collections

Limit the export to specific document collections, or leave everything selected to export all localizable text. Narrowing the scope keeps vendor files small when only part of the content changed.

Step 4. Summary

Review the choices and start the export. The resulting file is downloaded through the browser.

CLI equivalent

```
dnx dotnet-charon -- DATA I18N EXPORT \
  --dataBase "gamedata.json" \
  --sourceLanguage en-US \
  --targetLanguage fr \
  --output "en_fr_texts.xliff" \
  --outputFormat xliff
```

Key parameters:

- --sourceLanguage - language of the original text.
- --targetLanguage - language the content is translated into.
- --schemas - optional list of schemas, matching the *Select Collections* step.
- --outputFormat - xliff (alias of xliff2), xliff1, xslx or json.

See DATA I18N EXPORT for the full parameter list.

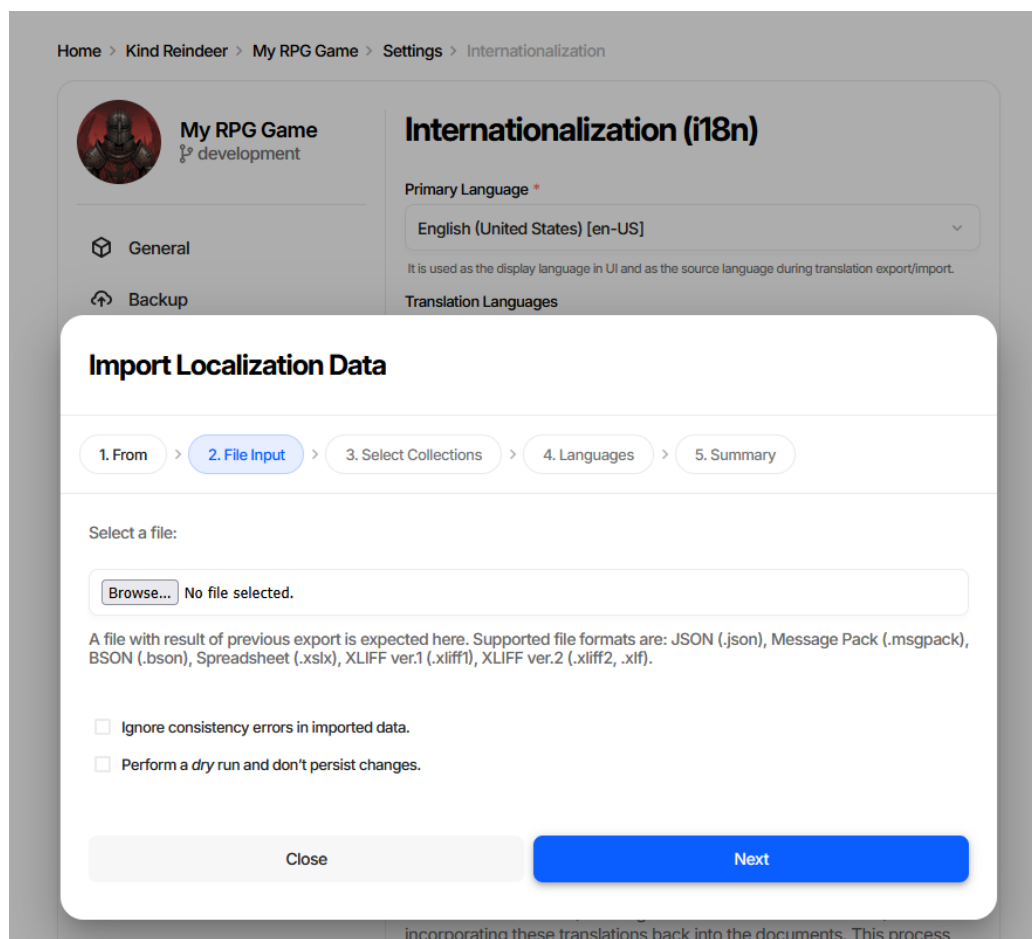
Importing Translated Data

Click **Import Text** to open the **Import Localization Data** wizard.

Step 1. From

Choose where the translated data comes from - a **File** on your device, or the **Clipboard** for pasted JSON text.

Step 2. File Input



Browse for the file returned by the translators. Accepted formats are JSON (.json), MessagePack (.msgpack), BSON (.bson), Spreadsheet (.xlsx), XLIFF ver.1 (.xliff1) and XLIFF ver.2 (.xliff2, .xlf).

Two options are available on this step:

- **Ignore consistency errors in imported data** - accepts data that fails validation. Useful for work-in-progress translations.
- **Perform a dry run and don't persist changes** - runs the whole import and reports what would happen without writing anything. Always worth doing before the first import of a vendor's file.

Step 3. Select Collections

Restrict the import to specific collections, if the file covers more than you want to update.

Step 4. Languages

Import Localization Data

1. From > 2. File Input > 3. Select Collections > **4. Languages** > 5. Summary

Choose languages to import:

☐ English (United States)

☒ Spanish (Spain)

☐ Translation Notes

The language list is taken from the data being imported. Select only the languages relevant to your current import scenario.
New translation languages will be added to Project's settings upon import completion.

Close Import

The language list is read from the file being imported, not from project settings. Tick only the languages relevant to this import - typically the target language alone, leaving the source language untouched. **Translation Notes** carries translator comments back into the data. Languages that are not yet in project settings are added automatically when the import completes.

Step 5. Summary

Review and run the import.

CLI equivalent

```
dnx dotnet-charon -- DATA I18N IMPORT \  
--dataBase "gamedata.json" \  
--input "en_fr_translated.xliff" \  
--inputFormat xliff \  
--languages fr \  
--dryRun
```

Key parameters:

- --inputFormat - auto by default, detected from the file extension.
- --languages - languages to import; * imports all languages found in the file.
- --validationOptions None - the CLI equivalent of *Ignore consistency errors*.
- --dryRun - the CLI equivalent of *Perform a dry run*. Drop it to persist the changes.

See DATA I18N IMPORT for the full parameter list.

Machine Translation

Machine translation pre-populates target language fields as a **first draft** for translators to review. It speeds up the initial pass on large amounts of text but does not replace professional translation for final copy.

The feature is available in the cloud and server editions and is enabled per-project by an Administrator. When it is off, the **Machine Translation** card is not shown.

Translating the Whole Project

Home > Kind Reindeer > My RPG Game > Settings > Internationalization

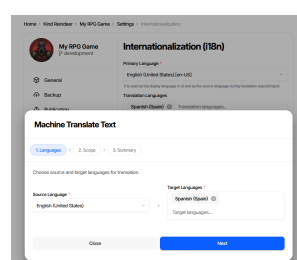
The screenshot shows the 'Internationalization (I18n)' settings page for a project named 'My RPG Game'. On the left is a sidebar with navigation links: General, Backup, Publication, Extensions, Features, Source Code, Internationalization (highlighted), Branches, and Members. The main content area has a header 'Internationalization (I18n)' and a description of the feature. Below the description is a large blue button labeled 'Translate Text' with a translation icon.

Machine Translation

Machine Translation (MT) uses software to automatically translate text between languages. The process involves scanning the documents for all localizable text entries, sending them to the translation service, and incorporating these translations back into the documents. This process may take some time.

[Translate Text](#)

Translate Text opens the **Machine Translate Text** wizard.



Step 1 selects the **Source Language** and **Target Languages**. Step 2 selects the scope:

Scope	Effect
Translate all texts	Overwrites every target text, including reviewed translations.
Translate stale texts	Retranslates entries whose primary language text changed after the translation was made, plus missing ones.
Translate only missing texts	Fills empty entries only, never touching existing translations.

Step 3 shows progress. A project-wide translation scans all documents and may take a while; the progress bar on the settings card reflects the running operation.

Translating a Single Document

The same three scopes are available for one document from the document form's **Actions** menu → **Translate Texts**.

The screenshot shows the Charon document editor interface for a document titled 'Crossbower'. The breadcrumb trail is 'Home > Kind Reindeer > My RPG Game > Heroes > Crossbower'. The document form includes fields for 'Id' (Crossbower), 'Name' (Crossbower), 'Bio' (Enveloped in weighty armor and capable of obliterating adversaries with...), 'Religious' (No), 'Dislike Heroes' (Search for Dislike Heroes...), and 'Provisions'. The 'Actions' menu is open, showing options: Design, Create New, Clone, Translate Texts (highlighted), Delete, Undo, Redo, and View Mode. A tooltip for 'Translate Texts' explains: 'Translate all localizable fields in the document to all available languages where translation text is missing.' The 'Translate all texts' option is also visible in the background.

This is the fast path when a designer edits one document's primary text and wants the other languages refreshed immediately.

Note

Machine translation has no CLI command. For automated pipelines, use the export/import workflow with an external translation service.

Working with Translation Teams

The standard human-translation workflow with Charon:

1. Designers fill the **primary language** for all content.
2. Optionally run **Machine Translation** with *Translate only missing texts* to give translators a draft to edit instead of a blank file.
3. **Export Text** in XLIFF ver.2 for each target language, or run:

```

dnx dotnet-charon -- DATA I18N EXPORT \
  --dataBase "gamedata.json" \
  --sourceLanguage en-US --targetLanguage fr \
  --output en_fr.xliff --outputFormat xliff

```

4. Send the .xliff file to your localization vendor or translation management system (Crowdin, Lokalise, POEditor, or any XLIFF-compatible tool).
5. Receive the translated file back from the vendor.
6. **Import Text** with *Perform a dry run* first, then again without it, or run:

```

dnx dotnet-charon -- DATA I18N IMPORT \
  --dataBase "gamedata.json" \
  --input en_fr_translated.xliff \
  --languages fr

```

7. Repeat for each target language. CI/CD scripts can automate steps 3 and 6 for continuous localization pipelines.

Best Practices

- Use **XLIFF** for integration with professional translation software or localization platforms.
- Use **Spreadsheet (.xlsx)** when working with translators familiar with spreadsheet tools.
- Always run a **dry run** before importing a vendor's file into production data.
- Export a language before removing it from **Translation Languages**.
- Keep schema changes and translation rounds apart - restructuring documents between export and import loses translated entries.

Unsupported Formats

While Charon may accept other serialization formats (e.g., BSON, MsgPack) on import, compatibility for internationalization workflows is only guaranteed for XLIFF, XLSX and JSON.

Additional Resources

- Importing and Exporting Data
- DATA EXPORT
- DATA IMPORT
- DATA I18N EXPORT
- DATA I18N IMPORT
- DATA I18N LANGUAGES
- DATA I18N ADDLANGUAGE

Validation

Charon validates game data at several points in the editing lifecycle: while a document is being edited, on every save, on import, before publication, and on demand via DATA VALIDATE. The editor surfaces problems where you will run into them; the CLI exposes the same engine through a flag system you can tune for bulk workflows such as CI pipelines or large data migrations.

Where Validation Appears in the Editor

Errors in the Document Form	180
Finding Documents with Errors	181
Reviewing Before Publication	182

When Validation Runs	184
Validation Flags	184
Integrity check flags	184
Repair flags	185
Validation Report Format	185
Exit Code Behaviour	186
Common Recipes	186
Skip all checks (import prototype data)	186
Auto-repair + full check on import	186
Fix union conflicts after schema refactor	186
Check for untranslated strings	186
Gate a build on a clean publication review	186
Dry Run vs Validate	187
See also	187

Where Validation Appears in the Editor

Three places, in the order you normally meet them:

- **The document form** - per-field errors while editing and after a rejected save.
- **A collection's error view** - the list of documents in one collection that currently have errors.
- **The Publication wizard** - a project-wide review of every collection before you ship.

Errors in the Document Form

Crate

Save

Discard Changes

English (United States)

Actions

Id*

Crate

Type*

Treasure

Name

Crate

Missing translation for: Spanish (Spain)

Description

English (United States)

A value is required.

Cleansing Options

> Chance: 75% Loot: Heirloom v1 None

Two kinds of message appear under a field, and they behave differently:

- **Red - an error.** A value is required. in the screenshot. The server rejects the save until it is fixed, or until you explicitly bypass validation.
- **Orange - a warning.** Missing translation for: Spanish (Spain). Checked in the browser only, never blocks a save. See Internationalization (i18n) for filling these in.

When a save is rejected, the **Save** button changes to **Save Anyway** and shows a badge with the number of errors. Clicking it again writes the document with checks turned off - repairs still run, but nothing is reported. Use it for work in progress you intend to fix later; the document keeps its error count and shows up in the views below until it is fixed.

Keyboard shortcuts on the Save button:

Shortcut	Action
Ctrl + S	Save
Ctrl + click	Save & close
Shift + click	Save ignoring errors

Saving a document runs `repair`, `repairRequiredWithDefaults` and all six integrity checks. It does **not** run `checkTranslations` - that is why a missing translation is only a warning here.

Finding Documents with Errors

Every collection has a built-in errors view: the document list with an **Error Count** column, filtered to `Error Count > 0`.

>
2026.3.5

[Home](#) > [Kind Reindeer](#) > [My RPG Game](#) > [Diseases](#) > [Collection](#)

Diseases

+ Create
Edit
Delete
Filter
Actions

☐	Id	Display Name	Error Count
☐	BadHumors	Bad Humors	1
☐	CreepingCough	Creeping Cough	1
☐	Ennui	Ennui	1
☐	Hemophilia	Hemophilia	1
☐	Rabies	Rabies	1
☐	TheRuns	The Runs	1

+ New Disease

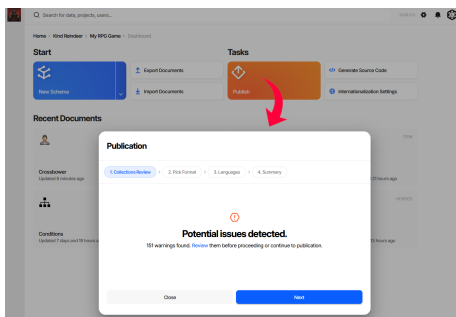
Is It Free?

The fastest way to reach it is the Publication wizard - clicking a collection in the review tree opens that collection already filtered this way. The filter chip stays visible in the search bar, so you can clear it to return to the full list.

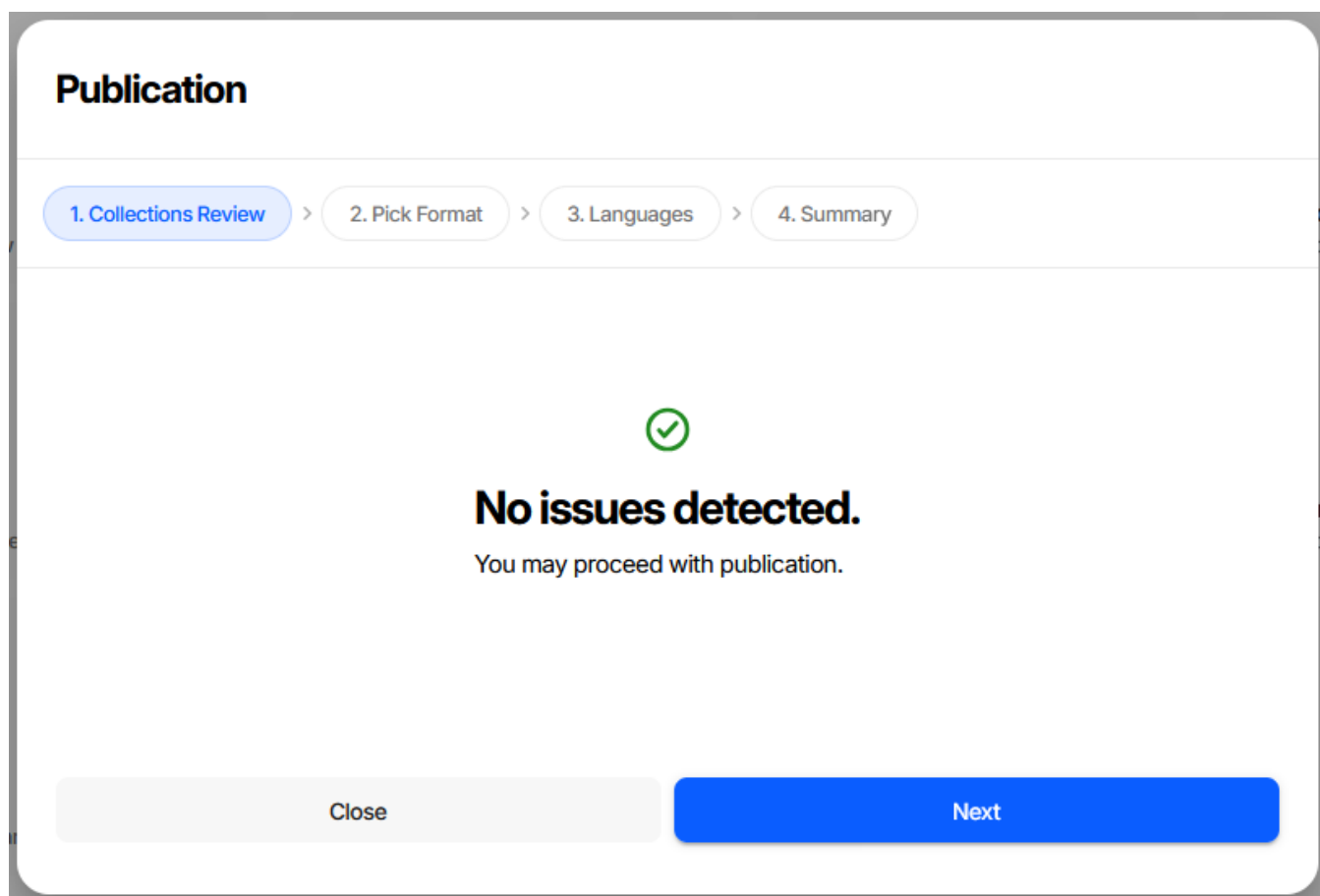
Error counts come from a background pass over the whole database using the six integrity checks **plus** checkTranslations. This is a wider profile than the one used when saving, so a document can be clean at save time and still be counted here - untranslated text being the usual reason.

Reviewing Before Publication

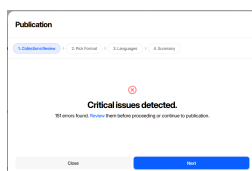
Publish on the dashboard opens the Publication wizard. Its first step, *Collections Review*, is a project-wide validation report.



The step opens with a summary in one of three states. Above is the middle one - **warnings**, meaning missing translations only. Publication will succeed; the shipped data will have gaps in the affected languages.

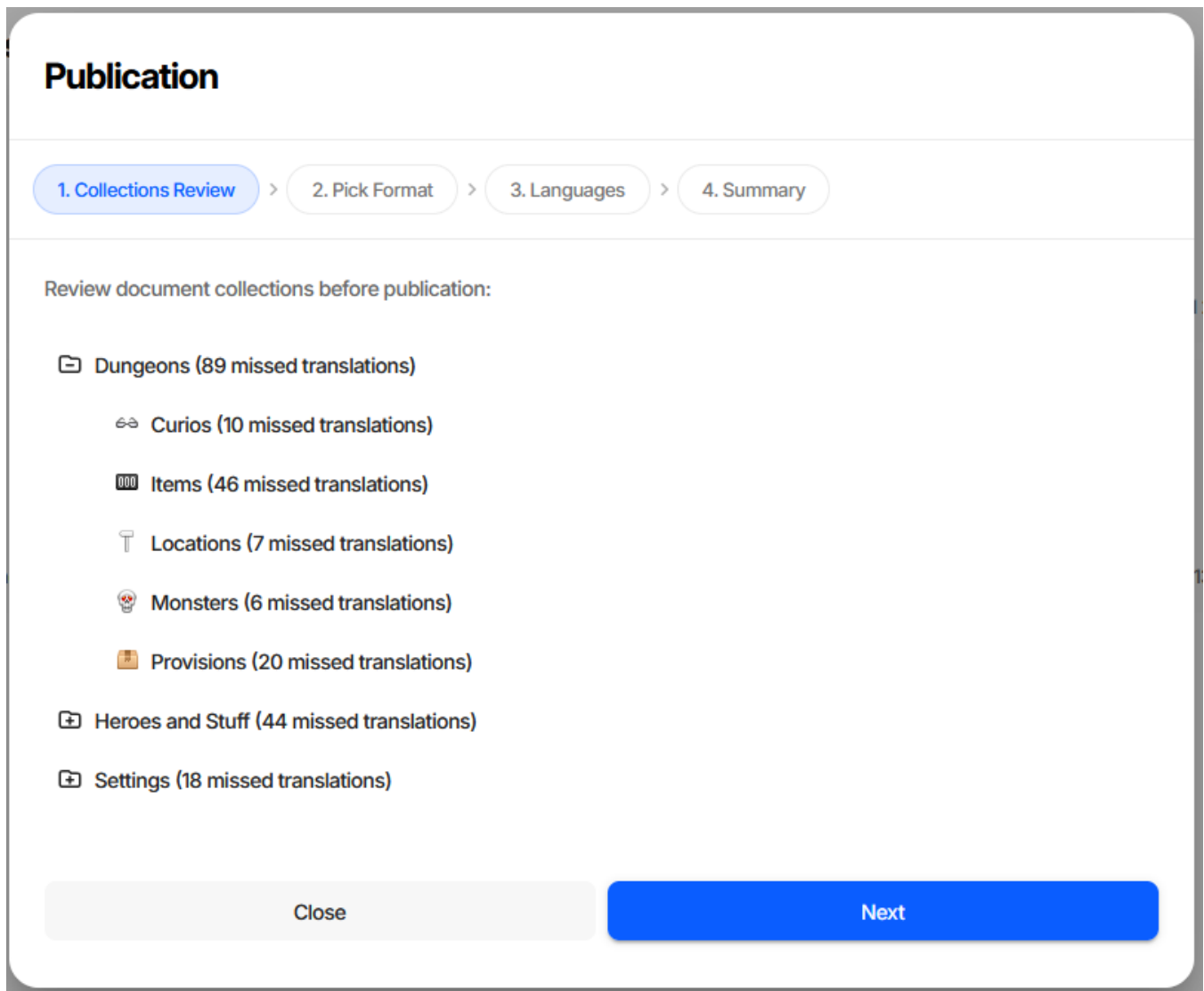


No issues - nothing to fix, proceed.



Critical issues - real integrity errors: broken references, missing required values, format problems. Fix these before publishing. Generated code and game-side loaders assume the data satisfies the schema, so a dangling reference or a missing required value usually shows up at runtime as a null reference or a failed load rather than as anything visible in the editor.

Review replaces the summary with a per-collection tree:



Each node is annotated with what was found: `N issues` counts integrity errors, `N missed translations` counts untranslated text. Collections with nothing wrong are marked with a check. Clicking a collection opens its errors view, described above.

Warning

Publication is never blocked - **Next** stays enabled in all three states. Treat the two severities differently. *Missed translations* ship as visible gaps in the affected languages: undesirable, but the game runs. *Critical issues* are broken data that will most likely crash the game or fail to load, and should be fixed before publishing rather than after. To enforce a clean state automatically, run `DATA VALIDATE --output err` in CI; see [Exit Code Behaviour](#).

CLI equivalent

`DATA EXAMINE` returns the same per-collection statistics that populate the review tree:

Added in version 2026.4.0.

```
dnx dotnet-charon -- DATA EXAMINE --dataBase "gamedata.json"
```

DATA VALIDATE returns the individual errors behind those counts:

```
dnx dotnet-charon -- DATA VALIDATE \
  --dataBase "gamedata.json" \
  --validationOptions checkRequirements checkFormat checkUniqueness \
    checkReferences checkSpecification checkConstraints checkTranslations
  --output out --outputFormat json
```

When Validation Runs

Throughout this page, *the six integrity checks* means checkRequirements, checkFormat, checkUniqueness, checkReferences, checkSpecification and checkConstraints.

Trigger	Checks and repairs applied
Document save (UI / API CREATE or UPDATE)	repair, repairRequiredWithDefaults, and the six integrity checks
Document save with <i>Save Anyway</i>	repairs only - no integrity checks
DATA IMPORT	repair, repairRequiredWithDefaults, deduplicateIds, resolveConflictingUnions - no integrity checks
DATA I18N IMPORT	repair, repairRequiredWithDefaults, deduplicateIds, resolveConflictingUnions - no integrity checks
DATA VALIDATE (on demand)	the six integrity checks
Error counts and publication review	the six integrity checks plus checkTranslations

Warning

Both import commands default to repairs **without integrity checks**. They fix what they can and report nothing else - broken references or missing required values land in the data silently. Pass `--validationOptions` explicitly, or follow the import with a DATA VALIDATE run, whenever the input is not fully trusted.

All defaults can be overridden with `--validationOptions` on the relevant command.

Validation Flags

Flags are combined to form a validation profile. Pass multiple flags as space-separated values:

```
dnx dotnet-charon -- DATA VALIDATE --dataBase gamedata.json \
  --validationOptions checkRequirements checkReferences checkFormat
```

Integrity check flags

These flags **report errors** without modifying data.

checkRequirements

Every property marked **Not Null** or **Not Empty** must have a value.

checkFormat

Values must match the declared data type format - e.g. Date must be ISO 8601, Number must not be a string, SelectList value must be in the defined list.

checkUniqueness

Properties marked **Unique** or **Unique In Collection** must have no duplicates.

checkReferences

Every Reference and ReferenceCollection must point to a document that exists.

checkSpecification

Values must satisfy constraints declared in the Specification field of the schema or property (used by extension editors and code-generation plugins).

checkConstraints

Structural invariants: valid pick-list values, formula syntax, collection size limits defined via Size on the property.

checkTranslations

Every LocalizedText field must have a translation for each language declared in Project Settings. Detects untranslated strings before shipping. Not included in any default profile except the background pass - request it explicitly.

Repair flags

Repair flags **automatically fix** certain problem classes before integrity checks run. They modify the data in-place (or within the import transaction).

repair

Master switch - without it the repair pass is skipped entirely and the other repair flags have no effect.

deduplicateIds

If multiple documents in the same collection share the same Id, each duplicate gets a new auto-generated Id.

repairRequiredWithDefaults

A missing required field is filled with the property's **Default Value** if one is defined.

eraseInvalidValues

A field whose value cannot be parsed as the declared data type is set to null.

resolveConflictingUnions

Fixes malformed Union values: a union with two or more variants set keeps the **last** variant in schema order and drops the rest; an empty union with no variant set is erased to null.

Validation Report Format

Both DATA VALIDATE and DATA IMPORT can emit a structured validation report:

```
{
  "records": [
    {
      "id": "<document-id>",
      "schemaId": "<schema-id>",
      "schemaName": "<schema-name>",
      "errors": [
        {
          "path": "<json-pointer to field>",
          "message": "<human-readable description>",
          "code": "<machine-readable error code>"
        }
      ]
    }
  ]
}
```

Documents with no errors are omitted. An empty records array means the data is valid.

Exit Code Behaviour

The exit code is 1 **only when** the report contains errors **and** --output is set to err (standard error). In all other cases the exit code is 0, even if the report file contains errors.

This design makes it easy to use validation in CI:

```
# Fail the pipeline if there are any integrity errors
dnx dotnet-charon -- DATA VALIDATE --dataBase gamedata.json \
  --validationOptions checkRequirements checkReferences checkFormat \
  --output err
# Exit code 1 → CI step fails

# Write report to a file without failing the step
dnx dotnet-charon -- DATA VALIDATE --dataBase gamedata.json \
  --output validation_report.json --outputFormat json
# Exit code 0 → CI step passes; inspect the file separately
```

Common Recipes

Skip all checks (import prototype data)

```
dnx dotnet-charon -- DATA IMPORT --dataBase gamedata.json --input prototype.json \
  --validationOptions none
```

The UI equivalent is the *Ignore consistency errors in imported data* checkbox in the import wizard.

Auto-repair + full check on import

```
dnx dotnet-charon -- DATA IMPORT --dataBase gamedata.json --input data.json \
  --validationOptions repair deduplicateIds repairRequiredWithDefaults \
  checkRequirements checkReferences checkFormat
```

Fix union conflicts after schema refactor

```
dnx dotnet-charon -- DATA IMPORT --dataBase gamedata.json --input data.json \
  --validationOptions repair resolveConflictingUnions
```

Check for untranslated strings

Reproduces what the publication review counts as *missed translations*:

```
dnx dotnet-charon -- DATA VALIDATE --dataBase gamedata.json \
  --validationOptions checkTranslations --output out --outputFormat json
```

Gate a build on a clean publication review

```
dnx dotnet-charon -- DATA VALIDATE --dataBase gamedata.json \
  --validationOptions checkRequirements checkFormat checkUniqueness \
  checkReferences checkSpecification checkConstraints \
  --output err
```

Dry Run vs Validate

- DATA VALIDATE checks the **current state** of the database.
- DATA IMPORT --dryRun simulates the full import pipeline inside a transaction, then rolls back. Use this to preview the post-import state and catch referential errors before committing. The UI equivalent is the *Perform a dry run and don't persist changes* checkbox.

See also

- Publishing Game Data
- Internationalization (i18n)
- Importing and Exporting Data
- DATA VALIDATE Command
- DATA EXAMINE Command
- DATA IMPORT Command
- Schema Types
- Property

Backup and Restore

Charon can create a full snapshot of a game data file and restore it later. Backups capture the complete state of the database - all documents, schemas, project settings, and metadata - in a single portable file.

Both operations are available from the editor UI and from the CLI. The UI is the quickest way to take a snapshot before a risky change; the CLI is what you schedule and run from a build server.

Where Backup and Restore Live	187
Creating a Backup	188
Step 1. Destination	188
Step 2. Pick Format	188
Step 3. Summary	189
What is included	190
What is not included	190
Restoring from a Backup	190
Step 1. From	190
Step 2. File Input	191
Step 3. Summary	191
Backup vs Export	191
Scheduled Automated Backups	192
Pre-Publish Safety Snapshot	192
See also	193

Where Backup and Restore Live

Open **Project Settings** → **Backup**. The **Manual Backup & Restore** card holds both buttons.

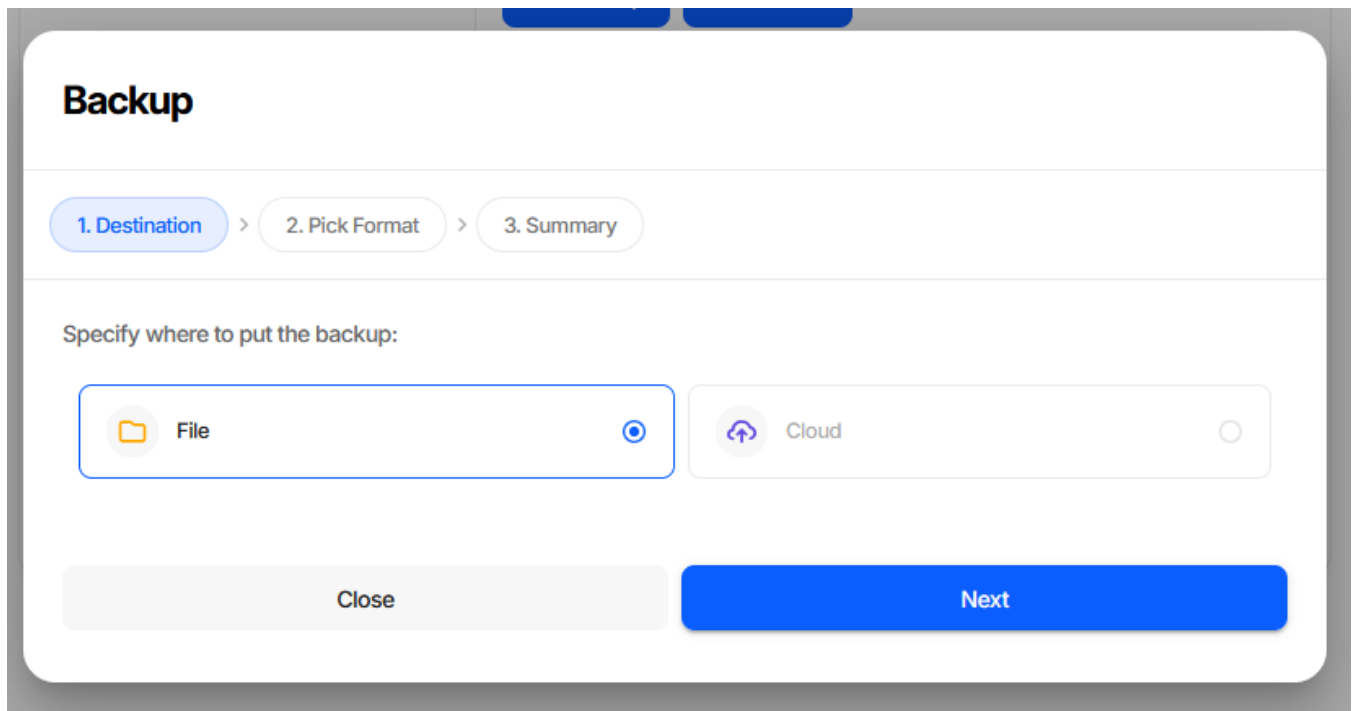


The page is part of every edition - standalone, the Unity and Unreal Engine plugins, and the web editor. Both buttons require the **Administrator** permission and stay disabled for every other role; see Roles and Permissions.

Creating a Backup

Backup opens a three-step wizard.

Step 1. Destination



Backup

1. Destination > 2. Pick Format > 3. Summary

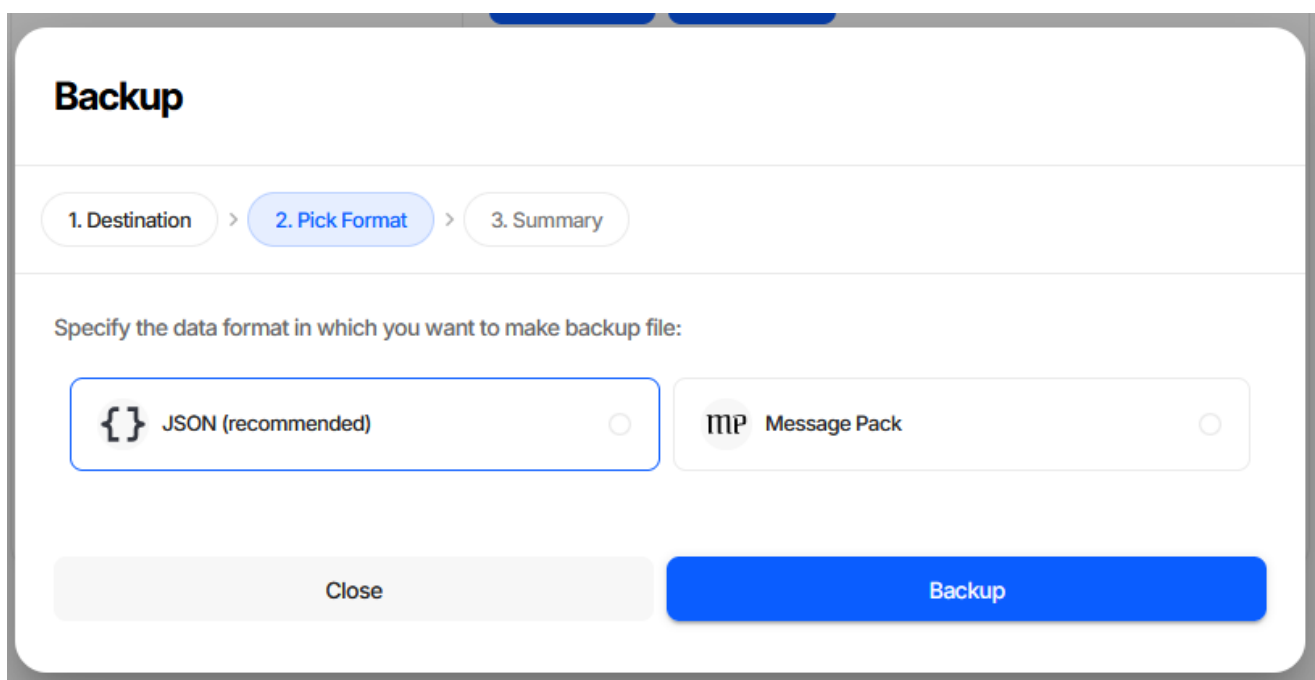
Specify where to put the backup:

☒ File ☐ Cloud

Close Next

File downloads the snapshot through the browser. **Cloud** is shown but not selectable.

Step 2. Pick Format



Backup

1. Destination > 2. Pick Format > 3. Summary

Specify the data format in which you want to make backup file:

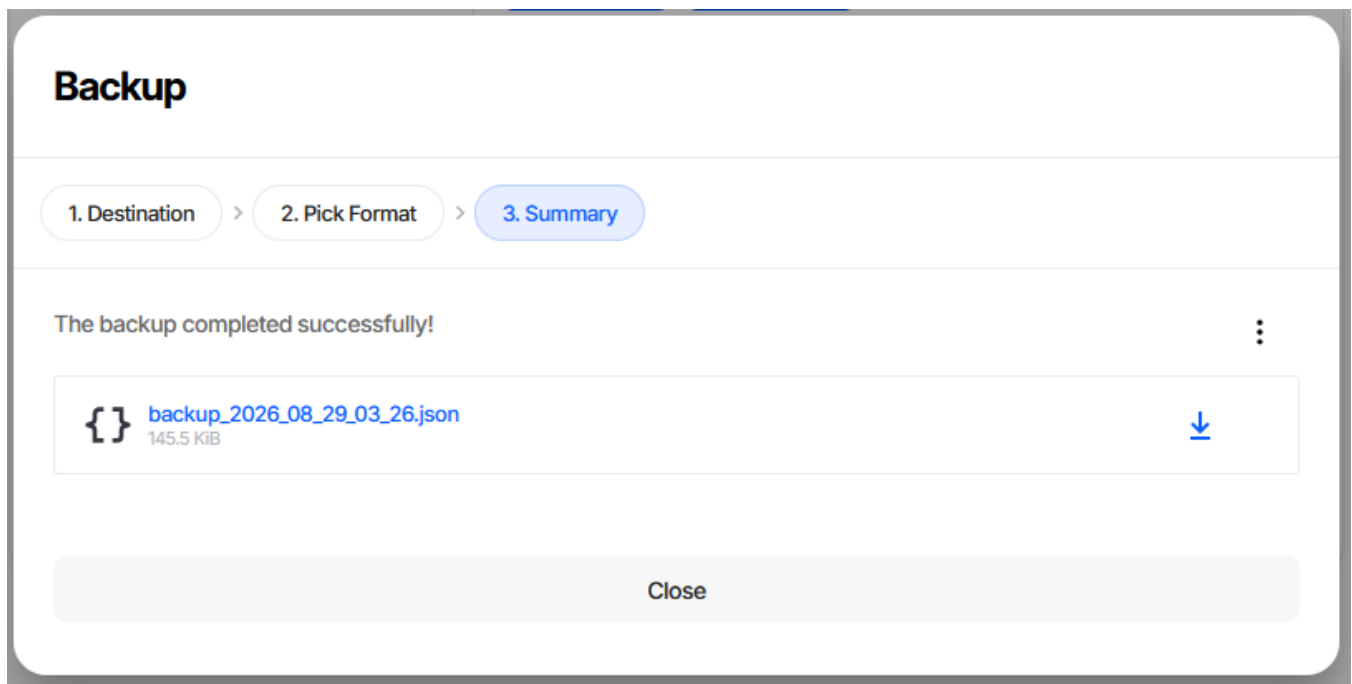
☒ JSON (recommended) ☐ Message Pack

Close Backup

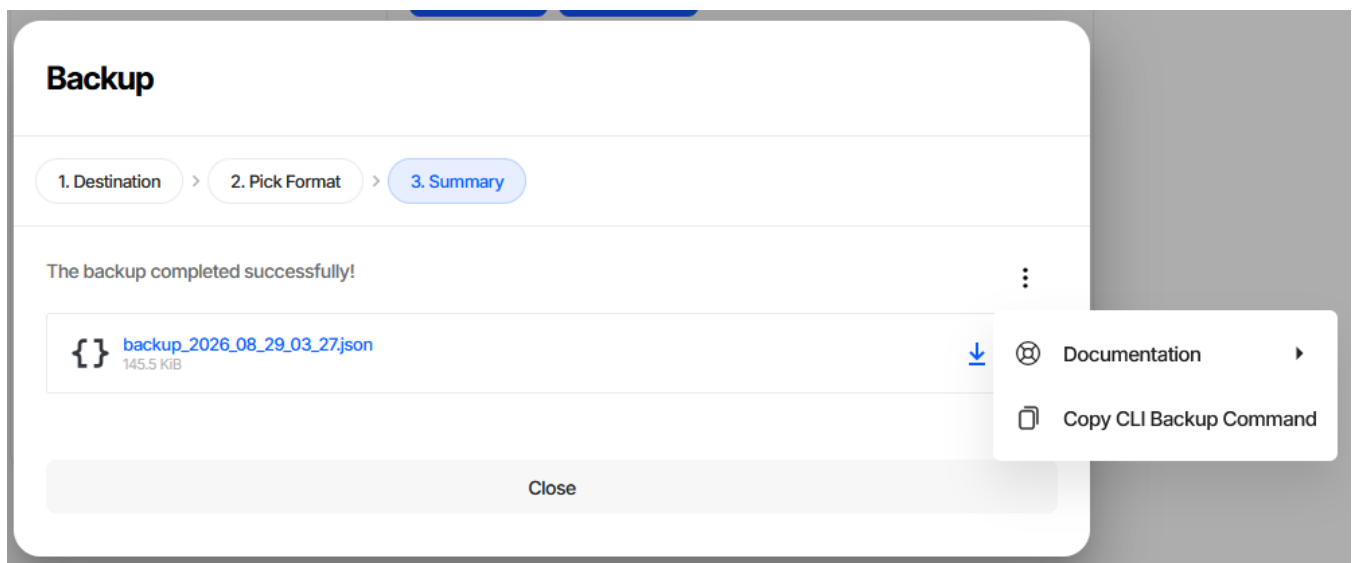
Format	When to use
JSON (recommended)	Default. Readable and diffable, so a backup can be inspected or committed to version control alongside the data it snapshots.
Message Pack	Compact binary. Noticeably smaller files for large databases, at the cost of not being human-readable.

Both formats restore identically, so the choice only affects file size and readability. **Backup** on this step starts the process.

Step 3. Summary



The snapshot is offered as a download, named after the date and time it was taken and annotated with its size. Nothing is written until you click the link.



The  menu has **Copy CLI Backup Command**, which puts the CLI equivalent of this backup on the clipboard - the fastest way to turn a manual snapshot into a scheduled one.

CLI equivalent

```
# Local file
dnx dotnet-charon -- DATA BACKUP \
  --dataBase gamedata.json \
  --output backup.json \
  --outputFormat json

# Remote project
dnx dotnet-charon -- DATA BACKUP \
  --dataBase "https://charon.live/view/data/MyGame/develop/" \
  --output backup.msgpack \
  --outputFormat msgpack \
  --credentials "$CHARON_API_KEY"
```

Warning

--outputFormat is **not** inferred from the --output file name; it defaults to json. So --output backup.msgpack without a matching --outputFormat msgpack writes JSON into a file named .msgpack - and DATA RESTORE, which *does* infer the format from the extension, then fails to read that file back. Always pass --outputFormat explicitly.

What is included

- All document collections (user data, schema definitions, project settings).
- ToolsVersion, RevisionHash, and ChangeNumber file-level fields.

What is not included

- User accounts, API keys, and access control settings (server edition - managed separately).
- Binary resource files (images, audio) attached to documents via Asset Path properties.

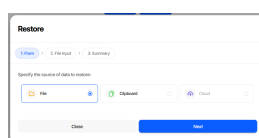
Restoring from a Backup

Warning

Restore **replaces all content** in the target database, from both the UI and the CLI. The operation is not incremental - the entire existing state is overwritten. Confirm you are targeting the correct project and branch before starting.

Restore opens a three-step wizard.

Step 1. From



File reads a previously downloaded backup. **Clipboard** accepts pasted JSON text, which is useful for a snapshot kept in a ticket or a chat message. **Cloud** is shown but not selectable.

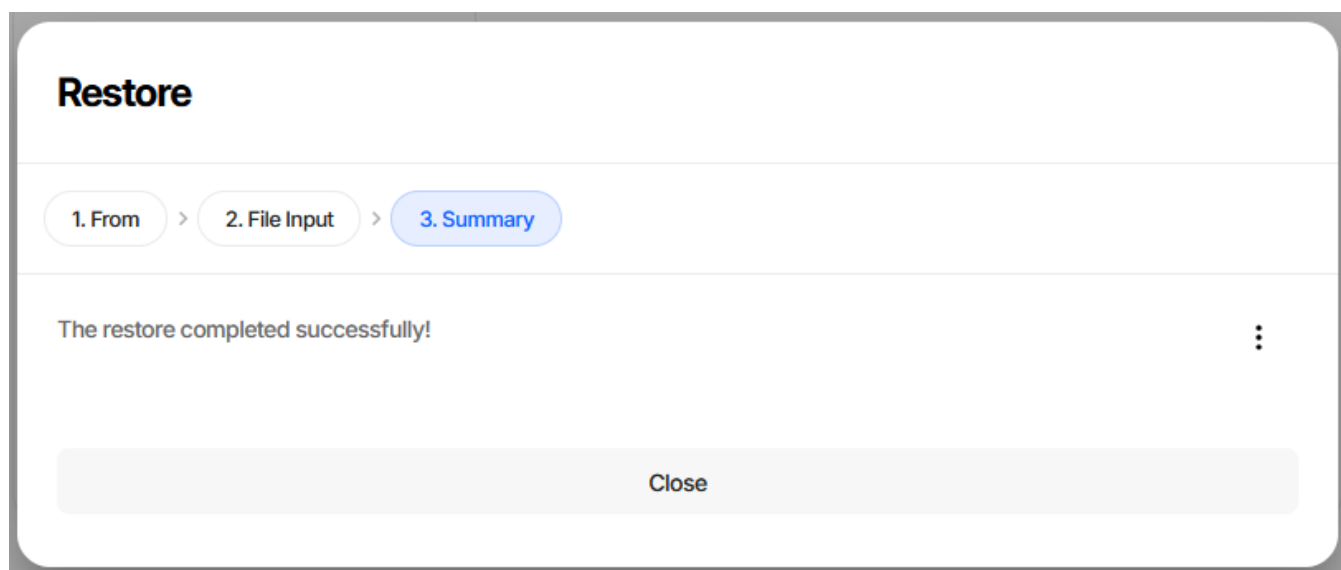
Step 2. File Input

Browse for the backup file. The step is labelled **Clipboard Input** and shows a text box instead when *Clipboard* was chosen on step 1; pasted text must be a valid JSON object.

Format	Accepted extensions
JSON	.json, .gdjs
Message Pack	.msgpack, .gdmp, .msgpkg
BSON	.bson, .gdbson

A file with any other extension is rejected on this step, before anything is sent to the server. **Restore** on this step starts the process.

Step 3. Summary



On success the  menu offers **Copy CLI Restore Command**. On failure the step shows the server's error message and the data is left untouched.

CLI equivalent

Local file

```
dnx dotnet-charon -- DATA RESTORE \
  --dataBase gamedata.json \
  --input backup.json
```

Remote project

```
dnx dotnet-charon -- DATA RESTORE \
  --dataBase "https://charon.live/view/data/MyGame/develop/" \
  --input backup.msgpack \
  --credentials "$CHARON_API_KEY"
```

--inputFormat defaults to auto, which - unlike DATA BACKUP - is taken from the input file's extension. Pass json, msgpack or bson explicitly when the file has no extension or carries a name that does not match its contents.

Backup vs Export

Choose the right tool for the job:

	DATA BACKUP	DATA EXPORT
Purpose	Full snapshot for disaster recovery	Filtered slice for runtime or tooling

Format	JSON / MessagePack	JSON / BSON / MessagePack / XLSX
Includes schemas	Always	Optional (<code>--schemas Schema</code>)
Strips unused data	No	Yes <code>--mode publication</code> strips unused languages
Restorable	Yes, via <code>DATA RESTORE</code>	Partially - via <code>DATA IMPORT</code>

Use **Backup** when you need a full, restorable point-in-time snapshot. Use **Export** when preparing data for the game runtime or for external tools.

Scheduled Automated Backups

Linux / macOS (cron)

```
# Daily at 02:00
0 2 * * * dnx dotnet-charon -- DATA BACKUP \
  --dataBase /var/game/gamedata.json \
  --output /backups/gamedata_$(date +%Y%m%d).msgpack \
  --outputFormat msgpack
```

GitHub Actions

```
- name: Backup game data
  run: |
    dnx dotnet-charon -- DATA BACKUP \
      --dataBase "${{ secrets.CHARON_DB_URL }}" \
      --output backups/gamedata_${{ github.run_id }}.msgpack \
      --outputFormat msgpack \
      --credentials "${{ secrets.CHARON_API_KEY }}"

- name: Upload artifact
  uses: actions/upload-artifact@v4
  with:
    name: gamedata-backup
    path: backups/
```

Pre-Publish Safety Snapshot

Take a backup immediately before publishing so that a known-good restore point exists:

```
# 1. Snapshot
dnx dotnet-charon -- DATA BACKUP \
  --dataBase gamedata.json \
  --output pre_publish_backup.json \
  --outputFormat json

# 2. Validate
dnx dotnet-charon -- DATA VALIDATE \
  --dataBase gamedata.json \
  --validationOptions checkRequirements checkReferences \
  --output err

# 3. Publish
dnx dotnet-charon -- DATA EXPORT \
  --dataBase gamedata.json \
  --mode publication \
```

```
--output StreamingAssets/gamedata.json \
--outputFormat json
```

Tip

The same three steps run from the UI as **Project Settings** → **Backup**, the Publish wizard’s collections review, and **Publish** itself.

See also

- DATA BACKUP Command
- DATA RESTORE Command
- Importing and Exporting Data
- Publishing Game Data
- Roles and Permissions
- CI/CD Integration
- Patch and Diff Workflow

Patch and Diff Workflow

Charon can compute the **difference** between two game data files and represent it as a reusable patch. That patch can later be applied to any compatible target file. This enables branching workflows, incremental DLC updates, audit trails, and automated data merging in CI/CD pipelines.

Overview	193
Creating a Patch	194
Patch file format	194
Applying a Patch	194
Validation on apply	195
End-to-End Example: Feature Branch Merge	195
Storing Patches in Version Control	195
See also	195

Overview

Two commands form the workflow:

DATA CREATEPATCH

Compares two game data sources and produces a *patch file* describing every addition, modification, and deletion between them.

DATA APPLYPATCH

Applies a patch file to a target game data file.

dataBase1 (before) ■■■

■■■ CREATEPATCH ■→ patch.json

dataBase2 (after) ■■■

target.json ■■■■■ APPLYPATCH ■■→ patched target.json

patch.json ■■■

Common use cases:

- **Feature branch merge** - keep a stable baseline; apply a feature branch as a patch.
 - **DLC / live updates** - ship incremental changes without distributing the full file.
 - **Code-review diff** - generate a human-readable changelog of data edits for review.
 - **Automated reconciliation** - merge concurrent edits from multiple environments in CI.
-

Creating a Patch

```
dnx dotnet-charon -- DATA CREATEPATCH \
  --dataBase1 gamedata_before.json \
  --dataBase2 gamedata_after.json \
  --output changes.patch.json \
  --outputFormat json
```

- --dataBase1 is the **baseline** (before).
- --dataBase2 is the **modified** version (after).
- Both can be local files or remote server URLs. A single --credentials value is used for both when they are remote.

Tip

When both sources are remote and the dataset is large, download them first with DATA BACKUP and run CREATEPATCH on the local copies to avoid network round-trips.

Patch file format

The patch file follows the standard game data JSON format.:

```
{
  "Collections": {
    "Item": {
      "IronSword": { Id: "IronSword", "Name": "Iron Sword", "Damage": 20 }, // addition
      "OldSword": { "Damage": 18 }, // modification
      "RustySword": null // deletion
    }
  }
}
```

Applying a Patch

```
dnx dotnet-charon -- DATA APPLYPATCH \
  --dataBase target.json \
  --input changes.patch.json \
  --inputFormat json
```

Validation on apply

DATA APPLYPATCH validates the target after applying the patch with repairs only - repair, deduplicateIds, repairRequiredWithDefaults and resolveConflictingUnions - and no integrity checks. The command takes no --validationOptions parameter, so check the result separately whenever the patch is not fully trusted:

```
dnx dotnet-charon -- DATA VALIDATE \
  --dataBase target.json \
  --validationOptions checkRequirements checkFormat checkUniqueness \
                        checkReferences checkSpecification checkConstraints \
  --output err
```

End-to-End Example: Feature Branch Merge

1. Backup the stable main branch

```
dnx dotnet-charon -- DATA BACKUP \
  --dataBase "https://charon.live/view/data/MyGame/main/" \
  --output main.json --outputFormat json --credentials "$CHARON_KEY"
```

2. Backup the feature branch

```
dnx dotnet-charon -- DATA BACKUP \
  --dataBase "https://charon.live/view/data/MyGame/feature-loot/" \
  --output feature.json --outputFormat json --credentials "$CHARON_KEY"
```

3. Compute the diff

```
dnx dotnet-charon -- DATA CREATEPATCH \
  --dataBase1 main.json \
  --dataBase2 feature.json \
  --output loot_changes.patch.json
```

4. (Optional) review the patch in a text editor or commit it to version control

5. Apply to a staging branch

```
dnx dotnet-charon -- DATA APPLYPATCH \
  --dataBase staging/gamedata.json \
  --input loot_changes.patch.json
```

Storing Patches in Version Control

Patch files are plain JSON (or BSON / MessagePack). They can be:

- Committed alongside game source code to track data changes per feature branch.
- Applied conditionally during a build - e.g., apply a hotfix patch only in production builds.
- Accumulated as a migration log: v1.0→v1.1.patch.json, v1.1→v1.2.patch.json, and so on.

See also

- Document Merge Logic
- DATA CREATEPATCH Command
- DATA APPLYPATCH Command
- Backup and Restore
- CI/CD Integration

- Importing and Exporting Data

Modding Support

The free version of Charon can be distributed together with your shipped game, letting players edit game data with the same editor your team used to build it. Combined with patch files, this enables a full modding pipeline: players edit a copy of the game data, changes are captured as a small patch, and the game loads the base data with any number of patches applied on top.

Overview	196
Shipping the Editor with Your Game	196
Capturing Mod Changes as a Patch	197
Loading Mods at Runtime	197
How Patches Are Merged at Load Time	197
See also	198

Overview

A typical modding setup consists of three parts:

1. **The base game data file** (e.g. `gamedata.json`), shipped with the game or provided in a separate mod kit.
2. **The Charon editor**, launched locally by the player (or by your game) against a copy of that file, with schema editing disabled.
3. **Patch files** describing the mod's changes relative to the base data, loaded by the game at startup through the generated `Load()` code.

Shipping the Editor with Your Game

Charon is a .NET tool, so the player's machine needs .NET installed. What exactly is required depends on which of the two options below you pick. Verify what is present with:

```
dotnet --list-sdks
dotnet --list-runtimes
```

If .NET is missing, direct the player to the [.NET download page](#). Some distribution platforms (e.g. Steam) can install the .NET Runtime automatically as a game dependency.

There are two ways to run the editor:

On-demand via the .NET SDK (simplest, requires the .NET SDK 10 or later):

```
dnx dotnet-charon -- SERVER START --dataBase "./gamedata.json" --denySchemaEditing --launchD
```

A bundled copy of a specific version (full control, only the .NET 8 Runtime required):

Download the `dotnet-charon` package from [nuget.org](#) (a .nupkg file is a ZIP archive) and ship the contents of `tools/net8.0/any/` with your game, then launch it via the runtime:

```
dotnet "<game folder>/Charon/Charon.dll" SERVER START \
  --dataBase "./mods/gamedata_copy.json" \
  --port 8080 \
  --denySchemaEditing \
  --launchDefaultBrowser
```

The `--denySchemaEditing` flag is important for modding scenarios: it locks the data *structure* (schemas), so mods can change values and documents but cannot break the shape of the data your compiled game expects. See `SERVER START` for all flags.

Capturing Mod Changes as a Patch

Instead of shipping a whole modified game data file, a mod can be a small **patch file** - the difference between the base data and the modified copy. Creating patches is currently a CLI-only operation - there is no UI action for it:

```
dnx dotnet-charon -- DATA CREATEPATCH \
  --dataBase1 "./gamedata.json" \
  --dataBase2 "./mods/gamedata_copy.json" \
  --output "./mods/my_mod.gdpatch" \
  --outputFormat json
```

Patch files are small, composable, and independent of each other as long as they touch different documents. See Patch and Diff Workflow for the format details and conflict behavior.

Loading Mods at Runtime

The generated source code loads patches alongside the base game data - pass patch streams in the load options. Patches are applied in the order given, and each patch is merged into the data following the default merge rules. Patching during load is supported by all Tier 1 code generators: **C# 4.0**, **C# 7.3**, **TypeScript**, and **Unreal Engine C++** (see Generating Source Code).

C# example:

```
var gameData = new GameData(gameDataStream, new Formatters.GameDataLoadOptions {
    Format = Formatters.Format.Json,
    Patches = new[] { modPatchStream1, modPatchStream2 }
});
```

A simple mod-loading convention: scan a mods/ folder for patch files at startup and pass them all to Load() in a deterministic order (e.g. alphabetical, or driven by a mod manifest).

How Patches Are Merged at Load Time

Runtime patching follows the same deterministic principles as the editor-side document merge logic. The core rules, in short:

- If the patch **provides a value** for a property → the result uses the patch value.
- If the patch **does not provide a value** (the property key is absent) → the original value is kept.
- A property present with an explicit null is an **intentional deletion**, not a missing value.
- The rules apply recursively: embedded documents are merged property-by-property, and collections are merged item-by-item using the document's Id as the key.

On top of that, for whole documents inside a patch, the **presence of the Id field** decides how the entry is applied - the Id value is irrelevant:

No Id field - a *partial update*. The entry's fields are merged into the existing document. A partial update cannot resurrect a deleted document - if the document was removed by an earlier patch, it stays deleted:

```
original    patch 1    patch 2    patch 3    result
{ Id: 1 } → null → { ... } → { ... } = null
```

Has an Id field - a *full replacement*. The document is replaced entirely. A replacement can also resurrect a previously deleted document:

```
original    patch 1    patch 2    patch 3    result
{ Id: 1 } → null → { Id: 2 } → { Id: 3 } = { Id: 3 }
```

See Document Merge Logic for the full property-level rules (LocalizedText per-language merging, collection modes, and validation caveats when a patch recreates a document with missing required fields).

See also

- SERVER START Command
- DATA CREATEPATCH Command
- Patch and Diff Workflow
- Merge Logic
- Generating Source Code
- Publishing Game Data

CI/CD Integration

Charon's CLI supports full automation of game data workflows - validation, export, translation, code generation, and backup - making it straightforward to integrate into any continuous integration or deployment pipeline.

Installation in CI	198
Version Pinning and Compatibility	199
Tool version vs. format version	199
Authentication	199
Exit Codes	200
Common Pipeline Recipes	200
Validate on every pull request	200
Export and publish on tag	201
Apply a patch on merge	202
Export translation files for a localization vendor	202
Import translated files from a vendor	202
Recommended Pipeline Structure	203
See also	203

Installation in CI

Charon requires the **.NET SDK 10 or later** on the runner. With SDK 10 there is no install step at all - dnx fetches and runs the tool on demand, and NuGet caches it for the rest of the job.

GitHub Actions

```
- name: Setup .NET
  uses: actions/setup-dotnet@v5
  with:
    dotnet-version: '10.x'

- name: Check Charon
  run: dnx dotnet-charon -- VERSION
```

GitLab CI

```
before_script:
  - apt-get install -y dotnet-sdk-10.0
  - dnx dotnet-charon -- VERSION
```


Shell script (any runner)

```
dnx dotnet-charon -- VERSION
```

If your runner is pinned to .NET SDK 8 or 9, dnx is unavailable. Install the tool globally instead and call it as charon:

```
dotnet tool install -g dotnet-charon
export PATH="$PATH:$HOME/.dotnet/tools"
charon VERSION
```

Version Pinning and Compatibility

dnx dotnet-charon without a version always fetches the latest release. For reproducible builds, pin the tool version.

On-demand with a pinned version (recommended, .NET SDK 10+). Nothing to install or commit:

```
dnx dotnet-charon@2026.3.4 -- DATA VALIDATE --dataBase gamedata.json ...
```

Local tools manifest. Create a manifest once and commit `.config/dotnet-tools.json` to the repository:

```
dotnet new tool-manifest
dotnet tool install dotnet-charon --version 2026.3.4
```

CI then restores the exact pinned version and invokes the tool through dotnet:

```
dotnet tool restore
dotnet charon DATA VALIDATE --dataBase gamedata.json ...
```

Global install with a pinned version:

```
dotnet tool install -g dotnet-charon --version 2026.3.4
charon DATA VALIDATE --dataBase gamedata.json ...
```

Tool version vs. format version

Two versions matter for compatibility:

- **Tool version** (e.g. 2026.3.4) - the actual release version of dotnet-charon. It changes with every release: bug fixes, new features, UI updates.
- **Format version** - the version of the game data file format. It changes only when the system schemas are updated or system fields are added or changed, which is rare. Tool releases overwhelmingly ship with an unchanged format version.

Game data files are **auto-migrated to the newest format version on save**. Read-only operations - `DATA VALIDATE`, `DATA EXPORT`, `GENERATE` ... - never mutate the file, so running a newer tool against an older-format file in CI is safe and leaves the file untouched.

If you want to deliberately migrate a file to the current format version, apply any change and save - for example, update the *Copyright* field in *Project Settings* from the editor UI. The save rewrites the file in the newest format.

Authentication

Provide the API key for remote data sources via the `CHARON_API_KEY` environment variable. Store it as a CI secret; never hard-code it in pipeline files.

```
# GitHub Actions - secrets.CHARON_API_KEY defined in repository settings
env:
  CHARON_API_KEY: ${ secrets.CHARON_API_KEY }
```

```
# Shell - set before running any charon command
export CHARON_API_KEY="your-api-key-here"
dnx dotnet-charon -- DATA EXPORT --dataBase "https://charon.live/view/data/MyGame/main/" ...
```

Alternatively, pass credentials explicitly with `--credentials`. The environment variable takes precedence when both are supplied.

Exit Codes

Charon exits with 0 on success. It exits with 1 or any positive code in these cases:

- A fatal error occurred (bad parameters, file not found, network failure).
- `DATA VALIDATE` produced errors **and** `--output` was set to `err`.

All other cases - including a validation report that contains errors written to a file - exit with 0. Design your pipeline steps accordingly.

Common Pipeline Recipes

Validate on every pull request

```
# .github/workflows/validate.yml
name: Validate game data

on: [pull_request]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-dotnet@v5
        with: { dotnet-version: '10.x' }

      - name: Validate integrity
        run: |
          dnx dotnet-charon -- DATA VALIDATE \
            --dataBase gamedata.json \
            --validationOptions checkRequirements checkReferences checkFormat \
            --output err
          # Exits 1 and fails the workflow if errors are found
```

Export and publish on tag

```
# .github/workflows/publish.yml
name: Publish game data

on:
  push:
    tags: ['v*']

jobs:
  publish:
    runs-on: ubuntu-latest
    env:
      CHARON_API_KEY: ${ secrets.CHARON_API_KEY }

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v5
        with: { dotnet-version: '10.x' }

      - name: Backup before publish
        run: |
          dnx dotnet-charon -- DATA BACKUP \
            --dataBase "https://charon.live/view/data/MyGame/main/" \
            --output backup_${ github.ref_name }.json \
            --outputFormat json

      - name: Validate
        run: |
          dnx dotnet-charon -- DATA VALIDATE \
            --dataBase "https://charon.live/view/data/MyGame/main/" \
            --validationOptions checkRequirements checkReferences \
            --output err

      # DATA UPDATEPROJECTSETTINGS is available since 2026.4.0
      - name: Stamp the data version from the tag
        run: |
          dnx dotnet-charon -- DATA UPDATEPROJECTSETTINGS \
            --dataBase "https://charon.live/view/data/MyGame/main/" \
            --property Version \
            --value "${GITHUB_REF_NAME#v}"

      - name: Export published data (JSON)
        run: |
          dnx dotnet-charon -- DATA EXPORT \
            --dataBase "https://charon.live/view/data/MyGame/main/" \
            --mode publication \
            --output StreamingAssets/gamedata.json \
            --outputFormat json

      - name: Generate C# source code
        run: |
          dnx dotnet-charon -- GENERATE CSHARPCODE \
            --dataBase "https://charon.live/view/data/MyGame/main/" \
            --namespace MyGame.Parameters \
            --outputDirectory Assets/Scripts/Generated \
            --clearOutputDirectory

      - name: Commit generated files
        run: |
```

```
git config user.name "CI Bot"
git config user.email "ci@example.com"
git add Assets/Scripts/Generated StreamingAssets/gamedata.json
git commit -m "chore: regenerate game data for ${github.ref_name}"
git push
```

Apply a patch on merge

```
# In a merge script or CI job
dnx dotnet-charon -- DATA APPLYPATCH \
  --dataBase gamedata.json \
  --input feature_changes.patch.json

# APPLYPATCH takes no validation options - validate as a separate step
dnx dotnet-charon -- DATA VALIDATE \
  --dataBase gamedata.json \
  --validationOptions checkRequirements checkReferences checkFormat \
  --output err # Fail CI if post-merge data is invalid
```

Export translation files for a localization vendor

```
for lang in de fr ja ko; do
  dnx dotnet-charon -- DATA I18N EXPORT \
    --dataBase gamedata.json \
    --sourceLanguage en \
    --targetLanguage "$lang" \
    --output "translations/en_${lang}.xliff" \
    --outputFormat xliff
done
```

Import translated files from a vendor

```
for file in translations/en_*.xliff; do
  dnx dotnet-charon -- DATA I18N IMPORT \
    --dataBase gamedata.json \
    --input "$file" \
    --dryRun # Preview first

  dnx dotnet-charon -- DATA I18N IMPORT \
    --dataBase gamedata.json \
    --input "$file"
done
```

Recommended Pipeline Structure

```
[PR open]
  ■■ DATA VALIDATE (checkRequirements + checkReferences + checkFormat → err)
```

```
[Merge to main]
  ■■ DATA BACKUP
  ■■ DATA VALIDATE (as above)
  ■■ GENERATE <LANG>CODE --clearOutputDirectory
  ■■ commit generated files
```

```
[Release tag]
  ■■ DATA BACKUP (tagged archive)
  ■■ DATA UPDATEPROJECTSETTINGS --property Version
  ■■ DATA EXPORT --mode publication
  ■■ upload to CDN / game build
```

See also

- Command Line Interface
- Validation
- Backup and Restore
- Patch and Diff Workflow
- Internationalization
- DATA UPDATEPROJECTSETTINGS
- Universal Parameters
- URL-based I/O

Command Line Interface (CLI)

Most of Charon functionality could be accessed via CLI commands. The application uses [GNU-style long options](#) (`--parameterName <value>`). You should be familiar with the terminal on your OS to fully tap the potential of the CLI.

Requirements

Charon is distributed as a [.NET tool](#), so the [.NET SDK](#) is the only prerequisite.

- **.NET SDK 10 or later** — recommended. Enables dnx, which runs Charon without installing anything.
- **.NET SDK 8 or 9** — supported through bootstrap scripts or a classic tool install. dnx is not available on these versions.

Check which version you have:

```
dotnet --version
#> 10.0.100
```

If the command is not found, the SDK is not installed or not on PATH.

Installation

Option 1: dnx (recommended, .NET SDK 10+)

.NET SDK 10 introduced [dnx](#), a runner for .NET tools similar to npx. It downloads and executes the tool on demand without registering it in the global or local tool manifest. Nothing to install, nothing to add to PATH:

```
# run any charon command
dnx dotnet-charon -- DATA EXPORT --help

# use this form when dnx is not registered in PATH
dotnet dnx dotnet-charon -- DATA EXPORT --help
```

Everything after -- is passed to Charon. The tool is cached by NuGet after the first run, so subsequent invocations start immediately.

Pin a version for reproducible builds:

```
dnx dotnet-charon@2026.3.4 -- VERSION
```

This is the notation used throughout the rest of the documentation. If you installed Charon in another way, see [Choosing how to invoke Charon for the equivalent command](#).

Option 2: dotnet tool (.NET SDK 8+)

Installs Charon into the global tool store or into a local tool manifest next to your project.

```
# install charon globally
dotnet tool install --global dotnet-charon

# install charon in current working directory
dotnet tool install dotnet-charon --local --create-manifest-if-needed

# pin a specific version (recommended for CI)
dotnet tool install --global dotnet-charon --version 2026.3.4
```

If the repository already contains a .config/dotnet-tools.json manifest, you do not install anything — just restore it:

```
dotnet tool restore
```

To update an installed tool:

```
# update global tool
dotnet tool update --global dotnet-charon

# update local tool
dotnet tool update dotnet-charon --local
```

To remove it:

```
dotnet tool uninstall --global dotnet-charon
dotnet tool uninstall dotnet-charon --local
```

Adding the global tool to PATH

A **global** tool is launched by the bare charon command, but only if the .NET tools directory is on PATH. On Windows this normally happens automatically; on macOS and Linux you have to do it yourself.

Windows

The installer adds %USERPROFILE%.dotnet\tools to PATH. Open a **new** terminal window after installing so it picks up the change. To add it manually:

```
setx PATH "%PATH%;%USERPROFILE%.dotnet\tools"
```

macOS

```
echo 'export PATH="$PATH:$HOME/.dotnet/tools"' >> ~/.zshrc
source ~/.zshrc
```

Linux

Is It Free?

```
echo 'export PATH="$PATH:$HOME/.dotnet/tools"' >> ~/.bashrc
source ~/.bashrc
```

A **local** tool is never placed on PATH. It is always invoked through dotnet from the directory containing the manifest — see Choosing how to invoke Charon.

Option 3: Bootstrap scripts (.NET SDK 8+)

Use this when dnx is unavailable (.NET SDK 8 or 9) and you would rather not manage the tool manifest yourself. The scripts create a local tool manifest, install or update Charon in it, and then run it.

- [RunCharon.bat \(Windows\)](#)
- [RunCharon.sh \(Linux, macOS\)](#)

Both scripts require the [dotnet](#) tool to be included in the system PATH.

Windows

```
mkdir Charon
cd Charon
curl -O https://raw.githubusercontent.com/gamedevware/charon/main/scripts/bootstrap/RunCharon.bat
REM everything after the script name is your command
.\RunCharon.bat DATA EXPORT --help
```

Linux, macOS

```
mkdir Charon
cd Charon
curl -O https://raw.githubusercontent.com/gamedevware/charon/main/scripts/bootstrap/RunCharon.sh
chmod +x ./RunCharon.sh
# everything after the script name is your command
./RunCharon.sh DATA EXPORT --help
```

Verifying the installation

Run the VERSION command. It prints the tool version and exits with code 0:

dnx

```
dnx dotnet-charon -- VERSION
```

Global tool

```
charon VERSION
dotnet tool list --global
```

Local tool

```
dotnet charon VERSION
dotnet tool list --local
```

Bootstrap script

```
# Windows
.\RunCharon.bat VERSION

# Linux, macOS
./RunCharon.sh VERSION
```

If charon is reported as an unknown command after a global install, the .NET tools directory is missing from PATH — see Adding the global tool to PATH.

Choosing how to invoke Charon

Every example in this documentation is written as `dnx dotnet-charon -- <VERB>`. If you installed Charon in another way, substitute the leading part of the command:

Installation	Command prefix
dnx (.NET SDK 10+)	<code>dnx dotnet-charon --</code>
Global tool	<code>charon</code>
Local tool	<code>dotnet charon</code>
Bootstrap script	<code>.\RunCharon.bat</code> or <code>./RunCharon.sh</code>

For example, these four commands are equivalent:

```
dnx dotnet-charon -- DATA VALIDATE --dataBase ./gamedata.json
charon DATA VALIDATE --dataBase ./gamedata.json
dotnet charon DATA VALIDATE --dataBase ./gamedata.json
./RunCharon.sh DATA VALIDATE --dataBase ./gamedata.json
```

Command Syntax

Commands have the following syntax:

```
dnx dotnet-charon -- VERB [SUB-VERB] --parameterName <parameter-value>
```

parameters can have more than one value.

Use space to separate values

```
dnx dotnet-charon -- DATA EXPORT --schemas Item Armor "Project Settings" Quest
```

if your value contains a space, put it inside the quotation marks.

```
dnx dotnet-charon -- DATA EXPORT --dataBase "c:\my application\gamedata.json"
```

some parameters don't require a value (e.g. flag).

```
dnx dotnet-charon -- VERSION --verbose
```

Available verbs are INIT, DATA, GENERATE, SERVER, MCP and VERSION. Verbs are case-insensitive; they are written in upper case in this documentation to stand out from parameters.

Running on Windows, macOS and Linux

Charon behaves identically on all platforms, but the *shell* around it does not. Examples in this documentation use Bash syntax. Keep the following in mind.

Splitting a long command across lines. The trailing `\` used in the examples is a Bash/Zsh continuation character. It is not understood by Windows shells:

Bash / Zsh (macOS, Linux)

```
dnx dotnet-charon -- DATA EXPORT \
--dataBase ./gamedata.json \
--output ./characters.json
```

PowerShell

```
dnx dotnet-charon -- DATA EXPORT `
--dataBase ./gamedata.json `
--output ./characters.json
```

Command Prompt

```
dnx dotnet-charon -- DATA EXPORT ^
--dataBase .\gamedata.json ^
--output .\characters.json
```

When in doubt, put the whole command on a single line — that works in every shell.

Wildcards. Parameters like `--schemas` accept `*` wildcards. Bash and Zsh expand `*` against file names before Charon ever sees it, so always quote such values:

```
# correct
dnx dotnet-charon -- DATA EXPORT --schemas "Item*"

# wrong on macOS/Linux – the shell replaces Item* with matching file names
dnx dotnet-charon -- DATA EXPORT --schemas Item*
```

Environment variables. Setting `CHARON_API_KEY` differs per shell:

Bash / Zsh

```
export CHARON_API_KEY=87758CC0D7C745D0948F2A8AFE61BC81
```

PowerShell

```
$env:CHARON_API_KEY = "87758CC0D7C745D0948F2A8AFE61BC81"
```

Command Prompt

```
set CHARON_API_KEY=87758CC0D7C745D0948F2A8AFE61BC81
```

Absolute and relative paths

When running commands, it's crucial to be aware of whether you are using [absolute or relative paths](#) to files.

Understanding Paths

1. **Absolute Path:** An absolute path defines a file or directory's location in relation to the root directory. In Linux and macOS, it starts from the root `/`, while in Windows, it begins with a drive letter (like `C:\`).
 - Example for Linux/macOS: `/usr/local/share/mygame`
 - Example for Windows: `C:\Program Files\MyGame`
2. **Relative Path:** A relative path references a file or directory in relation to the current working directory, without starting with a root slash or drive letter.
 - Example: If currently in `/home/user/Documents`, a file in `/home/user/Documents/Projects` would have the relative path `Projects/FileName`.

Usage in Terminals

- **Windows (Command Prompt and PowerShell):** Paths use backslashes (`\`). Absolute paths start with a drive letter (like `C:\Users\Name`), while relative paths use the file name or paths like `subfolder\file.txt`. Forward slashes are also accepted, so `./gamedata.json` works too.
- **macOS/Linux Terminal:** Paths are denoted with forward slashes (`/`). Absolute paths begin from the root (`/`), and relative paths use `./` for the current directory or `../` to go up one level.

Getting Help Text

To display the list of available commands add `--help` or `/?`.

```
dnx dotnet-charon -- --help
```

```
#> Verbs:
#> INIT           Initialize a new, empty game data file at the specified path.
#> MCP            Start an MCP (Model Context Protocol) server for AI assistant integration v
#>               Usage: claudes mcp add --transport stdio charon -- charon MCP
#> SERVER         Commands for starting and managing the embedded HTTP server.
#> DATA          Commands for reading, writing, importing, and exporting game data.
#> GENERATE       Commands for generating typed source code from game data schemas.
#> VERSION        Print current product version.
```

The same flag works for any verb and sub-verb:

```
dnx dotnet-charon -- DATA --help
```

```
dnx dotnet-charon -- DATA EXPORT --help
```

```
#> Usage:
```

```
#> DATA EXPORT --dataBase <URI> [--schemas [<TEXT>]] [--properties [<TEXT>]] [--languages  
#> ] [--outputFormat <TEXT>] [--outputFormattingOptions [<TEXT>]] [--mode <EXP  
#> TEXT>]]
```

Apply Patch (Merge)

Applies patch created with DATA CREATEPATCH command to a game data.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
```

```
dnx dotnet-charon -- DATA APPLYPATCH \  
--dataBase "c:\my app\gamedata.json" \  
--input "c:\my app\gamedata_patch.json" \  
--inputFormat json
```

```
# remote game data
```

```
dnx dotnet-charon -- DATA APPLYPATCH \  
--dataBase "https://charon.live/view/data/My_Game/develop/" \  
--input "./gamedata_patch.json" \  
--inputFormat json \  
--credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <pre># local file --dataBase "c:\my app\gamedata.json"</pre> <pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of <code>--dataBase</code> being URL.

`--input`

Path to a file with patch to apply. Alternatively, you can use [Standard Input](#) or [URL <remote_input_output>](#).

```
# standard input (default)
```

```
--input in
```

```
--input con
```

```
# absolute path (windows)
```

```
--input "c:\my app\gamedata_patch.json"
```

```
# absolute path (unix)
```

```
--input "/user/data/gamedata_patch.json"
```

```
# relative path (universal)
```

```
--input "./gamedata_patch.json"
```

```
# remote location (HTTP)
```

```
--input "http://example.com/gamedata_patch.json"
```

```
# remote location with authentication (FTP)
```

```
--input "ftp://user:password@example.com/gamedata_patch.json"
```

`--inputFormat`

Format of imported data.

```
# Auto-detect by extension (default)
```

```
--inputFormat auto
```

```
# JSON
```

```
--inputFormat json
```

```
# BSON
```

```
--inputFormat bson
```

```
# Message Pack
```

```
--inputFormat msgpack
```

```
# XML (removed in 2025.1.1)
```

```
--inputFormat xml
```

`--inputFormattingOptions`

Additional options for specified format.

This command supports universal parameters.

See also

- Patch and Diff Workflow
- DATA CREATEPATCH
- Merge Logic
- CI/CD Integration

Create Backup

Backs up game data to a specified file. Saved data could be later used with DATA RESTORE command. Also this command can be used to convert game data into different format.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA BACKUP \
  --dataBase "c:\my app\gamedata.json" \
  --output "c:\my app\backup.msgpkg" \
  --outputFormat msgpack

# remote game data
dnx dotnet-charon -- DATA BACKUP \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --output "./backup.msgpkg" \
  --outputFormat msgpack \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of <code>--dataBase</code> being URL.
--output	Path to a backup file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to Standard Error , Standard Output , <code>/dev/null</code> , or a URL. <pre># standard output (default) --output out --output con # standard error --output err # null device --output null # absolute path (windows) --output "c:\my app\backup.json" # absolute path (unix) --output "/user/data/backup.json" # relative path (universal) --output "./backup.json" # remote location (HTTP) --output "http://example.com/backup.json" # remote location with authentication (FTP) --output "ftp://user:password@example.com/backup.json"</pre>

--outputFormat

Format of backed up data.

```
# JSON (default)
```

```
--outputFormat json
```

```
# Message Pack
```

```
--outputFormat msgpack
```

--outputFormattingOptions

Additional options for specified format.

This command supports universal parameters.

Output

The back up data follows the general game data structure.

See also

- Backup and Restore
- DATA RESTORE
- DATA EXPORT
- CI/CD Integration

Create Document

Creates a new document. For a bulk creations use DATA IMPORT command with --mode create. Only the first document from the --input will be processed.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA CREATE \
  --dataBase "c:\my app\gamedata.json" \
  --schema Item \
  --input "c:\my app\item.json" \
  --inputFormat json

# remote game data
dnx dotnet-charon -- DATA CREATE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schema Item \
  --input "./item.json" \
  --inputFormat json \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <pre># local file --dataBase "c:\my app\gamedata.json"</pre> <pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of --dataBase being URL.
--schema	Name or identifier of the type (schema) of the new document. <pre># name --schema Item</pre> <pre># id --schema 55a4f32faca22e191098f3d9</pre>
--validationOptions	List of validation checks and repairs to perform during document creation. <pre># no checks --validationOptions none</pre> <pre># repairs --validationOptions repair --validationOptions repair deduplicateIds --validationOptions repair repairRequiredWithDefaultValue --validationOptions repair eraseInvalidValue</pre> <pre># checks (default) --validationOptions checkTranslation --validationOptions checkRequirements --validationOptions checkFormat --validationOptions checkUniqueness --validationOptions checkReferences --validationOptions checkSpecification --validationOptions checkConstraints</pre>
--input	Path to a file with document. Alternatively, you can use Standard Input or URL. <pre># standard input (default) --input in --input con</pre> <pre># absolute path (windows) --input "c:\my app\item.json"</pre> <pre># absolute path (unix) --input "/user/data/item.json"</pre> <pre># relative path (universal) --input "./item.json"</pre> <pre># remote location (HTTP) --input "http://example.com/item.json"</pre> <pre># remote location with authentication (FTP) --input "ftp://user:password@example.com/item.json"</pre>

--inputFormat	Format of imported data. <pre># Auto-detect by extension (default) --inputFormat auto # JSON --inputFormat json # BSON --inputFormat bson # Message Pack --inputFormat msgpack # XML (removed in 2025.1.1) --inputFormat xml</pre>
--inputFormattingOptions	Additional options for specified format.
--output	Path to a created document file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to Standard Error, Standard Output, /dev/null, or a URL. <pre># standard output --output out --output con # standard error --output err # null device (default) --output null # absolute path (windows) --output "c:\my app\created_item.json" # absolute path (unix) --output /user/data/created_item.json # relative path (universal) --output "./created_item.json" # remote location (HTTP) --output "http://example.com/created_item.json" # remote location with authentication (FTP) --output "ftp://user:password@example.com/created_item.json"</pre>
--outputFormat	Format of created data. <pre># JSON (default) --outputFormat json # BSON --outputFormat bson # Message Pack --outputFormat msgpack # XML (removed in 2025.1.1) --outputFormat xml</pre>
--outputFormattingOptions	Additional options for specified format.

This command supports universal parameters.

Input Data Schema

The data you input should follow this schema (recommended):

```
{
  "Collections": {
    "<Schema-Name>": [
      {
        // <Document>
      }
    ]
  }
}
```

This schema is also accepted:

```
{
  "<Schema-Name>": [
    {
      // <Document>
    }
  ]
}
```

A list of documents is accepted:

```
[
  {
    // <Document>
  }
]
```

And single document too:

```
{
  // <Document>
}
```

Output

Outputs the created document with all the edits that were made to make it conform to the schema.

```
{
  "Id": "Sword"

  /* rest of properties of created document */
}
```

See also

- DATA IMPORT
- DATA UPDATE
- DATA DELETE
- Importing and Exporting Data

Create Patch (Diff)

Outputs the differences between two game datas as a file that can be used later to DATA APPLYPATCH to another game data.

- CLI Installation

- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA CREATEPATCH \
  --dataBase1 "c:\my app\gamedata_base.json" \
  --dataBase2 "c:\my app\gamedata_modified.json" \
  --output "./gamedata_patch.json" \
  --outputFormat json

# remote game data
dnx dotnet-charon -- DATA CREATEPATCH \
  --dataBase1 "https://charon.live/view/data/My_Game/develop/" \
  --dataBase2 "https://charon.live/view/data/My_Game/staging/" \
  --output "./gamedata_staging_patch.json" \
  --outputFormat json \
  --credentials "<API-Key>"
```

Parameters

--dataBase1	Absolute or relative path to a first game data. Use quotation marks if your path contains spaces. # local file --dataBase1 "c:\my app\gamedata.json" # remote server --dataBase1 "https://charon.live/view/data/My_Game/develop/"
--dataBase2	Absolute or relative path to a second game data. Use quotation marks if your path contains spaces. # local file --dataBase2 "c:\my app\gamedata.json" # remote server --dataBase2 "https://charon.live/view/data/My_Game/develop/"

`--output`

Path to a patch file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

```
# standard output (default)
```

```
--output out
```

```
--output con
```

```
# standard error
```

```
--output err
```

```
# null device
```

```
--output null
```

```
# absolute path (windows)
```

```
--output "c:\my app\gamedata_patch.json"
```

```
# absolute path (unix)
```

```
--output /user/data/gamedata_patch.json
```

```
# relative path (universal)
```

```
--output "./gamedata_patch.json"
```

```
# remote location (HTTP)
```

```
--output "http://example.com/gamedata_patch.json"
```

```
# remote location with authentication (FTP)
```

```
--output "ftp://user:password@example.com/gamedata_patch.json"
```

`--outputFormat`

Format of exported data.

```
# JSON (default)
```

```
--outputFormat json
```

```
# BSON
```

```
--outputFormat bson
```

```
# Message Pack
```

```
--outputFormat msgpack
```

```
# XML (removed in 2025.1.1)
```

```
--outputFormat xml
```

`--outputFormattingOptions`

Additional options for specified format.

`--credentials`

This parameter sets the API key used to access *BOTH* remote servers. If this is not suitable, consider downloading the data locally and running this command on local files instead.

This command supports universal parameters.

See also

- Patch and Diff Workflow
- DATA APPLYPATCH
- DATA BACKUP
- CI/CD Integration

Delete Document

Deletes a document. For a bulk deletion use DATA IMPORT command with `--mode delete`.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA DELETE \
  --dataBase "c:\my app\gamedata.json" \
  --schema Item \
  --id "Sword"

# remote game data
dnx dotnet-charon -- DATA DELETE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schema Item \
  --id "Sword" \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of <code>--dataBase</code> being URL.
--schema	Name or identifier of the type (schema) of deleting document.
	<pre># name --schema Item # id --schema 55a4f32faca22e191098f3d9</pre>
--id	Identifier of deleting document.
	<pre># text --id Sword # number --id 101</pre>

--output

The path to a file where the *deleted document* should be placed. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

```
# standard output
--output out
--output con

# standard error
--output err

# null device (default)
--output null

# absolute path (windows)
--output "c:\my app\deleted_item.json"

# absolute path (unix)
--output /user/data/deleted_item.json

# relative path (universal)
--output "../deleted_item.json"

# remote location (HTTP)
--output "http://example.com/deleted_item.json"

# remote location with authentication (FTP)
--output "ftp://user:password@example.com/deleted_item.json"
```

--outputFormat

Format for deleted document.

```
# JSON (default)
--outputFormat json

# BSON
--outputFormat bson

# Message Pack
--outputFormat msgpack

# XML (removed in 2025.1.1)
--outputFormat xml
```

--outputFormattingOptions

Additional options for specified format.

This command supports universal parameters.

Output

Outputs the deleted document in its state at the time of deletion.

```
{
  "Id": "Sword"

  /* rest of properties of deleted document */
}
```

See also

- DATA IMPORT
- DATA CREATE
- DATA UPDATE

- DATA FIND

Examine Game Data

Added in version 2026.4.0.

Returns statistics about game data: document and validation error counts per schema, metadata hash, and revision hash.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA EXAMINE \
  --dataBase "c:\my app\gamedata.json"

# remote game data
dnx dotnet-charon -- DATA EXAMINE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --credentials "<API-Key>"
```

Use this command to verify that a file is valid game data and get a content overview - document counts per schema, a metadata hash, and a revision hash - before running a heavier command such as DATA LIST or DATA SEARCH.

Examine runs a full validation pass over the data and reports how many errors of each kind every collection has. This is the same pass that produces the error counts shown in the editor's document lists and in the publication review, and it uses the same profile: all integrity checks plus checkTranslations. Use DATA VALIDATE instead when you need the individual errors rather than the counts, or when you need to choose which checks run.

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of --dataBase being URL.

--output

Path to the statistics output file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

```
# standard output (default)
```

```
--output out
```

```
--output con
```

```
# standard error
```

```
--output err
```

```
# null device
```

```
--output null
```

```
# absolute path (windows)
```

```
--output "c:\my app\stats.json"
```

```
# absolute path (unix)
```

```
--output /user/data/stats.json
```

```
# relative path (universal)
```

```
--output "./stats.json"
```

```
# remote location (HTTP)
```

```
--output "http://example.com/stats.json"
```

```
# remote location with authentication (FTP)
```

```
--output "ftp://user:password@example.com/stats.json"
```

--outputFormat

Format for serializing the output.

```
# JSON (default)
```

```
--outputFormat json
```

```
# BSON
```

```
--outputFormat bson
```

```
# Message Pack
```

```
--outputFormat msgpack
```

--outputFormattingOptions

Additional formatting options for the chosen output format.

This command supports universal parameters.

Output

```

{
  "metadataHash": "3f1c2a9e7b0d4c5e8a1f6d2b",
  "revisionHash": "678f998993a22d1f54b7fa80",
  "toolsVersion": "2025.1.1.0",
  "changeNumber": 42,
  "collections":
  [
    {
      "schemaName": "Character",
      "schemaId": "592fc86c983a36266c0912b1",
      "documentCount": 128,
      "rootDocumentCount": 128,
      "validationErrorCount": {}
    },
    {
      "schemaName": "Item",
      "schemaId": "592fc86c983a36266c0912a0",
      "documentCount": 340,
      "rootDocumentCount": 210,
      "validationErrorCount": {}
    }
    // ...
  ]
}

```

documentCount includes documents embedded inside other documents; rootDocumentCount counts only top-level documents in the collection. validationErrorCount maps validation error codes to their occurrence count and is empty when no errors were found during the examine pass.

See also

- DATA FINDFILES
- DATA VALIDATE
- DATA LIST

Export Data

Exports documents into a file.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA EXPORT \
  --dataBase "c:\my app\gamedata.json" \
  --schemas Character \
  --output "c:\my app\characters.json" \
  --outputFormat json

# remote game data
dnx dotnet-charon -- DATA EXPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schemas Character \
  --output "./characters.json" \
  --outputFormat json \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of <i>--dataBase</i> being URL.
--schemas	A list of types of documents (schemas) to export. By default all schemas <i>EXCEPT</i> metadata are exported. • Use space to separate multiple schemas. • You can use wildcards (*) at the beginning and end of names. • You can use identifiers in {} instead of names. • You can exclude certain names by using an exclamation mark (!) at the beginning of their names. <pre># schema name --schemas Character --schemas Character Item # all (default) --schemas * # masks --schemas Char* --schemas *Modifier --schemas *Mod* # schema id --schemas {18d4bf318f3c49688087dbed} # negation --schemas Char* !Character --schemas !*Item* # excluding system schemas (Schema, SchemaProperty, ProjectSettings) --schemas ![system]</pre>

--properties

A list of properties or property types to export. By default all properties are exported.

- *Id* property always included
- Use space to separate multiple properties.
- You can use wildcards (*) at the beginning and end of names.
- You can use identifiers in {} instead of names.
- You can exclude certain names by using an exclamation mark (!) at the beginning of their names.
- You can use data type in [] instead of names.

--languages

List of languages to keep in exported data. Language's [english name](#) is used or [language tag \(BCP 47\)](#).

Use DATA I18N LANGUAGES to get list of used languages.

- Use space to separate multiple languages
- You can use wildcards (*) at the beginning and end of names.
- You can use LCID or [CultureInfo.Name](#) in {} instead of the name.
- You can exclude certain names by using an exclamation mark (!) at the beginning of their names.

language tag (BCP 47)

--languages {en-US}

language name

--languages "Spanish (Spain)"

language name mask

--languages Spanish*

language LCID

--languages {3082}

negation and masks

--languages !Spanish*

--languages Spanish* !{es-Es}

--mode

Export mode controls stripping and inclusion rules for exported data.

```
# (default)
```

```
--mode normal
```

```
--mode publication
```

```
--mode extraction
```

```
--mode localization
```

normal

Export all specified documents defined in *--schemas*. This mode ensures that the exported graph of documents remains valid by including any necessary additional documents to avoid any broken references.

publication

Same as *--mode normal*, but shaped for the game: localization for languages outside *--languages* is stripped, and the file level `ToolsVersion`, `RevisionHash` and `ChangeNumber` fields are written into the output. System schemas and the project settings document are kept - the generated code requires them and refuses to load a file without them.

extraction

Export only the specified *--schemas* without exporting any referenced documents. In this mode, the exported graph of documents may contain broken references. It is recommended to use the import *--mode safeupdate* when importing this data back.

localization

Same as *--mode extraction* but only `LocalizedText` properties are exported.

`--output`

Path to a exported data file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), `/dev/null`, or a URL.

```
# standard output (default)
```

```
--output out
```

```
--output con
```

```
# standard error
```

```
--output err
```

```
# null device
```

```
--output null
```

```
# absolute path (windows)
```

```
--output "c:\my app\document.json"
```

```
# absolute path (unix)
```

```
--output /user/data/document.json
```

```
# relative path (universal)
```

```
--output "./document.json"
```

```
# remote location (HTTP)
```

```
--output "http://example.com/document.json"
```

```
# remote location with authentication (FTP)
```

```
--output "ftp://user:password@example.com/document.json"
```

`--outputFormat`

Format of exported data.

```
# JSON (default)
```

```
--outputFormat json
```

```
# BSON
```

```
--outputFormat bson
```

```
# Message Pack
```

```
--outputFormat msgpack
```

```
# XML (removed in 2025.1.1)
```

```
--outputFormat xml
```

```
# XLSX Spreadsheet
```

```
--outputFormat xlsx
```

`--outputFormattingOptions`

Additional options for specified format.

This command supports universal parameters.

Output

The exported data follows the general game data structure, but omits *ToolsVersion*, *RevisionHash*, and *ChangeNumber* when the export mode is **not** set to publication.

```
{
  "Collections":
  {
    "Character":
    [
      {
        "Id": "Knight"

        /* rest of properties of document */
      }
    ]
  }
}
```

```

    },
    {
      "Id": "Templar"

      /* rest of properties of document */
    },
    // ...
  ]
}
}

```

Modifying Exported Data with *yq*

The exported data can be accessed or modified using the *yq* tool, a lightweight and portable command-line YAML, JSON, and XML processor. *yq* uses *jq*-like syntax and supports common operations for manipulating structured data.

To use *yq* with exported JSON data:

1. **Install *yq***: Follow the installation instructions from the official *yq* documentation: <https://mikefarah.gitbook.io/yq/>.

2. **Query Data**: Use *yq* to query specific fields or values from the exported JSON file.

```

# Query a specific field
yq '.Collections.Character[0].name' characters.json

```

3. **Modify Data**: Use *yq* to update or add fields in the exported JSON file.

```

# Export data
dnx dotnet-charon -- DATA EXPORT \
  --dataBase gamedata.json \
  --schemas Character \
  --output characters.json

```

```

# Update a field
yq -i '.Collections.Character[0].name = "New Name"' characters.json

```

```

# Add a new field
yq -i '.Collections.Character[0].level = 10' characters.json

```

```

# Import data back
dnx dotnet-charon -- DATA IMPORT \
  --dataBase gamedata.json \
  --schemas Character \
  --input characters.json \
  --mode safeUpdate

```

4. **Convert Formats**: *yq* can also convert between JSON, YAML, and other supported formats.

```

# Convert JSON to YAML
yq -o=yaml characters.json > characters.yaml

```

For more advanced usage, refer to the *yq* documentation: <https://mikefarah.gitbook.io/yq/>.

See also

- Importing and Exporting Data
- DATA IMPORT
- DATA I18N EXPORT
- DATA BACKUP
- Game Data Structure
- CI/CD Integration

Find Document

Seaches for a document.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA FIND \
  --dataBase "c:\my app\gamedata.json" \
  --schema Character \
  --id John \
  --output "c:\my app\character.json"

# remote game data
dnx dotnet-charon -- DATA FIND \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schema Character \
  --id John \
  --output out \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of <i>--dataBase</i> being URL.
--schema	Name or identifier of the type (schema) of document.
	<pre># name --schema Item # id --schema 55a4f32faca22e191098f3d9</pre>
--id	Identifier of document.
	<pre># text --id Sword # number --id 101</pre>

`--output`

Path to a found document file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), `/dev/null`, or a URL.

```
# standard output (default)
--output out
--output con

# standard error
--output err

# null device
--output null

# absolute path (windows)
--output "c:\my app\document.json"

# absolute path (unix)
--output /user/data/document.json

# relative path (universal)
--output "./document.json"

# remote location (HTTP)
--output "http://example.com/document.json"

# remote location with authentication (FTP)
--output "ftp://user:password@example.com/document.json"
```

`--outputFormat`

Format of exported data.

```
# JSON (default)
--outputFormat json

# BSON
--outputFormat bson

# Message Pack
--outputFormat msgpack

# XML (removed in 2025.1.1)
--outputFormat xml
```

`--outputFormattingOptions`

Additional options for specified format.

This command supports universal parameters.

Output

Outputs the found document.

```
{
  "Id": "John"

  /* rest of properties of found document */
}
```

See also

- DATA LIST
- DATA SEARCH
- DATA CREATE

- DATA UPDATE
- DATA DELETE
- DATA PROJECTSETTINGS

Find Game Data Files

Added in version 2026.4.0.

Scans a folder for game data files and returns discovered paths with their collection statistics.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# scan a folder tree, JSON output (default)
dnx dotnet-charon -- DATA FINDFILES \
  --folder "c:\my app\game-content"

# scan a folder tree, one path per line for shell piping
dnx dotnet-charon -- DATA FINDFILES \
  --folder "c:\my app\game-content" \
  --outputFormat list \
  --output out | xargs -I{} dnx dotnet-charon -- DATA EXAMINE --dataBase "{}"
```

Use this command to bootstrap work in an unfamiliar directory tree - “what game data lives here?” - before picking a file to pass to DATA EXAMINE, DATA LIST, or DATA SEARCH. Only files with recognized formatter extensions (.json, .bson, .msgpack, .xlsx, ...) are considered, and each candidate is probed - reading only its first 4 KB for the Collections marker - before a full load, so the scan stays fast even on large directory trees full of unrelated files.

Parameters

--folder

Path to the folder to scan.

local folder

```
--folder "c:\my app\game-content"
```

```
--folder /home/user/game-content
```

--recursive

When true (default), scans all sub-folders recursively.

scan sub-folders too (default)

```
--recursive true
```

scan only the top-level folder

```
--recursive false
```

--outputFormat **O** Output shape.

JSON array of `{path, collections}` objects, matching the MCP `data\_find\_files`

```
--outputFormat json
```

one file path per line, for shell piping (e.g. `xargs`, `for` loops)

```
--outputFormat list
```

--output

Specifies where to write the output data.

```

# standard output (default)
--output out
--output con

# standard error
--output err

# null device
--output null

# absolute path (windows)
--output "c:\my app\found-files.json"

# absolute path (unix)
--output /user/data/found-files.json

# relative path (universal)
--output "./found-files.json"

# remote location (HTTP)
--output "http://example.com/found-files.json"

# remote location with authentication (FTP)
--output "ftp://user:password@example.com/found-files.json"

```

--credentials

Supplies authentication credentials for accessing discovered files if they're located on a remote server. Credentials can also be provided via the CHARON_API_KEY environment variable.

This command supports universal parameters.

Output

--outputFormat json (default) writes an array of {path, collections} objects, where collections maps each schema name found in that file to its document count:

```

[
  {
    "path": "C:/my app/game-content/characters.json",
    "collections":
    {
      "Character": 128,
      "Item": 340
    }
  },
  {
    "path": "C:/my app/game-content/quests.xlsx",
    "collections":
    {
      "Quest": 56
    }
  }
]

```

--outputFormat list writes one absolute file path per line, with no other formatting - suitable for piping into xargs or a shell for loop:

```

C:/my app/game-content/characters.json
C:/my app/game-content/quests.xlsx

```


See also

- DATA EXAMINE
- DATA LIST
- DATA SEARCH

Add Translation Languages

Add translation languages to specified game data.

- CLI Installation
- Universal Parameters
- Commands Reference

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA I18N ADDLANGUAGE \
  --dataBase "c:\my app\gamedata.json" \
  --languages "es-ES" "en-GB"

# remote game data
dnx dotnet-charon -- DATA I18N ADDLANGUAGE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --languages "es-ES" "en-GB" \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--languages	The list of languages to add. Values are language tags (BCP 47) separated by space..
--credentials	The API key used to access remote server in case of <code>--dataBase</code> being URL.
--output	Path to the file receiving the updated language list. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to Standard Error , Standard Output , <code>/dev/null</code> , or a URL.
	<pre># standard output (default) --output out --output con # standard error --output err # null device --output null # relative path (universal) --output "./languages.json"</pre>

`--outputFormat`

Format of the returned language list.

JSON (default)`--outputFormat json`*# space separated list*`--outputFormat list`*# new line (OS specific) separated list*`--outputFormat table``--outputFormattingOptions`

Additional options for specified format.

This command supports universal parameters.

See also

- Internationalization
- DATA I18N LANGUAGES
- DATA I18N EXPORT
- DATA I18N IMPORT
- [BCP 47 language tags](#)

Export Translated Data

Export text that can be translated into a file.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA I18N EXPORT \
  --dataBase "c:\my app\gamedata.json" \
  --schemas Character \
  --sourceLanguage en-US \
  --targetLanguage fr \
  --output "c:\my app\character_loc.xliff" \
  --outputFormat xliff

# remote game data
dnx dotnet-charon -- DATA I18N EXPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schemas Character \
  --sourceLanguage en-US \
  --targetLanguage fr \
  --output "./character_loc.xliff" \
  --outputFormat xliff \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <pre># local file --dataBase "c:\my app\gamedata.json"</pre> <pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of --dataBase being URL.
--schemas	A list of types of documents (schemas) to export. By default all schemas <i>EXCEPT</i> metadata are exported. • Use space to separate multiple schemas. • You can use wildcards (*) at the beginning and end of names. • You can use identifiers in {} instead of names. • You can exclude certain names by using an exclamation mark (!) at the beginning of their names. <pre># schema name --schemas Character --schemas Character Item</pre> <pre># all (default) --schemas *</pre> <pre># masks --schemas Char* --schemas *Modifier --schemas *Mod*</pre> <pre># schema id --schemas {18d4bf318f3c49688087dbed}</pre> <pre># negation --schemas Char* !Character --schemas !*Item*</pre> <pre># excluding system schemas (Schema, SchemaProperty, ProjectSettings) --schemas ![system]</pre>
--sourceLanguage	Source (original) language for translation. Value is language tag (BCP 47). Use DATA I18N LANGUAGES to get list of used languages. <pre># it is used as <source> in XLIFF --sourceLanguage en-US</pre>
--targetLanguage	Target language for translation. Value is language tag (BCP 47). <pre># it is used as <target> in XLIFF --targetLanguage es-ES</pre>

--output

Path to a file to which data will be exported. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

```
# standard output (default)
```

```
--output out
```

```
--output con
```

```
# standard error
```

```
--output err
```

```
# null device
```

```
--output null
```

```
# absolute path (windows)
```

```
--output "c:\my app\input.json"
```

```
# absolute path (unix)
```

```
--output /user/data/input.json
```

```
# relative path (universal)
```

```
--output "./input.json"
```

```
# remote location (HTTP)
```

```
--output "http://example.com/input.json"
```

```
# remote location with authentication (FTP)
```

```
--output "ftp://user:password@example.com/input.json"
```

--outputFormat

Format of exported data.

```
# XLIFF v2 (default)
```

```
--outputFormat xliiff
```

```
--outputFormat xliiff2
```

```
# XLIFF v1
```

```
--outputFormat xliiff1
```

```
# XSLX Spreadsheet
```

```
--outputFormat xslx
```

```
# JSON
```

```
--outputFormat json
```

--outputFormattingOptions

Additional options for specified format.

This command supports universal parameters.

Output

The exported data follows the general game data structure, but omits *ToolsVersion*, *RevisionHash*, and *ChangeNumber* fields.

See also

- [Internationalization](#)
- [DATA I18N IMPORT](#)
- [DATA I18N LANGUAGES](#)
- [DATA EXPORT](#)
- [XLIFF standard](#)
- [BCP 47 language tags](#)

Importing Translated Data

Import translated text from a specified file into game data.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA I18N IMPORT \
  --dataBase "c:\my app\gamedata.json" \
  --input "c:\my app\character_loc.xliff" \
  --inputFormat xliff

# remote game data
dnx dotnet-charon -- DATA I18N IMPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --input "./character_loc.xliff" \
  --inputFormat xliff \
  --credentials "<API-Key>"
```

Parameters

<code>--dataBase</code>	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json"</pre>
	<pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
<code>--credentials</code>	The API key used to access remote server in case of <code>--dataBase</code> being URL.

`--schemas`

A list of types of documents (schemas) to import. By default all schemas *EXCEPT* metadata are imported.

- Use space to separate multiple schemas.
- You can use wildcards (*) at the beginning and end of names.
- You can use identifiers in {} instead of names.
- You can exclude certain names by using an exclamation mark (!) at the beginning of their names.

```
# schema name
```

```
--schemas Character
```

```
--schemas Character Item
```

```
# all (default)
```

```
--schemas *
```

```
# masks
```

```
--schemas Char*
```

```
--schemas *Modifier
```

```
--schemas *Mod*
```

```
# schema id
```

```
--schemas {18d4bf318f3c49688087dbed}
```

```
# negation
```

```
--schemas Char* !Character
```

```
--schemas !*Item*
```

```
# excluding system schemas (Schema, SchemaProperty, ProjectSettings)
```

```
--schemas ![system]
```

`--languages`

The list of languages to import. Values are [language tags \(BCP 47\)](#).

```
# Import specific language
```

```
--languages en-US es-ES
```

```
# Import all languages
```

```
--languages *
```

`--validationOptions`

List of validation checks and repairs to perform during import.

```
# no checks
```

```
--validationOptions none
```

```
# repairs
```

```
--validationOptions repair
```

```
--validationOptions repair deduplicateIds
```

```
--validationOptions repair repairRequiredWithDefaultValue
```

```
--validationOptions repair eraseInvalidValue
```

```
# checks (default)
```

```
--validationOptions checkTranslation
```

```
--validationOptions checkRequirements
```

```
--validationOptions checkFormat
```

```
--validationOptions checkUniqueness
```

```
--validationOptions checkReferences
```

```
--validationOptions checkSpecification
```

```
--validationOptions checkConstraints
```

--input

Path to a file with data to import. Alternatively, you can use [Standard Input](#) or URL.

See input data structure requirements.

```
# standard input (default)
```

```
--input in
```

```
--input con
```

```
# absolute path (windows)
```

```
--input "c:\my app\input.json"
```

```
# absolute path (unix)
```

```
--input /user/data/input.json
```

```
# relative path (universal)
```

```
--input "./input.json"
```

```
# remote location (HTTP)
```

```
--input "http://example.com/input.json"
```

```
# remote location with authentication (FTP)
```

```
--input "ftp://user:password@example.com/input.json"
```

--inputFormat

Format of imported data.

```
# Auto-detect by extension (default)
```

```
--inputFormat auto
```

```
# XLIFF v2
```

```
--inputFormat xliiff
```

```
--inputFormat xliiff2
```

```
# XLIFF v1
```

```
--inputFormat xliiff1
```

```
# XSLX Spreadsheet
```

```
--inputFormat xslx
```

--inputFormattingOptions

Additional options for specified format.

`--output`

Optional path to a import report file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

```
# standard output
--output out
--output con

# standard error
--output err

# null device (default)
--output null

# absolute path (windows)
--output "c:\my app\document.json"

# absolute path (unix)
--output /user/data/document.json

# relative path (universal)
--output "./document.json"

# remote location (HTTP)
--output "http://example.com/document.json"

# remote location with authentication (FTP)
--output "ftp://user:password@example.com/document.json"
```

`--outputFormat`

Format of import report.

```
# JSON (default)
--outputFormat json

# BSON
--outputFormat bson

# Message Pack
--outputFormat msgpack

# XLSX Spreadsheet
--outputFormat xlsx
```

`--outputFormattingOptions`

Additional options for specified format.

`--dryRun`

Allows you to run the command without actually making any changes to the game data, providing a preview of what would happen.

This command supports universal parameters.

See also

- [Internationalization](#)
- [DATA I18N EXPORT](#)
- [DATA I18N LANGUAGES](#)
- [DATA IMPORT](#)
- [XLIFF standard](#)
- [BCP 47 language tags](#)

List Translation Languages

Get a list of supported translation languages. Primary language always shows up first in the list.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA I18N LANGUAGES \
  --dataBase "c:\my app\gamedata.json" \
  --output out \
  --outputFormat table

# remote game data
dnx dotnet-charon -- DATA I18N LANGUAGES \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --output out \
  --outputFormat table \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json"</pre>
	<pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of <i>--dataBase</i> being URL.

`--output`

Path to language list file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), `/dev/null`, or a URL.

```
# standard output (default)
```

```
--output out
```

```
--output con
```

```
# standard error
```

```
--output err
```

```
# null device
```

```
--output null
```

```
# absolute path (windows)
```

```
--output "c:\my app\input.json"
```

```
# absolute path (unix)
```

```
--output /user/data/input.json
```

```
# relative path (universal)
```

```
--output "./input.json"
```

```
# remote location (HTTP)
```

```
--output "http://example.com/input.json"
```

```
# remote location with authentication (FTP)
```

```
--output "ftp://user:password@example.com/input.json"
```

`--outputFormat`

Format of exported data.

```
# JSON (default)
```

```
--outputFormat json
```

```
#> [
```

```
#>   "en-US",
```

```
#>   "es-ES",
```

```
#> ]
```

```
# Space separated list
```

```
--outputFormat list
```

```
#> en-US es-ES
```

```
# New line (OS specific) separated list
```

```
--outputFormat table
```

```
#> en-US
```

```
#> es-ES
```

`--outputFormattingOptions`

Additional options for specified format.

This command supports universal parameters.

See also

- Internationalization
- DATA I18N EXPORT
- DATA I18N IMPORT
- DATA I18N ADDLANGUAGE

- [BCP 47 language tags](#)

Import Data

Imports documents from file to a game data.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA IMPORT \
  --dataBase "c:\my app\gamedata.json" \
  --schemas Character \
  --input "c:\my app\characters.json" \
  --inputFormat json \
  --mode safeUpdate

# remote game data
dnx dotnet-charon -- DATA IMPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schemas Character \
  --input "./characters.json" \
  --inputFormat json \
  --mode safeUpdate \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json"</pre>
	<pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of <code>--dataBase</code> being URL.

--schemas

A list of types of documents (schemas) to import. By default all schemas *EXCEPT* metadata are imported.

- Use space to separate multiple schemas.
- You can use wildcards (*) at the beginning and end of names.
- You can use identifiers in {} instead of names.
- You can exclude certain names by using an exclamation mark (!) at the beginning of their names.

schema name

--schemas Character

--schemas Character Item

all (default)

--schemas *

masks

--schemas Char*

--schemas *Modifier

--schemas *Mod*

schema id

--schemas {18d4bf318f3c49688087dbed}

negation

--schemas Char* !Character

--schemas !*Item*

excluding system schemas (Schema, SchemaProperty, ProjectSettings)

--schemas ![system]

--mode

Import mode controls merge behavior during import.

(default)

--mode createAndUpdate

--mode create

--mode update

--mode safeUpdate

--mode replace

--mode delete

createAndUpdate

creates new documents and updates existing ones

create

only creates new documents, existing documents are kept unchanged

update

only updates existing documents, no new ones are created

safeUpdate

same as *update* but without creating, moving and erasing embedded documents

replace

replaces the entire collection with the imported documents

delete

deletes documents found in the imported data

```

--validationOptions List of validation checks and repairs to perform during import. Defaults to repair repairRequiredWithDefaults deduplicateIds resolveConflictingUnions - repairs only, with no integrity checks. Pass the check* flags explicitly to have the import verify the data, and see Validation for what each flag does.

# no checks
--validationOptions none

# all repairs
--validationOptions repair

# deduplicate documents with the same Id
--validationOptions repair deduplicateIds

# fill required fields that have a default value
--validationOptions repair repairRequiredWithDefaults

# erase values whose type does not match the property's data type
--validationOptions repair eraseInvalidValues

# fix malformed Union values:
#   empty union (no variant set)      → erased to null
#   conflicting union (2+ variants)   → last variant in schema order is
--validationOptions repair resolveConflictingUnions

# integrity checks - applied only when requested
--validationOptions checkTranslations
--validationOptions checkRequirements
--validationOptions checkFormat
--validationOptions checkUniqueness
--validationOptions checkReferences
--validationOptions checkSpecification
--validationOptions checkConstraints

--input Path to a data file. Alternatively, you can use Standard Input or URL.

# standard input (default)
--input in
--input con

# absolute path (windows)
--input "c:\my app\characters.json"

# absolute path (unix)
--input "/user/data/characters.json"

# relative path (universal)
--input "./characters.json"

# remote location (HTTP)
--input "http://example.com/characters.json"

# remote location with authentication (FTP)
--input "ftp://user:password@example.com/characters.json"

```

`--inputFormat`

Format of imported data.

Auto-detect by extension (default)`--inputFormat auto`*# JSON*`--inputFormat json`*# BSON*`--inputFormat bson`*# Message Pack*`--inputFormat msgpack`*# XML (removed in 2025.1.1)*`--inputFormat xml`*# XLSX Spreadsheet*`--inputFormat xlsx``--inputFormattingOptions`

Additional options for specified format.

`--output`

Optional path to a import report file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), `/dev/null`, or a URL.

standard output`--output out``--output con`*# standard error*`--output err`*# null device (default)*`--output null`*# absolute path (windows)*`--output "c:\my app\document.json"`*# absolute path (unix)*`--output /user/data/document.json`*# relative path (universal)*`--output "./document.json"`*# remote location (HTTP)*`--output "http://example.com/document.json"`*# remote location with authentication (FTP)*`--output "ftp://user:password@example.com/document.json"``--outputFormat`

Format of import report.

JSON (default)`--outputFormat json`*# BSON*`--outputFormat bson`*# Message Pack*`--outputFormat msgpack`*# XLSX Spreadsheet*`--outputFormat xlsx`

Is It Free?

--outputFormattingOptions
--dryRun

Additional options for specified format.

Allows you to run the command without actually making any changes to the game data, providing a preview of what would happen.

This command supports universal parameters.

Input Data Structure

The data you input should follow this structure (recommended):

```
{
  "Collections": {
    "<Schema-Name>": [
      {
        // <Document>
      }
    ]
  }
}
```

This structure is also accepted:

```
{
  "<Schema-Name>": [
    {
      // <Document>
    }
  ]
}
```

A list of documents is accepted if only one name in --schemas is specified:

```
[
  {
    // <Document>
  }
]
```

And single document is accepted too if only one name in --schemas is specified:

```
{
  // <Document>
}
```

See also

- Importing and Exporting Data
- DATA EXPORT
- DATA VALIDATE
- DATA I18N IMPORT
- Game Data Structure
- CI/CD Integration

List Documents

Seaches for a documents.

- CLI Installation

- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA LIST \
  --dataBase "c:\my app\gamedata.json" \
  --schema Character

# remote game data
dnx dotnet-charon -- DATA LIST \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schema Character \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<i># local file</i>
	--dataBase "c:\my app\gamedata.json"
	<i># remote server</i>
	--dataBase "https://charon.live/view/data/My_Game/develop/"
--credentials	The API key used to access remote server in case of --dataBase being URL.
--schema	Name or identifier of the type (schema) of document.
	<i># name</i>
	--schema Item
	<i># id</i>
	--schema 55a4f32faca22e191098f3d9

--filters Document filter expressions.

```
# patterns
--filters <Field> <Operator> <Value> [<Field> <Operator> <Value>...]

# single expression
--filters Id > 10
--filters Name like "Zombie"

# multiple expressions
--filters Id > 10 Name like "Zombie"

# greater than
--filters Id > 0
--filters Id greaterThan 0

# greater than or equal
--filters Id >= 0
--filters Id greaterThanOrEqual 0

# less than
--filters Id < 0
--filters Id lessThan 0

# less than or equal
--filters Id <= 0
--filters Id LessThanOrEqual 0

# equal
--filters Id = 0
--filters Id == 0
--filters Id equal 0

# not equal
--filters Id <> 0
--filters Id != 0
--filters Id notEqual 0

# like - is used to search for specific patterns in a field, allowing for partial
--filters Name like "Zombie"
```

--sorters

Document sort expressions.

```
# patterns
--sorters <Field> ASC|DESC [<Field> ASC|DESC ...]

# ascending
--sorters Name ASC

# descending
--sorters Name DESC
```

--select

Projection of specific fields to include in the output. Use the format <PropertyName>, e.g., Name or Id. Useful for reducing output size.

```
# only names and ids
--select Id Name
```

--path	Embeddane path filter. Could be used to get only embedded documents. <pre># any path --path *</pre> <pre># root documents (default) --path ""</pre> <pre># in 'Item' property --path /Item</pre>
--skip	Number of found documents to skip. <pre># skip first ten documents after applying --filter and --sort --skip 10</pre>
--take	Max amount to documents return. <pre># limit to first 100 documents after --skip --take 100</pre>
--output	Path to a found document file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to Standard Error, Standard Output, /dev/null, or a URL. <pre># standard output (default) --output out --output con</pre> <pre># standard error --output err</pre> <pre># null device --output null</pre> <pre># absolute path (windows) --output "c:\my app\document.json"</pre> <pre># absolute path (unix) --output /user/data/document.json</pre> <pre># relative path (universal) --output "./document.json"</pre> <pre># remote location (HTTP) --output "http://example.com/document.json"</pre> <pre># remote location with authentication (FTP) --output "ftp://user:password@example.com/document.json"</pre>
--outputFormat	Format of exported data. <pre># JSON (default) --outputFormat json</pre> <pre># BSON --outputFormat bson</pre> <pre># Message Pack --outputFormat msgpack</pre> <pre># XML (removed in 2025.1.1) --outputFormat xml</pre>
--outputFormattingOptions	Additional options for specified format.

This command supports universal parameters.

Output

The exported data follows the general game data structure, but omits *ToolsVersion*, *RevisionHash*, and *ChangeNumber*.

```
{
  "Collections":
  {
    "Character":
    [
      {
        "Id": "Knight"

        /* rest of properties of document */
      },
      {
        "Id": "Templar"

        /* rest of properties of document */
      },
      // ...
    ]
  }
}
```

See also

- DATA FIND
- DATA SEARCH
- DATA LISTSCHEMAS
- DATA EXPORT
- Game Data Structure

List Schemas

Added in version 2026.4.0.

Lists schema definitions (document types) configured in game data.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA LISTSCHEMAS \
  --dataBase "c:\my app\gamedata.json"

# remote game data
dnx dotnet-charon -- DATA LISTSCHEMAS \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --credentials "<API-Key>"
```

This command lists the built-in Schema collection itself - the schema *documents* (definitions: name, display name, properties, id generator, and so on), not the documents of a particular schema. It is the same listing capability as DATA LIST, just scoped internally to Schema instead of an arbitrary --schema you provide. For full metadata (including computed/derived information used when generating source code), use the data_get_metadata MCP tool or the metadata endpoint of a running server instead.

Parameters

```
--dataBase          Absolute or relative path to game data. Use quotation marks if your path
                    contains spaces.

                    # local file
                    --dataBase "c:\my app\gamedata.json"

                    # remote server
                    --dataBase "https://charon.live/view/data/My_Game/develop/"

--credentials        The API key used to access remote server in case of
                    --dataBase being URL.

--filters            Schema filter expressions. Use the format
                    <PropertyName> <Operation> <Value>, e.g., Name == "Item". Multiple
                    filters are ANDed together. For deeper paths use JSON Pointer format. Example:
                    ['/Item/Level', '>=', '10'].

                    # patterns
                    --filters <Field> <Operator><Value> [<Field> <Operator> <Value>...]

                    # single expression
                    --filters Name == "Item"
                    --filters DisplayName like "Item*"

                    # multiple expressions
                    --filters Type == 0 Name like "*Item*"

--sorters            Sorting rules for the result set. Use the format
                    <PropertyName> <ASC|DESC>, e.g., Name ASC. You can
                    specify multiple sorters to sort by multiple fields. For deeper
                    paths use JSON Pointer format. Example:
                    ['/Item/Name', 'ASC'].

                    # patterns
                    --sorters <Field> ASC|DESC [<Field> ASC|DESC ...]

                    # ascending
                    --sorters Name ASC

                    # descending
                    --sorters Name DESC

--select            Projection of specific fields to include in the output. Use
                    the format <PropertyName>, e.g., Name or Id. Useful
                    for reducing output size.

                    # only names and ids
                    --select Id Name

--skip              Number of schema definitions to skip from the start of the result set. Use for
                    pagination.

                    # skip first ten schemas after applying --filters and --sorters
                    --skip 10

--take              Number of schema definitions to return after skipping.
                    Use for limiting result size in combination with --skip.

                    # limit to first 100 schemas after --skip
                    --take 100
```

--output

Path to a found document file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

```
# standard output (default)
```

```
--output out
```

```
--output con
```

```
# standard error
```

```
--output err
```

```
# null device
```

```
--output null
```

```
# absolute path (windows)
```

```
--output "c:\my app\schemas.json"
```

```
# absolute path (unix)
```

```
--output /user/data/schemas.json
```

```
# relative path (universal)
```

```
--output "./schemas.json"
```

```
# remote location (HTTP)
```

```
--output "http://example.com/schemas.json"
```

```
# remote location with authentication (FTP)
```

```
--output "ftp://user:password@example.com/schemas.json"
```

--outputFormat

Format of exported data.

```
# JSON (default)
```

```
--outputFormat json
```

```
# BSON
```

```
--outputFormat bson
```

```
# Message Pack
```

```
--outputFormat msgpack
```

```
# XLSX Spreadsheet
```

```
--outputFormat xlsx
```

--outputFormattingOptions

Additional options for specified format.

This command supports universal parameters.

Output

The result follows the general game data structure, with schema documents under the Schema collection key. See the Schema section for the full field reference.

```
{
  "Collections":
  {
    "Schema":
    [
      {
        "Id": "592fc86c983a36266c0912a0",
        "Name": "Item",
        "DisplayName": "Items",
        "Type": 0,
        "Description": "An item.",
        "IdGenerator": 1,

```

```

    "Specification": "icon=fugue16%2Fabacus&group=Metagame",
    "Properties": [
        // schema property
    ]
},
{
    "Id": "592fc86c983a36266c0912b1",
    "Name": "Character"

    /* rest of properties of schema document */
},
// ...
]
}
}

```

See also

- DATA LIST
- DATA SEARCH
- DATA FINDFILES
- Game Data Structure

Project Settings

Added in version 2026.4.0.

Retrieves the project settings document (languages, primary language, and other project-level configuration) from game data.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```

# local game data (windows)
dnx dotnet-charon -- DATA PROJECTSETTINGS \
  --dataBase "c:\my app\gamedata.json"

# remote game data
dnx dotnet-charon -- DATA PROJECTSETTINGS \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --credentials "<API-Key>"

```

There is always exactly one project settings document per game data source, so unlike DATA FIND this command takes no --schema/--id parameters - its identity is fixed internally.

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <pre># local file --dataBase "c:\my app\gamedata.json"</pre> <pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of --dataBase being URL.
--output	Path to the retrieved document file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to Standard Error, Standard Output, /dev/null, or a URL. <pre># standard output (default) --output out --output con # standard error --output err # null device --output null # absolute path (windows) --output "c:\my app\settings.json" # absolute path (unix) --output /user/data/settings.json # relative path (universal) --output "./settings.json" # remote location (HTTP) --output "http://example.com/settings.json" # remote location with authentication (FTP) --output "ftp://user:password@example.com/settings.json"</pre>
--outputFormat	Specifies the format of the output data. <pre># JSON (default) --outputFormat json # BSON --outputFormat bson # Message Pack --outputFormat msgpack</pre>
--outputFormattingOptions	Additional options for formatting the output.

This command supports universal parameters.

Output

Outputs the project settings document. See the Project Settings section for the full field reference.

```
{
  "Id": "55a4f32faca22e191098f3d9",
  "Name": "My Project",
  "PrimaryLanguage": "en-US",
```

```

"Languages": "en-GB;fr-FR",
"Copyright": "My Company (c) 2025",
"Version": "1.0.0.0"
}

```

See also

- DATA UPDATEPROJECTSETTINGS
- DATA I18N LANGUAGES
- DATA FIND
- Game Data Structure

Restore from Backup

Restores game data from a file created by DATA BACKUP command.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```

# local game data (windows)
dnx dotnet-charon -- DATA RESTORE \
  --dataBase "c:\my app\gamedata.json" \
  --input "c:\my app\backup.msgpkg" \
  --inputFormat msgpack

# remote game data
dnx dotnet-charon -- DATA RESTORE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --input "./backup.msgpkg" \
  --inputFormat msgpack \
  --credentials "<API-Key>"

```

Parameters

- | | |
|----------------------------|--|
| <code>--dataBase</code> | <p>Absolute or relative path to game data. Use quotation marks if your path contains spaces.</p> <pre># local file --dataBase "c:\my app\gamedata.json"</pre> <pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre> |
| <code>--credentials</code> | <p>The API key used to access remote server in case of <code>--dataBase</code> being URL.</p> |

--input

Path to a backup file. Alternatively, you can use [Standard Input](#) or URL.

```
# standard input (default)
--input in
--input con

# absolute path (windows)
--input "c:\my app\backup.json"

# absolute path (unix)
--input "/user/data/backup.json"

# relative path (universal)
--input "./backup.json"

# remote location (HTTP)
--input "http://example.com/backup.json"

# remote location with authentication (FTP)
--input "ftp://user:password@example.com/backup.json"
```

--inputFormat

Format of imported data.

```
# Auto-detect by extension (default)
--inputFormat auto

# JSON
--inputFormat json

# Message Pack
--inputFormat msgpack
```

--inputFormattingOptions

Additional options for specified format.

This command supports universal parameters.

See also

- Backup and Restore
- DATA BACKUP
- CI/CD Integration

Search Documents

Added in version 2026.4.0.

Searches for documents across all schemas using a text query with wildcard matching.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows), default table output
dnx dotnet-charon -- DATA SEARCH \
  --dataBase "c:\my app\gamedata.json" \
  --query Dragon

# remote game data, full documents as JSON
dnx dotnet-charon -- DATA SEARCH \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --query Dragon \
  --outputFormat json \
  --credentials "<API-Key>"
```

Unlike DATA LIST, which requires a `--schema` and matches one field at a time via `--filters`, DATA SEARCH matches the query text against *every field of every document*, across schemas, using a contains (LIKE) operator. Use it to find where a value shows up when you don't know which schema or field it lives in; use DATA LIST once you do.

Parameters

<code>--dataBase</code>	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
<code>--credentials</code>	The API key used to access remote server in case of <code>--dataBase</code> being URL.
<code>--query</code>	Text to search for across all document fields. Matches using 'contains' (LIKE) operator against every field.
	<pre># matches any document with 'Dragon' in any field value --query Dragon</pre>
<code>--caseSensitive</code>	Whether the search should be case-sensitive. Default is false (case-insensitive).
	<pre>--caseSensitive true --caseSensitive false</pre>
<code>--schemas</code>	Defines which schemas to search. Accepts a list of schema names. If omitted, all schemas are searched.
	<pre># single schema --schemas Item # multiple schemas --schemas Item Character # masks --schemas *Item* # negation --schemas !DeprecatedSchema</pre>
<code>--skip</code>	Number of matches to skip from the start of the result set. Use for pagination.
	<pre># skip first ten matches --skip 10</pre>

Is It Free?

--take	Number of matches to return after skipping. Use for limiting result size in combination with --skip. <i># limit to first 100 matches after --skip</i> --take 100
--output	Specifies where to write the output data. <i># standard output (default)</i> --output out --output con <i># standard error</i> --output err <i># null device</i> --output null <i># absolute path (windows)</i> --output "c:\my app\results.json" <i># absolute path (unix)</i> --output /user/data/results.json <i># relative path (universal)</i> --output "./results.json" <i># remote location (HTTP)</i> --output "http://example.com/results.json" <i># remote location with authentication (FTP)</i> --output "ftp://user:password@example.com/results.json"
--outputFormat	Format for serializing the output. Only used when --outputFormat json; ignored for table. <i># flat `Schema Document Id Path Value` rows, script/grep-friendly (default)</i> --outputFormat table <i># JSON (default)</i> --outputFormat json <i># BSON</i> --outputFormat bson <i># Message Pack</i> --outputFormat msgpack <i># XLSX Spreadsheet</i> --outputFormat xlsx
--outputFormattingOptions	Additional formatting options for the chosen output format.

This command supports universal parameters.

Output

--outputFormat table (default) writes one line per match, pipe-delimited, with a header row - convenient for grep/cut/awk:

```
Schema|Document Id|Path|Value
Character|Knight|/Description|A knight who slays dragons for gold.
Item|DragonScale|/Name|Dragon Scale
Quest|SlayTheDragon|/Name|Slay the Dragon
```

The Path column is empty for root documents and holds the embedding property's JSON Pointer path (e.g. /Item) for documents embedded inside another document.

--outputFormat json writes the matched documents in full, grouped back into their schema collections, in the same shape as DATA EXPORT's ExportResult:

```
{
  "Collections":
  {
    "Character":
    [
      {
        "Id": "Knight"

        /* rest of properties of document */
      }
    ],
    "Item":
    [
      {
        "Id": "DragonScale"

        /* rest of properties of document */
      }
    ]
  }
}
```

See also

- DATA LIST
- DATA LISTSCHEMAS
- DATA FIND
- DATA FINDFILES
- DATA EXPORT

Update Document

Updates a document. For a bulk updates use DATA IMPORT command with --mode update. The update document in --input may be partial, with non-included fields being omitted. Only the first document from the --input will be processed.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA UPDATE \
  --dataBase "c:\my app\gamedata.json" \
  --schema Item \
  --input "c:\my app\item.json" \
  --inputFormat json

# remote game data
dnx dotnet-charon -- DATA UPDATE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schema Item \
  --input "./item.json" \
  --inputFormat json \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces. # local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"
--credentials	The API key used to access remote server in case of --dataBase being URL.
--schema	Name or identifier of the type (schema) of updated document. # name --schema Item # id --schema 55a4f32faca22e191098f3d9
--id	Identifier of updated document. Could be omitted if <i>Id</i> is specified in --input document. # text --id Sword # number --id 101

```
--validationOptions      List of validation checks and repairs to perform during document update.

                           # no checks
                           --validationOptions none

                           # repairs
                           --validationOptions repair
                           --validationOptions repair deduplicateIds
                           --validationOptions repair repairRequiredWithDefaultValue
                           --validationOptions repair eraseInvalidValue

                           # checks (default)
                           --validationOptions checkTranslation
                           --validationOptions checkRequirements
                           --validationOptions checkFormat
                           --validationOptions checkUniqueness
                           --validationOptions checkReferences
                           --validationOptions checkSpecification
                           --validationOptions checkConstraints

--input                  Path to a file with update data. Alternatively, you can use Standard Input or URL.

                           # standard input (default)
                           --input in
                           --input con

                           # absolute path (windows)
                           --input "c:\my app\item.json"

                           # absolute path (unix)
                           --input "/user/data/item.json"

                           # relative path (universal)
                           --input "./item.json"

                           # remote location (HTTP)
                           --input "http://example.com/item.json"

                           # remote location with authentication (FTP)
                           --input "ftp://user:password@example.com/item.json"

--inputFormat            Format of update data.

                           # Auto-detect by extension (default)
                           --inputFormat auto

                           # JSON
                           --inputFormat json

                           # BSON
                           --inputFormat bson

                           # Message Pack
                           --inputFormat msgpack

                           # XML (removed in 2025.1.1)
                           --inputFormat xml

--inputFormattingOptions Additional options for specified format.
```

`--output`

Path to a updated document file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), `/dev/null`, or a URL.

```
# standard output
--output out
--output con

# standard error
--output err

# null device (default)
--output null

# absolute path (windows)
--output "c:\my app\updated_item.json"

# absolute path (unix)
--output /user/data/updated_item.json

# relative path (universal)
--output "./updated_item.json"

# remote location (HTTP)
--output "http://example.com/updated_item.json"

# remote location with authentication (FTP)
--output "ftp://user:password@example.com/updated_item.json"
```

`--outputFormat`

Format of updated data.

```
# JSON (default)
--outputFormat json

# BSON
--outputFormat bson

# Message Pack
--outputFormat msgpack

# XML (removed in 2025.1.1)
--outputFormat xml
```

`--outputFormattingOptions`

Additional options for specified format.

This command supports universal parameters.

Input Data Schema

The data you input should follow this schema (recommended):

```
{
  "Collections": {
    "<Schema-Name>": [
      {
        // <Document>
      }
    ]
  }
}
```

This schema is also accepted:

Is It Free?

```
{
  "<Schema-Name>": [
    {
      // <Document>
    }
  ]
}
```

A list of documents is accepted:

```
[
  {
    // <Document>
  }
]
```

And single document too:

```
{
  // <Document>
}
```

Output

Outputs the updated document with all the edits that were made to make it conform to the schema.

```
{
  "Id": "Sword"

  /* rest of properties of updated document */
}
```

See also

- DATA IMPORT
- DATA CREATE
- DATA DELETE
- Importing and Exporting Data

Update Project Settings

Added in version 2026.4.0.

Updates a single property of the project settings document (project name, copyright, languages, and other project-level configuration) in game data.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# bump the game data version (windows)
dnx dotnet-charon -- DATA UPDATEPROJECTSETTINGS \
  --dataBase "c:\my app\gamedata.json" \
  --property Version \
  --value "1.2.0.0"

# the same property addressed by JSON Pointer
dnx dotnet-charon -- DATA UPDATEPROJECTSETTINGS \
  --dataBase "c:\my app\gamedata.json" \
  --property /Version \
  --value "1.2.0.0"

# rename the project
dnx dotnet-charon -- DATA UPDATEPROJECTSETTINGS \
  --dataBase "c:\my app\gamedata.json" \
  --property Name \
  --value "My Project"

# remote game data
dnx dotnet-charon -- DATA UPDATEPROJECTSETTINGS \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --property Copyright \
  --value "My Company (c) 2025" \
  --credentials "<API-Key>"
```

There is always exactly one project settings document per game data source, so unlike DATA UPDATE this command takes no --schema/-id parameters - its identity is fixed internally.

Available Properties

One property is updated per command run. All project settings properties hold text values.

Property	Description
Id	Unique identifier of the project settings document (BSON ObjectId). Read-only - the command refuses to change it.
Name	Name of the project.
PrimaryLanguage	Primary language for localizable text (language ID in BCP-47 format, e.g. en-US).
Languages	Alternative languages for localizable text (semicolon-delimited list of BCP-47 language IDs, e.g. en-GB; fr-FR). Prefer DATA I18N ADD LANGUAGE when adding a language, because it keeps the primary language out of the list and preserves existing entries.
Copyright	Copyright information for the project.
Version	Version of the current file as four dot-separated numbers (Major.Minor.Build.Revision).
Extensions	Installed extension packages as a semicolon-delimited list of name=version pairs.

See the Project Settings section for the full field reference. Any other property present in the document can be addressed the same way, by name or by [JSON Pointer](#).

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <pre># local file --dataBase "c:\my app\gamedata.json"</pre> <pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--property	Project settings property to update. Use a property name for top level values, or a JSON Pointer starting with / to address nested values. <pre># top level property by name --property PrimaryLanguage</pre> <pre># the same property as a JSON Pointer --property /PrimaryLanguage</pre> <pre># nested value --property /Extensions/0</pre>
--value	New value of the property. It is stored as text unless <code>--valueIsJson</code> is specified. <pre>--value "en-GB;fr-FR"</pre>
--valueIsJson	Treats <code>--value</code> as a JSON document and parses it, which allows storing numbers, booleans, arrays, and objects instead of text. <pre>--property Version --valueIsJson --value "\"1.2.0.0\""</pre> <pre>--property Extensions --valueIsJson --value "null"</pre>
--credentials	The API key used to access remote server in case of <code>--dataBase</code> being URL.
--output	Path to the file receiving the updated document. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to Standard Error, Standard Output, /dev/null, or a URL. <pre># standard output (default) --output out --output con</pre> <pre># standard error --output err</pre> <pre># null device --output null</pre> <pre># absolute path (windows) --output "c:\my app\settings.json"</pre> <pre># absolute path (unix) --output /user/data/settings.json</pre> <pre># relative path (universal) --output "./settings.json"</pre> <pre># remote location (HTTP) --output "http://example.com/settings.json"</pre> <pre># remote location with authentication (FTP) --output "ftp://user:password@example.com/settings.json"</pre>

`--outputFormat`

Specifies the format of the output data.

JSON (default)

`--outputFormat json`

BSON

`--outputFormat bson`

Message Pack

`--outputFormat msgpack`

`--outputFormattingOptions`

Additional options for formatting the output.

This command supports universal parameters.

Output

Outputs the updated project settings document.

```
{
  "Id": "55a4f32faca22e191098f3d9",
  "Name": "My Project",
  "PrimaryLanguage": "en-US",
  "Languages": "en-GB;fr-FR",
  "Copyright": "My Company (c) 2025",
  "Version": "1.2.0.0"
}
```

See also

- DATA PROJECTSETTINGS
- DATA I18N ADD LANGUAGE
- DATA UPDATE
- Game Data Structure

Validate Game Data

Checks the game data for validity and produces a report.

The exit code will be 1 if the report contains errors and the `--output` is set to `err`. Otherwise, the exit code will be 0.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA DELETE \
  --dataBase "c:\my app\gamedata.json" \
  --schema Item \
  --id "Sword"

# remote game data
dnx dotnet-charon -- DATA DELETE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schema Item \
  --id "Sword" \
  --credentials "<API-Key>"
```

Parameters

--dataBase Absolute or relative path to game data. Use quotation marks if your path contains spaces.

```
# local file
--dataBase "c:\my app\gamedata.json"

# remote server
--dataBase "https://charon.live/view/data/My_Game/develop/"
```

--credentials The API key used to access remote server in case of --dataBase being URL.

--validationOptions List of validation checks and repairs to perform.

```
# no checks
--validationOptions none

# all repairs
--validationOptions repair

# deduplicate documents with the same Id
--validationOptions repair deduplicateIds

# fill required fields that have a default value
--validationOptions repair repairRequiredWithDefaults

# erase values whose type does not match the property's data type
--validationOptions repair eraseInvalidValues

# fix malformed Union values:
#   empty union (no variant set)      → erased to null
#   conflicting union (2+ variants)  → last variant in schema order is
--validationOptions repair resolveConflictingUnions

# checks (default)
--validationOptions checkTranslations
--validationOptions checkRequirements
--validationOptions checkFormat
--validationOptions checkUniqueness
--validationOptions checkReferences
--validationOptions checkSpecification
--validationOptions checkConstraints
```

`--output`

Path to a validation report file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

```
# standard output
--output out
--output con

# standard error
--output err

# null device (default)
--output null

# absolute path (windows)
--output "c:\my app\document.json"

# absolute path (unix)
--output /user/data/document.json

# relative path (universal)
--output "./document.json"

# remote location (HTTP)
--output "http://example.com/document.json"

# remote location with authentication (FTP)
--output "ftp://user:password@example.com/document.json"
```

`--outputFormat`

Format of exported data.

```
# JSON (default)
--outputFormat json

# BSON
--outputFormat bson

# Message Pack
--outputFormat msgpack

# XML (removed in 2025.1.1)
--outputFormat xml
```

`--outputFormattingOptions`

Additional options for specified format.

This command supports universal parameters.

Output Data Schema

The report follow this pattern:

```
{
  records:
  [
    {
      id: "<document-id>",
      schemaId: "<schema-id>",
      schemaName: "<schema-name>",
      errors: // could be null if no errors
      [
        {
          path: "<path-in-document>",
          message: "<error-message>",

```

```

        code: "<error-code>"
      },
      // ...
    ]
  },
  // ...
]
}

```

or [JSON schema](#):

```

{
  "type": "object",
  "x-name": "ValidationReport",
  "additionalProperties": false,
  "properties": {
    "records": {
      "type": "array",
      "items": {
        "type": "object",
        "x-name": "ValidationRecord",
        "additionalProperties": false,
        "properties": {
          "id": { },
          "schemaName": {
            "type": "string"
          },
          "schemaId": {
            "type": "string"
          },
          "errors": {
            "type": "array",
            "items": {
              "type": "object",
              "x-name": "ValidationError",
              "additionalProperties": true,
              "readOnly": true,
              "properties": {
                "path": {
                  "type": "string"
                },
                "message": {
                  "type": "string"
                },
                "code": {
                  "type": "string"
                }
              }
            }
          }
        }
      }
    },
    "metadataHashCode": {
      "type": "integer",
      "format": "int32"
    }
  }
}

```

See also

- Validation
- DATA IMPORT
- DATA EXPORT
- CI/CD Integration
- [JSON Schema specification](#)

Generate C# Source Code

Generates C# source code for game data into output directory.

By default, this command does not delete previously generated files. Use the `--clearOutputDirectory` parameter to automatically remove prior generated files before creating new ones.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- GENERATE CSHARPCODE \
  --dataBase "c:\my app\gamedata.json" \
  --namespace "MyGame.Parameters" \
  --outputDirectory "c:\my app\scripts"

# remote game data
dnx dotnet-charon -- GENERATE CSHARPCODE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --namespace "MyGame.Parameters" \
  --outputDirectory "./scripts" \
  --credentials "<API-Key>"
```

Parameters

<code>--dataBase</code>	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<i># local file</i>
	<code>--dataBase "c:\my app\gamedata.json"</code>
	<i># remote server</i>
	<code>--dataBase "https://charon.live/view/data/My_Game/develop/"</code>
<code>--credentials</code>	The API key used to access remote server in case of <code>--dataBase</code> being URL.

--outputDirectory	Specifies the path where the source code should be written. It can be either an absolute or relative path to a directory. The specified directory must already be present. # Windows --outputDirectory "c:\my app\scripts" # Linux or OSX --outputDirectory "~/my app/scripts" # Relative path --outputDirectory "./my app/scripts"
--languageVersion	Target C# version. By default it is 4.0. --languageVersion CSharp40 --languageVersion CSharp73
--documentClassName	Name for base class for all documents. # name (default) --documentClassName Document # in case of name collision --documentClassName GameDataDocument
--gameDataClassName	Name for class containing whole in-memory game data. # name (default) --gameDataClassName GameData # in case of name collision --gameDataClassName MyGameData
--namespace	Namespace for all generated classes. # name (default) --namespace GameParameters
--defineConstants	Preprocessor constants to define. Use semicolon(;) to separate multiple values. # Use GameDevWare.Dynamic.Expressions.dll for formulas --defineConstants USE_DYNAMIC_EXPRESSIONS # Exclude all formula related code from compilation --defineConstants SUPPRESS_BUILD_IN_FORMULAS # Enable some JSON formatting optimizations using System.Memory.dll and System. --defineConstants BUFFERS_TEXT_DLL
--indentation	Indentation style for generated code. # Tabs (default) --indentation Tabs # Two spaces --indentation TwoSpaces # Four spaces --indentation FourSpaces


```

--lineEndings                               Line ending symbols for generated code.
                                           # Windows \\r\\n (default)
                                           --lineEndings Windows

                                           # Unix style \\n
                                           --lineEndings Unix

--splitFiles                               Set this flag to lay out generated classes into separate
                                           files. If not set, then one giant file with the name of
                                           --gameDataClassName.cs will be generated.

--optimizations                             List of enabled optimization in generated code.
                                           # Eagerly resolves and validates all references in loaded documents.
                                           # When enabled, this optimization ensures that all references in documents are
                                           # during loading. This comes with a performance cost but guarantees the validity
                                           --optimizations eagerReferenceResolution

                                           # Opts for raw references without generating helper methods for referenced docu
                                           # With this optimization, the generated code will not include helper methods fo
                                           # referenced documents, keeping only accessors that work with raw references.
                                           --optimizations rawReferences

                                           # Avoids generating helper methods for localized strings, keeping only raw acce
                                           # This optimization eliminates helper methods for accessing localized text, ins
                                           # accessors that deal directly with lists of localized texts.
                                           --optimizations rawLocalizedTexts

                                           # Keeps text properties carrying a vector, rectangle or tags editor as plain st
                                           # By default such a property is renamed to '<Name>Raw' and a parsed accessor ta
                                           # is generated. This optimization skips the parsing helpers and leaves the prop
                                           --optimizations rawCompositeTypes

                                           # Disables string pooling during game data loading.
                                           # Turning off string pooling can yield a minor performance improvement at the c
                                           # memory usage, as it avoids reusing short strings.
                                           --optimizations disableStringPooling

                                           # Disables generation of code for loading game data from JSON formatted files.
                                           # This optimization omits code related to JSON serialization, useful when JSON
                                           # game data is not used.
                                           --optimizations disableJsonSerialization

                                           # Disables generation of code for loading game data from Message Pack formatted
                                           # Similar to DisableJsonSerialization, this option removes code related to load
                                           # from Message Pack formatted files.
                                           --optimizations disableMessagePackSerialization

                                           # Disables generation of code related to applying patches during game data load
                                           # This removes a significant portion of code that is mainly used for modding su
                                           # where patches are applied to game data at runtime.
                                           --optimizations disablePatching

                                           # Disables generation of enums with known document IDs.
                                           # This removes a significant portion of code that contains listings of IDs for
                                           # documents known at the moment of code generation, which improves compilation
                                           --optimizations disableDocumentIdEnums

--clearOutputDirectory                     Clear the output directory from generated files.
                                           Generated files are identified by the presence of the
                                           <auto-generated> tag inside.

```

This command supports universal parameters.

See also

- Generating Source Code
- Working with Source Code (C# 7.3)
- Working with Source Code (C# 4.0)
- Publishing Game Data
- GENERATE TYPESCRIPTCODE
- GENERATE UECPP
- GENERATE HAXE

Generate Haxe Source Code

Generates Haxe source code for game data into output directory.

By default, this command does not delete previously generated files. Use the `--clearOutputDirectory` parameter to automatically remove prior generated files before creating new ones.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- GENERATE HAXECODE \
  --dataBase "c:\my app\gamedata.json" \
  --packageName "" \
  --outputDirectory "c:\my app\scripts"

# remote game data
dnx dotnet-charon -- GENERATE HAXECODE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --packageName "" \
  --outputDirectory "./scripts" \
  --credentials "<API-Key>"
```

Parameters

- | | |
|----------------------------|---|
| <code>--dataBase</code> | Absolute or relative path to game data. Use quotation marks if your path contains spaces. |
| | <i># local file</i> |
| | <code>--dataBase "c:\my app\gamedata.json"</code> |
| | <i># remote server</i> |
| | <code>--dataBase "https://charon.live/view/data/My_Game/develop/"</code> |
| <code>--credentials</code> | The API key used to access remote server in case of <code>--dataBase</code> being URL. |

--outputDirectory	Specifies the path where the source code should be written. It can be either an absolute or relative path to a directory. The specified directory must already be present. <pre> # Windows --outputDirectory "c:\my app\scripts" # Linux or OSX --outputDirectory "~/my app/scripts" # Relative path --outputDirectory "./my app/scripts" </pre>
--documentClassName	Name for base class for all documents. <pre> # name (default) --documentClassName Document # in case of name collision --documentClassName GameDataDocument </pre>
--gameDataClassName	Name for class containing whole in-memory game data. <pre> # name (default) --gameDataClassName GameData # in case of name collision --gameDataClassName MyGameData </pre>
--packageName	Package name for all generated classes. <pre> # empty package (default) --packageName "" # named --packageName GameParameters </pre>
--defineConstants	Preprocessor constants to define. Use semicolon(;) to separate multiple values. <pre>--defineConstants NO_OPTIMIZATIONS</pre>
--indentation	Indentation style for generated code. <pre> # Tabs (default) --indentation Tabs # Two spaces --indentation TwoSpaces # Four spaces --indentation FourSpaces </pre>
--lineEndings	Line ending symbols for generated code. <pre> # Windows \r\n (default) --lineEndings Windows # Unix style \n --lineEndings Unix </pre>
--splitFiles	Set this flag to lay out generated classes into separate files. If not set, then one giant file with the name of --gameDataClassName.hx will be generated.

--optimizations **list** of enabled optimization in generated code.

```
# Eagerly resolves and validates all references in loaded documents.
# When enabled, this optimization ensures that all references in documents are
# during loading. This comes with a performance cost but guarantees the validity
--optimizations eagerReferenceResolution

# Opts for raw references without generating helper methods for referenced documents.
# With this optimization, the generated code will not include helper methods for
# referenced documents, keeping only accessors that work with raw references.
--optimizations rawReferences

# Avoids generating helper methods for localized strings, keeping only raw accessors.
# This optimization eliminates helper methods for accessing localized text, instead
# accessors that deal directly with lists of localized texts.
--optimizations rawLocalizedTexts

# Keeps text properties carrying a vector, rectangle or tags editor as plain strings.
# By default such a property is renamed to '<Name>Raw' and a parsed accessor tag
# is generated. This optimization skips the parsing helpers and leaves the property
--optimizations rawCompositeTypes

# Disables string pooling during game data loading.
# Turning off string pooling can yield a minor performance improvement at the cost of
# memory usage, as it avoids reusing short strings.
# --optimizations disableStringPooling (N/A)

# Disables generation of code for loading game data from JSON formatted files.
# This optimization omits code related to JSON serialization, useful when JSON
# game data is not used.
--optimizations disableJsonSerialization

# Disables generation of code for loading game data from Message Pack formatted files.
# Similar to DisableJsonSerialization, this option removes code related to loading
# from Message Pack formatted files.
--optimizations disableMessagePackSerialization

# Disables generation of code related to applying patches during game data loading.
# This removes a significant portion of code that is mainly used for modding situations
# where patches are applied to game data at runtime.
--optimizations disablePatching

# Disables generation of enums with known document IDs.
# This removes a significant portion of code that contains listings of IDs for
# documents known at the moment of code generation, which improves compilation time.
--optimizations disableDocumentIdEnums
```

--clearOutputDirectory Clear the output directory from generated files.
Generated files are identified by the presence of the
<auto-generated> tag inside.

This command supports universal parameters.

See also

- Generating Source Code
- Working with Haxe Code
- GENERATE CSHARPCODE
- GENERATE TYPESCRIPTCODE
- GENERATE UECPPCODE

- Publication

Generate Lua Source Code

Generates Lua source code for game data into output directory.

By default, this command does not delete previously generated files. Use the `--clearOutputDirectory` parameter to automatically remove prior generated files before creating new ones.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- GENERATE LUACODE \
  --dataBase "c:\my app\gamedata.json" \
  --outputDirectory "c:\my app\scripts"

# remote game data
dnx dotnet-charon -- GENERATE LUACODE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --outputDirectory "./scripts" \
  --credentials "<API-Key>"
```

Parameters

<code>--dataBase</code>	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
<code>--credentials</code>	The API key used to access remote server in case of <code>--dataBase</code> being URL.
<code>--outputDirectory</code>	Specifies the path where the source code should be written. It can be either an absolute or relative path to a directory. The specified directory must already be present.
	<pre># Windows --outputDirectory "c:\my app\scripts" # Linux or OSX --outputDirectory "~/my app/scripts" # Relative path --outputDirectory "./my app/scripts"</pre>
<code>--documentClassName</code>	Name for base class for all documents.
	<pre># name (default) --documentClassName GameDataDocument # in case of name collision --documentClassName MyDocument</pre>

<code>--gameDataClassName</code>	Name for class containing whole in-memory game data. <i># name (default)</i> <code>--gameDataClassName GameData</code> <i># in case of name collision</i> <code>--gameDataClassName MyGameData</code>
<code>--indentation</code>	Indentation style for generated code. <i># Tabs (default)</i> <code>--indentation Tabs</code> <i># Two spaces</i> <code>--indentation TwoSpaces</code> <i># Four spaces</i> <code>--indentation FourSpaces</code>
<code>--lineEndings</code>	Line ending symbols for generated code. <i># Windows \r\n (default)</i> <code>--lineEndings Windows</code> <i># Unix style \n</i> <code>--lineEndings Unix</code>
<code>--splitFiles</code>	Set this flag to lay out generated classes into separate files. If not set, then one giant file with the name of <code>--gameDataClassName.lua</code> will be generated. Default is true.

--optimizations **list** of enabled optimization in generated code.

```
# Eagerly resolves and validates all references in loaded documents.
# When enabled, this optimization ensures that all references in documents are
# during loading. This comes with a performance cost but guarantees the validity
--optimizations eagerReferenceResolution

# Opts for raw references without generating helper methods for referenced documents.
# With this optimization, the generated code will not include helper methods for
# referenced documents, keeping only accessors that work with raw references.
--optimizations rawReferences

# Avoids generating helper methods for localized strings, keeping only raw accessors.
# This optimization eliminates helper methods for accessing localized text, instead
# accessors that deal directly with lists of localized texts.
--optimizations rawLocalizedTexts

# Keeps text properties carrying a vector, rectangle or tags editor as plain strings.
# By default such a property is renamed to '<Name>Raw' and a parsed accessor tag
# is generated. This optimization skips the parsing helpers and leaves the property
--optimizations rawCompositeTypes

# Disables string pooling during game data loading.
# Turning off string pooling can yield a minor performance improvement at the cost of
# memory usage, as it avoids reusing short strings.
# --optimizations disableStringPooling (N/A)

# Disables generation of code for loading game data from JSON formatted files.
# This optimization omits code related to JSON serialization, useful when JSON
# game data is not used.
# --optimizations disableJsonSerialization (N/A)

# Disables generation of code for loading game data from Message Pack formatted files.
# Similar to DisableJsonSerialization, this option removes code related to loading
# from Message Pack formatted files.
# --optimizations disableMessagePackSerialization (N/A)

# Disables generation of code related to applying patches during game data loading.
# This removes a significant portion of code that is mainly used for modding situations
# where patches are applied to game data at runtime.
# --optimizations disablePatching (N/A)

# Disables generation of enums with known document IDs.
# This removes a significant portion of code that contains listings of IDs for
# documents known at the moment of code generation, which improves compilation time.
--optimizations disableDocumentIdEnums
```

--clearOutputDirectory

Clear the output directory from generated files.
Generated files are identified by the presence of the
<auto-generated> tag inside.

This command supports universal parameters.

See also

- Generating Source Code
- Working with Lua Code
- GENERATE CSHARPCODE
- GENERATE TYPESCRIPTCODE
- GENERATE UECPP

- GENERATE HAXE
- Publication

Export Code Generation Templates (Obsolete)

Exports [T4](#) code generation templates to a specified directory. These templates can be used with Visual Studio, Rider, Visual Studio Code with plugin, dotnet [tool](#) or other tools to generate source code.

Warning

This command will be removed in future versions. Download templates directly from the [repository](#).

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# Windows
dnx dotnet-charon -- GENERATE TEMPLATES --outputDirectory "c:\templates"
```

Parameters

--outputDirectory

Specifies the path where the templates should be written. It can be either an absolute or relative path to a directory. The specified directory must already be present.

```
# Windows
--outputDirectory "c:\templates"

# Linux or OSX
--outputDirectory "~/templates"

# Relative path
--outputDirectory "../templates"
```

See also

- Generating Source Code
- GENERATE CSHARPCODE
- GENERATE TYPESCRIPTCODE
- GENERATE UECPPCODE
- [T4 Text Templates](#)

Generate Text from Templates (Obsolete)

Generates text from T4 templates into output directory.

Warning

This command was removed since 2025.1.1 version. It is recommended to use an IDE or open-source alternatives for generating text with T4 templates. See: [dotnet-t4](#)

See GENERATE TEMPLATES to get actual T4 templates.

- CLI Installation
- Commands Reference

See also

- Generating Source Code
- GENERATE TEMPLATES
- GENERATE CSHARPCODE
- [dotnet-t4 tool](#)

Generate TypeScript Source Code

Generates TypeScript source code for game data into output directory.

By default, this command does not delete previously generated files. Use the `--clearOutputDirectory` parameter to automatically remove prior generated files before creating new ones.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- GENERATE TYPESCRIPTCODE \
  --dataBase "c:\my app\gamedata.json" \
  --outputDirectory "c:\my app\scripts"

# remote game data
dnx dotnet-charon -- GENERATE TYPESCRIPTCODE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --outputDirectory "./scripts" \
  --credentials "<API-Key>"
```

Parameters

- | | |
|----------------------------|---|
| <code>--dataBase</code> | Absolute or relative path to game data. Use quotation marks if your path contains spaces. |
| | <i># local file</i> |
| | <code>--dataBase "c:\my app\gamedata.json"</code> |
| | <i># remote server</i> |
| | <code>--dataBase "https://charon.live/view/data/My_Game/develop/"</code> |
| <code>--credentials</code> | The API key used to access remote server in case of <code>--dataBase</code> being URL. |

--outputDirectory	Specifies the path where the source code should be written. It can be either an absolute or relative path to a directory. The specified directory must already be present. <pre> # Windows --outputDirectory "c:\my app\scripts" # Linux or OSX --outputDirectory "~/my app/scripts" # Relative path --outputDirectory "./my app/scripts" </pre>
--documentClassName	Name for base class for all documents. <pre> # name (default) --documentClassName Document # in case of name collision --documentClassName GameDataDocument </pre>
--gameDataClassName	Name for class containing whole in-memory game data. <pre> # name (default) --gameDataClassName GameData # in case of name collision --gameDataClassName MyGameData </pre>
--indentation	Indentation style for generated code. <pre> # Tabs (default) --indentation Tabs # Two spaces --indentation TwoSpaces # Four spaces --indentation FourSpaces </pre>
--lineEndings	Line ending symbols for generated code. <pre> # Windows \r\n (default) --lineEndings Windows # Unix style \n --lineEndings Unix </pre>
--splitFiles	Set this flag to lay out generated classes into separate files. If not set, then one giant file with the name of --gameDataClassName.ts will be generated.

--optimizations **list** of enabled optimization in generated code.

```
# Eagerly resolves and validates all references in loaded documents.
# When enabled, this optimization ensures that all references in documents are
# during loading. This comes with a performance cost but guarantees the validity
--optimizations eagerReferenceResolution

# Opts for raw references without generating helper methods for referenced documents.
# With this optimization, the generated code will not include helper methods for
# referenced documents, keeping only accessors that work with raw references.
--optimizations rawReferences

# Avoids generating helper methods for localized strings, keeping only raw accessors.
# This optimization eliminates helper methods for accessing localized text, instead
# accessors that deal directly with lists of localized texts.
--optimizations rawLocalizedTexts

# Keeps text properties carrying a vector, rectangle or tags editor as plain strings.
# By default such a property is renamed to '<Name>Raw' and a parsed accessor tag
# is generated. This optimization skips the parsing helpers and leaves the property
--optimizations rawCompositeTypes

# Disables string pooling during game data loading.
# Turning off string pooling can yield a minor performance improvement at the cost of
# memory usage, as it avoids reusing short strings.
# --optimizations disableStringPooling (N/A)

# Disables generation of code for loading game data from JSON formatted files.
# This optimization omits code related to JSON serialization, useful when JSON
# game data is not used.
--optimizations disableJsonSerialization

# Disables generation of code for loading game data from Message Pack formatted files.
# Similar to DisableJsonSerialization, this option removes code related to loading
# from Message Pack formatted files.
--optimizations disableMessagePackSerialization

# Disables generation of code related to applying patches during game data loading.
# This removes a significant portion of code that is mainly used for modding situations
# where patches are applied to game data at runtime.
--optimizations disablePatching

# Disables generation of enums with known document IDs.
# This removes a significant portion of code that contains listings of IDs for
# documents known at the moment of code generation, which improves compilation time.
--optimizations disableDocumentIdEnums
```

--clearOutputDirectory Clear the output directory from generated files.
Generated files are identified by the presence of the
<auto-generated> tag inside.

This command supports universal parameters.

See also

- Generating Source Code
- Working with TypeScript Code
- GENERATE CSHARPCODE
- GENERATE UECPPCODE
- GENERATE HAXECODE

- Publication

Generate Unreal Engine C++ Source Code

Generates C++ for Unreal Engine source code for game data into output directory.

By default, this command does not delete previously generated files. Use the `--clearOutputDirectory` parameter to automatically remove prior generated files before creating new ones.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- GENERATE UECPPCODE \
  --dataBase "c:\My Project\Content\gamedata.json" \
  --outputDirectory "c:\My Project\Source\Gamedata"

# remote game data
dnx dotnet-charon -- GENERATE UECPPCODE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --outputDirectory "../My Project/Source/Gamedata" \
  --credentials "<API-Key>"
```

Parameters

<code>--dataBase</code>	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\My Project\Content\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
<code>--credentials</code>	The API key used to access remote server in case of <code>--dataBase</code> being URL.
<code>--outputDirectory</code>	Specifies the path where the source code should be written. It can be either an absolute or relative path to a directory. The specified directory must already be present.
	<pre># Windows --outputDirectory "c:\My Project\Source\Gamedata" # Linux or OSX --outputDirectory "~/My Project/Source/Gamedata" # Relative path --outputDirectory "../My Project/Source/Gamedata"</pre>
<code>--documentClassName</code>	Name for base class for all documents.
	<pre># name (default) --documentClassName Document # became UDocument in generated code # in case of custom inheritance chain # class SHOULD publicly inherit UDocument --documentClassName GameDataDocument # became UGameDataDocument in generated code</pre>

```
--gameDataClassName Name for class containing whole in-memory game data.
    # name (default)
    --gameDataClassName GameData # became UGameData in generated code

    # in case of name collision
    --gameDataClassName MyGameData # became UMyGameData in generated code

--namespace
    Accepted for symmetry with the other generators, but
    ignored by the Unreal Engine C++ target - generated
    types are named with the U/F prefixes Unreal expects
    instead of being scoped into a namespace. Use
    --documentClassName and
    --gameDataClassName to control the generated
    names.

--defineConstants
    Preprocessor constants to define. Use semicolon(;) to separate
    multiple values.
    --defineConstants NO_OPTIMIZATIONS;USE_FSTRING_ONLY

--indentation
    Indentation style for generated code.
    # Tabs (default)
    --indentation Tabs

    # Two spaces
    --indentation TwoSpaces

    # Four spaces
    --indentation FourSpaces

--lineEndings
    Line ending symbols for generated code.
    # Windows \r\n (default)
    --lineEndings Windows

    # Unix style \n
    --lineEndings Unix

--splitFiles
    Accepted for symmetry with the other generators, but
    ignored by the Unreal Engine C++ target - it always
    writes one header per type into Public/ with the
    matching sources in Private/.
```

--optimizations **list** of enabled optimization in generated code.

```
# Eagerly resolves and validates all references in loaded documents.
# When enabled, this optimization ensures that all references in documents are
# during loading. This comes with a performance cost but guarantees the validity
--optimizations eagerReferenceResolution

# Opts for raw references without generating helper methods for referenced documents.
# With this optimization, the generated code will not include helper methods for
# referenced documents, keeping only accessors that work with raw references.
--optimizations rawReferences

# Avoids generating helper methods for localized strings, keeping only raw accessors.
# This optimization eliminates helper methods for accessing localized text, instead
# accessors that deal directly with lists of localized texts.
--optimizations rawLocalizedTexts

# Not applicable. Unreal Engine C++ never renames a property, so a text property,
# vector, rectangle or tags editor always keeps its name and gets a separate Get
# accessor beside it.
# --optimizations rawCompositeTypes (N/A)

# Disables string pooling during game data loading.
# Turning off string pooling can yield a minor performance improvement at the cost of
# memory usage, as it avoids reusing short strings.
# --optimizations disableStringPooling (N/A)

# Disables generation of code for loading game data from JSON formatted files.
# This optimization omits code related to JSON serialization, useful when JSON
# game data is not used.
--optimizations disableJsonSerialization

# Disables generation of code for loading game data from Message Pack formatted files.
# Similar to DisableJsonSerialization, this option removes code related to loading
# from Message Pack formatted files.
--optimizations disableMessagePackSerialization

# Disables generation of code related to applying patches during game data loading.
# This removes a significant portion of code that is mainly used for modding situations
# where patches are applied to game data at runtime.
--optimizations disablePatching

# Disables generation of enums with known document IDs.
# This removes a significant portion of code that contains listings of IDs for
# documents known at the moment of code generation, which improves compilation time.
--optimizations disableDocumentIdEnums
```

--clearOutputDirectory Clear the output directory from generated files.
Generated files are identified by the presence of the
<auto-generated> tag inside.

This command supports universal parameters.

See also

- Generating Source Code
- Working with Unreal Engine C++ Code
- GENERATE CSHARPCODE
- GENERATE TYPESCRIPTCODE
- GENERATE HAXECODE

- Publication

Initialize Game Data

Initializes an empty or missing file with initial data. Path to game data should be local file system's file.

- CLI Installation
- Commands Reference

Command

```
# full path (windows)
dnx dotnet-charon -- INIT "c:\my app\gamedata.gdjs"

# full path (linux)
dnx dotnet-charon -- INIT "/var/mygame/gamedata.json"

# relative path
dnx dotnet-charon -- INIT mygame/gamedata.json
```

Note

The parent directory must already exist - INIT creates the file, not the folders leading to it.

Parameters

--fileName Absolute or relative path to game data file. The directory must already exist. Use quotation marks if your path contains spaces.

```
# local file
--fileName "c:\my app\gamedata.json"
```

See also

- Command Line Reference
- Game Data Structure
- SERVER START

URL input/output parameters

Some command accept [URL](#) as input/output parameter.

Supported URL Schemes

Scheme	Input parameter	Output parameter
HTTP[S]	A GET request will be sent	A POST request with body containing output will be sent
FTP(S)	A RETR command will be sent	A STOR command with output content will be sent
FILE	File will be read	File will be written

Authentication

Authentication data could be passed in *user* part of [URL](#). More advanced authentication schemes are not supported.

Examples

```
# publish data to FTP
dnx dotnet-charon -- DATA EXPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/dashboard" \
  --output "ftp://user:password@example.com/public/gamedata.json" \
  --mode publication \
  --outputFormat json \
  --credentials "<API-Key>"

# import localization from remote HTTP server
dnx dotnet-charon -- DATA I18N IMPORT \
  --dataBase "file:///c:/my app/gamedata.json" \
  --input "https://example.com/translated/gamedata.xliff" \
  --inputFormat xliff

# print languages for game data in local file
dnx dotnet-charon -- DATA I18N LANGUAGES --dataBase "file:///c:/my app/gamedata.json"

# print languages for game data in local file relative to current working directory
dnx dotnet-charon -- DATA I18N LANGUAGES --dataBase "file:///./gamedata.json"

# print languages for game data at remote server using API Key
export CHARON_API_KEY=87758CC0D7C745D0948F2A8AFE61BC81
dnx dotnet-charon -- DATA I18N LANGUAGES --dataBase "https://charon.live/view/data/My_Game/d
```

See also

- Command Line Reference
- DATA EXPORT
- DATA IMPORT
- DATA I18N EXPORT
- [URI \(Wikipedia\)](#)
- [FTP protocol \(Wikipedia\)](#)

Start in Standalone Mode

Starts Charon in standalone mode for specified game data. Path to game data could be local file system's file or remote server address.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- SERVER START \
  --dataBase "c:\my app\gamedata.json" \
  --port 8080 \
  --launchDefaultBrowser

# shortcut version
charon "c:\my app\gamedata.json"
```


Parameters

<code>--dataBase</code>	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <i># local file</i> <code>--dataBase "c:\my app\gamedata.json"</code>
<code>--port</code>	Number of an IP port (1-65535) to be used to listen for browser based UI. <code>--port 8080</code>
<code>--launchDefaultBrowser</code>	Set this flag to open system-default browser on successful start.
<code>--denySchemaEditing</code>	Disable the ability to change the data structure (metadata) for editable game data.
<code>--resetPreferences</code>	Set this flag to reset UI preferences on successful start.
<code>--changesFile</code>	<i>Added in version 2026.4.0.</i> Path to a changes file. When set, the game data file is opened read-only and every edit is written into this file as a patch document instead of into the game data file. Overrides the <code>changesFile</code> query parameter of <code>--dataBase</code> . Apply the patch later with <code>DATA APPLYPATCH</code> . <code>--changesFile "./player-changes.patch.json"</code>

This command supports universal parameters.

See also

- [Command Line Reference](#)
- [DATA APPLYPATCH](#)
- [Web Application Overview](#)
- [IP port \(Wikipedia\)](#)

Universal parameters

All commands accept universal parameters and environment variables.

<code>--verbose</code>	Set this flag to get additional diagnostic information in logs.
<code>--log <path></code>	Add additional file logging to the existing logging configuration from <code>appsettings.json</code> . <code>--log "./logs/charon.log"</code>
<code>--log out</code>	Add additional terminal (standard output) logging to the existing logging configuration from <code>appsettings.json</code> . <code>--log out</code> <i># or</i> <code>--log con</code>
<code>--pause</code>	Wait for user prompt before the application exits.

Environment variables

In addition to the standard configuration redefinition [mechanism](#) using environment variables, the following environment variables are also supported.

CHARON_API_KEY

Is It Free?

The API key which is used to access the remote server. This environment variable is usually used in conjunction with `--dataBase`, which points to a remote server.

```
# Windows
set CHARON_API_KEY=87758CC0D7C745D0948F2A8AFE61BC81

# OSX or Linux
export CHARON_API_KEY=87758CC0D7C745D0948F2A8AFE61BC81
```

See also

- Command Line Reference
- CLI Access to charon.live
- URL-based Input/Output
- Working with Logs

Get Charon Version

Gets version of charon application.

- CLI Installation
- Commands Reference

Command

```
# Windows, Linux or OSX
dnx dotnet-charon -- VERSION
#> 2023.2.3-alpha
```

Parameters

This command supports universal parameters.

See also

- Command Line Reference
- Universal Parameters

See also

- CI/CD Integration
- URL-based Input/Output
- Universal Parameters
- Game Data Structure
- [dnx documentation](#)
- [dotnet tool documentation](#)

MCP Server (AI Assistant Integration)

Charon ships a built-in [Model Context Protocol \(MCP\)](#) server that lets AI assistants - Claude, Cursor, GitHub Copilot, Windsurf, Cline, Zed, and others - read and write game data directly. The server communicates over **stdio** using the standard JSON-RPC MCP transport, so no network port is opened and no extra infrastructure is required.

With the MCP server running, an AI assistant can:

- Discover and inspect schemas

• Query, create, update, and delete game data documents	
• Import and export data in multiple formats	
• Generate typed source code (C#, TypeScript, Unreal C++, Haxe, Lua)	
• Manage localization (XLIFF / JSON round-trips)	
• Compare and patch game data between two sources	
• Read project settings (languages, primary language)	
Prerequisites	289
Starting the MCP Server	289
Logging	290
Configuring AI Clients	290
Claude Desktop	290
Claude Code (CLI)	290
Cursor	290
VS Code (GitHub Copilot)	290
Windsurf	291
JetBrains IDEs (Rider, IntelliJ IDEA, WebStorm, etc.)	291
Cline (VS Code Extension)	291
Zed	291
Authentication for Remote Sources	292
Available Tools	292
Limitations and Remote-Only Use Case	293

Prerequisites

Install the [.NET SDK 10 or later](#). Nothing else is required - dnx downloads and runs Charon on demand:

```
dnx dotnet-charon -- VERSION
```

Note

On **.NET SDK 8 or 9** dnx is unavailable. Install the tool globally instead:

```
dotnet tool install -g dotnet-charon
charon VERSION
```

Then in every client config below replace "command": "dnx" / "args": ["dotnet-charon", "--", "MCP"] with "command": "charon" / "args": ["MCP"]. See Adding the global tool to PATH if charon is not found.

Starting the MCP Server

Run the MCP verb with no additional arguments:

```
dnx dotnet-charon -- MCP
```

The process stays running and communicates over **stdin / stdout**. Do not write anything else to stdout while the server is active - the transport is the raw MCP JSON-RPC stream and any extra output will corrupt the protocol.

All diagnostic logging should be going to a log file (see [Logging](#) below), so it never interferes with the transport.

Logging

Because stdout is reserved for the MCP transport, use the `--log` option to write diagnostics to a file:

```
dnx dotnet-charon -- MCP --log /tmp/charon-mcp.log
```

Pass `--verbose` to include debug-level messages:

```
dnx dotnet-charon -- MCP --log /tmp/charon-mcp.log --verbose
```

Specifying `--log con` or `--log out` in MCP mode is silently ignored to protect the transport.

Configuring AI Clients

Claude Desktop

Add the following entry to your `claude_desktop_config.json` (usually at `~/Library/Application Support/Claude/claude_desktop_config.json` on macOS or `%APPDATA%\Claude\claude_desktop_config.json` on Windows):

```
{
  "mcpServers": {
    "charon": {
      "command": "dnx",
      "args": ["dotnet-charon", "--", "MCP"]
    }
  }
}
```

Restart Claude Desktop after editing the file.

Claude Code (CLI)

```
claude mcp add --transport stdio charon -- dnx dotnet-charon -- MCP
```

Cursor

Open **Settings** → **MCP** → **Add MCP Server** and fill in:

- **Command:** `dnx`
- **Arguments:** `dotnet-charon -- MCP`

VS Code (GitHub Copilot)

Add to your workspace `.vscode/mcp.json`:

```
{
  "servers": {
    "charon": {
      "type": "stdio",
      "command": "dnx",
      "args": ["dotnet-charon", "--", "MCP"]
    }
  }
}
```

Windsurf

Add to ~/.codeium/windsurf/mcp_config.json (or open **Windsurf Settings** → **Cascade** → **Model Context Protocol**):

```
{
  "mcpServers": {
    "charon": {
      "command": "dnx",
      "args": ["dotnet-charon", "--", "MCP"]
    }
  }
}
```

JetBrains IDEs (Rider, IntelliJ IDEA, WebStorm, etc.)

Open **Settings** → **Tools** → **AI Assistant** → **Model Context Protocol (MCP)**, click **Add**, and paste the following configuration:

```
{
  "mcpServers": {
    "charon": {
      "command": "dnx",
      "args": ["dotnet-charon", "--", "MCP"]
    }
  }
}
```

Note

MCP client support requires JetBrains AI Assistant plugin and IDE version 2025.1 or later.

Cline (VS Code Extension)

Open the **MCP Servers** panel in Cline, click **Configure**, then **Configure MCP Servers**, and add the following to cline_mcp_settings.json:

```
{
  "mcpServers": {
    "charon": {
      "command": "dnx",
      "args": ["dotnet-charon", "--", "MCP"]
    }
  }
}
```

Zed

Add to your Zed settings file (open with **Zed** → **Settings** → **Open Settings** or Cmd+,):

```
{
  "context_servers": {
    "charon": {
      "command": "dnx",
      "args": ["dotnet-charon", "--", "MCP"]
    }
  }
}
```

Authentication for Remote Sources

When a tool's `dataBase` parameter points to a remote Charon service URL (e.g. `https://charon.live/view/data/MyGame/main`), set the `CHARON_API_KEY` environment variable before starting the server:

```
export CHARON_API_KEY=your_api_key_here
dnx dotnet-charon -- MCP
```

In client configuration files you can pass the variable directly. Most clients (Claude Desktop, Windsurf, Cursor, Cline, JetBrains) use the same env block:

```
{
  "mcpServers": {
    "charon": {
      "command": "dnx",
      "args": ["dotnet-charon", "--", "MCP"],
      "env": {
        "CHARON_API_KEY": "your_api_key_here"
      }
    }
  }
}
```

For **VS Code (GitHub Copilot)**, add the `env` field inside the server entry in `.vscode/mcp.json`:

```
{
  "servers": {
    "charon": {
      "type": "stdio",
      "command": "dnx",
      "args": ["dotnet-charon", "--", "MCP"],
      "env": {
        "CHARON_API_KEY": "your_api_key_here"
      }
    }
  }
}
```

For **Zed**, add the `env` field inside your `context_servers` entry:

```
{
  "context_servers": {
    "charon": {
      "command": "dnx",
      "args": ["dotnet-charon", "--", "MCP"],
      "env": {
        "CHARON_API_KEY": "your_api_key_here"
      }
    }
  }
}
```

Local file sources do not require an API key.

Available Tools

Use the following command to inspect the MCP server:

```
npx @modelcontextprotocol/inspector dnx dotnet-charon -- MCP
```

Limitations and Remote-Only Use Case

The built-in MCP server works with **local files** and with **remote Charon service URLs** that are reachable from the machine running the Charon process. The game data file is loaded locally by the CLI tool.

If you need a **fully-remote** MCP setup where Charon itself runs as a cloud service and the CLI is not installed on the AI client's machine, the MCP server cannot be used directly. As a fallback, `charon.live` exposes its full API through an OpenAPI 3 specification:

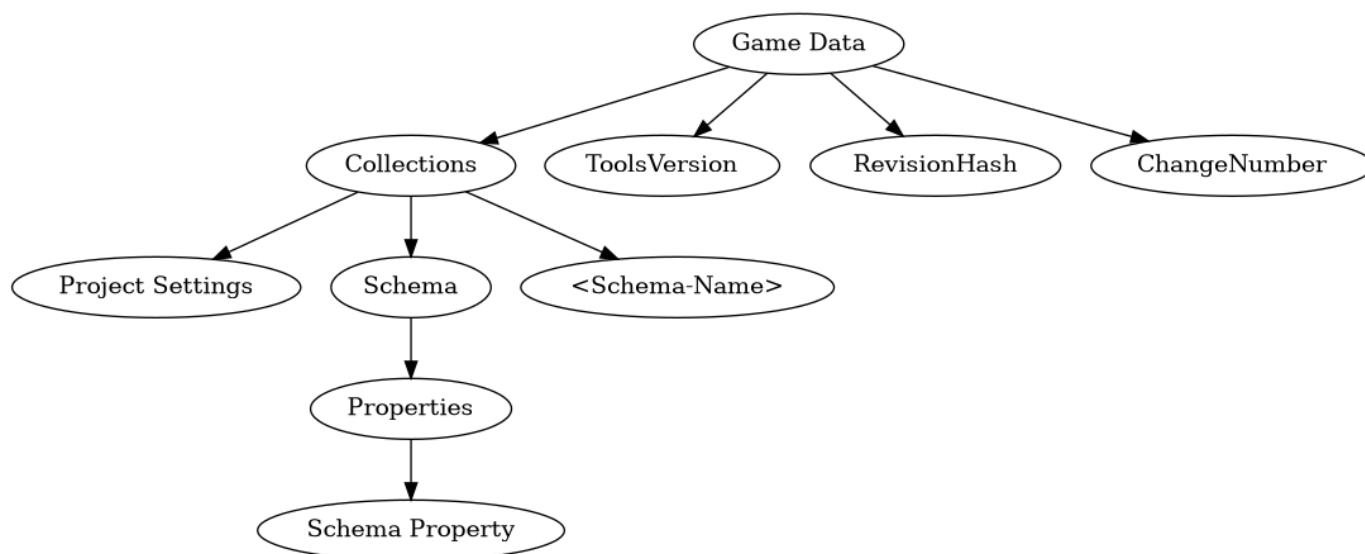
<https://charon.live/api/v1/open.api.server.yml>

AI clients that support OpenAPI-based tool use (such as those using function-calling from an OpenAPI schema) can configure that URL as an API tool source. Authentication is done with a bearer token equal to your `CHARON_API_KEY` value.

Seealso

- REST API - REST API reference for the `charon.live` service
- Command Line Interface (CLI) - full CLI reference, including all `DATA` and `GENERATE` verbs
- CI/CD Integration - using Charon in automated pipelines

Game Data Structure



Game Data

Fields

- `ToolsVersion` (string): Version of the application used to create this file.
- `RevisionHash` (string): Current changeset hash value.
- `ChangeNumber` (number): Current changeset ordinal number.
- `Collections` (object): List of document collections identified by schema name.
- `ProjectSettings` (array): Project-related settings for the current file.

- See Project Settings section below.
- Schema (array): Project-related schemas for the current file.
- See Schema section below.
- <Schema-Name> (array): Other document collections listed in alphabetical order.

Example

```
{
  "ToolsVersion": "2023.1.2.0",
  "RevisionHash": "678f998993a22d1f54b7fa80",
  "ChangeNumber": 1,
  "Collections": {
    "ProjectSettings": [
      {
        /* see project settings section below */
      }
    ],
    "Schema": [
      {
        /* see schema section below */
      }
    ],
    "SchemaProperty": [ /* always empty */ ],
    "<Schema-Name>": [
      {
        "Id": "<Id>" // All documents have Id

        /* rest of properties of document */
      },
      // ...
    ]
  }
}
```

Project Settings**Fields**

- Id (string): Unique identifier for the project settings (BSON ObjectId).
- Name (string): Name of the project.
- PrimaryLanguage (string): Primary language for localizable text in the project (language ID in BCP-47 format).
- Languages (string): Alternative languages for localizable text in the project (semicolon-delimited list of language IDs in BCP-47 format).
- Copyright (string): Copyright information for the project.
- Version (string): Version of the current file, represented as four numbers separated by dots (Major.Minor.Build.Revision).

Example

```
{
  "Id": "049bc0604c363a980b0000088",
  "Name": "My Project",
```



```

"PrimaryLanguage": "en-US",
"Languages": "en-GB;fr-FR",
"Copyright": "My Company (■) 2025",
"Version": "1.0.0.0"
}

```

Schema

Fields

- **Id** (string): Unique identifier for the schema (BSON ObjectId).
- **Name** (string): Name of the schema (valid C identifier).
- **DisplayName** (string): Display name of the schema for UI purposes.
- **Description** (string): Schema description used in generated documentation.
- **Specification** (string): Extension data for the schema in *application/x-www-form-urlencoded* format (RFC-1867).
- **IdGenerator** (number): ID generation method for documents created by this schema:
 - 0 : None - ID must be provided manually by the user.
 - 1 : ObjectId - Generates a new BSON ObjectId.
 - 2 : Guid - Generates a new UUID.
 - 3 : Sequence - Uses an incrementing number unique to each schema.
 - 4 : GlobalSequence - Uses an incrementing number shared across all schemas.
- **Type** (number): Schema type:
 - 0 : Normal - Documents can be created in *Collections* or embedded in another document.
 - 1 : Component - Documents are always embedded in another document and never appear in *Collections*.
 - 2 : Settings - Only one document of this schema can exist in *Collections*.
- **Properties** (array): List of schema properties. Always includes the **Id** property.
 - See Schema Property section below.

Example

```

{
  "Id": "592fc86c983a36266c0912a0",
  "Name": "Item",
  "DisplayName": "Items",
  "Type": 0,
  "Description": "An item.",
  "IdGenerator": 1,
  "Specification": "icon=fugue16%2Fabacus&group=Metagame",
  "Properties": [
    // property
  ]
}

```

Schema Property

Structure

- **Id** (string): Unique identifier for the property (BSON ObjectId).
- **Name** (string): Name of the property (valid C identifier).
- **DisplayName** (string): Display name for UI and documentation purposes.

- **Description** (string): Property description used in generated documentation.
- **DataType** (number): Data type of values stored in documents:
 - 0: Text - Line of text.
 - 1: LocalizedText - Lines of localized text.
 - 5: Logical - Boolean value.
 - 8: Time - Time span.
 - 9: Date - Specific date.
 - 12 : Number - Decimal number.
 - 13 : Integer - Whole number.
 - 18 : PickList - Predefined value list.
 - 19 : MultiPickList - Multiple selections from predefined values.
 - 22 : Document - Embedded document.
 - 23 : DocumentCollection - Collection of embedded documents.
 - 28 : Reference - Reference to another document.
 - 29 : ReferenceCollection - References to multiple documents.
 - 35 : Formula - C#-like expression for calculations.
- **DefaultValue** (vary|null): Default value for the property used when a new document is created.
- **Uniqueness** (number): Uniqueness requirement for the property:
 - 0 : None - Value does not need to be unique.
 - 1 : Unique - Value must be unique across all documents of this type.
 - 2 : UniqueInCollection - Value must be unique within the containing collection.
- **Requirement** (number): Value requirement for the property:
 - 0 : None - Value is optional and can be null.
 - 1 : <UNUSED> - Remapped to 2 (NotNull) during saving.
 - 2 : NotNull - Value is required but can be an empty string/collection.
 - 3 : NotEmpty - Value is required and cannot be empty.
- **ReferenceType** (object|null): Referenced schema for certain data types (*Document*, *DocumentCollection*, *Reference*, *ReferenceCollection*):
 - **Id** (string): Identifier of the referenced schema.
 - **DisplayName** (string): Optional display name of the referenced schema.
- **Size** (number): Maximum or exact size of the data type. For variable-length types (e.g., text, collections), this defines the size; for others, it is zero.
- **Specification** (string): Extension data for the property in *application/x-www-form-urlencoded* format (RFC-1867).

Example

```
{
  "Id": "592fc9f8983a36266c0912aa",
  "Name": "TextField",
  "DisplayName": "Text Field",
  "Description": "",
  "DataType": 0,
  "DefaultValue": null,
```

```

    "Uniqueness": 0,
    "Requirement": 0,
    "ReferenceType": null,
    "Size": 0,
    "Specification": null
  }

```

See also

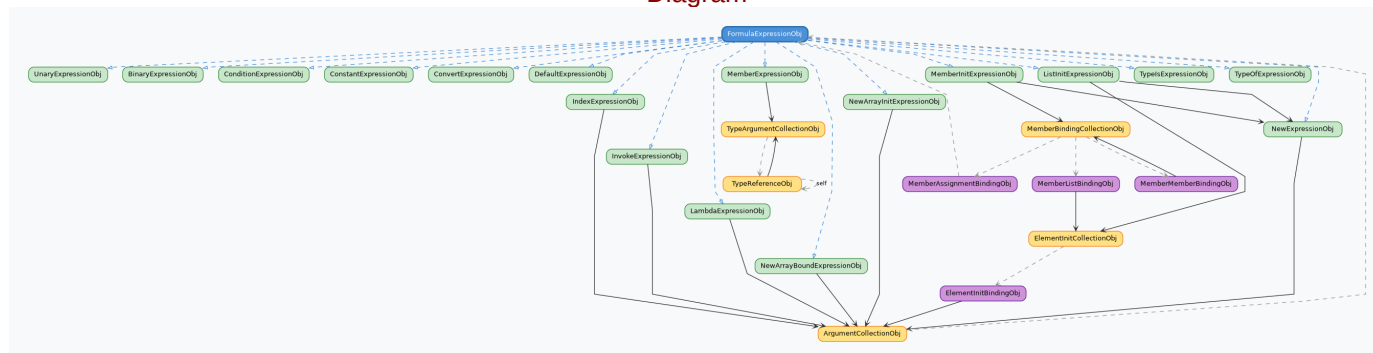
- DATA EXPORT
- DATA IMPORT
- DATA LIST
- REST API
- Importing and Exporting Data
- [BSON ObjectId](#)
- [BCP-47 language tags](#)

Formula Expression AST

Formula values are stored as a JSON-encoded Abstract Syntax Tree (AST). Each node in the tree represents an expression and is identified by the `expressionType` field, which acts as a discriminator for the node type. An interpreter recursively evaluates the tree starting from the root node.

Reference implementations exist for C#, TypeScript, and C++.

Diagram



TypeScript Interfaces

```

export type FormulaExpressionObj = UnaryExpressionObj | BinaryExpressionObj | ConditionExpr
  ConvertExpressionObj | DefaultExpressionObj | IndexExpressionObj | InvokeExpressionObj |
  NewArrayBoundExpressionObj | NewArrayInitExpressionObj | ListInitExpressionObj | MemberI
  TypeIsExpressionObj | TypeOfExpressionObj;

```

```

export interface UnaryExpressionObj {
  'expressionType': 'UncheckedScope' | 'CheckedScope' | 'Group' | 'UnaryPlus' | 'Negate' |
  'expression': FormulaExpressionObj;
}

```

```

export interface BinaryExpressionObj {
  'expressionType': 'Divide' | 'MultiplyChecked' | 'Multiply' | 'Power' | 'Modulo' | 'AddO
  'left': FormulaExpressionObj;
  'right': FormulaExpressionObj;
};

```

```

export interface ConditionExpressionObj {
  'expressionType': 'Condition';
  'test': FormulaExpressionObj;
}

```

```

    'ifTrue': FormulaExpressionObj;
    'ifFalse': FormulaExpressionObj;
}

export interface ConstantExpressionObj {
    'expressionType': 'Constant';
    'type': TypeReferenceObj | string;
    'value': any;
};

export interface ConvertExpressionObj {
    'expressionType': 'TypeAs' | 'Convert' | 'ConvertChecked';
    'type': TypeReferenceObj | string;
    'expression': FormulaExpressionObj;
}

export interface DefaultExpressionObj {
    'expressionType': 'Default';
    'type': TypeReferenceObj | string;
}

export interface IndexExpressionObj {
    'expressionType': 'Index';
    'expression': FormulaExpressionObj;
    'arguments': ArgumentCollectionObj;
    'useNullPropagation': boolean;
}

export interface InvokeExpressionObj {
    'expressionType': 'Invoke';
    'expression': FormulaExpressionObj;
    'arguments': ArgumentCollectionObj;
}

export interface LambdaExpressionObj {
    'expressionType': 'Lambda';
    'arguments': ArgumentCollectionObj;
    'expression': FormulaExpressionObj;
}

export interface MemberExpressionObj {
    'expressionType': 'MemberResolve';
    'name': string;
    'arguments'?: TypeArgumentCollectionObj;
    'expression'?: FormulaExpressionObj | null;
    'useNullPropagation': boolean;
};

export interface NewArrayBoundExpressionObj {
    'expressionType': 'NewArrayBounds';
    'type': TypeReferenceObj | string;
    'arguments': ArgumentCollectionObj;
};

export interface NewArrayInitExpressionObj {
    'expressionType': 'NewArrayInit';
    'type': TypeReferenceObj | string;
    'initializers': ArgumentCollectionObj;
};

```

```

export interface ListInitExpressionObj {
  'expressionType': 'ListInit';
  'new': NewExpressionObj;
  'initializers': ElementInitCollectionObj;
};

export interface MemberInitExpressionObj {
  'expressionType': 'MemberInit';
  'new': NewExpressionObj;
  'bindings': MemberBindingCollectionObj;
};

export interface NewExpressionObj {
  'expressionType': 'New';
  'type': TypeReferenceObj | string;
  'arguments': ArgumentCollectionObj;
}

export interface TypeIsExpressionObj {
  'expressionType': 'TypeIs';
  'type': TypeReferenceObj | string;
  'expression': FormulaExpressionObj;
}

export interface TypeOfExpressionObj {
  'expressionType': 'TypeOf';
  'type': TypeReferenceObj | string;
}

export interface TypeReferenceObj {
  'name': string;
  'expressionType'?: 'MemberResolve';
  'expression'?: TypeReferenceObj | string;
  'arguments'?: TypeArgumentCollectionObj;
}

export interface ElementInitBindingObj {
  'expressionType': 'ElementInitBinding';
  'initializers': ArgumentCollectionObj;
}

export interface MemberMemberBindingObj {
  'expressionType': 'MemberBinding';
  'name': string;
  'bindings': MemberBindingCollectionObj;
}

export interface MemberListBindingObj {
  'expressionType': 'ListBinding';
  'name': string;
  'initializers': ElementInitCollectionObj;
}

export interface MemberAssignmentBindingObj {
  'expressionType': 'AssignmentBinding';
  'name': string;
  'expression': FormulaExpressionObj;
}

```

```

export interface TypeArgumentCollectionObj {
  [name: string]: TypeReferenceObj | string;
}

export interface ArgumentCollectionObj {
  [name: string]: FormulaExpressionObj | null;
}

export interface ElementInitCollectionObj {
  [name: string]: ElementInitBindingObj;
}

export interface MemberBindingCollectionObj {
  [name: string]: MemberMemberBindingObj | MemberListBindingObj | MemberAssignmentBindingObj;
}

```

Structure

Every AST node is a JSON object with at least one field:

- `expressionType` - a string that identifies the node type and determines which other fields are present.

Nodes that refer to types use either a plain string (e.g. `"System.Int32"`) or a `TypeReference` object for generic or nested types.

Argument collections (`ArgumentCollectionObj`) are JSON objects where each key is either a zero-based positional index (`"0"`, `"1"`, ...) or a named parameter, and each value is a child expression node (or null for omitted optional arguments).

TypeReference

A type reference describes a type, including generic types. It is used wherever a type is needed (e.g. `Constant`, `Convert`, `New`).

- `name` (*string*) - The simple or fully-qualified type name.
- `expressionType` (*string, optional*) - When set to `"MemberResolve"`, the type is resolved relative to expression.
- `expression` (*TypeReference or string, optional*) - The containing namespace or type from which name is resolved.
- `arguments` (*object, optional*) - A map of generic type arguments, where keys are type parameter names and values are `TypeReference` objects or strings.

Example: `List<int>`

```

{
  "name": "List",
  "expression": { "expressionType": "MemberResolve", "name": "System.Collections.Generic" },
  "expressionType": "MemberResolve",
  "arguments": {
    "T": "System.Int32"
  }
}

```

Expression Types

Constant

Represents a literal value of a known type.

- `expressionType`: "Constant"
- `type` - The type of the value (TypeReference or string).
- `value` - The literal value. The JSON type matches the declared type (number, string, boolean, null, etc.).

Example: the integer literal 42

```
{
  "expressionType": "Constant",
  "type": "System.Int32",
  "value": 42
}
```

Example: the string literal "hello"

```
{
  "expressionType": "Constant",
  "type": "System.String",
  "value": "hello"
}
```

MemberResolve

Resolves a named member (variable, property, field, method, or type) optionally relative to another expression. This is also used to reference formula parameters by name.

- `expressionType`: "MemberResolve"
- `name` (*string*) - The member name to resolve.
- `expression` (*FormulaExpression or null, optional*) - The target object. When null or absent, the name is resolved in the current scope (e.g. a formula parameter or a known type).
- `arguments` (*object, optional*) - Generic type arguments for the member, keyed by type parameter name.
- `useNullPropagation` (*boolean*) - When true, the access uses null-safe semantics (equivalent to `?.` in C#). If expression evaluates to null, the result is null instead of throwing.

Example: accessing `target.HP`

```
{
  "expressionType": "MemberResolve",
  "name": "HP",
  "expression": {
    "expressionType": "MemberResolve",
    "name": "target",
    "expression": null,
    "useNullPropagation": false
  },
  "useNullPropagation": false
}
```

Example: null-safe access `target?.HP`

```
{
  "expressionType": "MemberResolve",
  "name": "HP",
  "expression": {
    "expressionType": "MemberResolve",
    "name": "target",
    "expression": null,
    "useNullPropagation": false
  },
}
```

```

    "useNullPropagation": true
  }

```

Unary

Applies a unary operator to a single operand.

- `expressionType` - One of:
 - "UnaryPlus" - identity (+x)
 - "Negate" - arithmetic negation (-x)
 - "NegateChecked" - arithmetic negation with overflow checking
 - "Not" - logical NOT (!x)
 - "Complement" - bitwise complement (~x)
 - "Group" - grouping parentheses, no operation change ((x))
 - "UncheckedScope" - disables overflow checking for the inner expression (unchecked(x))
 - "CheckedScope" - enables overflow checking for the inner expression (checked(x))
- `expression` (*FormulaExpression*) - The operand.

Example: -x

```

{
  "expressionType": "Negate",
  "expression": {
    "expressionType": "MemberResolve",
    "name": "x",
    "expression": null,
    "useNullPropagation": false
  }
}

```

Binary

Applies a binary operator to two operands.

- `expressionType` - One of:
 - Arithmetic: "Add", "AddChecked", "Subtract", "SubtractChecked", "Multiply", "MultiplyChecked", "Divide", "Modulo", "Power"
 - Bitwise / Shift: "And", "Or", "ExclusiveOr", "LeftShift", "RightShift"
 - Comparison: "Equal", "NotEqual", "LessThan", "LessThanOrEqual", "GreaterThan", "GreaterThanOrEqual"
 - Logical short-circuit: "AndAlso" (&&), "OrElse" (||)
 - Null coalescing: "Coalesce" (??)

Variants with Checked suffix perform overflow-checked arithmetic.

- `left` (*FormulaExpression*) - The left operand.
- `right` (*FormulaExpression*) - The right operand.

Example: `weaponPower * targetResistance`


```
{
  "expressionType": "Multiply",
  "left": {
    "expressionType": "MemberResolve",
    "name": "weaponPower",
    "expression": null,
    "useNullPropagation": false
  },
  "right": {
    "expressionType": "MemberResolve",
    "name": "targetResistance",
    "expression": null,
    "useNullPropagation": false
  }
}
```

Condition

A conditional (ternary) expression: test ? ifTrue : ifFalse.

- expressionType: "Condition"
- test (*FormulaExpression*) - The boolean condition.
- ifTrue (*FormulaExpression*) - Evaluated when test is true.
- ifFalse (*FormulaExpression*) - Evaluated when test is false.

Example: $x > 0 ? x : -x$

```
{
  "expressionType": "Condition",
  "test": {
    "expressionType": "GreaterThan",
    "left": { "expressionType": "MemberResolve", "name": "x", "expression": null, "useNullPropagation": false },
    "right": { "expressionType": "Constant", "type": "System.Int32", "value": 0 }
  },
  "ifTrue": {
    "expressionType": "MemberResolve",
    "name": "x",
    "expression": null,
    "useNullPropagation": false
  },
  "ifFalse": {
    "expressionType": "Negate",
    "expression": { "expressionType": "MemberResolve", "name": "x", "expression": null, "useNullPropagation": false }
  }
}
```

Convert / TypeAs

Converts or casts an expression to a different type.

- expressionType - One of:
 - "Convert" - explicit cast; throws on failure.
 - "ConvertChecked" - explicit cast with overflow checking.

- "TypeAs" - safe cast; returns null if the object is not of the target type (equivalent to as in C#).
- type - The target type (TypeReference or string).
- expression (*FormulaExpression*) - The expression to convert.

Example: (float)damage

```
{
  "expressionType": "Convert",
  "type": "System.Single",
  "expression": {
    "expressionType": "MemberResolve",
    "name": "damage",
    "expression": null,
    "useNullPropagation": false
  }
}
```

Default

Produces the default value for a type (e.g. 0 for numeric types, null for reference types).

- expressionType: "Default"
- type - The type whose default value to produce (TypeReference or string).

Example: default(Int32)

```
{
  "expressionType": "Default",
  "type": "Int32"
}
```

Types

Tests whether an expression is of a given type (equivalent to is in C#). Evaluates to a boolean.

- expressionType: "TypeIs"
- type - The type to test against (TypeReference or string).
- expression (*FormulaExpression*) - The expression whose type is tested.

Example: obj is Enemy

```
{
  "expressionType": "TypeIs",
  "type": "Enemy",
  "expression": {
    "expressionType": "MemberResolve",
    "name": "obj",
    "expression": null,
    "useNullPropagation": false
  }
}
```

typeof

Returns the runtime type descriptor for the given type (equivalent to `typeof(T)` in C#).

- `expressionType`: "typeof"
- `type` - The type to retrieve (`TypeReference` or `string`).

Example: `typeof(String)`

```
{
  "expressionType": "typeof",
  "type": "String"
}
```

Index

Accesses an element of an indexable object (array, list, dictionary, or any type with an indexer).

- `expressionType`: "Index"
- `expression` (*FormulaExpression*) - The collection or indexable object.
- `arguments` (*ArgumentCollectionObj*) - The index arguments, keyed by positional index ("0", "1", ...).
- `useNullPropagation` (*boolean*) - When true, the access is null-safe (`?[...]`). Returns null if expression is null.

Example: `items[0]`

```
{
  "expressionType": "Index",
  "expression": {
    "expressionType": "MemberResolve",
    "name": "items",
    "expression": null,
    "useNullPropagation": false
  },
  "arguments": {
    "0": { "expressionType": "Constant", "type": "Int32", "value": 0 }
  },
  "useNullPropagation": false
}
```

Invoke

Calls a method or invokes a delegate/function. The method itself is resolved as a `MemberResolve` node in the expression field.

- `expressionType`: "Invoke"
- `expression` (*FormulaExpression*) - The callable to invoke (typically a `MemberResolve` node).
- `arguments` (*ArgumentCollectionObj*) - Arguments passed to the call, keyed by positional index ("0", "1", ...) or parameter name.

Example: `target.DoDamage(100)`

```
{
  "expressionType": "Invoke",
  "expression": {
    "expressionType": "MemberResolve",
```

```

    "name": "DoDamage",
    "expression": {
      "expressionType": "MemberResolve",
      "name": "target",
      "expression": null,
      "useNullPropagation": false
    },
    "useNullPropagation": false
  },
  "arguments": {
    "0": { "expressionType": "Constant", "type": "Int32", "value": 100 }
  }
}

```

Lambda

Defines an anonymous function (lambda). The arguments field declares the parameters; expression is the body.

- `expressionType`: "Lambda"
- `arguments` (*ArgumentCollectionObj*) - Parameter declarations. Keys are parameter names; values are null (parameter type is inferred from context) or a `MemberResolve` expression denoting the declared type.
- `expression` (*FormulaExpression*) - The lambda body.

Example: `x => x * 2`

```

{
  "expressionType": "Lambda",
  "arguments": {
    "x": null
  },
  "expression": {
    "expressionType": "Multiply",
    "left": { "expressionType": "MemberResolve", "name": "x", "expression": null, "useNullPropagation": false },
    "right": { "expressionType": "Constant", "type": "System.Int32", "value": 2 }
  }
}

```

New

Instantiates a new object of a given type by calling a constructor.

- `expressionType`: "New"
- `type` - The type to instantiate (*TypeReference* or string).
- `arguments` (*ArgumentCollectionObj*) - Constructor arguments, keyed by positional index ("0", "1", ...) or parameter name.

Example: `new Vector2(1.0, 0.5)`

```

{
  "expressionType": "New",
  "type": "Vector2",
  "arguments": {
    "0": { "expressionType": "Constant", "type": "System.Single", "value": 1.0 },
    "1": { "expressionType": "Constant", "type": "System.Single", "value": 0.5 }
  }
}

```

```
}
}
```

NewArrayBounds

Creates a new array with specified dimension sizes but without initializers.

- `expressionType`: "NewArrayBounds"
- `type` - The element type of the array (TypeReference or string).
- `arguments` (*ArgumentCollectionObj*) - Dimension size expressions, keyed by dimension index ("0" for one-dimensional arrays).

Example: `new int[10]`

```
{
  "expressionType": "NewArrayBounds",
  "type": "System.Int32",
  "arguments": {
    "0": { "expressionType": "Constant", "type": "System.Int32", "value": 10 }
  }
}
```

NewArrayInit

Creates a new single-dimensional array and initializes it with a list of values.

- `expressionType`: "NewArrayInit"
- `type` - The element type of the array (TypeReference or string).
- `initializers` (*ArgumentCollectionObj*) - Element expressions, keyed by positional index ("0", "1", ...).

Example: `new int[] { 1, 2, 3 }`

```
{
  "expressionType": "NewArrayInit",
  "type": "System.Int32",
  "initializers": {
    "0": { "expressionType": "Constant", "type": "System.Int32", "value": 1 },
    "1": { "expressionType": "Constant", "type": "System.Int32", "value": 2 },
    "2": { "expressionType": "Constant", "type": "System.Int32", "value": 3 }
  }
}
```

MemberInit

Creates a new object and initializes its members. Combines a New node with a set of member bindings.

- `expressionType`: "MemberInit"
- `new` (*NewExpression*) - The object construction node.
- `bindings` (*MemberBindingCollectionObj*) - A map of member bindings, keyed by member name. Each value is one of:
 - `AssignmentBinding` - assigns an expression to a member.

- MemberBinding - recursively initializes members of a nested object.
- ListBinding - adds elements to a collection member.

Example: new Vector2 { X = 1.0, Y = 0.5 }

```
{
  "expressionType": "MemberInit",
  "new": {
    "expressionType": "New",
    "type": "Vector2",
    "arguments": {}
  },
  "bindings": {
    "X": {
      "expressionType": "AssignmentBinding",
      "name": "X",
      "expression": { "expressionType": "Constant", "type": "System.Single", "value": 1.0 }
    },
    "Y": {
      "expressionType": "AssignmentBinding",
      "name": "Y",
      "expression": { "expressionType": "Constant", "type": "System.Single", "value": 0.5 }
    }
  }
}
```

ListInit

Creates a new collection object and populates it using an initializer list. Each element is added by calling the collection's Add method (or equivalent).

- expressionType: "ListInit"
- new (*NewExpression*) - The collection construction node.
- initializers (*ElementInitCollectionObj*) - A map of element initializers, keyed by positional index. Each value is an ElementInitBinding object with:
 - expressionType: "ElementInitBinding"
 - initializers (*ArgumentCollectionObj*) - Arguments passed to the Add method for this element.

Example: new List<int> { 1, 2, 3 }

```
{
  "expressionType": "ListInit",
  "new": {
    "expressionType": "New",
    "type": {
      "name": "List",
      "expression": "System.Collections.Generic",
      "expressionType": "MemberResolve",
      "arguments": { "T": "System.Int32" }
    },
    "arguments": {}
  },
  "initializers": {
    "0": {
      "expressionType": "ElementInitBinding",
      "initializers": {
        "0": { "expressionType": "Constant", "type": "System.Int32", "value": 1 }
      }
    }
  }
}
```

```

    }
  },
  "1": {
    "expressionType": "ElementInitBinding",
    "initializers": {
      "0": { "expressionType": "Constant", "type": "System.Int32", "value": 2 }
    }
  },
  "2": {
    "expressionType": "ElementInitBinding",
    "initializers": {
      "0": { "expressionType": "Constant", "type": "System.Int32", "value": 3 }
    }
  }
}
}
}

```

Member Bindings

Member bindings are used inside MemberInit nodes to specify how object members are initialized.

AssignmentBinding

Assigns a value to a named member.

- `expressionType`: "AssignmentBinding"
- `name` (*string*) - Member name.
- `expression` (*FormulaExpression*) - The value to assign.

MemberBinding

Recursively initializes the members of a nested object member without reassigning the member itself.

- `expressionType`: "MemberBinding"
- `name` (*string*) - Member name of the nested object.
- `bindings` (*MemberBindingCollectionObj*) - Nested bindings applied to the member's sub-members.

ListBinding

Adds elements to a collection member using element initializers.

- `expressionType`: "ListBinding"
 - `name` (*string*) - Member name of the collection.
 - `initializers` (*ElementInitCollectionObj*) - Elements to add, keyed by positional index.
-

Implementing an Interpreter

An interpreter for this AST typically consists of a recursive `evaluate(node, context)` function that dispatches on `expressionType`. The context holds the current scope: parameter values, known types, and available members.

Suggested steps:

1. **Dispatch** - read `expressionType` from the node and select the appropriate handler.

- 2. **Recurse** - evaluate child nodes before computing the current node's result (bottom-up evaluation).
- 3. **Scope** - maintain a variable scope for formula parameters; MemberResolve with a null expression looks up names in this scope first, then in known types.
- 4. **Type resolution** - map type names (strings or TypeReference objects) to concrete types in your runtime. Built-in types follow .NET naming conventions (e.g. System.Int32, System.String).
- 5. **Null propagation** - for MemberResolve and Index nodes with useNullPropagation: true, short-circuit and return null when the target expression is null.
- 6. **Overflow checking** - Checked variants of arithmetic operators should raise an error on integer overflow; UncheckedScope / CheckedScope change the checking mode for a sub-tree.

See also

- Game Data Structure
- Data Types
- [Abstract Syntax Tree \(Wikipedia\)](#)
- [Expression tree pattern](#)

UI Extensions

An **extension** is a package of your own code that Charon loads into the editor at runtime. It can replace the editor of a single field, replace the whole document form with a graph or a node editor, add entries to Charon's menus, and add pages of its own to the side navigation.

Extensions are **Web Components** packaged as **NPM modules**. Nothing about them is framework-specific - React, Angular, Vue, or plain JavaScript all work - and they are installed per project, so everyone working on that project gets the same set.

What an Extension Can Add

How Extensions Are Declared

Choosing an Editor for a Property

Installing an Extension

Consent and Trust

When an Extension Fails to Load

Packaging and Publishing

Examples

See also

310

311

312

312

313

313

314

314

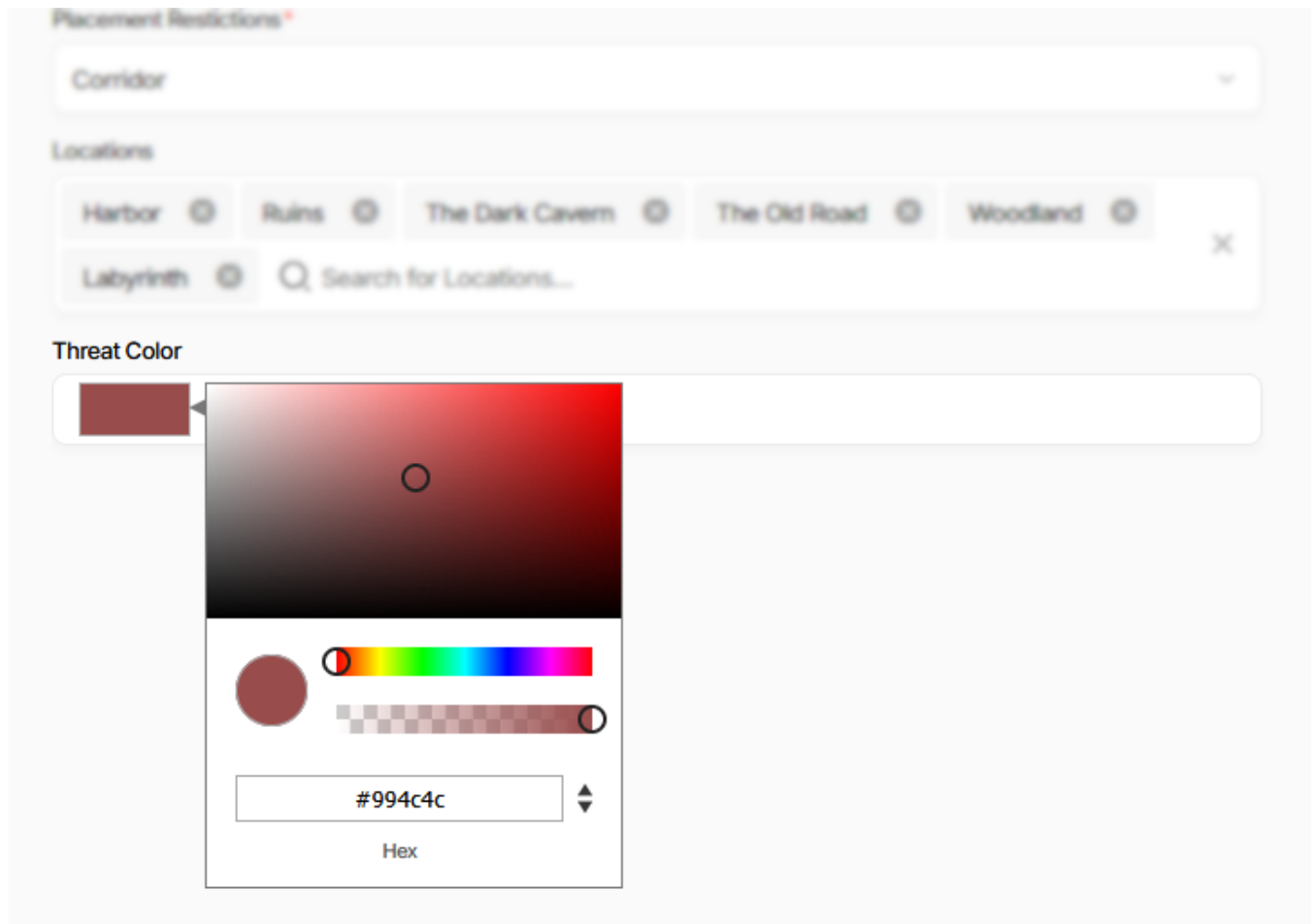
315

What an Extension Can Add

Kind	What it replaces or adds	Declared in package.json
Property editor	The input for one field of a document form.	config.customEditors, type: ["Property"]
Grid editor	The cell editor for the same property in the collection grid.	config.customEditors, type: ["Grid"]
Schema editor	The entire editing view of a document - see Implementing a Schema Editor.	config.customEditors, type: ["Schema"]
Custom action	A menu entry in one of four places in the UI - see Custom Actions.	config.customActions

Custom page	A routed, bookmarkable page of its own - see Custom Pages.	<code>config.customPages</code>
--------------------	--	---------------------------------

A property editor is the smallest of these and the usual starting point. The colour picker below is one - the field is an ordinary Text property, and the extension supplies the swatch and the picker popup:



Whatever the kind, the extension gets the host's services along with it: game data queries, dialogs and wizards, notifications, navigation, and persisted UI state. See Host Services.

How Extensions Are Declared

Everything Charon needs to know about a package is in its `package.json`, under `config`:

```
{
  "name": "charon-color-picker",
  "version": "1.0.24",
  "main": "main.js",
  "config": {
    "customEditors": [
      {
        "id": "ext-color-picker-hex",
        "selector": "ext-color-picker-editor",
        "name": "Color Picker (Hex)",
        "specification": "format=hex",
        "type": ["Property", "Grid"],
        "dataTypes": ["Text", "Integer"]
      }
    ]
  }
}
```

```
}
}
```

- `id` is how the editor is referenced from a schema or property, and must be unique across every extension installed in the project.
- `selector` is the custom element name the package registers with `customElements.define`.
- `name` is what the user sees when picking the editor.
- `dataTypes` limits which data types the editor is offered for.
- `specification` is extra configuration in `URLSearchParams` form. It is merged into the property's Specification when the editor is picked, which is how one custom element can back three entries - `format=hex`, `format=hsla`, `format=rgba`.
- `icon` is optional and takes a Charon icon reference, `<set>/<name>`: `svg/` for Charon's own icons, `emoji/` for an emoji by its short name (`emoji/speech_balloon`), `material/` for a Material icon. The default is `svg/extensions`.

The full JSON schema for the config section, including custom actions and custom pages, is [package.json.schema.json](#) in the examples repository. Point your editor at it for completion and validation while writing.

Choosing an Editor for a Property

Installed editors appear in the **Data Type** picker of the property form, nested under the data types they declared in `dataTypes`. Picking one writes `editor=<id>` into the property's Specification, together with whatever the extension declared in its own `specification` field.

A **Schema** editor is selected the same way but on the schema, by putting `editor=<id>` into the schema's Specification. An extension that ships a schema editor usually creates the schema for you through a custom action instead of asking you to type that in.

Installing an Extension

Open **Project Settings** → **Extensions**. Each row is an NPM package name and a version; leave the version empty to track the latest published one. The name field completes against the configured registry as you type.

Press **Update** to save the list. The status icon at the end of each row says what happened:

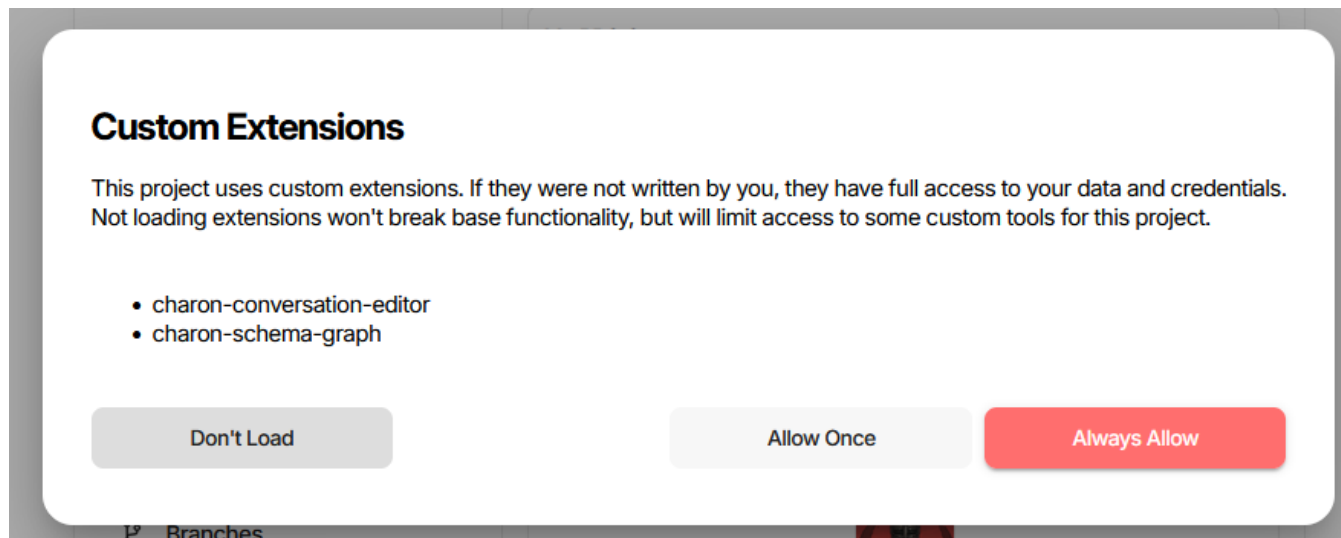
Status	Meaning
Loaded	The package was downloaded and its code is running. The tooltip names the exact version.
Extension not declared in project settings	The package is present on the server but not in this project's list.
Extension package not found	No such package in the registry.
No compatible version available	The requested version does not exist, or is not compatible with this Charon build.
Failed to download from registry	The registry was unreachable or rejected the request.
Package exceeds size limit	The archive is larger than the server allows.
Invalid or corrupted package archive	The <code>.tgz</code> could not be unpacked.

Re-Check asks the server whether newer versions of the installed packages are available. **Upload NPM Package...** takes a `.tgz` built locally with `npm pack` and installs it into the server's extension folder - convenient while developing, but the upload is per installation, so other users of a shared project will not get it. Publish to a registry for anything the team relies on.

The list itself is stored in the project settings inside the game data, so it travels with the data and applies to everyone who opens the project.

Consent and Trust

Extension code runs inside the editor with the same access as the editor itself - your game data and your credentials. Charon therefore asks before loading anything, the first time a project's extension list is seen:



- **Don't Load** - the editor starts without them. Nothing else breaks; only the extensions' own features are missing.
- **Allow Once** - loads them for this session and asks again next time.
- **Always Allow** - remembers the decision for that exact set of packages. A green tick next to a name in a later prompt means it was part of a previously allowed set.

Both allow buttons stay disabled for a few seconds so the dialog cannot be clicked through by accident. The remembered decision is tied to the list of package names: adding or removing an extension makes Charon ask again. **Allow loading these extensions without asking again** on the settings page is the same decision, and clearing it brings the prompt back.

Warning

Install extensions the way you would install a dependency into your build: from a source you trust, at a version you have looked at. A malicious extension can read everything you can read and act as you.

When an Extension Fails to Load

A failure is reported in the notifications area and does not stop the editor:



The browser console (F12) carries the actual error. Common causes:

- **the package could not be fetched or unpacked** - the row's status icon on the settings page says which of those it was;
- **an id or selector collision** - two installed packages declare the same editor id, the same element selector, or the same page id. Charon refuses to load the second one and logs which package it conflicted with;
- **the entry point threw** while registering its custom elements;
- **a missing export** - a custom action naming a functionName that the bundle does not export is skipped, with a warning naming the function;
- **a dangling page reference** - a custom action naming a pageId that is not in the same package's own customPages produces a link that does not resolve.

If nothing loads at all, check that the `uiExtensions` feature is enabled for your installation and that the consent prompt was not answered with **Don't Load**.

Packaging and Publishing

Build the package, then produce an archive:

```
npm run build
npm pack
```

That writes a `.tgz` next to the build output, ready for **Upload NPM Package....** To publish for the whole team instead:

```
npm publish
```

Remove `"private": true` first, and make sure `main` points at the built entry file. Any `.css` file listed in the package's `files` array is fetched and injected by Charon along with the code, which is how a bundled stylesheet reaches the page.

Versions are resolved when the project loads, so publishing a new version rolls out to everyone with an empty version field the next time they open the editor. Pin a version in the settings row when you need that not to happen.

Examples

The [charon-extensions repository](#) holds the type definitions and four complete example packages:

Package	What it demonstrates
charon-extensions	The API contract itself - every interface described in these pages.
charon-logical-toggle	The simplest property editor, in React.
charon-color-picker	A property editor in Angular, including the zoneless setup.
charon-conversation-editor	A full schema editor with a node graph, plus a custom action that creates its schema.
charon-schema-graph	A custom page with a side-navigation entry.

Two scaffolders generate a ready-to-build property editor package:

```
node src/create-charon-react-extension/index.js my-extension
node src/create-charon-angular-extension/index.js my-extension
```

See also

- Implementing a Property Editor
- Implementing a Schema Editor
- Custom Actions
- Custom Pages
- Host Services
- Creating a Custom Editor with React
- Creating a Custom Editor with Angular
- [Example Projects \(GitHub\)](#)

Implementing CharonPropertyEditorElement

Each custom property editor must implement the `CharonPropertyEditorElement` interface from the [charon-extensions](#) package:

```
export declare interface CharonPropertyEditorElement {
  valueControl: ValueControl;
}
```

When the editor is rendered, Charon sets `valueControl` to allow your component to:

- Subscribe to input value changes.
- Update the current value.
- Access schema metadata.
- Add custom validation.
- Respond to focus events.

ValueControl

The `ValueControl` interface loosely mirrors Angular's [AbstractControl](#). Here's a simplified definition:

```
export declare interface ValueControl<TValue = any> {
  readonly schemaProperty: SchemaProperty;

  readonly valueChanges: ObservableLike<TValue>;
  readonly value: TValue;

  setValue(value: TValue, options?: ControlEventEmitOptions): void;

  registerDoFocus(fn: (options?: FocusOptions) => void, targetName?: string): TeardownLogic;

  readonly errors: Object | null;

  addValidators(validators: ValueValidatorFn | ValueValidatorFn[]): void;
  removeValidators(validators: ValueValidatorFn | ValueValidatorFn[]): void;
  hasValidator(validator: ValueValidatorFn): boolean;
}
```

Purpose of ValueControl

The primary purpose of the `ValueControl<TValue>` interface is to act as a centralized, stateful container that manages a single piece of data (a form field's value) along with all its associated metadata, lifecycle, and interaction states.

It decouples the **data model** and **validation logic** from the specific user interface component (e.g., an HTML `<input>`).

Key responsibilities include:

- **Value Management:** Storing the current value, tracking its original/starting value, and handling updates (setValue, patchValue, reset).
- **Validation & Status:** Tracking the control's validation state (valid, invalid, pending, errors), providing methods to add/remove validators (addValidators, hasError), and updating the status (updateValueAndValidity).
- **Interaction State:** Tracking user interaction flags (touched, untouched, dirty, pristine) and providing methods to change them (markAsTouched).
- **Hierarchy/Structure:** Maintaining a tree structure with references to its parent and root control, allowing for complex, nested forms and path-based lookups (getByPath).
- **Observability:** Exposing change events via observables (valueChanges, statusChanges) to allow other parts of the application to react to state changes.
- **UI Integration (Focus/Toggle):** Providing methods and handlers (focus, toggle, registerDoFocus, registerDoToggle) to programmatically control the *view* component it represents.

Reacting to Value Changes

When your editor receives the `valueControl`, subscribe to `valueChanges`:

```
useEffect(() => {
  const subscription = valueControl.valueChanges.subscribe(value => {
    setLocalValue(value); // Update local state
  });

  return () => subscription.unsubscribe(); // Cleanup on unmount
}, [valueControl]);
```

When the user edits the field, call:

```
valueControl.setValue(newValue); // it will trigger `valueControl.valueChanges` even if value
```

The format and type of the value are described in `valueControl.schemaProperty`.

Handling Focus Events

Charon can request that your editor focuses the first interactive element. Use `registerDoFocus`:

```
useEffect(() => {
  const teardown = valueControl.registerDoFocus(() => {
    inputRef.current?.focus();
  });

  return () => teardown(); // Unregister on unmount or control change
}, [valueControl]);
```

Providing Custom Validators

To enable custom validation:

1. Register validators when `valueControl` is assigned.
2. Clean them up when your component is destroyed or `valueControl` changes.

Example:

```
useEffect(() => {
  const validateValue = (control: ValueControl): Object | null => {
    if (isValidColor(control.value)) {
      return null;
    }
    return {
      ERROR_CUSTOM: 'Invalid value for color in #XXXXXX format.'
    };
  };
}, [valueControl]);
```

Is It Free?

```
};  
};  
  
valueControl.addValidators(validateValue);  
return () => valueControl.removeValidators(validateValue);  
}, [valueControl]);
```

Reaching Host Services

A property editor is handed one value, but the rest of the editor is still reachable through the root of the control tree:

```
import { getRootDocumentControl } from 'charon-extensions';  
  
const root = getRootDocumentControl(valueControl);  
const languages = root.services.translationLanguage?.languages$;
```

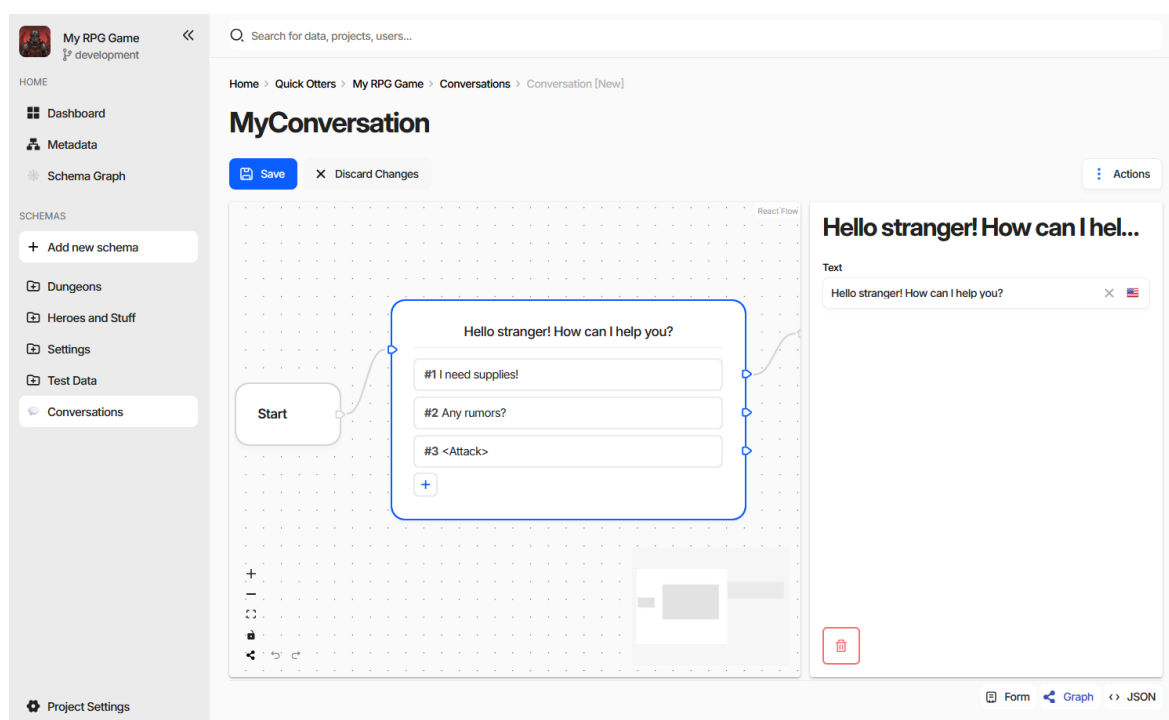
That gives the document being edited, its schema, and the document-level services - game data queries, dialogs, notifications, undo/redo, and persisted UI state. See Host Services.

See also

- UI Extensions
- Implementing a Schema Editor
- Host Services
- Creating a Custom Editor with React
- Creating a Custom Editor with Angular
- [charon-extensions package \(NPM\)](#)
- [Web Components \(MDN\)](#)

Implementing CharonSchemaEditorElement

A **schema editor** replaces the entire document form for one schema. Where a property editor is handed a single value, a schema editor is handed the whole document and decides how all of it is presented - as a node graph, a timeline, a board, a map, whatever the data actually is.



The screenshot is the `charon-conversation-editor` example: dialogue lines are nodes on a canvas, the links between them are edges, and the selected node's fields are in the panel on the right. Underneath it is an ordinary document - the nodes are entries of an embedded document collection - which is why the host's `Save`, `Discard Changes`, and the `Form` and `JSON` view modes keep working next to it.

Declaring the Editor	318
The Element Contract	318
Working with the Document	319
Read-Only Documents	320
Document Services	320
Undo and Redo	320
Development	320
See also	321

Declaring the Editor

A schema editor is a `customEditors` entry with `type: ["Schema"]`. `dataTypes` is meaningless here and is left out:

```
"config": {
  "customEditors": [
    {
      "id": "ext-conversation-editor",
      "selector": "ext-conversation-editor",
      "name": "Conversation Editor",
      "type": ["Schema"]
    }
  ]
}
```

A schema uses it when its `Specification` contains `editor=ext-conversation-editor`. Nothing else opts in - the same editor serves every schema whose specification names it, and a project can have several schema editors installed at once.

Because a schema editor usually needs the document laid out a particular way - specific properties, specific data types - packages rarely ask the user to write that specification by hand. They ship a custom action in the `new-schema-menu` that creates the schema, its properties, and the `editor=specification` in one step. That is what **Create Conversation Schema** does in the example.

Once a document of such a schema is opened, the editor appears as the **Graph** view mode in the bottom right corner of the form, alongside **Form** and **JSON**. The mode is absent for every other document.

The Element Contract

```
export declare interface CharonSchemaEditorElement {
  documentControl: RootDocumentControl;
}
```

Charon creates the element by its selector, sets `documentControl` on it, and appends it to the page. The setter runs **before** `connectedCallback`, so mount your framework on whichever of the two comes second:

```
export default class ConversationEditorElement extends HTMLElement implements CharonSchemaEditorElement {
  private _documentControl?: RootDocumentControl<ConversationTree>;
  private _root?: Root;

  get documentControl() { return this._documentControl!; }
```



```

set documentControl(value: RootDocumentControl<ConversationTree>) {
    if (Object.is(value, this._documentControl)) {
        return;
    }
    this._documentControl = value;
    this.render();
}

public connectedCallback() { this.render(); }
public disconnectedCallback() { this.unmount(); }
}

```

The control can be replaced while the element is alive - navigating to the next document with the < and > buttons does exactly that - so re-render on assignment rather than only on connect, and drop subscriptions to the previous control.

RootDocumentControl is generic over the document shape, so declaring your own interface for the schema you target gives the whole editor typed access to the document's properties.

Working with the Document

RootDocumentControl is a DocumentControl with the schema attached:

Member	Purpose
schema	The full schema - properties, data types, id generator, display text template.
controls	Child controls by property name. A Document property gives a DocumentControl, a document collection gives a DocumentCollectionControl, a reference collection gives a ReferenceCollectionControl, everything else a ValueControl.
value / setValue / patchValue	The document as data, and the two ways to write it back.
valueChanges / statusChanges	Streams to bind your own state to.
getByPath	A control by JSON Pointer, for editors that address the document by path.
errors / status / addValidators	Validation state, shared with the host's error counter.
readOnly / disabled / dirty / touched	UI state, also shared with the host.
services	The document-scoped host services, below.

Collections are edited through the collection control rather than by replacing the array: append, insertAt, swap, removeAt, and remove keep the child controls, their validation, and the host's change tracking intact. Adding a node to a graph is an append; dragging one node before another is a swap.

Note

A schema editor never saves. It edits controls; the host's Save button writes the document, and only the fields that actually changed. The same applies to Discard Changes, which resets the controls under you - another reason to render from valueChanges rather than from a copy taken once at mount.

Read-Only Documents

Honour `readOnly` and `disabled`. The host hides its own editing buttons when the game data is read-only or the user lacks permission, but it cannot stop a custom canvas from letting people drag things around - and edits made in that state are lost. See [When the Form Is Read-Only](#).

Document Services

`documentControl.services` carries what the host can offer for the document being edited. Every field is optional - check before use:

Service	What it gives you
<code>validationProvider</code>	<code>getDocumentValue(path, 'base' 'starting' 'current')</code> for comparing against the server state or the state at the start of the session, plus <code>listDocuments</code> , <code>listLocalDocuments</code> , and <code>findDocument</code> for validators that need to look elsewhere.
<code>translationLanguage</code>	<code>primaryLanguage\$</code> , <code>languages\$</code> , <code>currentLanguage\$</code> , and <code>selectLanguage</code> - so an editor showing localized text follows the language the rest of the form is on.
<code>gameData</code>	The project-wide game data service - querying, importing, exporting, validating.
<code>undoRedo</code>	The host's undo/redo history, below.
<code>ui.state</code>	Persisted UI state scoped to this document or its schema - see Host Services .
<code>ui.dialog / ui.snackBar</code>	Dialogs, wizards, and notifications.

Undo and Redo

The host's `Ctrl/Cmd+Z` and `Ctrl/Cmd+Y` drive the same `UndoRedoService` your editor sees, so changes made through the controls are undone by the toolbar and the keyboard without any work on your side. Reach for the service only when an interaction is not a plain control write:

- `push({ redo, undo }, batcher?)` records a step of your own - a node moved on the canvas, a layout recalculated. The optional `batcher` receives the previous entry and the milliseconds since it was pushed; returning that same object by reference merges the two into one undo step, which is how a drag becomes one step instead of forty.
- `barrier()` forces the next change to start a new step, ending any automatic batching.
- `clear()` throws the history away.

Warning

Do not maintain a private undo stack. Two histories over one document diverge as soon as the user presses the toolbar button instead of yours.

Development

A schema editor is harder to iterate on inside Charon than a property editor, so both example packages ship a development harness: `main.tsx` detects `import.meta.env.MODE === 'development'`, builds a stub `documentControl` over a sample document, persists it to `localStorage`, and mounts the element standalone. `npm run dev` then gives the editor in a browser with no Charon at all, and the same bundle is what gets packed for release.

Wrap the editor in an error boundary. An exception thrown while rendering a canvas takes the whole form down otherwise, and the user is left with a blank page and a console message.

When the editor does need to run inside Charon, build it, npm pack it, and use **Upload NPM Package...** on the Extensions settings page.

See also

- UI Extensions
- Implementing a Property Editor
- Custom Actions
- Host Services
- Editing a Document
- Schema
- [charon-conversation-editor \(GitHub\)](#)

Custom Actions

A **custom action** is an entry an extension adds to one of Charon’s menus. Clicking it either calls a function the package exports, or navigates to a custom page the package declares.

Actions are how an extension does work that is not tied to editing one value: generating schemas, running a bulk change, exporting into a format of your own, opening a tool. The function receives a context object with the host’s services, so it can read and write game data, show dialogs and wizards, report progress, and navigate.

Declaring an Action	321
Where Actions Appear	322
Grouping Side Navigation Entries	322
The Action Context	323
Reporting Progress	323
See also	324

Declaring an Action

```
"config": {
  "customActions": [
    {
      "name": "Create Conversation Schema",
      "functionName": "createConversationSchema",
      "location": "new-schema-menu",
      "icon": "emoji/speech_balloon"
    }
  ]
}
```

Field	Meaning
name	The menu entry’s text. Required.
location	Which menu it goes in. Required, one of the four below.

functionName	The function exported from the package entry point to call. Mutually exclusive with pageId.
pageId	The id of a page in this same package's customPages. Clicking navigates there instead of calling anything. Mutually exclusive with functionName.
icon	Optional icon reference, <set>/<name> - svg/ for Charon's own icons, emoji/ for an emoji by short name, material/ for a Material icon. Defaults to svg/extensions.
tooltip	Optional hover text.

Exactly one of functionName and pageId must be set. Declaring both makes Charon use pageId and warn; declaring neither leaves an entry that does nothing but log a warning when clicked.

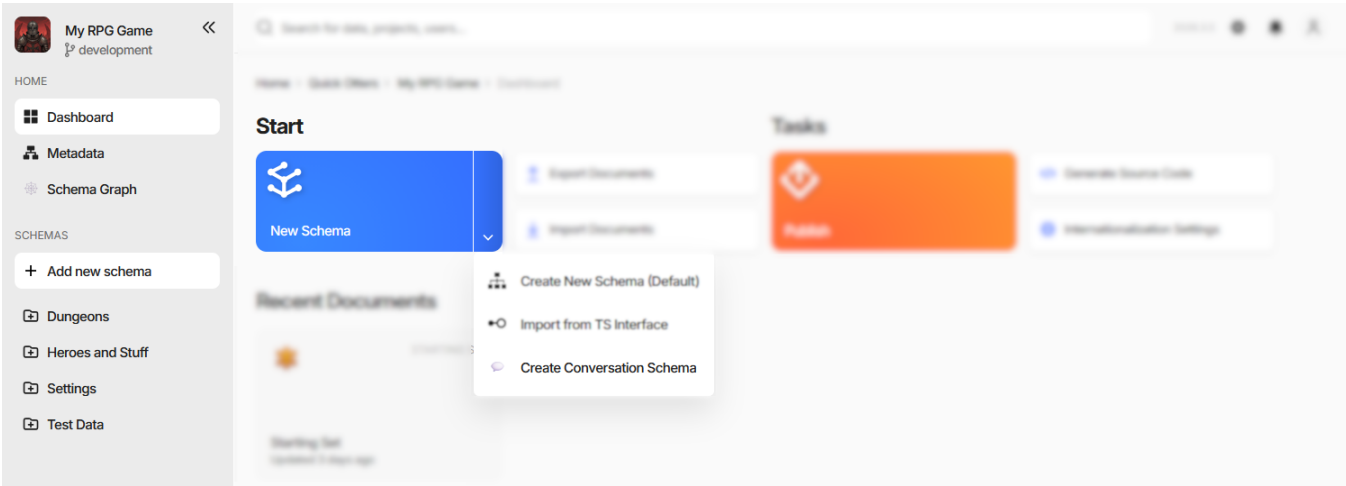
The named function must be exported from the module Charon loads - the file main points at. Bundlers tree-shake exports that nothing imports, so keep a reference to it; the example packages assign it to window for exactly that reason.

```
export { createConversationSchema } from './schema.validation/migrate.schema.ts';

// keeps the bundler from dropping it
(window as any).createConversationSchema = createConversationSchema;
```

Where Actions Appear

location	Where it shows up
new-schema-menu	The dropdown of the New Schema button on the project dashboard, below the built-in entries.
document-action-menu	The Actions menu of the document form, below the separator.
document-list-action-menu	The Actions menu of a document collection, below the separator.
side-navigation-menu	A permanent entry in the left side navigation.



Create Conversation Schema in the screenshot sits under **New Schema** next to Charon's own **Create New Schema (Default)** and **Import from TS Interface**, and creates a schema already configured to use the package's schema editor.

Grouping Side Navigation Entries

A side-navigation-menu action whose name contains a / is filed under a section: Tools/Schema Graph puts **Schema Graph** into a **Tools** section of the side navigation. A name without a / sits in the top group with **Dashboard** and **Metadata**.

The Action Context

The function is called with one argument, whose shape follows the location it was declared for:

Context	Extra fields beyond services
NewSchemaActionContext	none
DocumentActionContext	documentControl - the RootDocumentControl of the document open in the form
DocumentListActionContext	schema of the listed collection, and selection
SideNavigationActionContext	none

selection reports the grid's current rows: selected holds the ids, changed emits the ids added and removed by each change, and isAllSelected is true when the user chose *select all* for the whole filtered collection. In that case treat the selection as "every document in the current view" - selected can be empty while it is true, because the rows were never all loaded.

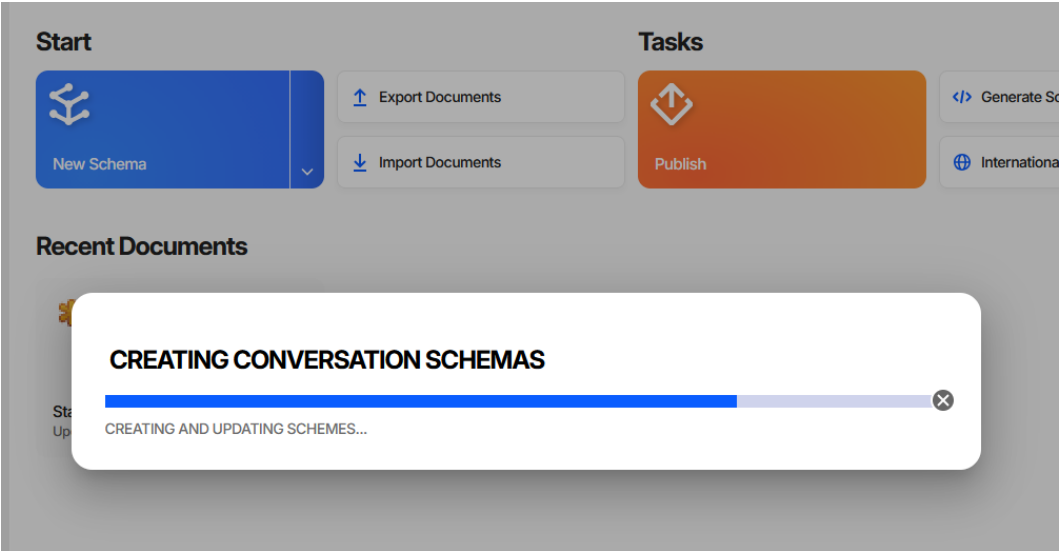
Every context carries services, and every field of it is optional. The host populates only what makes sense where the action was triggered, so check before use:

```
export async function createConversationSchema(context: ExtensionActionContext) {
  const progressRef = context.services.ui?.dialog?.showProgress({
    title: 'Creating Conversation Schemas',
    progressMode: 'determinate',
    cancellable: true,
    closedHandler: () => abortController.abort()
  });
  ...
}
```

The services themselves - game data, dialogs, wizards, notifications, navigation, persisted UI state - are described in Host Services.

Reporting Progress

Anything that takes more than a moment should say so. `services.ui.dialog.showProgress` returns a handle with `update(percent, message)`, `setFaulted()`, and `close(delayMs?)`:



With `cancellable: true` the dialog gets a close button, and `closedHandler(cancelled)` fires when the user uses it - abort the work there rather than letting it run on invisibly. The pattern the examples follow is to report success through the snack bar, and on failure to mark the dialog faulted and let it close itself a few seconds later:

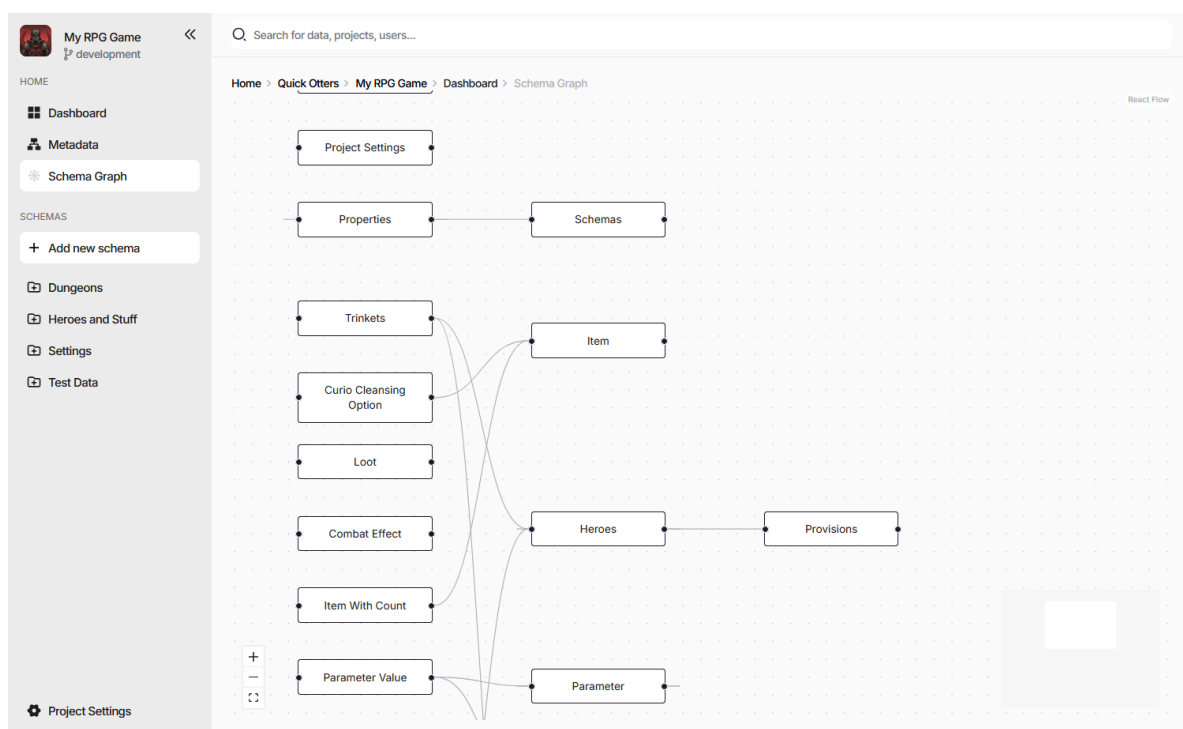
```
try {
  await doTheWork(progressRef, abortController.signal);
  context.services.ui?.snackBar?.saveSucceed();
  progressRef?.close();
} catch (error) {
  if (abortController.signal.aborted) { return; }
  context.services.ui?.snackBar?.saveFailed(error);
  progressRef?.setFaulted();
  progressRef?.close(3000);
}
```

See also

- UI Extensions
- Custom Pages
- Host Services
- Implementing a Schema Editor
- Filling Documents
- Editing a Document
- [charon-conversation-editor \(GitHub\)](#)

Custom Pages

A **custom page** is a page of the editor supplied by an extension. It has its own route, its own breadcrumb, and its own place in the browser history, and it is not tied to a document or a schema - which makes it the place for anything that looks at the project as a whole: a dashboard, a report, a graph of the data model, an importer with a UI of its own.



The screenshot is the charon-schema-graph example - a diagram of every schema in the project and the references between them, reached from a **Schema Graph** entry the same package added to the side navigation.

Declaring a Page

325

The Page Element

325

The Page Context

326

Persisting Page State

326

See also

326

Declaring a Page

```
"config": {
  "customPages": [
    {
      "id": "ext-schema-graph",
      "selector": "ext-schema-graph-page",
      "title": "Schema Graph",
      "breadcrumb": "Schema Graph"
    }
  ],
  "customActions": [
    {
      "name": "Schema Graph",
      "location": "side-navigation-menu",
      "pageId": "ext-schema-graph",
      "icon": "emoji/spider_web"
    }
  ]
}
```

Field	Meaning
id	Identifies the page and becomes a URL segment. Must be unique across every extension installed in the project. Required.
selector	The custom element Charon mounts at that route. Required.
title	Browser tab title, and the breadcrumb text when breadcrumb is absent. Required.
breadcrumb	Breadcrumb text, when it should differ from the title.

A page on its own is reachable only by URL. To give it a way in, pair it with a custom action that names it through pageId - most often in the side-navigation-menu, as above, but the document and collection Actions menus work the same way. The pageId must name a page in the same package.

Pages live under /ext/<package-name>/<page-id>, and everything after the page id is left for the page itself, so deep links into a page's own tabs, steps, or detail views are just longer paths.

The Page Element

Charon creates the element by its selector, sets context on it, and appends it to the working area. As with a schema editor, the property is assigned before connectedCallback, so render from whichever happens second:

```
export default class SchemaGraphPageElement extends HTMLElement {
  private _context?: ExtensionPageContext;
  private _root?: Root;
```

```

get context() { return this._context!; }
set context(value: ExtensionPageContext) {
    if (Object.is(value, this._context)) {
        return;
    }
    this._context = value;
    this.render();
}

public connectedCallback() { this.render(); }
public disconnectedCallback() { this.unmount(); }
}

customElements.define('ext-schema-graph-page', SchemaGraphPageElement);

```

Clean up in `disconnectedCallback`. A page is unmounted whenever the user navigates away, which happens far more often than for a field editor.

The Page Context

`ExtensionPageContext` carries the same services as an action context, plus what the URL said:

Field	Meaning
<code>params</code>	Query string parameters of the current URL.
<code>restOfRoute</code>	Path segments after the page id, empty when there are none.
<code>services</code>	Game data, dialogs and wizards, notifications, navigation, persisted UI state - see Host Services.

`restOfRoute` is what makes a page's internal state bookmarkable: read it on mount to restore a tab or a selection, and write it back through `services.navigation.customPage` as the user moves around.

```

context.services.navigation?.customPage(
    'charon-schema-graph', // package
    'ext-schema-graph',   // page id
    ['Item'],              // becomes /ext/charon-schema-graph/ext-schema-graph/Item
    { zoom: '1.5' }        // becomes ?zoom=1.5
);

```

The same call navigates to a page declared by *any* installed extension, not only the caller's own - which is how one extension can hand off to another. The rest of the navigation service reaches Charon's own screens: `dashboard()`, `settings(section?)`, `documentCollection(schemaName)`, `documentForm(schemaName, id)`, and `back()`.

Persisting Page State

Pages are unmounted on navigation, so anything the user arranged - a zoom level, a collapsed group, a chosen layout - is gone unless it is stored. `services.ui.state` persists state scoped to the extension package, at a layer you choose: the browser tab, the browser, the project, or the workspace, each of those either personal or shared with the team. See Host Services.

See also

- UI Extensions

- Custom Actions
- Host Services
- Implementing a Schema Editor
- [charon-schema-graph \(GitHub\)](#)

Host Services

Extension code does not talk to Charon's server or its UI directly. It is handed a set of **services** by the host, and reaches everything through them - reading and writing game data, opening dialogs and Charon's own wizards, showing notifications, navigating, and persisting UI state between sessions.

Which object carries them depends on what the extension is:

Extension kind	Where the services are
Custom action	<code>context.services</code> on the action context
Custom page	<code>context.services</code> on the page context
Schema editor	<code>documentControl.services</code>
Property editor	<code>getRootDocumentControl(valueControl).services</code>

The two sets overlap but are not identical: an action or a page gets the project-level services, a document gets the document-level ones. Both are declared `Partial` - **the host populates only what makes sense in the context, so check every field before use.**

```
const dialog = context.services.ui?.dialog;
if (!dialog) {
  return;
}
```

What Each Context Carries	327
Game Data	328
Dialogs and Wizards	329
Progress	329
Your Own Dialog	329
Charon's Wizards	329
Notifications	330
Navigation	330
Persisted UI State	330
Current User, Project, and Workspace	331
See also	331

What Each Context Carries

Service	Action / page context	Document control
<code>gameData</code>	yes	yes
<code>ui.dialog</code> / <code>ui.snackBar</code>	yes	yes
<code>ui.state</code>	scoped to the package	scoped to the document or its schema
<code>navigation</code>	yes	-

userService / projectService / workspaceService	yes	-
undoRedo	-	yes
validationProvider	-	yes
translationLanguage	-	yes
serverApiClient	yes	yes

`serverApiClient` is the raw HTTP client behind everything else. It is deliberately untyped and its surface is not stable - use it only for something the typed services do not cover, and expect it to change.

Note

`services.uiState` still exists next to `services.ui.state` and is the very same object, kept for packages written before `ui` was introduced. New code should use `ui.state`.

Game Data

`GameDataService` is the project's data, as the server sees it - not the document currently open in a form. Everything returns an observable.

Method	Purpose
<code>getMetadata()</code>	The project's schemas and settings.
<code>list(schema, parameters?)</code>	Documents of a schema, with skip, take, query, select, filters, sorters, and path for sub-documents. Returns the documents, their metadata, and the total before paging.
<code>find(schema, property, value)</code>	One document by a unique property value.
<code>query(requests)</code>	Several such lookups in one round trip.
<code>bulkChange(schema, documents, importMode, validationOptions?, dryRun?)</code>	Create, update, or delete many documents of one schema.
<code>import(collections, importMode?, schemas?, languages?, validationOptions?, dryRun?)</code>	The same across several schemas at once, taking a whole game data document.
<code>export(exportMode?, schemas?, properties?, languages?)</code>	Read data out in one of the export modes.
<code>validate(validationOptions)</code>	Run validation and get the errors per document.

`importMode` is the same choice the import wizard offers - `createAndUpdate`, `create`, `update`, `safeUpdate`, `replace`, `delete`. `exportMode` likewise: `normal`, `publication`, `localization`, `extraction`.

Tip

`dryRun: true` on `bulkChange` and `import` validates the whole change and reports what would happen without writing anything. Every result carries per-document errors, so an extension can show the user what it is about to do before it does it.

Results carry a `metadataHash` and a `revisionHash` - the state of the schemas and of the data at the time of the call. Comparing them across two calls is how an extension notices that someone else changed the project underneath it.

Dialogs and Wizards

`ui.dialog` opens modals in the host's own chrome.

Progress

`showProgress(options)` returns a handle for a long operation - `update(percent, message?, color?)`, `setFaulted()`, and `close(delayMs?)`. With `cancellable: true` the dialog can be dismissed and `closedHandler(cancelled)` fires. See Custom Actions for the full pattern.

Your Own Dialog

`showCustom(selector, data?, options?)` hosts a custom element your package registered, inside a Charon dialog:

```
const dialogRef = dialog.showCustom<MyResult>('ext-my-dialog', { itemId }, {
  title: 'Pick an item',
  width: '600px',
  disableClose: false,
});
const result = await firstValueFrom(dialogRef.afterClosed());
```

The element receives data and closes itself through `dialogRef.close(result)`; `afterClosed()` emits that result, or undefined when the dialog was dismissed.

`showCodeSnippet({ sourceCode })` is a ready-made read-only viewer with copy-to-clipboard, for showing a generated command line or a fragment of JSON.

Charon's Wizards

An extension can open the editor's own wizards rather than reimplementing them. Each returns a promise that resolves `true` when the wizard completed and `false` when it was cancelled:

Method	Opens
<code>showImportWizard(options?)</code>	Import, optionally pre-loaded with a file or text and a schema
<code>showExportWizard(options?)</code>	Export, optionally with a schema and format preselected
<code>showImportLocalizationWizard(options?)</code> / <code>showExportLocalizationWizard(options?)</code>	The translation round trip
<code>showPublicationWizard(options?)</code>	Publication
<code>showSourceCodeGenerationWizard(options?)</code>	Source code generation
<code>showBackupWizard(options?)</code> / <code>showRestoreWizard(options?)</code>	Backup and restore

All of them take `loadPreferences` and `savePreferences`, both defaulting to `true` - the wizard opens filled in the way the user last left it, and remembers what they do this time. Pass `false` when the extension is driving the wizard with its own values and should not disturb the user's saved choices.

Notifications

`ui.snackBar` is the host's transient notification strip, in matched sets of three - `loadStarted` / `loadSucceed` / `loadFailed(error)`, and the same for `save`, `reload`, and `delete`. Using them keeps an extension's feedback indistinguishable from the editor's own, including how errors are rendered.

Navigation

`navigation` moves around the editor without building URLs. Every method navigates immediately.

Method	Goes to
<code>dashboard()</code>	The project dashboard.
<code>settings(section?)</code>	Project settings - general, backup, publication, source-code, internationalization, branches, members, extensions, features.
<code>documentCollection(schemaName)</code>	A collection page.
<code>documentForm(schemaName, id)</code>	A single document form.
<code>customPage(packageName, pageId, restOfRoute?, params?)</code>	A custom page of any installed extension.
<code>errorPage(error?, retryRoute?)</code>	The generic error page.
<code>back()</code>	The previous entry in browser history.

Persisted UI State

`ui.state` stores small pieces of state - a zoom level, a collapsed panel, a chosen layout - and gives them back the next time the extension runs. On an action or page context it is scoped to the extension package; on a document control it takes a scope first, `'document'` or `'schema'`.

```
state.save('projectPersonal', { zoom, collapsed });
const restored = state.load('projectPersonal');
```

The layer decides where the value lives and how far it travels. On load, the requested layer is checked first and lower-priority layers after it, so a personal value shadows a shared one:

Layer	Stored in
<code>browserSession</code>	Session storage - gone when the tab closes.
<code>browserPersonal</code>	Local storage, this user.
<code>browserShared</code>	Local storage, everyone using this browser.
<code>projectPersonal</code>	The server, this user, this project.
<code>projectTeam</code>	The server, everyone in the project.
<code>workspacePersonal</code>	The server, this user, across the workspace.
<code>workspaceTeam</code>	The server, everyone in the workspace.
<code>default</code>	Fallback, lowest priority.

Use a browser* layer for view state nobody else cares about, and a *Team layer for something the whole project should share - a canvas layout that everyone should see the same way, for instance. save with null clears the value.

Current User, Project, and Workspace

userService.currentUser\$, projectService.currentProject\$, projectService.currentBranch\$, and workspaceService.currentWorkspace\$ are observables of who and where. The project carries its branches, and the branch says whether it is the primary one - enough for an extension to warn before writing to production data, or to label what it produces.

See also

- UI Extensions
- Custom Actions
- Custom Pages
- Implementing a Schema Editor
- Implementing a Property Editor
- Importing and Exporting Data
- Validation

Creating a Custom Editor with React

Charon is a flexible data modeling web application that supports user-defined custom editors for fields and documents. By leveraging **Web Components**, Charon allows developers to seamlessly integrate reusable UI components that can be packaged and distributed via NPM.

This guide walks you through the process of preparing and packaging a Charon-compatible extension. It assumes proficiency with React and modern frontend tooling.

Introduction to Web Components

Web Components are a set of standardized browser APIs that allow developers to create reusable and encapsulated HTML elements. Core technologies include:

- **Custom Elements:** Define new HTML tags using JavaScript.
- **Shadow DOM:** Encapsulate markup and styles.
- **HTML Templates:** Define reusable markup structures.

Web Components are framework-agnostic, making them ideal for embedding in applications like Charon.

Charon loads user-defined Web Components dynamically at runtime. Each component is registered via a known selector and associated with supported data types and usage contexts (e.g., form property, grid column).

Step-by-Step: Building a Logical Toggle Editor

This guide uses a custom logical toggle field editor as an example. You can [download and try](#) the example code referenced in this article.

1. Create a New React Project

```
npm create vite
> Project name: charon-logical-toggle
> Select a framework: React
> Select a variant: TypeScript
```

```
cd charon-logical-toggle
npm install
```

2. Install Required Dependencies

Install SASS, RxJS (recommended), the Web Components polyfill (for broader compatibility), and the Charon extension types package.

```
npm install sass --save
npm install rxjs --save
npm install @webcomponents/custom-elements --save
npm install charon-extensions --save
```

3. Write and Export Your Web Component

Follow a recommended guide for wrapping React components as Web Components:

- [Create a Web Component from a React Component](#)
- [LogicalToggleElement.tsx \(Web Component\)](#)
- [LogicalToggle.tsx \(React Component\)](#)

Key points when writing your component:

- Implement the `CharonPropertyEditorElement` interface in your Web Component to accept input properties from the main application.
- Ensure you correctly subscribe and unsubscribe when `valueControl` or `documentControl` input values change to avoid memory leaks.
- Export your Web Component using a unique tag name (e.g., `ext-logical-toggle-editor`).
- Your `main.js` script should only register the Web Component-it should not render anything directly.

4. Modify vite.config.ts for Packaging

Update the `vite.config.ts` file to control file naming and minimize output hashing:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  build: {
    rollupOptions: {
      output: {
        assetFileNames: 'assets/[name].[ext]',
        chunkFileNames: '[name].js',
        entryFileNames: '[name].js'
      }
    },
    minify: false // Disable minification during development
  }
})
```

5. Update package.json for Packaging

Ensure your package.json includes the following:

- A non-default version number (e.g., 1.0.0).
- A main field pointing to your generated JS bundle (e.g., index.js).
- A build script for bundling and packaging:

```
"scripts": {
  "build": "tsc -b && vite build && copy package.json dist\\package.json && cd dist && n"
}
```

- Include your stylesheet(s) in the files array:

```
"files": ["assets/index.css"]
```

- Declare the Charon extension metadata using the config.customEditors section:

```
"config": {
  "customEditors": [
    {
      "id": "ext-logical-toggle",
      "selector": "ext-logical-toggle-editor",
      "name": "Logical Toggle",
      "type": ["Property", "Grid"],
      "dataTypes": ["Logical"]
    }
  ]
}
```

- Add the Charon extension schema reference:

```
"$schema": "https://raw.githubusercontent.com/gamedevware/charon-extensions/refs/heads/main"

```

- Remove the "private": true flag before publishing.
- Fill in metadata fields like author, description, and license.

Debugging & Local Testing

Test Locally in Charon

Place the generated .tgz file in:

Windows

```
%PROGRAMDATA%\Charon\extensions\
C:\ProgramData\Charon\extensions\
```

MacOS

```
/Users/<username>/Library/Charon/extensions/
```

Linux

```
/home/<username>/Library/Charon/extensions/
```

Unity

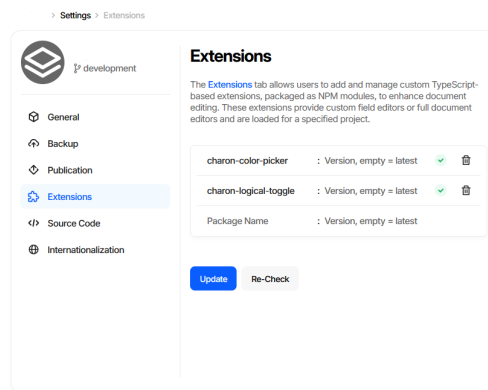
```
<project-directory>/Library/Charon/extensions/
```

Unreal Engine

```
<project-directory>/Intermediate/Charon/extensions/
```

This allows Charon to detect and load the extension during development, bypassing the NPM publication process.

Is It Free?



Disable Optimizations for Browser Debugging

During debugging, set the following in `vite.config.ts`:

```
build: {  
  minify: false  
}
```

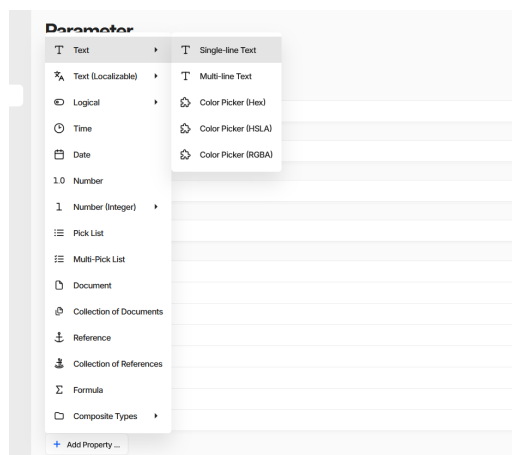
This improves source map quality and simplifies runtime behavior.

Versioning

Always increment the `version` field in `package.json` between builds to ensure Charon detects updates.

Publishing

Once your component is built, tested, and versioned, you can publish it to NPM and reference it in Charon's Project Settings → Extensions tab. Your custom editor will then be available in the schema designer for relevant property types.



See also

- [UI Extensions Overview](#)
- [Implementing Property Editor Interface](#)
- [Creating a Custom Editor with Angular](#)
- [charon-extensions package \(NPM\)](#)
- [Example Projects \(GitHub\)](#)
- [Web Components \(MDN\)](#)

Creating a Custom Editor with Angular

Charon is a flexible data modeling web application that supports user-defined custom editors for fields and documents. By leveraging **Web Components**, Charon allows developers to seamlessly integrate reusable UI components that can be packaged and distributed via NPM.

This guide walks you through the process of preparing and packaging a Charon-compatible extension. It assumes proficiency with Angular and modern frontend tooling.

Introduction to Web Components

Web Components are a set of standardized browser APIs that allow developers to create reusable and encapsulated HTML elements. Core technologies include:

- **Custom Elements:** Define new HTML tags using JavaScript.
- **Shadow DOM:** Encapsulate markup and styles.
- **HTML Templates:** Define reusable markup structures.

Web Components are framework-agnostic, making them ideal for embedding in applications like Charon.

Charon loads user-defined Web Components dynamically at runtime. Each component is registered via a known selector and associated with supported data types and usage contexts (e.g., form property, grid column).

Step-by-Step: Building a Color Picker Editor

This guide uses a custom color picker field editor as an example. You can [download and try](#) the example code referenced in this article.

1. Install the Latest Angular CLI

```
npm uninstall -g @angular/cli
npm install -g @angular/cli@latest
```

2. Create a New Angular Project

```
ng new charon-color-picker --standalone false --ssr false --style sass
```

Note

Ensure SSR is disabled and SCSS is used for styling.

3. Install Required Dependencies

Install Angular Elements for Web Component support, the Web Components polyfill (for broader compatibility), and the Charon extension types package.

```
npm install @angular/elements --save
npm install @webcomponents/custom-elements --save
npm install charon-extensions --save
```

4. Write and Export Your Web Component

Follow the official Angular guide to export your component as a custom element:

- [Angular Elements Guide](#)
- [color.picker.component.ts](#)

Key points when writing your component:

- Implement the `CharonPropertyEditorElement` interface in your Angular component to accept input properties from the main application.
- Ensure you correctly subscribe and unsubscribe when `valueControl` or `documentControl` input values change, to avoid memory leaks.
- Export your component with a unique tag name (e.g., `ext-color-picker-editor`) that is **not the same** as the Angular component's selector.

5. Modify `angular.json` for Packaging

Update the `angular.json` file:

- Set the build output path:
`"outputPath": "dist"`
- Disable output hashing:
`"outputHashing": "none"`
- Change the project type to `library`:
`"projectType": "library"`
- Optionally disable production optimizations during development:
`"configurations": {
 "production": {
 "optimizations": false
 }
}`

6. Update `package.json` for Packaging

Ensure your `package.json` includes the following:

- A non-default version number (e.g., `1.0.0`).
- A `main` field pointing to your generated JS bundle (e.g., `main.js`).
- A build script for bundling and packaging:
`"scripts": {
 "build": "ng build && copy package.json dist\\browser\\package.json && cd dist\\browser"`
- Include your stylesheet(s) in the `files` array:
`"files": ["styles.css"]`
- Declare the Charon extension metadata using the `config.customEditors` section:
`"config": {
 "customEditors": [
 {
 "id": "ext-color-picker",
 "selector": "ext-color-picker-editor",
 "name": "Color Picker",
 "type": ["Property", "Grid"],
 "dataTypes": ["Text", "Integer"]
 }
]
}`
- Add the Charon extension schema reference:
`"$schema": "https://raw.githubusercontent.com/gamedevware/charon-extensions/refs/heads/main"`
- Remove the `"private": true` flag before publishing.

Is It Free?

- Fill in other metadata: author, description, license, etc.

7. Enable Standalone/Zoneless Mode (Performance)

Ensure your component runs independently of Angular's *zone.js*. This improves performance and prevents interference between extensions. Refer to Angular documentation for zoneless application setup.

Debugging & Local Testing

Test Locally in Charon

Place the generated .tgz file in:

Windows

```
%PROGRAMDATA%\Charon\extensions\  
C:\ProgramData\Charon\extensions\
```

MacOS

```
/Users/<username>/.config/Charon/extensions/
```

Linux

```
/home/<username>/.config/Charon/extensions/
```

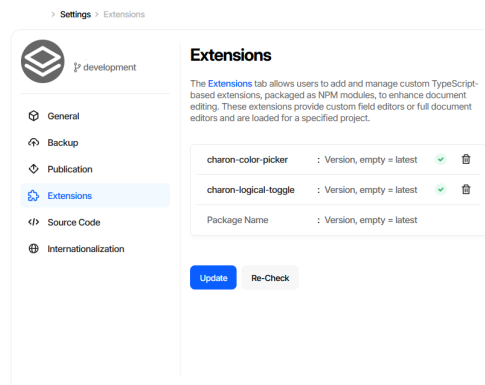
Unity

```
<project-directory>/Library/Charon/extensions/
```

Unreal Engine

```
<project-directory>/Intermediate/Charon/extensions/
```

This allows Charon to detect and load the extension during development, bypassing the NPM publication process.



Disable Optimizations for Browser Debugging

During debugging, set:

```
"optimizations": false
```

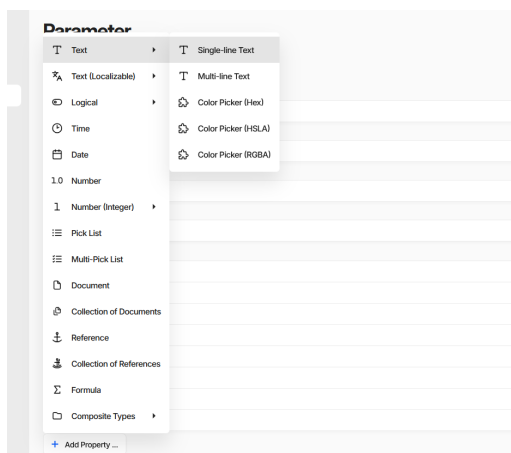
in the production build configuration to simplify source maps and runtime behavior.

Versioning

Always increment the version field in package.json between builds to ensure Charon detects updates.

Publishing

Once your component is built, tested, and versioned, you can publish it to NPM and reference it in Charon's Project Settings → Extensions tab. Your custom editor will then be available in the schema designer for relevant property types.



See also

- [UI Extensions Overview](#)
- [Implementing Property Editor Interface](#)
- [Creating a Custom Editor with React](#)
- [charon-extensions package \(NPM\)](#)
- [Example Projects \(GitHub\)](#)
- [Web Components \(MDN\)](#)

Troubleshooting Common Issues

This page collects the most frequently reported problems and their solutions. Each section starts with the symptom or the exact error message, followed by the cause and the fix.

If your problem is not listed here, check the log files for errors and see [Nothing Helped?](#) below for support channels.

Editor Fails to Launch or Gets Stuck Loading	339
Web UI Shows “Http failure response for api/v1/context/: 0 Unknown Error”	339
Blank Page or Extension Fails to Load	339
Game Data File Is Corrupted After Version Control Operations	340
Validation Errors on System Schemas (Schema, ProjectSettings, SchemaProperty)	340
“.NET SDK is not installed” During Setup	340
Exported XLSX File Contains an “Errors” Sheet	341
Game Data Is Read-Only in the Web Editor	341
Unreal Engine Compile Errors After an Engine Upgrade	341
Import Regenerates IDs of Nested Documents	341
Nothing Helped?	341
See also	342

Editor Fails to Launch or Gets Stuck Loading

Symptom: The editor window (or browser tab) never finishes loading, or the launch fails with a timeout error. Sometimes a retry succeeds.

Common causes and fixes, in order of likelihood:

1. **Large game data file exceeds the launch timeout.** The plugin waits a limited time (30 seconds by default) for the editor process to load the game data. For large files, increase the *Idle Close Timeout* in the Unity project settings: **Edit** → **Project Settings** → **Charon**.
2. **VPN or proxy software interferes with local connections.** The editor is a local web application served on localhost. Some VPN clients and system-wide proxies intercept or block this traffic. Temporarily disable them and retry.
3. **A stale editor process is holding the port.** Restart the Unity/Unreal editor. If that does not help, restart the computer to clear orphaned processes.
4. **Unreal Engine plugin: the randomly chosen port is unavailable.** The editor picks a random local port on each launch, and sometimes that port is already taken or blocked by OS permissions. Simply close and reopen the Charon window - a new port is picked on the next launch. Check the *Output Log* console for error messages if the problem persists.

If the problem persists, collect the log files and repeat the launch with verbose logging enabled.

Web UI Shows “Http failure response for api/v1/context/: 0 Unknown Error”

Symptom: The editor page opens but fails with the error above, while the same project may work in another git branch or after a reboot.

This indicates that the browser could not reach the local editor process at all. Try in order:

1. Hard-refresh the page and clear the cache: Ctrl+F5 or Ctrl+Shift+R.
2. Disable browser extensions (ad blockers and privacy tools are frequent offenders).
3. Disable VPN/proxy software.
4. Restart the Unity/Unreal editor, then the computer.

If the error reoccurs, save the log files from the failing session and send them to support - this class of problem is hard to reproduce and logs are essential.

Blank Page or Extension Fails to Load

Symptom: The editor opens as a blank page, or an installed extension (custom document editor, property editor) does not appear as an option in the UI.

1. Hard-refresh and clear the browser cache: Ctrl+F5 or Ctrl+Shift+R.
 2. Open the browser developer console (F12) and check for errors. A crashed or blocking browser extension is often visible at the top of the console output.
 3. Try a private/incognito window or another browser to rule out browser extensions.
 4. For Charon extensions: open **Project Settings** → **Extensions** and verify the package downloaded successfully (hover over the status icon, click *Recheck* if needed).
 5. Verify your browser is supported: **Chrome** released 2021 or later, **Firefox** released 2023 or later. Other Chromium-based browsers (Edge, Opera, Brave...) should be a recent version.
-

Game Data File Is Corrupted After Version Control Operations

Symptom: After a merge, rebase, or branch switch, Charon reports that the game data file is corrupted or is not valid JSON.

The file most likely contains unresolved merge conflict markers (<<<<<<, =====, >>>>>>). Charon detects **Git**, **Mercurial**, and **SVN** working copies and reloads the game data on checkouts, but it does not detect merge conflicts inside the file - a file with conflict markers is simply no longer valid JSON.

Fix:

- For **JSON** game data: resolve the conflict with your usual text merge tools, the same way as for source code.
- For **binary** formats (MessagePack): merging is not possible. Choose the correct side of the conflict entirely (ours or theirs) and re-apply the other side's changes in the editor.

Note

JSON is the default storage format precisely because it can be diffed and merged in version control. Keep binary formats for publication output rather than for the editable source file.

Validation Errors on System Schemas (Schema, ProjectSettings, SchemaProperty)

Symptom: Every schema and property in the project shows validation errors such as an invalid identifier, including freshly created schemas.

Charon stores the metadata that describes metadata (the Schema, ProjectSettings, and SchemaProperty system schemas) in the same file as your data. Hand-editing the game data file can accidentally damage them, which breaks validation for everything defined on top.

Warning

Never modify these system documents when editing the game data file manually:

- Schema - id 18d4bf318f3c49688087dbed
- ProjectSettings - id 55a4f32faca22e191098f3d9
- SchemaProperty - id d40d8eb1a23248b69b252eea

Fix: Create a new, empty game data file (e.g. via your engine plugin or `dnx dotnet-charon -- INIT`), open both files in a text editor, and copy the three system schema documents from the fresh file into your damaged one, replacing the broken versions. Make a backup first.

“.NET SDK is not installed” During Setup

Symptom: The engine plugin fails to set up the game data editor and complains about a missing .NET installation.

Charon is a .NET tool and requires the **.NET SDK 10 or later** (SDK 8 is the minimum supported, but it does not provide dnx). Download it from dotnet.microsoft.com and verify:

```
dotnet --list-sdks
```

After installing the SDK, restart the Unity/Unreal editor. If the plugin still does not detect it, restart the computer so the updated PATH is picked up.

Exported XLSX File Contains an “Errors” Sheet

Symptom: A DATA EXPORT or localization export produces a spreadsheet that looks empty except for a sheet named *Errors*.

The export failed and the error details were written into that sheet instead of the console. Open the *Errors* sheet and read the message - it typically points to the exact document, property, and reason.

Game Data Is Read-Only in the Web Editor

Symptom: The online editor shows the project as read-only and design changes are rejected, with a message about exceeded limits.

Your workspace has crossed one of its subscription limits. Go to **Workspace Settings** → **Subscriptions** and check which limit is exceeded - *Projects*, *Members*, or *Size*. Remove the excess or upgrade the plan; the read-only message disappears once the workspace is back within limits.

Unreal Engine Compile Errors After an Engine Upgrade

Symptom: After upgrading Unreal Engine, previously generated C++ code or the Charon plugin fails to compile.

New Unreal Engine releases regularly change engine APIs, so support for a new UE version ships in a Charon plugin update. The safe upgrade routine is:

1. **Wait** until the new UE version is officially supported (see the release #announcements channel on Discord or <https://www.gamedevware.com/whats-new>).
 2. **Regenerate** the C++ source code while still on your current engine version - generated code is backward compatible.
 3. **Upgrade** the engine and the Charon plugin together, then rebuild.
-

Import Regenerates IDs of Nested Documents

Symptom: After importing data (e.g. a localization round-trip), embedded documents receive new identifiers, which shows up as large diffs in version control.

The default import mode replaces embedded collections, recreating their documents. To update existing documents in place:

- Pass `--mode safeUpdate` to DATA IMPORT - it prevents creation and deletion of nested documents.
 - Or use DATA I18N IMPORT for localization round-trips - it is a specialized import that only touches translated text.
-

Nothing Helped?

If none of the above solves your problem, reach out directly:

- **Discord** - join the [Charon Discord server](#) and ask in the #issues channel. This is usually the fastest way to get help.
- **GitHub** - file a work item at github.com/gamedevware/charon/issues.
- **Email** - write to support@gamedevware.com.

Whichever channel you choose, attach the log files and mention your Charon version, engine (Unity/Unreal/standalone/web), and operating system - it saves a round-trip.

See also

- Working with Logs
- Reset UI Preferences
- Merge Logic
- Command Line Reference

Working with Logs

Charon creates a log files with various messages that may be useful for troubleshooting and debugging.

Unity Plugin

Log files are saved to <project-directory>/Library/Charon/logs/.

Unreal Engine Plugin

Log files are saved to <project-directory>/Intermediate/Charon/logs/.

CLI and Standalone

Log files are saved to:

- Windows: C:/ProgramData/Charon/logs/.
- MacOS: /Users/<username>/.config/Charon/logs
- Linux: /home/<username>/.config/Charon/logs

Note: Make sure to replace <project-directory> and <charon-directory> with the actual directories on your system.

Logging Levels

Normally only the most important events are logged. If you have trouble identifying an issue, you might want to change log to *verbose*. This way more information is included in logs.

Unity Plugin

In menu select Tools → Charon → Troubleshooting → Verbose Logs.

CLI and Standalone

Launch with `--verbose` parameter.

Then repeat the action that causes the bug (or the one you want analyzed anyway) and check log file again.

CLI Example:

```
dnx dotnet-charon -- SERVER START ./gamedata.json --launchDefaultBrowser --verbose
```

See also

- Reset UI Preferences
- Command Line Reference

Resetting UI Preferences

If for some reason editor behaves erratically (grids aren't displayed correctly or aren't displayed at all), you can restore default UI settings.

Unity plugin

Select in menu Tools → Charon → Troubleshooting → Reset Preferences.

CLI and Standalone

Launch with `--resetPreferences` parameter.

Web

Use the Preferences profile tab <User Icon> → Profile → Preferences.

CLI Example:

```
dnx dotnet-charon -- SERVER START ./gamedata.json --launchDefaultBrowser --resetPreferences
```

See also

- Working with Logs
- Command Line Reference

Apply Patch (Merge)

Applies patch created with DATA CREATEPATCH command to a game data.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA APPLYPATCH \
  --dataBase "c:\my app\gamedata.json" \
  --input "c:\my app\gamedata_patch.json" \
  --inputFormat json

# remote game data
dnx dotnet-charon -- DATA APPLYPATCH \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --input "./gamedata_patch.json" \
  --inputFormat json \
  --credentials "<API-Key>"
```

Parameters

<code>--dataBase</code>	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json"</pre>
	<pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
<code>--credentials</code>	The API key used to access remote server in case of <code>--dataBase</code> being URL.

--input

Path to a file with patch to apply. Alternatively, you can use [Standard Input](#) or *URL <remote_input_output>*.

```
# standard input (default)
--input in
--input con

# absolute path (windows)
--input "c:\my_app\gamedata_patch.json"

# absolute path (unix)
--input "/user/data/gamedata_patch.json"

# relative path (universal)
--input "../gamedata_patch.json"

# remote location (HTTP)
--input "http://example.com/gamedata_patch.json"

# remote location with authentication (FTP)
--input "ftp://user:password@example.com/gamedata_patch.json"
```

--inputFormat

Format of imported data.

```
# Auto-detect by extension (default)
--inputFormat auto

# JSON
--inputFormat json

# BSON
--inputFormat bson

# Message Pack
--inputFormat msgpack

# XML (removed in 2025.1.1)
--inputFormat xml
```

--inputFormattingOptions

Additional options for specified format.

This command supports universal parameters.

See also

- Patch and Diff Workflow
- DATA CREATEPATCH
- Merge Logic
- CI/CD Integration

Create Backup

Backs up game data to a specified file. Saved data could be later used with DATA RESTORE command. Also this command can be used to convert game data into different format.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA BACKUP \
  --dataBase "c:\my app\gamedata.json" \
  --output "c:\my app\backup.msgpkg" \
  --outputFormat msgpack

# remote game data
dnx dotnet-charon -- DATA BACKUP \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --output "./backup.msgpkg" \
  --outputFormat msgpack \
  --credentials "<API-Key>"
```

Parameters

<code>--dataBase</code>	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
<code>--credentials</code>	The API key used to access remote server in case of <code>--dataBase</code> being URL.
<code>--output</code>	Path to a backup file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to Standard Error, Standard Output, /dev/null, or a URL. <pre># standard output (default) --output out --output con # standard error --output err # null device --output null # absolute path (windows) --output "c:\my app\backup.json" # absolute path (unix) --output "/user/data/backup.json" # relative path (universal) --output "./backup.json" # remote location (HTTP) --output "http://example.com/backup.json" # remote location with authentication (FTP) --output "ftp://user:password@example.com/backup.json"</pre>

--outputFormat

Format of backed up data.

```
# JSON (default)
```

```
--outputFormat json
```

```
# Message Pack
```

```
--outputFormat msgpack
```

--outputFormattingOptions

Additional options for specified format.

This command supports universal parameters.

Output

The back up data follows the general game data structure.

See also

- Backup and Restore
- DATA RESTORE
- DATA EXPORT
- CI/CD Integration

Create Document

Creates a new document. For a bulk creations use DATA IMPORT command with --mode create. Only the first document from the --input will be processed.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA CREATE \
  --dataBase "c:\my app\gamedata.json" \
  --schema Item \
  --input "c:\my app\item.json" \
  --inputFormat json

# remote game data
dnx dotnet-charon -- DATA CREATE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schema Item \
  --input "./item.json" \
  --inputFormat json \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <i># local file</i> --dataBase "c:\my app\gamedata.json" <i># remote server</i> --dataBase "https://charon.live/view/data/My_Game/develop/"
--credentials	The API key used to access remote server in case of --dataBase being URL.
--schema	Name or identifier of the type (schema) of the new document. <i># name</i> --schema Item <i># id</i> --schema 55a4f32faca22e191098f3d9
--validationOptions	List of validation checks and repairs to perform during document creation. <i># no checks</i> --validationOptions none <i># repairs</i> --validationOptions repair --validationOptions repair deduplicateIds --validationOptions repair repairRequiredWithDefaultValue --validationOptions repair eraseInvalidValue <i># checks (default)</i> --validationOptions checkTranslation --validationOptions checkRequirements --validationOptions checkFormat --validationOptions checkUniqueness --validationOptions checkReferences --validationOptions checkSpecification --validationOptions checkConstraints
--input	Path to a file with document. Alternatively, you can use Standard Input or URL. <i># standard input (default)</i> --input in --input con <i># absolute path (windows)</i> --input "c:\my app\item.json" <i># absolute path (unix)</i> --input "/user/data/item.json" <i># relative path (universal)</i> --input "./item.json" <i># remote location (HTTP)</i> --input "http://example.com/item.json" <i># remote location with authentication (FTP)</i> --input "ftp://user:password@example.com/item.json"

`--inputFormat`

Format of imported data.

Auto-detect by extension (default)`--inputFormat auto`*# JSON*`--inputFormat json`*# BSON*`--inputFormat bson`*# Message Pack*`--inputFormat msgpack`*# XML (removed in 2025.1.1)*`--inputFormat xml``--inputFormattingOptions`

Additional options for specified format.

`--output`

Path to a created document file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), `/dev/null`, or a URL.

standard output`--output out``--output con`*# standard error*`--output err`*# null device (default)*`--output null`*# absolute path (windows)*`--output "c:\my app\created_item.json"`*# absolute path (unix)*`--output /user/data/created_item.json`*# relative path (universal)*`--output "./created_item.json"`*# remote location (HTTP)*`--output "http://example.com/created_item.json"`*# remote location with authentication (FTP)*`--output "ftp://user:password@example.com/created_item.json"``--outputFormat`

Format of created data.

JSON (default)`--outputFormat json`*# BSON*`--outputFormat bson`*# Message Pack*`--outputFormat msgpack`*# XML (removed in 2025.1.1)*`--outputFormat xml``--outputFormattingOptions`

Additional options for specified format.

This command supports universal parameters.

Input Data Schema

The data you input should follow this schema (recommended):

```
{
  "Collections": {
    "<Schema-Name>": [
      {
        // <Document>
      }
    ]
  }
}
```

This schema is also accepted:

```
{
  "<Schema-Name>": [
    {
      // <Document>
    }
  ]
}
```

A list of documents is accepted:

```
[
  {
    // <Document>
  }
]
```

And single document too:

```
{
  // <Document>
}
```

Output

Outputs the created document with all the edits that were made to make it conform to the schema.

```
{
  "Id": "Sword"

  /* rest of properties of created document */
}
```

See also

- DATA IMPORT
- DATA UPDATE
- DATA DELETE
- Importing and Exporting Data

Create Patch (Diff)

Outputs the differences between two game datas as a file that can be used later to DATA APPLYPATCH to another game data.

- CLI Installation

- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA CREATEPATCH \
  --dataBase1 "c:\my app\gamedata_base.json" \
  --dataBase2 "c:\my app\gamedata_modified.json" \
  --output "./gamedata_patch.json" \
  --outputFormat json

# remote game data
dnx dotnet-charon -- DATA CREATEPATCH \
  --dataBase1 "https://charon.live/view/data/My_Game/develop/" \
  --dataBase2 "https://charon.live/view/data/My_Game/staging/" \
  --output "./gamedata_staging_patch.json" \
  --outputFormat json \
  --credentials "<API-Key>"
```

Parameters

--dataBase1	Absolute or relative path to a first game data. Use quotation marks if your path contains spaces. <pre># local file --dataBase1 "c:\my app\gamedata.json"</pre> <pre># remote server --dataBase1 "https://charon.live/view/data/My_Game/develop/"</pre>
--dataBase2	Absolute or relative path to a second game data. Use quotation marks if your path contains spaces. <pre># local file --dataBase2 "c:\my app\gamedata.json"</pre> <pre># remote server --dataBase2 "https://charon.live/view/data/My_Game/develop/"</pre>

--output

Path to a patch file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

```
# standard output (default)
```

```
--output out
```

```
--output con
```

```
# standard error
```

```
--output err
```

```
# null device
```

```
--output null
```

```
# absolute path (windows)
```

```
--output "c:\my app\gamedata_patch.json"
```

```
# absolute path (unix)
```

```
--output /user/data/gamedata_patch.json
```

```
# relative path (universal)
```

```
--output "./gamedata_patch.json"
```

```
# remote location (HTTP)
```

```
--output "http://example.com/gamedata_patch.json"
```

```
# remote location with authentication (FTP)
```

```
--output "ftp://user:password@example.com/gamedata_patch.json"
```

--outputFormat

Format of exported data.

```
# JSON (default)
```

```
--outputFormat json
```

```
# BSON
```

```
--outputFormat bson
```

```
# Message Pack
```

```
--outputFormat msgpack
```

```
# XML (removed in 2025.1.1)
```

```
--outputFormat xml
```

--outputFormattingOptions

Additional options for specified format.

--credentials

This parameter sets the API key used to access *BOTH* remote servers. If this is not suitable, consider downloading the data locally and running this command on local files instead.

This command supports universal parameters.

See also

- Patch and Diff Workflow
- DATA APPLYPATCH
- DATA BACKUP
- CI/CD Integration

Delete Document

Deletes a document. For a bulk deletion use DATA IMPORT command with `--mode delete`.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA DELETE \
  --dataBase "c:\my app\gamedata.json" \
  --schema Item \
  --id "Sword"

# remote game data
dnx dotnet-charon -- DATA DELETE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schema Item \
  --id "Sword" \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of <code>--dataBase</code> being URL.
--schema	Name or identifier of the type (schema) of deleting document.
	<pre># name --schema Item # id --schema 55a4f32faca22e191098f3d9</pre>
--id	Identifier of deleting document.
	<pre># text --id Sword # number --id 101</pre>

`--output`

The path to a file where the *deleted document* should be placed. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

```
# standard output
--output out
--output con

# standard error
--output err

# null device (default)
--output null

# absolute path (windows)
--output "c:\my app\deleted_item.json"

# absolute path (unix)
--output /user/data/deleted_item.json

# relative path (universal)
--output "../deleted_item.json"

# remote location (HTTP)
--output "http://example.com/deleted_item.json"

# remote location with authentication (FTP)
--output "ftp://user:password@example.com/deleted_item.json"
```

`--outputFormat`

Format for deleted document.

```
# JSON (default)
--outputFormat json

# BSON
--outputFormat bson

# Message Pack
--outputFormat msgpack

# XML (removed in 2025.1.1)
--outputFormat xml
```

`--outputFormattingOptions`

Additional options for specified format.

This command supports universal parameters.

Output

Outputs the deleted document in its state at the time of deletion.

```
{
  "Id": "Sword"

  /* rest of properties of deleted document */
}
```

See also

- DATA IMPORT
- DATA CREATE
- DATA UPDATE

- DATA FIND

Examine Game Data

Added in version 2026.4.0.

Returns statistics about game data: document and validation error counts per schema, metadata hash, and revision hash.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA EXAMINE \
  --dataBase "c:\my app\gamedata.json"

# remote game data
dnx dotnet-charon -- DATA EXAMINE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --credentials "<API-Key>"
```

Use this command to verify that a file is valid game data and get a content overview - document counts per schema, a metadata hash, and a revision hash - before running a heavier command such as DATA LIST or DATA SEARCH.

Examine runs a full validation pass over the data and reports how many errors of each kind every collection has. This is the same pass that produces the error counts shown in the editor's document lists and in the publication review, and it uses the same profile: all integrity checks plus checkTranslations. Use DATA VALIDATE instead when you need the individual errors rather than the counts, or when you need to choose which checks run.

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of --dataBase being URL.

--output

Path to the statistics output file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

```
# standard output (default)
```

```
--output out
```

```
--output con
```

```
# standard error
```

```
--output err
```

```
# null device
```

```
--output null
```

```
# absolute path (windows)
```

```
--output "c:\my app\stats.json"
```

```
# absolute path (unix)
```

```
--output /user/data/stats.json
```

```
# relative path (universal)
```

```
--output "./stats.json"
```

```
# remote location (HTTP)
```

```
--output "http://example.com/stats.json"
```

```
# remote location with authentication (FTP)
```

```
--output "ftp://user:password@example.com/stats.json"
```

--outputFormat

Format for serializing the output.

```
# JSON (default)
```

```
--outputFormat json
```

```
# BSON
```

```
--outputFormat bson
```

```
# Message Pack
```

```
--outputFormat msgpack
```

--outputFormattingOptions

Additional formatting options for the chosen output format.

This command supports universal parameters.

Output

```
{
  "metadataHash": "3f1c2a9e7b0d4c5e8a1f6d2b",
  "revisionHash": "678f998993a22d1f54b7fa80",
  "toolsVersion": "2025.1.1.0",
  "changeNumber": 42,
  "collections":
  [
    {
      "schemaName": "Character",
      "schemaId": "592fc86c983a36266c0912b1",
      "documentCount": 128,
      "rootDocumentCount": 128,
      "validationErrorCount": {}
    },
    {
      "schemaName": "Item",
      "schemaId": "592fc86c983a36266c0912a0",
      "documentCount": 340,
      "rootDocumentCount": 210,
      "validationErrorCount": {}
    }
  ]
  // ...
}
```

documentCount includes documents embedded inside other documents; rootDocumentCount counts only top-level documents in the collection. validationErrorCount maps validation error codes to their occurrence count and is empty when no errors were found during the examine pass.

See also

- DATA FINDFILES
- DATA VALIDATE
- DATA LIST

Export Data

Exports documents into a file.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA EXPORT \
  --dataBase "c:\my app\gamedata.json" \
  --schemas Character \
  --output "c:\my app\characters.json" \
  --outputFormat json

# remote game data
dnx dotnet-charon -- DATA EXPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schemas Character \
  --output "./characters.json" \
  --outputFormat json \
  --credentials "<API-Key>"
```

Parameters

- dataBase Absolute or relative path to game data. Use quotation marks if your path contains spaces.
- ```
local file
--dataBase "c:\my app\gamedata.json"

remote server
--dataBase "https://charon.live/view/data/My_Game/develop/"
```
- credentials The API key used to access remote server in case of --dataBase being URL.
- schemas A list of types of documents (schemas) to export. By default all schemas *EXCEPT* metadata are exported.
- Use space to separate multiple schemas.
  - You can use wildcards (\*) at the beginning and end of names.
  - You can use identifiers in {} instead of names.
  - You can exclude certain names by using an exclamation mark (!) at the beginning of their names.
- ```
# schema name
--schemas Character
--schemas Character Item

# all (default)
--schemas *

# masks
--schemas Char*
--schemas *Modifier
--schemas *Mod*

# schema id
--schemas {18d4bf318f3c49688087dbed}

# negation
--schemas Char* !Character
--schemas !*Item*

# excluding system schemas (Schema, SchemaProperty, ProjectSettings)
--schemas ![system]
```

--properties

A list of properties or property types to export. By default all properties are exported.

- *Id* property always included
- Use space to separate multiple properties.
- You can use wildcards (*) at the beginning and end of names.
- You can use identifiers in {} instead of names.
- You can exclude certain names by using an exclamation mark (!) at the beginning of their names.
- You can use data type in [] instead of names.

--languages

List of languages to keep in exported data. Language's [english name](#) is used or [language tag \(BCP 47\)](#).

Use DATA I18N LANGUAGES to get list of used languages.

- Use space to separate multiple languages
- You can use wildcards (*) at the beginning and end of names.
- You can use LCID or [CultureInfo.Name](#) in {} instead of the name.
- You can exclude certain names by using an exclamation mark (!) at the beginning of their names.

language tag (BCP 47)

--languages {en-US}

language name

--languages "Spanish (Spain)"

language name mask

--languages Spanish*

language LCID

--languages {3082}

negation and masks

--languages !Spanish*

--languages Spanish* !{es-Es}

- -mode

Export mode controls stripping and inclusion rules for exported data.

```
# (default)
```

```
--mode normal
```

```
--mode publication
```

```
--mode extraction
```

```
--mode localization
```

normal

Export all specified documents defined in *--schemas*. This mode ensures that the exported graph of documents remains valid by including any necessary additional documents to avoid any broken references.

publication

Same as *--mode normal*, but shaped for the game: localization for languages outside *--languages* is stripped, and the file level `ToolsVersion`, `RevisionHash` and `ChangeNumber` fields are written into the output. System schemas and the project settings document are kept - the generated code requires them and refuses to load a file without them.

extraction

Export only the specified *--schemas* without exporting any referenced documents. In this mode, the exported graph of documents may contain broken references. It is recommended to use the import *--mode safeupdate* when importing this data back.

localization

Same as *--mode extraction* but only `LocalizedText` properties are exported.

`--output`

Path to a exported data file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), `/dev/null`, or a URL.

```
# standard output (default)
```

```
--output out
```

```
--output con
```

```
# standard error
```

```
--output err
```

```
# null device
```

```
--output null
```

```
# absolute path (windows)
```

```
--output "c:\my app\document.json"
```

```
# absolute path (unix)
```

```
--output /user/data/document.json
```

```
# relative path (universal)
```

```
--output "./document.json"
```

```
# remote location (HTTP)
```

```
--output "http://example.com/document.json"
```

```
# remote location with authentication (FTP)
```

```
--output "ftp://user:password@example.com/document.json"
```

`--outputFormat`

Format of exported data.

```
# JSON (default)
```

```
--outputFormat json
```

```
# BSON
```

```
--outputFormat bson
```

```
# Message Pack
```

```
--outputFormat msgpack
```

```
# XML (removed in 2025.1.1)
```

```
--outputFormat xml
```

```
# XLSX Spreadsheet
```

```
--outputFormat xlsx
```

`--outputFormattingOptions`

Additional options for specified format.

This command supports universal parameters.

Output

The exported data follows the general game data structure, but omits *ToolsVersion*, *RevisionHash*, and *ChangeNumber* when the export mode is **not** set to publication.

```
{
  "Collections":
  {
    "Character":
    [
      {
        "Id": "Knight"

        /* rest of properties of document */
      }
    ]
  }
}
```

```

    },
    {
      "Id": "Templar"

      /* rest of properties of document */
    },
    // ...
  ]
}
}

```

Modifying Exported Data with *yq*

The exported data can be accessed or modified using the *yq* tool, a lightweight and portable command-line YAML, JSON, and XML processor. *yq* uses *jq*-like syntax and supports common operations for manipulating structured data.

To use *yq* with exported JSON data:

1. **Install *yq***: Follow the installation instructions from the official *yq* documentation: <https://mikefarah.gitbook.io/yq/>.

2. **Query Data**: Use *yq* to query specific fields or values from the exported JSON file.

```

# Query a specific field
yq '.Collections.Character[0].name' characters.json

```

3. **Modify Data**: Use *yq* to update or add fields in the exported JSON file.

```

# Export data
dnx dotnet-charon -- DATA EXPORT \
  --dataBase gamedata.json \
  --schemas Character \
  --output characters.json

# Update a field
yq -i '.Collections.Character[0].name = "New Name"' characters.json

# Add a new field
yq -i '.Collections.Character[0].level = 10' characters.json

```

```

# Import data back
dnx dotnet-charon -- DATA IMPORT \
  --dataBase gamedata.json \
  --schemas Character \
  --input characters.json \
  --mode safeUpdate

```

4. **Convert Formats**: *yq* can also convert between JSON, YAML, and other supported formats.

```

# Convert JSON to YAML
yq -o=yaml characters.json > characters.yaml

```

For more advanced usage, refer to the *yq* documentation: <https://mikefarah.gitbook.io/yq/>.

See also

- Importing and Exporting Data
- DATA IMPORT
- DATA I18N EXPORT
- DATA BACKUP
- Game Data Structure
- CI/CD Integration

Find Document

Seaches for a document.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA FIND \
  --dataBase "c:\my app\gamedata.json" \
  --schema Character \
  --id John \
  --output "c:\my app\character.json"

# remote game data
dnx dotnet-charon -- DATA FIND \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schema Character \
  --id John \
  --output out \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of <code>--dataBase</code> being URL.
--schema	Name or identifier of the type (schema) of document.
	<pre># name --schema Item # id --schema 55a4f32faca22e191098f3d9</pre>
--id	Identifier of document.
	<pre># text --id Sword # number --id 101</pre>

`--output`

Path to a found document file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), `/dev/null`, or a URL.

```
# standard output (default)
--output out
--output con

# standard error
--output err

# null device
--output null

# absolute path (windows)
--output "c:\my app\document.json"

# absolute path (unix)
--output /user/data/document.json

# relative path (universal)
--output "./document.json"

# remote location (HTTP)
--output "http://example.com/document.json"

# remote location with authentication (FTP)
--output "ftp://user:password@example.com/document.json"
```

`--outputFormat`

Format of exported data.

```
# JSON (default)
--outputFormat json

# BSON
--outputFormat bson

# Message Pack
--outputFormat msgpack

# XML (removed in 2025.1.1)
--outputFormat xml
```

`--outputFormattingOptions`

Additional options for specified format.

This command supports universal parameters.

Output

Outputs the found document.

```
{
  "Id": "John"

  /* rest of properties of found document */
}
```

See also

- DATA LIST
- DATA SEARCH
- DATA CREATE

- DATA UPDATE
- DATA DELETE
- DATA PROJECTSETTINGS

Find Game Data Files

Added in version 2026.4.0.

Scans a folder for game data files and returns discovered paths with their collection statistics.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# scan a folder tree, JSON output (default)
dnx dotnet-charon -- DATA FINDFILES \
  --folder "c:\my app\game-content"

# scan a folder tree, one path per line for shell piping
dnx dotnet-charon -- DATA FINDFILES \
  --folder "c:\my app\game-content" \
  --outputFormat list \
  --output out | xargs -I{} dnx dotnet-charon -- DATA EXAMINE --dataBase "{}"
```

Use this command to bootstrap work in an unfamiliar directory tree - “what game data lives here?” - before picking a file to pass to DATA EXAMINE, DATA LIST, or DATA SEARCH. Only files with recognized formatter extensions (.json, .bson, .msgpack, .xlsx, ...) are considered, and each candidate is probed - reading only its first 4 KB for the Collections marker - before a full load, so the scan stays fast even on large directory trees full of unrelated files.

Parameters

--folder

Path to the folder to scan.

```
# local folder
--folder "c:\my app\game-content"
--folder /home/user/game-content
```

--recursive

When true (default), scans all sub-folders recursively.

```
# scan sub-folders too (default)
--recursive true
```

```
# scan only the top-level folder
--recursive false
```

--outputFormat Output shape.

```
# JSON array of `{path, collections}` objects, matching the MCP `data_find_files` command
--outputFormat json
```

```
# one file path per line, for shell piping (e.g. `xargs`, `for` loops)
--outputFormat list
```

`--output`

Specifies where to write the output data.

```

# standard output (default)
--output out
--output con

# standard error
--output err

# null device
--output null

# absolute path (windows)
--output "c:\my app\found-files.json"

# absolute path (unix)
--output /user/data/found-files.json

# relative path (universal)
--output "./found-files.json"

# remote location (HTTP)
--output "http://example.com/found-files.json"

# remote location with authentication (FTP)
--output "ftp://user:password@example.com/found-files.json"

```

`--credentials`

Supplies authentication credentials for accessing discovered files if they're located on a remote server. Credentials can also be provided via the CHARON_API_KEY environment variable.

This command supports universal parameters.

Output

`--outputFormat json` (default) writes an array of {path, collections} objects, where collections maps each schema name found in that file to its document count:

```

[
  {
    "path": "C:/my app/game-content/characters.json",
    "collections":
    {
      "Character": 128,
      "Item": 340
    }
  },
  {
    "path": "C:/my app/game-content/quests.xlsx",
    "collections":
    {
      "Quest": 56
    }
  }
]

```

`--outputFormat list` writes one absolute file path per line, with no other formatting - suitable for piping into xargs or a shell for loop:

```

C:/my app/game-content/characters.json
C:/my app/game-content/quests.xlsx

```

See also

- DATA EXAMINE
- DATA LIST
- DATA SEARCH

Add Translation Languages

Add translation languages to specified game data.

- CLI Installation
- Universal Parameters
- Commands Reference

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA I18N ADDLANGUAGE \
  --dataBase "c:\my app\gamedata.json" \
  --languages "es-ES" "en-GB"

# remote game data
dnx dotnet-charon -- DATA I18N ADDLANGUAGE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --languages "es-ES" "en-GB" \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--languages	The list of languages to add. Values are language tags (BCP 47) separated by space..
--credentials	The API key used to access remote server in case of <code>--dataBase</code> being URL.
--output	Path to the file receiving the updated language list. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to Standard Error, Standard Output, <code>/dev/null</code>, or a URL. <pre># standard output (default) --output out --output con # standard error --output err # null device --output null # relative path (universal) --output "./languages.json"</pre>

`--outputFormat`

Format of the returned language list.

JSON (default)`--outputFormat json`*# space separated list*`--outputFormat list`*# new line (OS specific) separated list*`--outputFormat table``--outputFormattingOptions`

Additional options for specified format.

This command supports universal parameters.

See also

- Internationalization
- DATA I18N LANGUAGES
- DATA I18N EXPORT
- DATA I18N IMPORT
- [BCP 47 language tags](#)

Export Translated Data

Export text that can be translated into a file.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA I18N EXPORT \
  --dataBase "c:\my app\gamedata.json" \
  --schemas Character \
  --sourceLanguage en-US \
  --targetLanguage fr \
  --output "c:\my app\character_loc.xliff" \
  --outputFormat xliff

# remote game data
dnx dotnet-charon -- DATA I18N EXPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schemas Character \
  --sourceLanguage en-US \
  --targetLanguage fr \
  --output "./character_loc.xliff" \
  --outputFormat xliff \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <pre># local file --dataBase "c:\my app\gamedata.json"</pre> <pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of --dataBase being URL.
--schemas	A list of types of documents (schemas) to export. By default all schemas <i>EXCEPT</i> metadata are exported. • Use space to separate multiple schemas. • You can use wildcards (*) at the beginning and end of names. • You can use identifiers in {} instead of names. • You can exclude certain names by using an exclamation mark (!) at the beginning of their names. <pre># schema name --schemas Character --schemas Character Item # all (default) --schemas *</pre> <pre># masks --schemas Char* --schemas *Modifier --schemas *Mod*</pre> <pre># schema id --schemas {18d4bf318f3c49688087dbed}</pre> <pre># negation --schemas Char* !Character --schemas !*Item*</pre> <pre># excluding system schemas (Schema, SchemaProperty, ProjectSettings) --schemas ![system]</pre>
--sourceLanguage	Source (original) language for translation. Value is language tag (BCP 47). Use DATA I18N LANGUAGES to get list of used languages. <pre># it is used as <source> in XLIFF --sourceLanguage en-US</pre>
--targetLanguage	Target language for translation. Value is language tag (BCP 47). <pre># it is used as <target> in XLIFF --targetLanguage es-ES</pre>

--output

Path to a file to which data will be exported. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

```
# standard output (default)
```

```
--output out
```

```
--output con
```

```
# standard error
```

```
--output err
```

```
# null device
```

```
--output null
```

```
# absolute path (windows)
```

```
--output "c:\my app\input.json"
```

```
# absolute path (unix)
```

```
--output /user/data/input.json
```

```
# relative path (universal)
```

```
--output "./input.json"
```

```
# remote location (HTTP)
```

```
--output "http://example.com/input.json"
```

```
# remote location with authentication (FTP)
```

```
--output "ftp://user:password@example.com/input.json"
```

--outputFormat

Format of exported data.

```
# XLIFF v2 (default)
```

```
--outputFormat xliiff
```

```
--outputFormat xliiff2
```

```
# XLIFF v1
```

```
--outputFormat xliiff1
```

```
# XSLX Spreadsheet
```

```
--outputFormat xslx
```

```
# JSON
```

```
--outputFormat json
```

--outputFormattingOptions

Additional options for specified format.

This command supports universal parameters.

Output

The exported data follows the general game data structure, but omits *ToolsVersion*, *RevisionHash*, and *ChangeNumber* fields.

See also

- Internationalization
- DATA I18N IMPORT
- DATA I18N LANGUAGES
- DATA EXPORT
- [XLIFF standard](#)
- [BCP 47 language tags](#)

Importing Translated Data

Import translated text from a specified file into game data.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA I18N IMPORT \
  --dataBase "c:\my app\gamedata.json" \
  --input "c:\my app\character_loc.xliff" \
  --inputFormat xliff

# remote game data
dnx dotnet-charon -- DATA I18N IMPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --input "./character_loc.xliff" \
  --inputFormat xliff \
  --credentials "<API-Key>"
```

Parameters

<code>--dataBase</code>	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json"</pre>
	<pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
<code>--credentials</code>	The API key used to access remote server in case of <code>--dataBase</code> being URL.

`--schemas`

A list of types of documents (schemas) to import. By default all schemas *EXCEPT* metadata are imported.

- Use space to separate multiple schemas.
- You can use wildcards (*) at the beginning and end of names.
- You can use identifiers in {} instead of names.
- You can exclude certain names by using an exclamation mark (!) at the beginning of their names.

```
# schema name
```

```
--schemas Character
```

```
--schemas Character Item
```

```
# all (default)
```

```
--schemas *
```

```
# masks
```

```
--schemas Char*
```

```
--schemas *Modifier
```

```
--schemas *Mod*
```

```
# schema id
```

```
--schemas {18d4bf318f3c49688087dbed}
```

```
# negation
```

```
--schemas Char* !Character
```

```
--schemas !*Item*
```

```
# excluding system schemas (Schema, SchemaProperty, ProjectSettings)
```

```
--schemas ![system]
```

`--languages`

The list of languages to import. Values are [language tags \(BCP 47\)](#).

```
# Import specific language
```

```
--languages en-US es-ES
```

```
# Import all languages
```

```
--languages *
```

`--validationOptions`

List of validation checks and repairs to perform during import.

```
# no checks
```

```
--validationOptions none
```

```
# repairs
```

```
--validationOptions repair
```

```
--validationOptions repair deduplicateIds
```

```
--validationOptions repair repairRequiredWithDefaultValue
```

```
--validationOptions repair eraseInvalidValue
```

```
# checks (default)
```

```
--validationOptions checkTranslation
```

```
--validationOptions checkRequirements
```

```
--validationOptions checkFormat
```

```
--validationOptions checkUniqueness
```

```
--validationOptions checkReferences
```

```
--validationOptions checkSpecification
```

```
--validationOptions checkConstraints
```

--input

Path to a file with data to import. Alternatively, you can use [Standard Input](#) or URL.

See input data structure requirements.

```
# standard input (default)
```

```
--input in
```

```
--input con
```

```
# absolute path (windows)
```

```
--input "c:\my app\input.json"
```

```
# absolute path (unix)
```

```
--input /user/data/input.json
```

```
# relative path (universal)
```

```
--input "./input.json"
```

```
# remote location (HTTP)
```

```
--input "http://example.com/input.json"
```

```
# remote location with authentication (FTP)
```

```
--input "ftp://user:password@example.com/input.json"
```

--inputFormat

Format of imported data.

```
# Auto-detect by extension (default)
```

```
--inputFormat auto
```

```
# XLIFF v2
```

```
--inputFormat xliiff
```

```
--inputFormat xliiff2
```

```
# XLIFF v1
```

```
--inputFormat xliiff1
```

```
# XSLX Spreadsheet
```

```
--inputFormat xslx
```

--inputFormattingOptions

Additional options for specified format.

`--output`

Optional path to a import report file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

```
# standard output
--output out
--output con

# standard error
--output err

# null device (default)
--output null

# absolute path (windows)
--output "c:\my app\document.json"

# absolute path (unix)
--output /user/data/document.json

# relative path (universal)
--output "./document.json"

# remote location (HTTP)
--output "http://example.com/document.json"

# remote location with authentication (FTP)
--output "ftp://user:password@example.com/document.json"
```

`--outputFormat`

Format of import report.

```
# JSON (default)
--outputFormat json

# BSON
--outputFormat bson

# Message Pack
--outputFormat msgpack

# XLSX Spreadsheet
--outputFormat xlsx
```

`--outputFormattingOptions`

Additional options for specified format.

`--dryRun`

Allows you to run the command without actually making any changes to the game data, providing a preview of what would happen.

This command supports universal parameters.

See also

- [Internationalization](#)
- [DATA I18N EXPORT](#)
- [DATA I18N LANGUAGES](#)
- [DATA IMPORT](#)
- [XLIFF standard](#)
- [BCP 47 language tags](#)

List Translation Languages

Get a list of supported translation languages. Primary language always shows up first in the list.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA I18N LANGUAGES \
  --dataBase "c:\my app\gamedata.json" \
  --output out \
  --outputFormat table

# remote game data
dnx dotnet-charon -- DATA I18N LANGUAGES \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --output out \
  --outputFormat table \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of <code>--dataBase</code> being URL.

--output

Path to language list file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

standard output (default)

--output out

--output con

standard error

--output err

null device

--output null

absolute path (windows)

--output "c:\my app\input.json"

absolute path (unix)

--output /user/data/input.json

relative path (universal)

--output "./input.json"

remote location (HTTP)

--output "http://example.com/input.json"

remote location with authentication (FTP)

--output "ftp://user:password@example.com/input.json"

--outputFormat

Format of exported data.

JSON (default)

--outputFormat json

#> [

#> "en-US",

#> "es-ES",

#>]

Space separated list

--outputFormat list

#> en-US es-ES

New line (OS specific) separated list

--outputFormat table

#> en-US

#> es-ES

--outputFormattingOptions

Additional options for specified format.

This command supports universal parameters.

See also

- Internationalization
- DATA I18N EXPORT
- DATA I18N IMPORT
- DATA I18N ADDLANGUAGE

- [BCP 47 language tags](#)

Import Data

Imports documents from file to a game data.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA IMPORT \
  --dataBase "c:\my app\gamedata.json" \
  --schemas Character \
  --input "c:\my app\characters.json" \
  --inputFormat json \
  --mode safeUpdate

# remote game data
dnx dotnet-charon -- DATA IMPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schemas Character \
  --input "./characters.json" \
  --inputFormat json \
  --mode safeUpdate \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json"</pre>
	<pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of <code>--dataBase</code> being URL.

--schemas

A list of types of documents (schemas) to import. By default all schemas *EXCEPT* metadata are imported.

- Use space to separate multiple schemas.
- You can use wildcards (*) at the beginning and end of names.
- You can use identifiers in {} instead of names.
- You can exclude certain names by using an exclamation mark (!) at the beginning of their names.

schema name

--schemas Character

--schemas Character Item

all (default)

--schemas *

masks

--schemas Char*

--schemas *Modifier

--schemas *Mod*

schema id

--schemas {18d4bf318f3c49688087dbed}

negation

--schemas Char* !Character

--schemas !*Item*

excluding system schemas (Schema, SchemaProperty, ProjectSettings)

--schemas ![system]

--mode

Import mode controls merge behavior during import.

(default)

--mode createAndUpdate

--mode create

--mode update

--mode safeUpdate

--mode replace

--mode delete

createAndUpdate

creates new documents and updates existing ones

create

only creates new documents, existing documents are kept unchanged

update

only updates existing documents, no new ones are created

safeUpdate

same as *update* but without creating, moving and erasing embedded documents

replace

replaces the entire collection with the imported documents

delete

deletes documents found in the imported data

```

--validationOptions List of validation checks and repairs to perform during import. Defaults to repair repairRequiredWithDefaults deduplicateIds resolveConflictingUnions - repairs only, with no integrity checks. Pass the check* flags explicitly to have the import verify the data, and see Validation for what each flag does.

# no checks
--validationOptions none

# all repairs
--validationOptions repair

# deduplicate documents with the same Id
--validationOptions repair deduplicateIds

# fill required fields that have a default value
--validationOptions repair repairRequiredWithDefaults

# erase values whose type does not match the property's data type
--validationOptions repair eraseInvalidValues

# fix malformed Union values:
#   empty union (no variant set)      → erased to null
#   conflicting union (2+ variants)   → last variant in schema order is
--validationOptions repair resolveConflictingUnions

# integrity checks - applied only when requested
--validationOptions checkTranslations
--validationOptions checkRequirements
--validationOptions checkFormat
--validationOptions checkUniqueness
--validationOptions checkReferences
--validationOptions checkSpecification
--validationOptions checkConstraints

--input Path to a data file. Alternatively, you can use Standard Input or URL.

# standard input (default)
--input in
--input con

# absolute path (windows)
--input "c:\my app\characters.json"

# absolute path (unix)
--input "/user/data/characters.json"

# relative path (universal)
--input "./characters.json"

# remote location (HTTP)
--input "http://example.com/characters.json"

# remote location with authentication (FTP)
--input "ftp://user:password@example.com/characters.json"

```

`--inputFormat`

Format of imported data.

```
# Auto-detect by extension (default)
--inputFormat auto
```

```
# JSON
--inputFormat json
```

```
# BSON
--inputFormat bson
```

```
# Message Pack
--inputFormat msgpack
```

```
# XML (removed in 2025.1.1)
--inputFormat xml
```

```
# XLSX Spreadsheet
--inputFormat xlsx
```

`--inputFormattingOptions`

Additional options for specified format.

`--output`

Optional path to a import report file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

```
# standard output
--output out
--output con
```

```
# standard error
--output err
```

```
# null device (default)
--output null
```

```
# absolute path (windows)
--output "c:\my app\document.json"
```

```
# absolute path (unix)
--output /user/data/document.json
```

```
# relative path (universal)
--output "./document.json"
```

```
# remote location (HTTP)
--output "http://example.com/document.json"
```

```
# remote location with authentication (FTP)
--output "ftp://user:password@example.com/document.json"
```

`--outputFormat`

Format of import report.

```
# JSON (default)
--outputFormat json
```

```
# BSON
--outputFormat bson
```

```
# Message Pack
--outputFormat msgpack
```

```
# XLSX Spreadsheet
--outputFormat xlsx
```

Is It Free?

--outputFormattingOptions
--dryRun

Additional options for specified format.

Allows you to run the command without actually making any changes to the game data, providing a preview of what would happen.

This command supports universal parameters.

Input Data Structure

The data you input should follow this structure (recommended):

```
{
  "Collections": {
    "<Schema-Name>": [
      {
        // <Document>
      }
    ]
  }
}
```

This structure is also accepted:

```
{
  "<Schema-Name>": [
    {
      // <Document>
    }
  ]
}
```

A list of documents is accepted if only one name in --schemas is specified:

```
[
  {
    // <Document>
  }
]
```

And single document is accepted too if only one name in --schemas is specified:

```
{
  // <Document>
}
```

See also

- Importing and Exporting Data
- DATA EXPORT
- DATA VALIDATE
- DATA I18N IMPORT
- Game Data Structure
- CI/CD Integration

List Documents

Seaches for a documents.

- CLI Installation

- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA LIST \
  --dataBase "c:\my app\gamedata.json" \
  --schema Character

# remote game data
dnx dotnet-charon -- DATA LIST \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schema Character \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json"</pre>
	<pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of <i>--dataBase</i> being URL.
--schema	Name or identifier of the type (schema) of document.
	<pre># name --schema Item</pre>
	<pre># id --schema 55a4f32faca22e191098f3d9</pre>

--filters Document filter expressions.

```
# patterns
--filters <Field> <Operator> <Value> [<Field> <Operator> <Value>...]

# single expression
--filters Id > 10
--filters Name like "Zombie"

# multiple expressions
--filters Id > 10 Name like "Zombie"

# greater than
--filters Id > 0
--filters Id greaterThan 0

# greater than or equal
--filters Id >= 0
--filters Id greaterThanOrEqual 0

# less than
--filters Id < 0
--filters Id lessThan 0

# less than or equal
--filters Id <= 0
--filters Id LessThanOrEqual 0

# equal
--filters Id = 0
--filters Id == 0
--filters Id equal 0

# not equal
--filters Id <> 0
--filters Id != 0
--filters Id notEqual 0

# like - is used to search for specific patterns in a field, allowing for partial
--filters Name like "Zombie"
```

--sorters

Document sort expressions.

```
# patterns
--sorters <Field> ASC|DESC [<Field> ASC|DESC ...]

# ascending
--sorters Name ASC

# descending
--sorters Name DESC
```

--select

Projection of specific fields to include in the output. Use the format <PropertyName>, e.g., Name or Id. Useful for reducing output size.

```
# only names and ids
--select Id Name
```


--path	Embeddane path filter. Could be used to get only embedded documents. <pre># any path --path *</pre> <pre># root documents (default) --path ""</pre> <pre># in 'Item' property --path /Item</pre>
--skip	Number of found documents to skip. <pre># skip first ten documents after applying --filter and --sort --skip 10</pre>
--take	Max amount to documents return. <pre># limit to first 100 documents after --skip --take 100</pre>
--output	Path to a found document file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to Standard Error, Standard Output, /dev/null, or a URL. <pre># standard output (default) --output out --output con</pre> <pre># standard error --output err</pre> <pre># null device --output null</pre> <pre># absolute path (windows) --output "c:\my app\document.json"</pre> <pre># absolute path (unix) --output /user/data/document.json</pre> <pre># relative path (universal) --output "./document.json"</pre> <pre># remote location (HTTP) --output "http://example.com/document.json"</pre> <pre># remote location with authentication (FTP) --output "ftp://user:password@example.com/document.json"</pre>
--outputFormat	Format of exported data. <pre># JSON (default) --outputFormat json</pre> <pre># BSON --outputFormat bson</pre> <pre># Message Pack --outputFormat msgpack</pre> <pre># XML (removed in 2025.1.1) --outputFormat xml</pre>
--outputFormattingOptions	Additional options for specified format.

This command supports universal parameters.

Output

The exported data follows the general game data structure, but omits *ToolsVersion*, *RevisionHash*, and *ChangeNumber*.

```
{
  "Collections":
  {
    "Character":
    [
      {
        "Id": "Knight"

        /* rest of properties of document */
      },
      {
        "Id": "Templar"

        /* rest of properties of document */
      },
      // ...
    ]
  }
}
```

See also

- DATA FIND
- DATA SEARCH
- DATA LISTSCHEMAS
- DATA EXPORT
- Game Data Structure

List Schemas

Added in version 2026.4.0.

Lists schema definitions (document types) configured in game data.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA LISTSCHEMAS \
  --dataBase "c:\my app\gamedata.json"

# remote game data
dnx dotnet-charon -- DATA LISTSCHEMAS \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --credentials "<API-Key>"
```

This command lists the built-in Schema collection itself - the schema *documents* (definitions: name, display name, properties, id generator, and so on), not the documents of a particular schema. It is the same listing capability as DATA LIST, just scoped internally to Schema instead of an arbitrary --schema you provide. For full metadata (including computed/derived information used when generating source code), use the data_get_metadata MCP tool or the metadata endpoint of a running server instead.

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <pre># local file --dataBase "c:\my app\gamedata.json"</pre> <pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of --dataBase being URL.
--filters	Schema filter expressions. Use the format <PropertyName> <Operation> <Value>, e.g., Name == "Item". Multiple filters are ANDed together. For deeper paths use JSON Pointer format. Example: ['/Item/Level', '>=', '10']. <pre># patterns --filters <Field> <Operator> <Value> [<Field> <Operator> <Value>...] # single expression --filters Name == "Item" --filters DisplayName like "Item*" # multiple expressions --filters Type == 0 Name like "*Item"</pre>
--sorters	Sorting rules for the result set. Use the format <PropertyName> <ASC DESC>, e.g., Name ASC. You can specify multiple sorters to sort by multiple fields. For deeper paths use JSON Pointer format. Example: ['/Item/Name', 'ASC']. <pre># patterns --sorters <Field> ASC DESC [<Field> ASC DESC ...] # ascending --sorters Name ASC # descending --sorters Name DESC</pre>
--select	Projection of specific fields to include in the output. Use the format <PropertyName>, e.g., Name or Id. Useful for reducing output size. <pre># only names and ids --select Id Name</pre>
--skip	Number of schema definitions to skip from the start of the result set. Use for pagination. <pre># skip first ten schemas after applying --filters and --sorters --skip 10</pre>

```

--take                               Number of schema definitions to return after skipping.
                                      Use for limiting result size in combination with --skip.
                                      # limit to first 100 schemas after --skip
                                      --take 100

--output                             Path to a found document file. If the file exists, it will be overwritten.
                                      The directory must already exist. Alternatively, you can output to
                                      Standard Error, Standard Output, /dev/null, or a URL.
                                      # standard output (default)
                                      --output out
                                      --output con

                                      # standard error
                                      --output err

                                      # null device
                                      --output null

                                      # absolute path (windows)
                                      --output "c:\my app\schemas.json"

                                      # absolute path (unix)
                                      --output /user/data/schemas.json

                                      # relative path (universal)
                                      --output "./schemas.json"

                                      # remote location (HTTP)
                                      --output "http://example.com/schemas.json"

                                      # remote location with authentication (FTP)
                                      --output "ftp://user:password@example.com/schemas.json"

--outputFormat                       Format of exported data.
                                      # JSON (default)
                                      --outputFormat json

                                      # BSON
                                      --outputFormat bson

                                      # Message Pack
                                      --outputFormat msgpack

                                      # XLSX Spreadsheet
                                      --outputFormat xlsx

--outputFormattingOptions            Additional options for specified format.

```

This command supports universal parameters.

Output

The result follows the general game data structure, with schema documents under the Schema collection key. See the Schema section for the full field reference.

```

{
  "Collections":
  {
    "Schema":
    [
      {
        "Id": "592fc86c983a36266c0912a0",

```

```

    "Name": "Item",
    "DisplayName": "Items",
    "Type": 0,
    "Description": "An item.",
    "IdGenerator": 1,
    "Specification": "icon=fugue16%2Fabacus&group=Metagame",
    "Properties": [
        // schema property
    ]
},
{
    "Id": "592fc86c983a36266c0912b1",
    "Name": "Character"

    /* rest of properties of schema document */
},
// ...
]
}
}

```

See also

- DATA LIST
- DATA SEARCH
- DATA FINDFILES
- Game Data Structure

Project Settings

Added in version 2026.4.0.

Retrieves the project settings document (languages, primary language, and other project-level configuration) from game data.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```

# local game data (windows)
dnx dotnet-charon -- DATA PROJECTSETTINGS \
  --dataBase "c:\my app\gamedata.json"

# remote game data
dnx dotnet-charon -- DATA PROJECTSETTINGS \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --credentials "<API-Key>"

```

There is always exactly one project settings document per game data source, so unlike DATA FIND this command takes no --schema/-id parameters - its identity is fixed internally.

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of --dataBase being URL.
--output	Path to the retrieved document file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to Standard Error, Standard Output, /dev/null, or a URL. <pre># standard output (default) --output out --output con # standard error --output err # null device --output null # absolute path (windows) --output "c:\my app\settings.json" # absolute path (unix) --output /user/data/settings.json # relative path (universal) --output "./settings.json" # remote location (HTTP) --output "http://example.com/settings.json" # remote location with authentication (FTP) --output "ftp://user:password@example.com/settings.json"</pre>
--outputFormat	Specifies the format of the output data. <pre># JSON (default) --outputFormat json # BSON --outputFormat bson # Message Pack --outputFormat msgpack</pre>
--outputFormattingOptions	Additional options for formatting the output.

This command supports universal parameters.

Output

Outputs the project settings document. See the Project Settings section for the full field reference.

```
{
  "Id": "55a4f32faca22e191098f3d9",
  "Name": "My Project",
  "PrimaryLanguage": "en-US",
```

```

"Languages": "en-GB;fr-FR",
"Copyright": "My Company (c) 2025",
"Version": "1.0.0.0"
}

```

See also

- DATA UPDATEPROJECTSETTINGS
- DATA I18N LANGUAGES
- DATA FIND
- Game Data Structure

Restore from Backup

Restores game data from a file created by DATA BACKUP command.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```

# local game data (windows)
dnx dotnet-charon -- DATA RESTORE \
  --dataBase "c:\my app\gamedata.json" \
  --input "c:\my app\backup.msgpkg" \
  --inputFormat msgpack

# remote game data
dnx dotnet-charon -- DATA RESTORE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --input "./backup.msgpkg" \
  --inputFormat msgpack \
  --credentials "<API-Key>"

```

Parameters

- | | |
|----------------------------|--|
| <code>--dataBase</code> | <p>Absolute or relative path to game data. Use quotation marks if your path contains spaces.</p> <pre> # local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/" </pre> |
| <code>--credentials</code> | <p>The API key used to access remote server in case of <code>--dataBase</code> being URL.</p> |

--input

Path to a backup file. Alternatively, you can use [Standard Input](#) or URL.

```
# standard input (default)
--input in
--input con

# absolute path (windows)
--input "c:\my app\backup.json"

# absolute path (unix)
--input "/user/data/backup.json"

# relative path (universal)
--input "./backup.json"

# remote location (HTTP)
--input "http://example.com/backup.json"

# remote location with authentication (FTP)
--input "ftp://user:password@example.com/backup.json"
```

--inputFormat

Format of imported data.

```
# Auto-detect by extension (default)
--inputFormat auto

# JSON
--inputFormat json

# Message Pack
--inputFormat msgpack
```

--inputFormattingOptions

Additional options for specified format.

This command supports universal parameters.

See also

- Backup and Restore
- DATA BACKUP
- CI/CD Integration

Search Documents

Added in version 2026.4.0.

Searches for documents across all schemas using a text query with wildcard matching.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows), default table output
dnx dotnet-charon -- DATA SEARCH \
  --dataBase "c:\my app\gamedata.json" \
  --query Dragon

# remote game data, full documents as JSON
dnx dotnet-charon -- DATA SEARCH \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --query Dragon \
  --outputFormat json \
  --credentials "<API-Key>"
```

Unlike DATA LIST, which requires a `--schema` and matches one field at a time via `--filters`, DATA SEARCH matches the query text against *every field of every document*, across schemas, using a contains (LIKE) operator. Use it to find where a value shows up when you don't know which schema or field it lives in; use DATA LIST once you do.

Parameters

<code>--dataBase</code>	Absolute or relative path to game data. Use quotation marks if your path contains spaces. # local file <code>--dataBase "c:\my app\gamedata.json"</code> # remote server <code>--dataBase "https://charon.live/view/data/My_Game/develop/"</code>
<code>--credentials</code>	The API key used to access remote server in case of <code>--dataBase</code> being URL.
<code>--query</code>	Text to search for across all document fields. Matches using 'contains' (LIKE) operator against every field. # matches any document with 'Dragon' in any field value <code>--query Dragon</code>
<code>--caseSensitive</code>	Whether the search should be case-sensitive. Default is false (case-insensitive). <code>--caseSensitive true</code> <code>--caseSensitive false</code>
<code>--schemas</code>	Defines which schemas to search. Accepts a list of schema names. If omitted, all schemas are searched. # single schema <code>--schemas Item</code> # multiple schemas <code>--schemas Item Character</code> # masks <code>--schemas *Item*</code> # negation <code>--schemas !DeprecatedSchema</code>
<code>--skip</code>	Number of matches to skip from the start of the result set. Use for pagination. # skip first ten matches <code>--skip 10</code>

```

--take                               Number of matches to return after skipping. Use for
                                     limiting result size in combination with --skip.
                                     # limit to first 100 matches after --skip
                                     --take 100

--output                             Specifies where to write the output data.
                                     # standard output (default)
                                     --output out
                                     --output con

                                     # standard error
                                     --output err

                                     # null device
                                     --output null

                                     # absolute path (windows)
                                     --output "c:\my app\results.json"

                                     # absolute path (unix)
                                     --output /user/data/results.json

                                     # relative path (universal)
                                     --output "./results.json"

                                     # remote location (HTTP)
                                     --output "http://example.com/results.json"

                                     # remote location with authentication (FTP)
                                     --output "ftp://user:password@example.com/results.json"

--outputFormat                       Format for serializing the output. Only used when --outputFormat json; ignored
                                     for table.
                                     # flat `Schema|Document Id|Path|Value` rows, script/grep-friendly (default)
                                     --outputFormat table

                                     # JSON (default)
                                     --outputFormat json

                                     # BSON
                                     --outputFormat bson

                                     # Message Pack
                                     --outputFormat msgpack

                                     # XLSX Spreadsheet
                                     --outputFormat xlsx

--outputFormattingOptions             Additional formatting options for the chosen output
                                     format.

```

This command supports universal parameters.

Output

`--outputFormat table` (default) writes one line per match, pipe-delimited, with a header row - convenient for `grep/cut/awk`:

```

Schema|Document Id|Path|Value
Character|Knight|/Description|A knight who slays dragons for gold.
Item|DragonScale|/Name|Dragon Scale
Quest|SlayTheDragon|/Name|Slay the Dragon

```

The Path column is empty for root documents and holds the embedding property's JSON Pointer path (e.g. /Item) for documents embedded inside another document.

--outputFormat json writes the matched documents in full, grouped back into their schema collections, in the same shape as DATA EXPORT's ExportResult:

```
{
  "Collections":
  {
    "Character":
    [
      {
        "Id": "Knight"

        /* rest of properties of document */
      }
    ],
    "Item":
    [
      {
        "Id": "DragonScale"

        /* rest of properties of document */
      }
    ]
  }
}
```

See also

- DATA LIST
- DATA LISTSCHEMAS
- DATA FIND
- DATA FINDFILES
- DATA EXPORT

Update Document

Updates a document. For a bulk updates use DATA IMPORT command with --mode update. The update document in --input may be partial, with non-included fields being omitted. Only the first document from the --input will be processed.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA UPDATE \
  --dataBase "c:\my app\gamedata.json" \
  --schema Item \
  --input "c:\my app\item.json" \
  --inputFormat json

# remote game data
dnx dotnet-charon -- DATA UPDATE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schema Item \
  --input "./item.json" \
  --inputFormat json \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces. # local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"
--credentials	The API key used to access remote server in case of --dataBase being URL.
--schema	Name or identifier of the type (schema) of updated document. # name --schema Item # id --schema 55a4f32faca22e191098f3d9
--id	Identifier of updated document. Could be omitted if <i>Id</i> is specified in --input document. # text --id Sword # number --id 101

```
--validationOptions      List of validation checks and repairs to perform during document update.

                           # no checks
                           --validationOptions none

                           # repairs
                           --validationOptions repair
                           --validationOptions repair deduplicateIds
                           --validationOptions repair repairRequiredWithDefaultValue
                           --validationOptions repair eraseInvalidValue

                           # checks (default)
                           --validationOptions checkTranslation
                           --validationOptions checkRequirements
                           --validationOptions checkFormat
                           --validationOptions checkUniqueness
                           --validationOptions checkReferences
                           --validationOptions checkSpecification
                           --validationOptions checkConstraints

--input                  Path to a file with update data. Alternatively, you can use Standard Input or URL.

                           # standard input (default)
                           --input in
                           --input con

                           # absolute path (windows)
                           --input "c:\my app\item.json"

                           # absolute path (unix)
                           --input "/user/data/item.json"

                           # relative path (universal)
                           --input "./item.json"

                           # remote location (HTTP)
                           --input "http://example.com/item.json"

                           # remote location with authentication (FTP)
                           --input "ftp://user:password@example.com/item.json"

--inputFormat            Format of update data.

                           # Auto-detect by extension (default)
                           --inputFormat auto

                           # JSON
                           --inputFormat json

                           # BSON
                           --inputFormat bson

                           # Message Pack
                           --inputFormat msgpack

                           # XML (removed in 2025.1.1)
                           --inputFormat xml

--inputFormattingOptions Additional options for specified format.
```

`--output`

Path to a updated document file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), `/dev/null`, or a URL.

```
# standard output
--output out
--output con

# standard error
--output err

# null device (default)
--output null

# absolute path (windows)
--output "c:\my app\updated_item.json"

# absolute path (unix)
--output /user/data/updated_item.json

# relative path (universal)
--output "./updated_item.json"

# remote location (HTTP)
--output "http://example.com/updated_item.json"

# remote location with authentication (FTP)
--output "ftp://user:password@example.com/updated_item.json"
```

`--outputFormat`

Format of updated data.

```
# JSON (default)
--outputFormat json

# BSON
--outputFormat bson

# Message Pack
--outputFormat msgpack

# XML (removed in 2025.1.1)
--outputFormat xml
```

`--outputFormattingOptions`

Additional options for specified format.

This command supports universal parameters.

Input Data Schema

The data you input should follow this schema (recommended):

```
{
  "Collections": {
    "<Schema-Name>": [
      {
        // <Document>
      }
    ]
  }
}
```

This schema is also accepted:

Is It Free?

```
{
  "<Schema-Name>": [
    {
      // <Document>
    }
  ]
}
```

A list of documents is accepted:

```
[
  {
    // <Document>
  }
]
```

And single document too:

```
{
  // <Document>
}
```

Output

Outputs the updated document with all the edits that were made to make it conform to the schema.

```
{
  "Id": "Sword"

  /* rest of properties of updated document */
}
```

See also

- DATA IMPORT
- DATA CREATE
- DATA DELETE
- Importing and Exporting Data

Update Project Settings

Added in version 2026.4.0.

Updates a single property of the project settings document (project name, copyright, languages, and other project-level configuration) in game data.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# bump the game data version (windows)
dnx dotnet-charon -- DATA UPDATEPROJECTSETTINGS \
  --dataBase "c:\my app\gamedata.json" \
  --property Version \
  --value "1.2.0.0"

# the same property addressed by JSON Pointer
dnx dotnet-charon -- DATA UPDATEPROJECTSETTINGS \
  --dataBase "c:\my app\gamedata.json" \
  --property /Version \
  --value "1.2.0.0"

# rename the project
dnx dotnet-charon -- DATA UPDATEPROJECTSETTINGS \
  --dataBase "c:\my app\gamedata.json" \
  --property Name \
  --value "My Project"

# remote game data
dnx dotnet-charon -- DATA UPDATEPROJECTSETTINGS \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --property Copyright \
  --value "My Company (c) 2025" \
  --credentials "<API-Key>"
```

There is always exactly one project settings document per game data source, so unlike DATA UPDATE this command takes no --schema/- -id parameters - its identity is fixed internally.

Available Properties

One property is updated per command run. All project settings properties hold text values.

Property	Description
Id	Unique identifier of the project settings document (BSON ObjectId). Read-only - the command refuses to change it.
Name	Name of the project.
PrimaryLanguage	Primary language for localizable text (language ID in BCP-47 format, e.g. en-US).
Languages	Alternative languages for localizable text (semicolon-delimited list of BCP-47 language IDs, e.g. en-GB; fr-FR). Prefer DATA I18N ADD LANGUAGE when adding a language, because it keeps the primary language out of the list and preserves existing entries.
Copyright	Copyright information for the project.
Version	Version of the current file as four dot-separated numbers (Major.Minor.Build.Revision).
Extensions	Installed extension packages as a semicolon-delimited list of name=version pairs.

See the Project Settings section for the full field reference. Any other property present in the document can be addressed the same way, by name or by [JSON Pointer](#).

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <i># local file</i> --dataBase "c:\my app\gamedata.json" <i># remote server</i> --dataBase "https://charon.live/view/data/My_Game/develop/"
--property	Project settings property to update. Use a property name for top level values, or a JSON Pointer starting with / to address nested values. <i># top level property by name</i> --property PrimaryLanguage <i># the same property as a JSON Pointer</i> --property /PrimaryLanguage <i># nested value</i> --property /Extensions/0
--value	New value of the property. It is stored as text unless --valueIsJson is specified. --value "en-GB;fr-FR"
--valueIsJson	Treats --value as a JSON document and parses it, which allows storing numbers, booleans, arrays, and objects instead of text. --property Version --valueIsJson --value "\"1.2.0.0\""
--credentials	--property Extensions --valueIsJson --value "null" The API key used to access remote server in case of --dataBase being URL.

`--output`

Path to the file receiving the updated document. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

```
# standard output (default)
```

```
--output out
```

```
--output con
```

```
# standard error
```

```
--output err
```

```
# null device
```

```
--output null
```

```
# absolute path (windows)
```

```
--output "c:\my app\settings.json"
```

```
# absolute path (unix)
```

```
--output /user/data/settings.json
```

```
# relative path (universal)
```

```
--output "./settings.json"
```

```
# remote location (HTTP)
```

```
--output "http://example.com/settings.json"
```

```
# remote location with authentication (FTP)
```

```
--output "ftp://user:password@example.com/settings.json"
```

`--outputFormat`

Specifies the format of the output data.

```
# JSON (default)
```

```
--outputFormat json
```

```
# BSON
```

```
--outputFormat bson
```

```
# Message Pack
```

```
--outputFormat msgpack
```

`--outputFormattingOptions`

Additional options for formatting the output.

This command supports universal parameters.

Output

Outputs the updated project settings document.

```
{
  "Id": "55a4f32faca22e191098f3d9",
  "Name": "My Project",
  "PrimaryLanguage": "en-US",
  "Languages": "en-GB;fr-FR",
  "Copyright": "My Company (c) 2025",
  "Version": "1.2.0.0"
}
```

See also

- DATA PROJECTSETTINGS
- DATA I18N ADD LANGUAGE
- DATA UPDATE

- Game Data Structure

Validate Game Data

Checks the game data for validity and produces a report.

The exit code will be 1 if the report contains errors and the `--output` is set to `err`. Otherwise, the exit code will be 0.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- DATA DELETE \
  --dataBase "c:\my app\gamedata.json" \
  --schema Item \
  --id "Sword"

# remote game data
dnx dotnet-charon -- DATA DELETE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --schema Item \
  --id "Sword" \
  --credentials "<API-Key>"
```

Parameters

- | | |
|----------------------------|--|
| <code>--dataBase</code> | <p>Absolute or relative path to game data. Use quotation marks if your path contains spaces.</p> <pre># local file --dataBase "c:\my app\gamedata.json"</pre> <pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre> |
| <code>--credentials</code> | <p>The API key used to access remote server in case of <code>--dataBase</code> being URL.</p> |

--validationOptions list of validation checks and repairs to perform.

```
# no checks
--validationOptions none

# all repairs
--validationOptions repair

# deduplicate documents with the same Id
--validationOptions repair deduplicateIds

# fill required fields that have a default value
--validationOptions repair repairRequiredWithDefaults

# erase values whose type does not match the property's data type
--validationOptions repair eraseInvalidValues

# fix malformed Union values:
#   empty union (no variant set)      → erased to null
#   conflicting union (2+ variants)   → last variant in schema order is
--validationOptions repair resolveConflictingUnions

# checks (default)
--validationOptions checkTranslations
--validationOptions checkRequirements
--validationOptions checkFormat
--validationOptions checkUniqueness
--validationOptions checkReferences
--validationOptions checkSpecification
--validationOptions checkConstraints
```

--output

Path to a validation report file. If the file exists, it will be overwritten. The directory must already exist. Alternatively, you can output to [Standard Error](#), [Standard Output](#), [/dev/null](#), or a URL.

```
# standard output
```

```
--output out
--output con
```

```
# standard error
```

```
--output err
```

```
# null device (default)
```

```
--output null
```

```
# absolute path (windows)
```

```
--output "c:\my app\document.json"
```

```
# absolute path (unix)
```

```
--output /user/data/document.json
```

```
# relative path (universal)
```

```
--output "./document.json"
```

```
# remote location (HTTP)
```

```
--output "http://example.com/document.json"
```

```
# remote location with authentication (FTP)
```

```
--output "ftp://user:password@example.com/document.json"
```

`--outputFormat`

Format of exported data.

JSON (default)`--outputFormat json`*# BSON*`--outputFormat bson`*# Message Pack*`--outputFormat msgpack`*# XML (removed in 2025.1.1)*`--outputFormat xml``--outputFormattingOptions`

Additional options for specified format.

This command supports universal parameters.

Output Data Schema

The report follow this pattern:

```
{
  records:
  [
    {
      id: "<document-id>",
      schemaId: "<schema-id>",
      schemaName: "<schema-name>",
      errors: // could be null if no errors
      [
        {
          path: "<path-in-document>",
          message: "<error-message>",
          code: "<error-code>"
        },
        // ...
      ]
    },
    // ...
  ]
}
```

or [JSON schema](#):

```
{
  "type": "object",
  "x-name": "ValidationReport",
  "additionalProperties": false,
  "properties": {
    "records": {
      "type": "array",
      "items": {
        "type": "object",
        "x-name": "ValidationRecord",
        "additionalProperties": false,
        "properties": {
          "id": { },
          "schemaName": {
            "type": "string"
          },
          // ...
        },
        "schemaId": {

```

```

    "type": "string"
  },
  "errors": {
    "type": "array",
    "items": {
      "type": "object",
      "x-name": "ValidationError",
      "additionalProperties": true,
      "readOnly": true,
      "properties": {
        "path": {
          "type": "string"
        },
        "message": {
          "type": "string"
        },
        "code": {
          "type": "string"
        }
      }
    }
  }
},
"metadataHashCode": {
  "type": "integer",
  "format": "int32"
}
}

```

See also

- Validation
- DATA IMPORT
- DATA EXPORT
- CI/CD Integration
- **JSON Schema specification**

Generate C# Source Code

Generates C# source code for game data into output directory.

By default, this command does not delete previously generated files. Use the `--clearOutputDirectory` parameter to automatically remove prior generated files before creating new ones.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- GENERATE CSHARPCODE \
  --dataBase "c:\my app\gamedata.json" \
  --namespace "MyGame.Parameters" \
  --outputDirectory "c:\my app\scripts"

# remote game data
dnx dotnet-charon -- GENERATE CSHARPCODE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --namespace "MyGame.Parameters" \
  --outputDirectory "./scripts" \
  --credentials "<API-Key>"
```

Parameters

--dataBase	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
--credentials	The API key used to access remote server in case of --dataBase being URL.
--outputDirectory	Specifies the path where the source code should be written. It can be either an absolute or relative path to a directory. The specified directory must already be present.
	<pre># Windows --outputDirectory "c:\my app\scripts" # Linux or OSX --outputDirectory "~/my app/scripts" # Relative path --outputDirectory "./my app/scripts"</pre>
--languageVersion	Target C# version. By default it is 4.0.
	<pre>--languageVersion CSharp40 --languageVersion CSharp73</pre>
--documentClassName	Name for base class for all documents.
	<pre># name (default) --documentClassName Document # in case of name collision --documentClassName GameDataDocument</pre>
--gameDataClassName	Name for class containing whole in-memory game data.
	<pre># name (default) --gameDataClassName GameData # in case of name collision --gameDataClassName MyGameData</pre>

```

--namespace                               Namespace for all generated classes.
                                           # name (default)
                                           --namespace GameParameters
--defineConstants                         One or more preprocessor constants to define. Use semicolon(;) to separate multiple values.
                                           # Use GameDevWare.Dynamic.Expressions.dll for formulas
                                           --defineConstants USE_DYNAMIC_EXPRESSIONS

                                           # Exclude all formula related code from compilation
                                           --defineConstants SUPPRESS_BUILD_IN_FORMULAS

                                           # Enable some JSON formatting optimizations using System.Memory.dll and System
                                           --defineConstants BUFFERS_TEXT_DLL
--indentation                             Indentation style for generated code.
                                           # Tabs (default)
                                           --indentation Tabs

                                           # Two spaces
                                           --indentation TwoSpaces

                                           # Four spaces
                                           --indentation FourSpaces
--lineEndings                             Line ending symbols for generated code.
                                           # Windows \r\n (default)
                                           --lineEndings Windows

                                           # Unix style \n
                                           --lineEndings Unix
--splitFiles                              Set this flag to lay out generated classes into separate
                                           files. If not set, then one giant file with the name of
                                           --gameDataClassName.cs will be generated.

```


--optimizations **list** of enabled optimization in generated code.

```
# Eagerly resolves and validates all references in loaded documents.
# When enabled, this optimization ensures that all references in documents are
# during loading. This comes with a performance cost but guarantees the validity
--optimizations eagerReferenceResolution

# Opts for raw references without generating helper methods for referenced documents.
# With this optimization, the generated code will not include helper methods for
# referenced documents, keeping only accessors that work with raw references.
--optimizations rawReferences

# Avoids generating helper methods for localized strings, keeping only raw accessors.
# This optimization eliminates helper methods for accessing localized text, instead
# accessors that deal directly with lists of localized texts.
--optimizations rawLocalizedTexts

# Keeps text properties carrying a vector, rectangle or tags editor as plain strings.
# By default such a property is renamed to '<Name>Raw' and a parsed accessor tag
# is generated. This optimization skips the parsing helpers and leaves the property
--optimizations rawCompositeTypes

# Disables string pooling during game data loading.
# Turning off string pooling can yield a minor performance improvement at the cost of
# memory usage, as it avoids reusing short strings.
--optimizations disableStringPooling

# Disables generation of code for loading game data from JSON formatted files.
# This optimization omits code related to JSON serialization, useful when JSON
# game data is not used.
--optimizations disableJsonSerialization

# Disables generation of code for loading game data from Message Pack formatted files.
# Similar to DisableJsonSerialization, this option removes code related to loading
# from Message Pack formatted files.
--optimizations disableMessagePackSerialization

# Disables generation of code related to applying patches during game data loading.
# This removes a significant portion of code that is mainly used for modding situations
# where patches are applied to game data at runtime.
--optimizations disablePatching

# Disables generation of enums with known document IDs.
# This removes a significant portion of code that contains listings of IDs for
# documents known at the moment of code generation, which improves compilation
--optimizations disableDocumentIdEnums
```

--clearOutputDirectory Clear the output directory from generated files.
Generated files are identified by the presence of the
<auto-generated> tag inside.

This command supports universal parameters.

See also

- Generating Source Code
- Working with Source Code (C# 7.3)
- Working with Source Code (C# 4.0)
- Publishing Game Data
- GENERATE TYPESCRIPTCODE

- GENERATE UECPP
- GENERATE HAXE

Generate Haxe Source Code

Generates Haxe source code for game data into output directory.

By default, this command does not delete previously generated files. Use the `--clearOutputDirectory` parameter to automatically remove prior generated files before creating new ones.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- GENERATE HAXECODE \
  --dataBase "c:\my app\gamedata.json" \
  --packageName "" \
  --outputDirectory "c:\my app\scripts"

# remote game data
dnx dotnet-charon -- GENERATE HAXECODE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --packageName "" \
  --outputDirectory "./scripts" \
  --credentials "<API-Key>"
```

Parameters

<code>--dataBase</code>	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
<code>--credentials</code>	The API key used to access remote server in case of <code>--dataBase</code> being URL.
<code>--outputDirectory</code>	Specifies the path where the source code should be written. It can be either an absolute or relative path to a directory. The specified directory must already be present.
	<pre># Windows --outputDirectory "c:\my app\scripts" # Linux or OSX --outputDirectory "~/my app/scripts" # Relative path --outputDirectory "./my app/scripts"</pre>

--documentClassName	Name for base class for all documents. <pre># name (default) --documentClassName Document # in case of name collision --documentClassName GameDataDocument</pre>
--gameDataClassName	Name for class containing whole in-memory game data. <pre># name (default) --gameDataClassName GameData # in case of name collision --gameDataClassName MyGameData</pre>
--packageName	Package name for all generated classes. <pre># empty package (default) --packageName "" # named --packageName GameParameters</pre>
--defineConstants	Preprocessor constants to define. Use semicolon(;) to separate multiple values. <pre>--defineConstants NO_OPTIMIZATIONS</pre>
--indentation	Indentation style for generated code. <pre># Tabs (default) --indentation Tabs # Two spaces --indentation TwoSpaces # Four spaces --indentation FourSpaces</pre>
--lineEndings	Line ending symbols for generated code. <pre># Windows \r\n (default) --lineEndings Windows # Unix style \n --lineEndings Unix</pre>
--splitFiles	Set this flag to lay out generated classes into separate files. If not set, then one giant file with the name of --gameDataClassName.hx will be generated.

--optimizations **list** of enabled optimization in generated code.

```
# Eagerly resolves and validates all references in loaded documents.
# When enabled, this optimization ensures that all references in documents are
# during loading. This comes with a performance cost but guarantees the validity
--optimizations eagerReferenceResolution

# Opts for raw references without generating helper methods for referenced documents.
# With this optimization, the generated code will not include helper methods for
# referenced documents, keeping only accessors that work with raw references.
--optimizations rawReferences

# Avoids generating helper methods for localized strings, keeping only raw accessors.
# This optimization eliminates helper methods for accessing localized text, instead
# accessors that deal directly with lists of localized texts.
--optimizations rawLocalizedTexts

# Keeps text properties carrying a vector, rectangle or tags editor as plain strings.
# By default such a property is renamed to '<Name>Raw' and a parsed accessor tag
# is generated. This optimization skips the parsing helpers and leaves the property
--optimizations rawCompositeTypes

# Disables string pooling during game data loading.
# Turning off string pooling can yield a minor performance improvement at the cost of
# memory usage, as it avoids reusing short strings.
# --optimizations disableStringPooling (N/A)

# Disables generation of code for loading game data from JSON formatted files.
# This optimization omits code related to JSON serialization, useful when JSON
# game data is not used.
--optimizations disableJsonSerialization

# Disables generation of code for loading game data from Message Pack formatted files.
# Similar to DisableJsonSerialization, this option removes code related to loading
# from Message Pack formatted files.
--optimizations disableMessagePackSerialization

# Disables generation of code related to applying patches during game data loading.
# This removes a significant portion of code that is mainly used for modding situations
# where patches are applied to game data at runtime.
--optimizations disablePatching

# Disables generation of enums with known document IDs.
# This removes a significant portion of code that contains listings of IDs for
# documents known at the moment of code generation, which improves compilation time.
--optimizations disableDocumentIdEnums
```

--clearOutputDirectory Clear the output directory from generated files.
Generated files are identified by the presence of the
<auto-generated> tag inside.

This command supports universal parameters.

See also

- Generating Source Code
- Working with Haxe Code
- GENERATE CSHARPCODE
- GENERATE TYPESCRIPTCODE
- GENERATE UECPPCODE

- Publication

Generate Lua Source Code

Generates Lua source code for game data into output directory.

By default, this command does not delete previously generated files. Use the `--clearOutputDirectory` parameter to automatically remove prior generated files before creating new ones.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- GENERATE LUACODE \
  --dataBase "c:\my app\gamedata.json" \
  --outputDirectory "c:\my app\scripts"

# remote game data
dnx dotnet-charon -- GENERATE LUACODE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --outputDirectory "./scripts" \
  --credentials "<API-Key>"
```

Parameters

<code>--dataBase</code>	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
<code>--credentials</code>	The API key used to access remote server in case of <code>--dataBase</code> being URL.
<code>--outputDirectory</code>	Specifies the path where the source code should be written. It can be either an absolute or relative path to a directory. The specified directory must already be present.
	<pre># Windows --outputDirectory "c:\my app\scripts" # Linux or OSX --outputDirectory "~/my app/scripts" # Relative path --outputDirectory "./my app/scripts"</pre>
<code>--documentClassName</code>	Name for base class for all documents.
	<pre># name (default) --documentClassName GameDataDocument # in case of name collision --documentClassName MyDocument</pre>

<code>--gameDataClassName</code>	Name for class containing whole in-memory game data. <i># name (default)</i> <code>--gameDataClassName GameData</code> <i># in case of name collision</i> <code>--gameDataClassName MyGameData</code>
<code>--indentation</code>	Indentation style for generated code. <i># Tabs (default)</i> <code>--indentation Tabs</code> <i># Two spaces</i> <code>--indentation TwoSpaces</code> <i># Four spaces</i> <code>--indentation FourSpaces</code>
<code>--lineEndings</code>	Line ending symbols for generated code. <i># Windows \r\n (default)</i> <code>--lineEndings Windows</code> <i># Unix style \n</i> <code>--lineEndings Unix</code>
<code>--splitFiles</code>	Set this flag to lay out generated classes into separate files. If not set, then one giant file with the name of <code>--gameDataClassName.lua</code> will be generated. Default is true.

--optimizations **list of enabled optimization in generated code.**

```
# Eagerly resolves and validates all references in loaded documents.
# When enabled, this optimization ensures that all references in documents are
# during loading. This comes with a performance cost but guarantees the validity
--optimizations eagerReferenceResolution

# Opts for raw references without generating helper methods for referenced documents.
# With this optimization, the generated code will not include helper methods for
# referenced documents, keeping only accessors that work with raw references.
--optimizations rawReferences

# Avoids generating helper methods for localized strings, keeping only raw accessors.
# This optimization eliminates helper methods for accessing localized text, instead
# accessors that deal directly with lists of localized texts.
--optimizations rawLocalizedTexts

# Keeps text properties carrying a vector, rectangle or tags editor as plain strings.
# By default such a property is renamed to '<Name>Raw' and a parsed accessor tag
# is generated. This optimization skips the parsing helpers and leaves the property
--optimizations rawCompositeTypes

# Disables string pooling during game data loading.
# Turning off string pooling can yield a minor performance improvement at the cost of
# memory usage, as it avoids reusing short strings.
# --optimizations disableStringPooling (N/A)

# Disables generation of code for loading game data from JSON formatted files.
# This optimization omits code related to JSON serialization, useful when JSON
# game data is not used.
# --optimizations disableJsonSerialization (N/A)

# Disables generation of code for loading game data from Message Pack formatted files.
# Similar to DisableJsonSerialization, this option removes code related to loading
# from Message Pack formatted files.
# --optimizations disableMessagePackSerialization (N/A)

# Disables generation of code related to applying patches during game data loading.
# This removes a significant portion of code that is mainly used for modding situations
# where patches are applied to game data at runtime.
# --optimizations disablePatching (N/A)

# Disables generation of enums with known document IDs.
# This removes a significant portion of code that contains listings of IDs for
# documents known at the moment of code generation, which improves compilation time.
--optimizations disableDocumentIdEnums
```

--clearOutputDirectory

Clear the output directory from generated files.
Generated files are identified by the presence of the
<auto-generated> tag inside.

This command supports universal parameters.

See also

- Generating Source Code
- Working with Lua Code
- GENERATE CSHARPCODE
- GENERATE TYPESCRIPTCODE
- GENERATE UECPP

- GENERATE HAXE
- Publication

Export Code Generation Templates (Obsolete)

Exports [T4](#) code generation templates to a specified directory. These templates can be used with Visual Studio, Rider, Visual Studio Code with plugin, dotnet [tool](#) or other tools to generate source code.

Warning

This command will be removed in future versions. Download templates directly from the [repository](#).

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# Windows
dnx dotnet-charon -- GENERATE TEMPLATES --outputDirectory "c:\templates"
```

Parameters

--outputDirectory

Specifies the path where the templates should be written. It can be either an absolute or relative path to a directory. The specified directory must already be present.

```
# Windows
--outputDirectory "c:\templates"

# Linux or OSX
--outputDirectory "~/templates"

# Relative path
--outputDirectory "../templates"
```

See also

- Generating Source Code
- GENERATE CSHARPCODE
- GENERATE TYPESCRIPTCODE
- GENERATE UECPPCODE
- [T4 Text Templates](#)

Generate Text from Templates (Obsolete)

Generates text from T4 templates into output directory.

Warning

This command was removed since 2025.1.1 version. It is recommended to use an IDE or open-source alternatives for generating text with T4 templates. See: [dotnet-t4](#)

See GENERATE TEMPLATES to get actual T4 templates.

- CLI Installation
- Commands Reference

See also

- Generating Source Code
- GENERATE TEMPLATES
- GENERATE CSHARPCODE
- [dotnet-t4 tool](#)

Generate TypeScript Source Code

Generates TypeScript source code for game data into output directory.

By default, this command does not delete previously generated files. Use the `--clearOutputDirectory` parameter to automatically remove prior generated files before creating new ones.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- GENERATE TYPESCRIPTCODE \
  --dataBase "c:\my app\gamedata.json" \
  --outputDirectory "c:\my app\scripts"

# remote game data
dnx dotnet-charon -- GENERATE TYPESCRIPTCODE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --outputDirectory "./scripts" \
  --credentials "<API-Key>"
```

Parameters

<code>--dataBase</code>	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\my app\gamedata.json"</pre>
	<pre># remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
<code>--credentials</code>	The API key used to access remote server in case of <code>--dataBase</code> being URL.

--outputDirectory

Specifies the path where the source code should be written. It can be either an absolute or relative path to a directory. The specified directory must already be present.

```
# Windows
--outputDirectory "c:\my app\scripts"
```

```
# Linux or OSX
--outputDirectory "~/my app/scripts"
```

```
# Relative path
--outputDirectory "./my app/scripts"
```

--documentClassName

Name for base class for all documents.

```
# name (default)
--documentClassName Document
```

```
# in case of name collision
--documentClassName GameDataDocument
```

--gameDataClassName

Name for class containing whole in-memory game data.

```
# name (default)
--gameDataClassName GameData
```

```
# in case of name collision
--gameDataClassName MyGameData
```

--indentation

Indentation style for generated code.

```
# Tabs (default)
--indentation Tabs
```

```
# Two spaces
--indentation TwoSpaces
```

```
# Four spaces
--indentation FourSpaces
```

--lineEndings

Line ending symbols for generated code.

```
# Windows \r\n (default)
--lineEndings Windows
```

```
# Unix style \n
--lineEndings Unix
```

--splitFiles

Set this flag to lay out generated classes into separate files. If not set, then one giant file with the name of --gameDataClassName.ts will be generated.

--optimizations **list** of enabled optimization in generated code.

```
# Eagerly resolves and validates all references in loaded documents.
# When enabled, this optimization ensures that all references in documents are
# during loading. This comes with a performance cost but guarantees the validity
--optimizations eagerReferenceResolution

# Opts for raw references without generating helper methods for referenced documents.
# With this optimization, the generated code will not include helper methods for
# referenced documents, keeping only accessors that work with raw references.
--optimizations rawReferences

# Avoids generating helper methods for localized strings, keeping only raw accessors.
# This optimization eliminates helper methods for accessing localized text, instead
# accessors that deal directly with lists of localized texts.
--optimizations rawLocalizedTexts

# Keeps text properties carrying a vector, rectangle or tags editor as plain strings.
# By default such a property is renamed to '<Name>Raw' and a parsed accessor tag
# is generated. This optimization skips the parsing helpers and leaves the property
--optimizations rawCompositeTypes

# Disables string pooling during game data loading.
# Turning off string pooling can yield a minor performance improvement at the cost of
# memory usage, as it avoids reusing short strings.
# --optimizations disableStringPooling (N/A)

# Disables generation of code for loading game data from JSON formatted files.
# This optimization omits code related to JSON serialization, useful when JSON
# game data is not used.
--optimizations disableJsonSerialization

# Disables generation of code for loading game data from Message Pack formatted files.
# Similar to DisableJsonSerialization, this option removes code related to loading
# from Message Pack formatted files.
--optimizations disableMessagePackSerialization

# Disables generation of code related to applying patches during game data loading.
# This removes a significant portion of code that is mainly used for modding situations
# where patches are applied to game data at runtime.
--optimizations disablePatching

# Disables generation of enums with known document IDs.
# This removes a significant portion of code that contains listings of IDs for
# documents known at the moment of code generation, which improves compilation time.
--optimizations disableDocumentIdEnums
```

--clearOutputDirectory Clear the output directory from generated files.
Generated files are identified by the presence of the
<auto-generated> tag inside.

This command supports universal parameters.

See also

- Generating Source Code
- Working with TypeScript Code
- GENERATE CSHARPCODE
- GENERATE UECPPCODE
- GENERATE HAXECODE

- Publication

Generate Unreal Engine C++ Source Code

Generates C++ for Unreal Engine source code for game data into output directory.

By default, this command does not delete previously generated files. Use the `--clearOutputDirectory` parameter to automatically remove prior generated files before creating new ones.

- CLI Installation
- Commands Reference
- Universal Parameters
- URL-based Input/Output

Command

```
# local game data (windows)
dnx dotnet-charon -- GENERATE UECPPCODE \
  --dataBase "c:\My Project\Content\gamedata.json" \
  --outputDirectory "c:\My Project\Source\Gamedata"

# remote game data
dnx dotnet-charon -- GENERATE UECPPCODE \
  --dataBase "https://charon.live/view/data/My_Game/develop/" \
  --outputDirectory "../My Project/Source/Gamedata" \
  --credentials "<API-Key>"
```

Parameters

<code>--dataBase</code>	Absolute or relative path to game data. Use quotation marks if your path contains spaces.
	<pre># local file --dataBase "c:\My Project\Content\gamedata.json" # remote server --dataBase "https://charon.live/view/data/My_Game/develop/"</pre>
<code>--credentials</code>	The API key used to access remote server in case of <code>--dataBase</code> being URL.
<code>--outputDirectory</code>	Specifies the path where the source code should be written. It can be either an absolute or relative path to a directory. The specified directory must already be present.
	<pre># Windows --outputDirectory "c:\My Project\Source\Gamedata" # Linux or OSX --outputDirectory "~/My Project/Source/Gamedata" # Relative path --outputDirectory "../My Project/Source/Gamedata"</pre>
<code>--documentClassName</code>	Name for base class for all documents.
	<pre># name (default) --documentClassName Document # became UDocument in generated code # in case of custom inheritance chain # class SHOULD publicly inherit UDocument --documentClassName GameDataDocument # became UGameDataDocument in generated code</pre>

```
--gameDataClassName Name for class containing whole in-memory game data.
    # name (default)
    --gameDataClassName GameData # became UGameData in generated code

    # in case of name collision
    --gameDataClassName MyGameData # became UMyGameData in generated code

--namespace
    Accepted for symmetry with the other generators, but
    ignored by the Unreal Engine C++ target - generated
    types are named with the U/F prefixes Unreal expects
    instead of being scoped into a namespace. Use
    --documentClassName and
    --gameDataClassName to control the generated
    names.

--defineConstants
    Preprocessor constants to define. Use semicolon(;) to separate
    multiple values.
    --defineConstants NO_OPTIMIZATIONS;USE_FSTRING_ONLY

--indentation
    Indentation style for generated code.
    # Tabs (default)
    --indentation Tabs

    # Two spaces
    --indentation TwoSpaces

    # Four spaces
    --indentation FourSpaces

--lineEndings
    Line ending symbols for generated code.
    # Windows \r\n (default)
    --lineEndings Windows

    # Unix style \n
    --lineEndings Unix

--splitFiles
    Accepted for symmetry with the other generators, but
    ignored by the Unreal Engine C++ target - it always
    writes one header per type into Public/ with the
    matching sources in Private/.
```

--optimizations **list of enabled optimization in generated code.**

```
# Eagerly resolves and validates all references in loaded documents.
# When enabled, this optimization ensures that all references in documents are
# during loading. This comes with a performance cost but guarantees the validity
--optimizations eagerReferenceResolution

# Opts for raw references without generating helper methods for referenced documents.
# With this optimization, the generated code will not include helper methods for
# referenced documents, keeping only accessors that work with raw references.
--optimizations rawReferences

# Avoids generating helper methods for localized strings, keeping only raw accessors.
# This optimization eliminates helper methods for accessing localized text, instead
# accessors that deal directly with lists of localized texts.
--optimizations rawLocalizedTexts

# Not applicable. Unreal Engine C++ never renames a property, so a text property
# vector, rectangle or tags editor always keeps its name and gets a separate Get
# accessor beside it.
# --optimizations rawCompositeTypes (N/A)

# Disables string pooling during game data loading.
# Turning off string pooling can yield a minor performance improvement at the cost of
# memory usage, as it avoids reusing short strings.
# --optimizations disableStringPooling (N/A)

# Disables generation of code for loading game data from JSON formatted files.
# This optimization omits code related to JSON serialization, useful when JSON
# game data is not used.
--optimizations disableJsonSerialization

# Disables generation of code for loading game data from Message Pack formatted files.
# Similar to DisableJsonSerialization, this option removes code related to loading
# from Message Pack formatted files.
--optimizations disableMessagePackSerialization

# Disables generation of code related to applying patches during game data loading.
# This removes a significant portion of code that is mainly used for modding situations
# where patches are applied to game data at runtime.
--optimizations disablePatching

# Disables generation of enums with known document IDs.
# This removes a significant portion of code that contains listings of IDs for
# documents known at the moment of code generation, which improves compilation time.
--optimizations disableDocumentIdEnums
```

--clearOutputDirectory Clear the output directory from generated files.
Generated files are identified by the presence of the
<auto-generated> tag inside.

This command supports universal parameters.

See also

- Generating Source Code
- Working with Unreal Engine C++ Code
- GENERATE CSHARPCODE
- GENERATE TYPESCRIPTCODE
- GENERATE HAXECODE

- Publication

Initialize Game Data

Initializes an empty or missing file with initial data. Path to game data should be local file system's file.

- CLI Installation
- Commands Reference

Command

```
# full path (windows)
dnx dotnet-charon -- INIT "c:\my app\gamedata.gdjs"

# full path (linux)
dnx dotnet-charon -- INIT "/var/mygame/gamedata.json"

# relative path
dnx dotnet-charon -- INIT mygame/gamedata.json
```

Note

The parent directory must already exist - INIT creates the file, not the folders leading to it.

Parameters

--fileName

Absolute or relative path to game data file. The directory must already exist. Use quotation marks if your path contains spaces.

```
# local file
--fileName "c:\my app\gamedata.json"
```

See also

- Command Line Reference
- Game Data Structure
- SERVER START

URL input/output parameters

Some command accept [URL](#) as input/output parameter.

Supported URL Schemes

Scheme	Input parameter	Output parameter
HTTP[S]	A GET request will be sent	A POST request with body containing output will be sent
FTP(S)	A RETR command will be sent	A STOR command with output content will be sent
FILE	File will be read	File will be written

Authentication

Authentication data could be passed in *user* part of [URL](#). More advanced authentication schemes are not supported.

Examples

```
# publish data to FTP
dnx dotnet-charon -- DATA EXPORT \
  --dataBase "https://charon.live/view/data/My_Game/develop/dashboard" \
  --output "ftp://user:password@example.com/public/gamedata.json" \
  --mode publication \
  --outputFormat json \
  --credentials "<API-Key>"

# import localization from remote HTTP server
dnx dotnet-charon -- DATA I18N IMPORT \
  --dataBase "file:///c:/my app/gamedata.json" \
  --input "https://example.com/translated/gamedata.xliff" \
  --inputFormat xliff

# print languages for game data in local file
dnx dotnet-charon -- DATA I18N LANGUAGES --dataBase "file:///c:/my app/gamedata.json"

# print languages for game data in local file relative to current working directory
dnx dotnet-charon -- DATA I18N LANGUAGES --dataBase "file:///./gamedata.json"

# print languages for game data at remote server using API Key
export CHARON_API_KEY=87758CC0D7C745D0948F2A8AFE61BC81
dnx dotnet-charon -- DATA I18N LANGUAGES --dataBase "https://charon.live/view/data/My_Game/d
```

See also

- [Command Line Reference](#)
- [DATA EXPORT](#)
- [DATA IMPORT](#)
- [DATA I18N EXPORT](#)
- [URI \(Wikipedia\)](#)
- [FTP protocol \(Wikipedia\)](#)

Start in Standalone Mode

Starts Charon in standalone mode for specified game data. Path to game data could be local file system's file or remote server address.

- [CLI Installation](#)
- [Commands Reference](#)
- [Universal Parameters](#)
- [URL-based Input/Output](#)

Command

```
# local game data (windows)
dnx dotnet-charon -- SERVER START \
  --dataBase "c:\my app\gamedata.json" \
  --port 8080 \
  --launchDefaultBrowser

# shortcut version
charon "c:\my app\gamedata.json"
```


Parameters

<code>--dataBase</code>	Absolute or relative path to game data. Use quotation marks if your path contains spaces. <i># local file</i> <code>--dataBase "c:\my app\gamedata.json"</code>
<code>--port</code>	Number of an IP port (1-65535) to be used to listen for browser based UI. <code>--port 8080</code>
<code>--launchDefaultBrowser</code>	Set this flag to open system-default browser on successful start.
<code>--denySchemaEditing</code>	Disable the ability to change the data structure (metadata) for editable game data.
<code>--resetPreferences</code>	Set this flag to reset UI preferences on successful start.
<code>--changesFile</code>	<i>Added in version 2026.4.0.</i> Path to a changes file. When set, the game data file is opened read-only and every edit is written into this file as a patch document instead of into the game data file. Overrides the <code>changesFile</code> query parameter of <code>--dataBase</code> . Apply the patch later with DATA APPLYPATCH. <code>--changesFile "./player-changes.patch.json"</code>

This command supports universal parameters.

See also

- [Command Line Reference](#)
- [DATA APPLYPATCH](#)
- [Web Application Overview](#)
- [IP port \(Wikipedia\)](#)

Universal parameters

All commands accept universal parameters and environment variables.

<code>--verbose</code>	Set this flag to get additional diagnostic information in logs.
<code>--log <path></code>	Add additional file logging to the existing logging configuration from <code>appsettings.json</code> . <code>--log "./logs/charon.log"</code>
<code>--log out</code>	Add additional terminal (standard output) logging to the existing logging configuration from <code>appsettings.json</code> . <code>--log out</code> <i># or</i> <code>--log con</code>
<code>--pause</code>	Wait for user prompt before the application exits.

Environment variables

In addition to the standard configuration redefinition [mechanism](#) using environment variables, the following environment variables are also supported.

CHARON_API_KEY

The API key which is used to access the remote server. This environment variable is usually used in conjunction with `--dataBase`, which points to a remote server.

```
# Windows
set CHARON_API_KEY=87758CC0D7C745D0948F2A8AFE61BC81

# OSX or Linux
export CHARON_API_KEY=87758CC0D7C745D0948F2A8AFE61BC81
```

See also

- [Command Line Reference](#)
- [CLI Access to charon.live](#)
- [URL-based Input/Output](#)
- [Working with Logs](#)

Get Charon Version

Gets version of charon application.

- [CLI Installation](#)
- [Commands Reference](#)

Command

```
# Windows, Linux or OSX
dnx dotnet-charon -- VERSION
#> 2023.2.3-alpha
```

Parameters

This command supports universal parameters.

See also

- [Command Line Reference](#)
- [Universal Parameters](#)

Frequently Asked Questions (FAQ)

What kind of game data can be modeled in Charon?

Charon is designed for modeling static game data. Typical use cases include Quests, Dialogs, Levels, Loot Tables, Items, Monsters, Abilities, Units, Research Trees, Weapons, Spells, Goods, Buildings, and similar structured data used to drive gameplay logic and content.

What data types are supported in Charon?

Charon provides a broad set of data types to suit a variety of needs. These include primitive types such as text, localized text, booleans, time, dates, numbers, enums (pick lists), and flags. More advanced structures are also supported, including embedded documents, collections, references to other documents, and computed formulas.

Is schema inheritance supported?

Yes, schema inheritance is supported in Charon, although it is implemented using a composition-based approach rather than traditional inheritance. Existing schemas can be extended by including them as Document properties within other schemas. Read more about inheritance.

Is Charon free to use?

The offline version, including the command-line interface (CLI) and associated plugins, is free to use and distributed under a permissive license. It imposes no restrictions and may be bundled with games to support modding workflows.

The [online](#) version, which enables collaborative editing of shared game data in teams, is available under a subscription model.

Are there limitations in the Unity/Unreal plugins or CLI version?

No, there are no imposed limitations on the CLI version or the Unity and Unreal Engine plugins. This includes file size, number of documents, or schema complexity.

Does Charon support localization?

Yes, Charon supports multi-language content through the localized text type. Localizable text can be exported for translation and imported back once translated. In-game, language switching is supported globally with a single function call, ensuring a consistent localization experience.

How does Charon integrate with Unity or Unreal Engine?

Charon provides dedicated plugins for both Unity and Unreal Engine. These plugins integrate natively into each engine's workflow and expose game data as strongly typed assets. Full source code is included, along with example projects demonstrating how to extend and use the plugins within game code.

Can I define relationships between documents (e.g., parent-child or references)?

Yes. Charon supports embedding documents within each other to model hierarchical structures (e.g., trees). It also allows referencing other documents through reference fields or reference collections. These references can be automatically validated for consistency, avoiding the common pitfalls of spreadsheet-based data modeling.

Can I generate custom editors or tools with Charon?

Yes. Charon is extensible through custom field and document editors. It can also be embedded in other applications using an embedded browser such as CEF. For example, the Unreal Engine plugin demonstrates how to run the Charon UI inside an in-editor browser, allowing you to build a seamless experience around your tools.

Does Charon support formulas or computed fields?

Yes. Charon includes support for C#-like formulas as a first-class data type. These formulas can be evaluated at runtime with user-defined parameters, allowing you to move balance logic and other calculations out of compiled code and into editable data structures.

Can Charon be integrated into CI/CD pipelines or custom toolchains?

Yes. Charon includes a command-line interface (CLI) that supports full CRUD operations and data import/export. The CLI can work with local game data stored in JSON or MessagePack formats, as well as connect to the online service. You can script uploads, downloads, merges, and validation steps as part of your CI/CD workflow.

What export formats does Charon support?

Charon supports multiple export and import formats for game data, including JSON, MessagePack, BSON, and XLSX (spreadsheets). For localization, the XLIFF format is also supported. Runtime and asset storage formats are limited to JSON and MessagePack.

Can I control user access and editing permissions?

Yes. In collaborative environments using the online version of Charon, a role-based access model is available. Roles include Administrator, Schema Designer, Editor, and Viewer, each with a defined set of permissions. For more information, see Roles and Permissions.

Is the editor's source code available?

The source code for the editor itself is not publicly available. However, the plugin source code is available on GitHub: [Charon GitHub Repositories](#)

Charon can be extended with custom field editors or even full document editors, such as node-based interfaces.

For teams requiring a fully private, self-contained collaborative workspace, an **on-premises installation** of the online service is available. It provides the same features as the hosted version - real-time collaboration, access control, multiple projects, AI features, Machine Translation, and team workflows - while keeping all data and infrastructure under your control. If you are interested in an on-premises deployment, please contact us support@gamedevware.com for licensing and installation options.

What happens if the project is discontinued or no longer maintained?

In the unlikely event that Charon is no longer maintained or all original contributors become unavailable, a contingency plan is in place. The online service domain would eventually become inaccessible; however, the full source code of the server, as well as private Docker images, will be published under the MIT license by a designated successor. This ensures that the system can be self-hosted or forked by the community.

The CLI and engine plugins rely on published NuGet packages, which are already public and will remain unaffected. Users working in offline mode will experience no disruption. For users of the online version, all project data is periodically cloned and stored locally. These local copies remain fully compatible with the standalone application, allowing teams to continue working without interruption.

Glossary

Game Data

The static information for the game, such as items, quests, dialogues, etc., is stored as game data. Schemas are also included to organize and structure game data.

Schema

A schema is a description of the structure for documents in game data. It defines the properties and structure of each document in the game data.

Schema Type

A field of a schema that controls how its documents are created, stored, and navigated. Available types are Normal, Component, Settings, and Union.

Component

A schema type for sub-objects that are always embedded inside another document and never exist as standalone root-level entries. Documents of a component schema do not appear in the navigation menu and are not exported on their own.

Settings Schema

A singleton schema type. Exactly one document of this schema exists in the root-level collection; it is created automatically and cannot be deleted. Suited for project-wide configuration.

Tagged Union

A schema type (also known as a discriminated union) that allows documents of different shapes, called variants, to coexist within the same collection or be embedded inside other documents.

Schema Property

A part of a schema that defines a specific property or attribute of a document in the game data.

Id Property

A required schema property, always named `Id`, that holds the unique identifier of a document. Its value can be entered manually or produced automatically by an Id generator.

Shared Property

A schema property that synchronizes its data type-related configuration with properties of the same name in other schemas. Modifying a shared property in one schema updates all other synced properties. A shared property can be unlinked at any time, making it schema-specific again.

Data Type

The kind of value a schema property holds, such as Text, Integer, Number, Logical, Date, Pick List, Localized Text, Reference, Document, or Formula.

Formula

A property data type in a schema for a C# expression that can be executed at runtime to calculate a value for a field.

Reference

A property data type in a schema for a pointer to another document.

Pick List

A property data type for a single value chosen from a predefined list of options. The Multi-Pick List variant allows several options to be selected at once. In generated source code, pick list properties produce enum types. Making a pick list a Shared Property keeps its options synchronized across schemas, producing a single shared enum type instead of a separate one per schema.

Localized Text

A property data type that stores text in multiple translation languages, enabling localization workflows.

Asset Path

A sub-type of the Text data type used to store a path to a game asset (e.g. a texture or prefab) within game data. Paths are typically relative to the project root and can be used to load assets at runtime.

Specification

A flexible extension field available on schemas and schema properties. It stores URL-encoded key-value pairs (e.g. `icon=table&group=Combat`) used for custom logic in UI editors, source code generation, and other tooling.

Display Text Template

A schema-level template that defines how documents are labeled in the UI, using replaceable expressions in curly braces (e.g. `Quest #{Id}: {Description}`). Without it, documents are labeled by the `DisplayName`, `Name`, `Title`, or `Id` property.

Document

A specific instance of a schema in the game data. It represents a single item or entity in the game, such as an item, quest, or dialogue.

Document Collection

A list of documents of the same schema. Root-level collections appear in the navigation menu; embedded collections are stored inside another document as a property value.

Field

A named part of a document that holds a specific value.

Source Code

The code generated by Charon that represents the game data. This code can be used to load the game data at runtime.

Metadata

All the schemas and relations between them. This data is used by Charon to generate the source code for the game data.

Validation

The process of checking game data against schema constraints and integrity rules. Validation runs on every document save, on import, and on demand; selected checks can automatically repair common problems.

Import

The process of loading documents into game data from an external file in a supported format (JSON, BSON, MessagePack, XLSX, etc.).

Export

The process of writing documents from game data to an external file in a supported format, optionally filtered by schemas, properties, or languages.

Patch

A file describing the difference between two game data files as a set of additions, modifications, and deletions. A patch can be applied to any compatible target file, enabling branching workflows, incremental updates, and modding.

Backup

A complete snapshot of game data and metadata saved to a single file, which can later be restored to replace the current content.

Internationalization (i18n)

The workflow of translating game data. Localized Text fields are exported to XLSX, XLIFF, or JSON, translated externally, and imported back.

Translation Language

A language configured in the project settings for which Localized Text fields can store values.

Publication

The process of exporting game data in a format that can be loaded into the game.

Modding

A workflow where the free version of Charon is shipped with the game, letting players edit a copy of the game data. Player changes are captured as patch files that the game applies on top of the base data at startup.

CLI

The command-line interface of the `dotnet-charon` tool. It provides commands for editing, importing, exporting, validating, and patching game data, as well as generating source code — used for automation and CI/CD pipelines.

REST API

An HTTP API exposed by Charon that lets external tools perform the same data operations available in the UI and CLI.

MCP Server

A built-in Model Context Protocol server that lets AI assistants read and write game data, generate source code, and manage localization. It communicates over stdio using the standard JSON-RPC MCP transport.

UI Extension

An external NPM package containing Web Components that add custom field or document editors to the document form.

Standalone Edition

The locally installed distribution of Charon that works with game data files on the local disk, without requiring a game engine integration or a server.

Web Application

The hosted, collaborative version of Charon at charon.live, where teams work on shared projects organized into workspaces.

Server Edition

The team collaboration layer built on top of the standalone editor, giving studios a shared game data repository accessed through a browser. Available as the hosted web application or self-hosted on your own infrastructure.

Workspace

In the web application, the workspace is the place where users can manage their projects and subscription.

Project

In the web application, a project is a container for organizing related game data. It can contain multiple branches.

Branch

In the web application, a branch is a specific variant of game data within a project. It can be used to manage different versions or stages of the game data.

API Key

Is It Free?

A unique identifier generated for a user in the User's Profile "API Keys" section, which can be used to access the REST API and CLI for various operations. It allows for automation of game build processes, such as pushing game data to local GIT repositories.